CLAUDE

Helix Constitution — Universal CLAUDE.md

Field	Value
Revision	68
Created	2026-05-14
Last modified	2026-07-26T18:27:34Z
Status	active

Field	Value
Status summary	Amendment of 2026-07-26 — NEW universal anchor §11.4.234 (operator mandate 2026-07-26, verbatim: "We MUST have dedicated script to run hook validations, and commit and push mechanism ALWAYS unblocked!"): hook validations run as an EXPLICIT stage of a dedicated, idempotent commit/push script (consumer-bound path as DATA per §11.4.35 — Lava binding: scripts/commit-push-all.sh); automatic git hooks wired via core.hooksPath MUST NOT gate routine commit/push (disconnect = hooksPath unset, hook file preserved UNMODIFIED, --no-verify never the routine path); NO gate is lost (every disconnected check remains executed at a named script stage or release/tag gate, silently dropping one is a §11.4 PASS-bluff at the process layer); the commit/push mechanism is ALWAYS unblocked (failing validation = clear per-check report + documented remediation path, long multi-minute gates separable from cheap checks and skippable ONLY via an explicit recorded deferral flag — Lava binding: LAVA_SYNC_SKIP_CI=1 — never an opaque hung/rejected push). Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-234-PROPAGATION + recommended mechanism gates CM-DEDICATED-HOOK-VALIDATION-SCRIPT / CM-HOOKS-NEVER-BLOCK-PUSH / CM-NO-GATE-LOST-ON-HOOK-DISCONNECT / CM-COMMIT-PUSH-ALWAYS-UNBLOCKED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). Amendment region byte-identical across all 5 governance files (md5-verified); §11.4.234 exactly-once block-start per file (§11.4.227(B)), no anchor-number collision. CLAUDE.md + AGENTS.md + QWEN.md + GEMINI.md mirrors updated in lockstep (§11.4.157). Generated .html/.pdf/.docx twins NOT regenerated (exporter still not a runnable in-repo script — honestly STALE per §11.4.106(E)). Prior round: Amendment of 2026-07-26 — the EFFORT-tiering + 5-field-label batch (operator IMPORTANT mandate 2026-07-26, verbatim: "Besides dynamic determining proper models to use, we MUST decide the effort as well and switch between the effort settings same as we do now for chosen model! Labeling MUST BE extended for this details as well. Example: (T1/main - claude4 - opus - high)."). §11.4.182 EXTENSION amended (in-place, no re-mint): the work-stream label form extends from `(T<N>/<branch> - <alias> - <model>)` to `(T<N>/<branch> - <alias> - <model> - <effort>)`, effort ∈ {low
Issues	none

Field	Value
Issues summary	—
Fixed	§11.4.108 mirror
Fixed summary	§11.4.107 lands in lockstep with the Constitution.md §11.4.107 addition.
Continuation	—

Table of contents

- When in doubt

> This is the **base CLAUDE.md** imported by every project that includes the Helix Constitution submodule. Project-level `CLAUDE.md` may extend or tighten any rule by adding an explicit `Override: <section>` block — but MUST NOT weaken them.
>
> Last revision: 2026-05-14

How inheritance works

A consuming project's root `CLAUDE.md` MUST start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md`
it references) apply unconditionally. The project-specific rules below
extend them.
```

Claude Code supports the `@path/to/file` import syntax natively, so a consuming project can also write `@constitution/CLAUDE.md` at the top of its own CLAUDE.md and Claude Code will resolve it recursively. For agents that do not support `@imports`, the pointer-block pattern above ensures the inheritance is at least readable.

MANDATORY DEVELOPMENT PRINCIPLES

NO BLUFF. Every change ships with positive-evidence validation on the real target environment. A test that passes without exercising the user-visible behaviour is a critical defect.

(Constitution §7.1 + §11.4 — these references point to `constitution/Constitution.md`.)

CRITICAL: All code changes MUST follow these principles WITHOUT EXCEPTION:

1. **Solutions MUST NOT be error-prone.** Every fix must be robust, not introduce new failure modes. If a fix solves problem A but creates problem B, it is NOT acceptable. Test the fix against ALL existing functionality before committing.
2. **No blocking operations inside synchronized / shared-lock regions.** Long computations, network calls, or sleeps inside `synchronized` / `Lock`-held regions cause deadlocks or timeouts in other threads.
3. **Always consider concurrent callers.** Any method called from multiple threads must be safe for rapid consecutive calls.
4. **Test the fix, not just the symptom.** Verify the fix works AND doesn't break anything else.
5. **Anti-bluff is mandatory** (Constitution §7.1) — every runtime test MUST source the project anti-bluff helper, call at least one explicit user-visible action before any PASS, capture state delta, and assert positive evidence.
6. **Real captured evidence for audio/video** (Constitution §7.1 + §11.4.5) — features producing audio output validated via captured audio; features producing video output validated via captured frames + OCR or pixel-diff.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA, integration suites, smoke tests, acceptance suites) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

Canonical authority: `constitution/Constitution.md` §11.4 and its sub-sections §11.4.1 through §11.4.16.

Non-compliance is a release blocker regardless of context.

§ . . — FAIL-bluffs are equally forbidden

A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

§ . . — Recorded-evidence requirement

A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Every PASS for a user-visible feature MUST be cross-checked against captured recording + action timeline.

§ . . — Per-environment-topology test dispatch

Tests that depend on environment topology MUST detect topology at test entry and dispatch the topology-appropriate variant. SKIP-with-reason is the correct fallback when the required topology is absent; PASS-by-default is forbidden.

§ . . — Test-interrupt-on-discovery + retest-from-clean-baseline

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full rebuild → re-deploy on every target → full retest from beginning.

§ . . — Captured-evidence quality analysis

Audio: presence (RMS amplitude), channel count, sample rate + bit depth, glitch census, coexistence-artifact census. Video: presence (non-zero frame count), routing target, frame health (drops / jitter / freeze), obstruction census (OCR for hostile overlays), resolution + codec. Every check is required for every PASS.

§ . . — No-guessing mandate

Forensic anchor — verbatim user mandate (2026-05-08):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Forbidden vocabulary in tests / gates / status reports / closure narratives / commit messages when describing causes: `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`, `guess`, `seemingly`, `apparently`, `perhaps`, `supposedly`, `conjectured`, and synonyms.

Either prove the cause with captured forensic evidence and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

§ . . — Demotion-evidence rule

Demotion from a FAIL classification to a lower-severity classification requires positive evidence captured under the SAME CONDITIONS (same target, same build, same cycle position, same load profile) that originally exposed the defect. "I cannot reproduce in isolation" is a hypothesis, not a finding.

§ . . — Deep-web-research-before-implementation

Before designing a non-trivial fix, perform deep web research: official docs, vendor technical guides, open-source codebases, coding tutorials, issue trackers. Every non-trivial fix's commit message (or accompanying entry) MUST cite at least one external source OR the literal "NO external solution found — original work".

§ . . — Batch-source-fixes-before-rebuild

All source-side fixes that DO NOT require runtime validation to design MUST be landed BEFORE the next artifact rebuild. The anti-pattern of "fix A → rebuild → flash → cycle → fix B → rebuild → ..." serializes operator time onto rebuild latency.

§ . . — Credentials-handling mandate

Forensic anchor — verbatim user mandate (2026-05-12):

> "Credentials or any secret and sensitive data MUST NOT leak!"

Credentials MUST NEVER be tracked in git. `.env` / `.env.*` / `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + README.md exception) git-ignored project-wide. Test scripts MUST NEVER print or log credentials. Per-service file separation limits blast radius. `chmod 600` on credential files, `chmod 700` on parent directory. Rotation on suspected leak.

§ . . .A — Pre-store credential leak audit (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-17):

> "Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!"

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project's gitignored configuration, the storing agent MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing: (1) `git ls-files | xargs grep -l <value>` for tree-leaks, (2) `git log -S<value> --all --source --remotes` for history-leaks, (3) surface findings to operator BEFORE storing (operator may rotate, accept-as-compromise, or abort), (4) on finding open a §6/§7 sixth-law-incidents record + redact tracked files in-place to `<redacted-per-§11.4.10>` + record OPERATOR ACTION REQUIRED for rotation per §11.4.10 sub-clause 7, (5) extend pre-push hook[11] credential-pattern grep[4] to catch the escaped class in the same commit.[11] See Constitution §11.4.10.A for the full mandate.

§ . . — File-layout discipline

Project files organised by purpose, not historical accident. Source under canonical project roots. Tests under canonical test directories. Logs and forensic artifacts under[11] operator-controlled[4] directories[12] — never scattered at repo root, never tracked unless they are reference assets.

§ . . — Auto-generated docs sync

Every auto-generated document MUST be regenerated in the same commit as any edit to its source. All output formats (.md + .html + .pdf) MUST stay in sync at all times.

§ . . — Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud monitor, downstream service) provides a network-accessible introspection API that reports what was actually[11] [4] [14] received, the test suite MUST consume that report as captured-evidence. The on-source-side view alone is insufficient.

§ . . — Test playback cleanup

Every test MUST leave the target in a quiescent state. Cleanup mandatory on every exit[11] [4] [15] path (`trap '<cleanup>' EXIT`). The orchestrator MUST run a post-test sanity check and FAIL the just-completed test if it left orphan state.

§ . . — Item-status tracking

Every active item in the project Issues file carries a `**Status:**` line within five lines of its heading. Six-state vocabulary: `Queued` , `In progress` , `Ready for testing` , `In testing` , `Reopened` , `Fixed (→ Fixed.md)` . Status updated as the item[11] [4] [16] progresses. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF).

§ . . — Item-type tracking

Every active item in the project Issues file carries a `**Type:**` line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The Issues_Summary file carries the Type column for every active item. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF). Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION` enforce the mandate.

§ . . — Universal-vs-project classification of new rules (User mandate, - -)

Before adding ANY new rule, mandatory constraint, covenant clause, gate, or "MUST"-bearing statement to a project's Constitution / CLAUDE.md / AGENTS.md (or to a submodule's equivalents), the author MUST classify it as **universal** (reusable across any project → goes into this constitution submodule) or **project-specific** (references particular hardware / vendor / package / region → stays in the project / submodule layer). The commit message MUST carry a `Classification:` line stating the choice + one-sentence rationale. Universal rules that leak project-specific assumptions (hardware part numbers, vendor names, geographic regions, internal asset names) MUST be genericised first or downgraded to project-specific. When uncertain, default to project-specific (the narrower scope — lifting to universal later is cheap; the reverse is expensive). Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits new rule commits for the classification statement; paired mutation strips it and asserts gate FAILs.

11 4 20

2026 05 14

§ . . — Subagent-driven-by-default mandate (User mandate, - -)

When the runtime supports subagent delegation (Claude Code Agent tool, Cursor task-runners, Aider sub-sessions, etc.), the primary agent MUST default to subagent delegation for any task that has multi-step scope (≥3 phases), parallelisable independent subtasks, long-running diagnostic loops, OR specialised domain workflows (code review, security audit, doc propagation). Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks so quick that subagent overhead exceeds the work. Sub-discipline: **tight scope** (4-6 tasks, not "do everything"), **checkpoint commits** after each major task, **anti-stall protection** explicit in prompts, **anti-bluff verification** of subagent claims via repo state. Parallel subagents MUST partition non-overlapping files; `commit_all.sh --auto-cascade` bundles both via `git add -A`. Gate `CM-SUBAGENT-DELEGATION-AUDIT` (when implemented in consuming project) flags foreground multi-step work as a §11.4.20 violation.

§ . . — Script documentation mandate (User mandate, - -)

Every Bash / shell / POSIX-sh script anywhere in a project (`scripts/` , `bin/` , `tests/` , library directories, deployment hooks, CI helpers — depth-N recursive) MUST carry: (1) an in-source documentation block (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) at the top of the file; (2) an external user guide under `docs/scripts/<script-name>.md` covering Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date. When a script is modified, BOTH the in-source block AND the external user guide MUST be updated in the SAME commit. **No documentation ever can be out of sync with its codebase.** Pre-build gate `CM-SCRIPT-DOCS-SYNC` walks every `*.sh` / `*.bash` under script directories, verifies a companion `docs/scripts/<name>.md` exists, AND verifies doc was modified in the same commit (or doc-mtime ≥ script-mtime as a softer floor). Paired mutation strips the doc-sync invariant and asserts gate FAILs.

11 4 19

2026 05 14

§ . . — Fixed-document column-alignment mandate (User mandate, - -)

Every project that maintains an open-work tracker AND a closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes (Status + Type). For the Fixed archive that means: (1) every `###` / `####` heading in `Fixed.md` (or equivalent) carries a `**Status:**` line and a `**Type:**` line within 8 non-blank lines of its heading; (2) a `Fixed_Summary.md` companion exists with the same column structure as `Issues_Summary.md` (`# | Level | Status | Type | One-line description`); (3) all three file formats (`.md` + `.html` + `.pdf`) for BOTH `Fixed.md` and `Fixed_Summary.md` stay in sync via the same single-shot wrapper that handles Issues + Issues_Summary; (4) closure migration is atomic — when an Issues entry resolves it moves to Fixed.md in the same commit, disappears from Issues_Summary (open-only), and appears in Fixed_Summary (closed-only). Status values for closed items are drawn from `{Fixed (→ Fixed.md) | Fixed — pending device verification | Fixed — RECLASSIFIED}` . Type values follow §11.4.16: `{Bug | Feature | Task}` . Pre-build gate `CM-FIXED-COLUMN-ALIGNMENT` (5+ invariants) — Fixed_Summary.md exists, table header carries Status+Type columns, mtime (Fixed_Summary ≥

Fixed), generator script present, sync wrapper invokes it, HTML+PDF exports for both Fixed and Fixed_Summary present. Paired mutation strips Status column from Fixed_Summary table header → gate FAILs. Classification: universal (per §11.4.17). No escape hatch.
2026 05 14 11 4 21

§ . . — Operator-blocked status + self-resolution exhaustion (User mandate, - -)

`Operator-blocked` is the §11.4.15 Status closed-set's 7th value: `{Queued | In progress | Ready for testing | In testing | Reopened | Operator-blocked | Fixed (→ Fixed.md)}`. It is a **last-resort classification**, earned only after the agent documents exhaustion of every applicable self-resolution path: (a) CLI / ADB / SSH / API access already available, (b) subagent delegation per §11.4.20, (c) existing repo tooling (scripts / helpers / libraries), (d) captured fallback (synthetic event, asset substitution, mock, §11.4.3 topology SKIP), (e) external research per §11.4.8. Every `Operator-blocked` item MUST carry an `**Operator-Block-Details:**` line within 8 non-blank lines of its heading stating: **WHAT** (concrete action), **WHY** (each exhausted alternative enumerated), **UNBLOCK CONDITION** (observable signal), **WHO** (handle / contact / doc pointer). `Issues_Summary.md` lists `Operator-blocked` as a sortable Status value. Items MUST be re-evaluated every Nth tag cycle (project- defined, recommended ≥3rd cycle) — operator dependencies change. Fake `Operator-blocked` (no exhaustion audit) is a §11.4 covenant violation at the planning layer, severity-equivalent to a PASS-bluff. Gates: `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (every Operator-blocked heading has the details line), `CM-OPERATOR-BLOCKED-SELF-RESOLUTION- AUDIT` (NEW Operator-blocked commits contain "Attempted: a — ...; b — ...; c — ..." trail). Classification: universal (per §11.4.17). No escape hatch.
11 4 22
2026 05 14

§ . . — Document-sync commit discipline (User mandate, - -)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes ONLY the status-tracking doc set — Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + any auto-generated audit artifact — so doc-status never

drifts behind working-tree reality when the full-repo wrapper is unavailable (in-flight rebase, large submodule churn, partial network). The wrapper MUST: (a) auto-invoke the project's export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list — NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper's lock; (d) push to every parent-repo remote; (e) exit `3` on nothing-to-commit (informational, not error). The wrapper MUST be invocable standalone OR as a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model. Inherits the project's §9 preflight discipline (refuses to run mid-meta-test). Gate `CM-COMMIT-DOCS-EXISTS` verifies wrapper + guide + flag + doc-set enumeration. Paired mutation strips the doc-set array → gate FAILs. Classification: universal (per §11.4.17). Composes with §11.4.12 (export-sync), §11.4.15 (status tracking), §11.4.18 (script documentation), §12.10 (CONTINUATION maintenance). No escape hatch — doc-status drift is a §11.4 PASS-bluff at the documentation layer.

11 4 24

2026 05 14

§ . . — Build-resource stats tracking mandate (User mandate, – –)

Every project under this Constitution with a build exceeding 1 minute wall-clock MUST run a host-side resource sampler for every build that captures memory used, CPU%, load average, disk read/write at a fixed interval (recommended 5 s) and computes per-metric **min / max / mean / p95** at stop. Per-build summaries appended to a TSV registry; the registry is the single source of truth — the human-readable Markdown report (and its HTML + PDF exports per §11.4.12) is derived. Top of the report MUST surface **ever-values** (min / max / mean across all tracked builds). Per-build entries sorted most-recent-first; each row carries SUCCESS / FAIL / UNKNOWN + reason for FAIL. Sampler MUST itself stay under 50 MB RSS and 5% CPU (Heisenberg-class observer constraint). Stop hook MUST be called from both success AND failure paths of the build wrapper. The Stats.{md,html,pdf} triple is committed via the project's §11.4.22 lightweight doc-sync wrapper. Gate `CM-BUILD-RESOURCE-STATS-TRACKER` + paired mutation hiding the monitor aside → gate FAILs. Classification: mixed (per §11.4.17) — the discipline universal, the implementation paths project-specific. Composes with §11.4.12 / §11.4.18 /

§11.4.22 / §12.6 / §12.7 / §12.9 (the host-safety forensic anchors are the empirical motivation for this telemetry). No escape hatch — build-resource debugging without time-series data is the bluff this anchor forbids.

§ . . — Full-Automation-Coverage Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product!"

For every consuming project, no feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered **deliverable** until it is covered by automation tests proving six invariants: (1) anti-bluff posture (captured runtime evidence per §7.1 + §11.4); (2) proof of working capability end-to-end on the target topology (per §11.4.3, not in a mock); (3) working implementation matching the documented promise; (4) no open issues / bugs surfaced by the suite (cross-checked against §11.4.15 / §11.4.16 trackers); (5) full documentation (user manual entry + §11.4.18 for scripts) kept in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build

- runtime + paired mutation). Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status), regenerated as part of release-gate sweeps, with gaps tracked per §11.4.15. Classification: universal (§11.4.17). No escape hatch. A project that ships a feature without all six invariants is **not delivering a stable product** — severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. See Constitution §11.4.25 for the full mandate (cross-cutting reach, composition, audit requirements).

§ . . — Constitution-Submodule Update Workflow Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Before ANY modification to `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md`, the agent or operator MUST execute the following pipeline in order:

1. **Fetch + pull first** — inside the constitution submodule worktree run `git fetch` against every remote, then `git pull --ff-only` (or `--rebase` if non-FF-mergeable; never `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization). The submodule MUST be at upstream tip BEFORE any local edit.
2. **Apply the change** — classify per §11.4.17 (only universal additions belong here; project-specific clauses stay in the consuming project's governance). Cite the verbatim user mandate if one originated the change.
3. **Validate before commit** — run `meta_test_inheritance.sh` (or equivalent); verify no merge-conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`); verify Constitution + CLAUDE + AGENTS cross-reference the new clause consistently.
4. **Commit + push to ALL upstreams** — stage only the governance files (NEVER `git add -A` inside the submodule); commit message cites the user mandate + §11.4.17 classification; push to every configured remote. A commit landing on one upstream but not others is a §2.1 violation AND a §11.4.26 violation.

5. **Conflict resolution** — if `pull --ff-only` reports non-fast-forward, merge carefully (preserve union of governance content, no clause silently dropped, re-classify, re-validate). Force-push to "make conflicts go away" is FORBIDDEN (§9.2). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable.
6. **Post-merge validation + verification** — after the push lands, `git submodule update --remote --init` and re-run the consumer project's cascade verifier (per CONST-047) to confirm the new clause reaches every owned submodule. Any cascade gap closed in the same change-window.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer MUST be advanced to the new constitution HEAD in the SAME commit as any cascade work. Out-of-sync pointers are §11.4.26 violations.

Classification: universal (§11.4.17). No escape hatch. A constitution-submodule change violating §11.4.26 is a release blocker for every consuming project, severity-equivalent to a force-push without §9.2 authorization. See Constitution §11.4.26 for the full mandate (operational scope, cross-cutting reach).

§ . . — Submodule-Dependency-Manifest Mandate (User mandate, - -)

Every owned-by-us submodule MUST ship a machine-readable dependency manifest at canonical path `helix-deps.yaml` (or .json/.toml) listing its own-org Git SSH dependencies. Schema (per Constitution §11.4.31): `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.recursive: true`, `transitive_handling.conflict_resolution: operator-required`, `language_specific_subtree: bool`.

Tooling: `incorporate-submodule <ssh-url>` adds the submodule at its declared canonical path (CONST-051(C) flat/grouped), reads its helix-deps.yaml, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a §11.4.31 violation.

§11.4.31 is the operational complement of §11.4.28 / CONST-051(C): nested own-org submodule chains are FORBIDDEN, manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Classification: universal (§11.4.17). Composes with §1, §3, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, §11.4.30, CONST-047. See Constitution §11.4.31 for the full mandate.

2026-05-15

§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.

Sweep contract (canonical script: `scripts/verify-all-constitution-rules.sh`): re-runs the governance- cascade verifier; for every rule with a programmatic gate (CONST-053 .gitignore audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke), runs the gate against post-pull tree. Failures produce directed FAIL entries → tracker per §11.4.15 with Status: `Reopened`, Type: `Bug`. Closure requires positive-evidence per §11.4 anti-bluff covenant.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook or commit wrapper); operator-explicit manual invocation also available.

Anti-bluff: sweep's own meta-test (paired mutation §1.1) plants a known violation of each enforced gate and asserts sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a §11.4.32 violation.

§11.4.32 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced in the codebase.

Classification: universal (§11.4.17). Composes with every rule that has a programmatic gate. See Constitution §11.4.32 for the full mandate.

§ . . — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!"

Every project module / owned-by-us submodule / service / application MUST ship a proper `.gitignore` covering the forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files (when the generator is committed).
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.nuxt/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.

4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.vscode/` (except shared settings), `.history/`.

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be currently tracked. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection: §11.4.30 composes tightly with §11.4.10

- §12.1 (CONST-042) — a `.env` leak is BOTH a §11.4.30 and a §11.4.10 violation, requiring rotation + post-mortem.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it's a build derivative and MUST be ignored. Generators MUST be committed so consumers regenerate on demand.

Classification: universal (§11.4.17). No escape hatch beyond enumerated exceptions. Severity-equivalent to §11.4 PASS-bluff at the repository-hygiene layer. Composes with §1, §2, §9.1, §11.4.10, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25[11 4 29 2026 05 15], §11.4.26, §11.4.27, §11.4.28, §11.4.29, CONST-047. See Constitution §11.4.30 for the full mandate.

§ . . — Lowercase-Snake_Case-Naming Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!"

Every directory, submodule, and file MUST use lowercase snake_case names (ASCII letters / digits / underscores, words separated by `_`). Existing non-compliant names MUST be renamed as part of the migration window opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a §11.4.29 violation of equal severity to the rename itself.

Exceptions (common-sense, must-not-break-technology). Language-mandated case (Java/Kotlin package paths, Android resource directories, Apple framework dirs, C#/Swift project layouts) is preserved inside the language-root. Submodule root directory follows our convention; language-specific subtree follows its own. Vendor/upstream third-party submodules keep their upstream names. Build artefacts (`node_modules/` , `__pycache__/` , `.git/` , `target/` , `build/` , `bin/`) keep tool-mandated names. "Does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition. Constitution submodule's `install_upstreams.sh` (exported via `.bashrc` / `.zshrc`) MUST support BOTH `Upstreams/` and `upstreams/` directory layouts during migration. Lowercase wins when both present. Uppercase fallback retires only by deliberate amendment.

Project-Toolkit Upstreamable synchronisation. Upstreamable / Project-Toolkit machinery MUST be fetched+pulled before any rename batch + MUST itself comply with this rule. Lacking BOTH- directory support is a release blocker.

Test coverage of renames. Each rename batch ships with: (i) regression test verifying every reference now resolves; (ii) full CONST-050(B) test-type matrix run on the post-rename tree; (iii) anti-bluff wire-evidence captured. All three or it's a §11.4.29 violation.

Phased execution. Comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. Phases run in parallel with mainstream work (§11.4.20 subagent delegation).

Classification: universal (§11.4.17). No escape hatch beyond the common-sense exceptions enumerated. Severity-equivalent to §11.4 PASS-bluff at the reference-integrity layer. Composes with §1, §1.1, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, CONST-047. See Constitution §11.4.29 for the full mandate.

§ . . — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project — directly from project's root project_name/ submodule_name or some more proper structure project_name/submodules/ submodule_name!"

Three cooperating invariants:

(A) Equal-codebase. Every owned-by-us submodule (orgs: `vasic-digital`, `HelixDevelopment`, `red-elf`, `ATMOSphere1234321`, `Bear-Suite`, `BoatOS123456`, `Helix-Flow`, `Helix-Track`, `Server-Factory` — dynamically

discoverable via `gh` / `glab` CLIs) is an **equal part** of the consuming project's codebase. Same engineering attention as main: analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, SQL, websites, all materials). A round that improves main while leaving an owned-submodule deficiency unaddressed is a §11.4.28 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. Coverage ledgers (§11.4.25) list every owned submodule as in-scope.

(B) Decoupling / reusability. Owned submodules MUST stay fully decoupled, project-not-aware, reusable, modular, completely testable. NEVER inject project-specific context (hardcoded paths, hostnames, asset names) INTO a submodule. When a submodule needs parent-project info, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Every dependency consumed by an owned submodule MUST be accessible from the parent project's root at:

```
<project_root>/<submodule_name>/
<project_root>/submodules/<submodule_name>/
```

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries pulling in further owned- by-us repos. Add the dependency at the parent's root path; the submodule reaches it via documented import / SDK / runtime resolver. Third-party submodules exempt. **EXCEPTION (operator Option-2, 2026-07-07; §11.4.28(C) carve-out):** the constitution submodule ITSELF MAY host depth-1 reusable-engine submodules under `constitution/submodules/<name>/`, PROVIDED each ships a §11.4.31 `helix-deps.yaml` AND nests zero further own-org submodules (depth-1 only, no recursive chain). This applies to NO other submodule — consumer submodules stay forbidden from nesting.

Gates: `CM-OWNED-SUBMODULE-EQUAL-ENGINEERING` (release-gate sweep audits parity), `CM-OWNED-SUBMODULE-DECOUPLING` (pre-commit greps for parent-project context inside the submodule diff), `CM-OWNED-SUBMODULE-LAYOUT` (pre-merge verifies canonical location + no nested own-org submodules + dependency lookup from root). Paired mutations (§1.1) for all three. Classification: universal (§11.4.17). No escape hatch. Composes with §1, §3, §11.4.17, §11.4.20, §11.4.25, §11.4.26, §11.4.27, CONST-047. See Constitution §11.4.28 for the full mandate.

§ . . — No-Fakes-Beyond-Unit-Tests + %-Test-Type-Coverage Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths. Gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` + paired mutation.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/ Challenges submodule fully incorporated), HelixQA (HelixDevelopment/ HelixQA submodule fully incorporated). HelixQA autonomous sessions drive end-to-end execution of every registered test bank with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047):

- Challenges — `git@github.com:vasic-digital/Challenges.git`
- HelixQA — `git@github.com:HelixDevelopment/HelixQA.git`
- Any other functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs the project depends on.

Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7); pointer drift = §11.4.27 violation.

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — this is its strict expansion into per-type-of-test territory). See Constitution §11.4.27 for full mandate.

§ . . — Type-aware closure-status vocabulary (User mandate, - -)

Every project that tracks work items by Type per §11.4.16 MUST close them with the Type-appropriate closure-status word, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh` , `generate_fixed_summary.sh` , status-counter helpers, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all map to "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate (recommended) `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal

values and asserts the Status-Type match. Composes with §11.4.15 (status tracking[11 4 34]), §11.4.16 (type tracking), §11.4.19 (Fixed-document column alignment), §11.4.23 (colorisation). Classification: universal[2026 05 15] (per §11.4.17). No escape hatch.

§ . . — Reopened-source attribution mandate (User mandate, - -)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values are permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`.

A `Reopened` entry without `**Reopened-Details:**` is a §11.4.34 violation. Pre-build gate (recommended) `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 (no-guessing — Reason from closed vocabulary), §11.4.7 (demotion-evidence —

reopen IS a demotion from Fixed), §11.4.15 (item-status tracking), §11.4.21 (Operator-blocked discipline — same audit-line pattern). Classification: universal (per §11.4.17). No escape hatch. 11 4 35 2026 05 15

§ . . — Canonical-root inheritance clarity (User mandate, - -)

The constitution submodule's three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the canonical root — also called the parent files. They contain only universal rules per §11.4.17.

The consuming project's repository-root files (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md` or equivalent) are consumer extensions. They open with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading defined in this file's "How inheritance works" section). They contain only project- specific rules per §11.4.17 — rules that reference particular hardware, vendor names, regulatory regions, internal asset names, or project-private conventions.

When in doubt about which file to edit: universal rule → edit constitution submodule's file; project-specific rule → edit consumer's file. Default consumer-side when uncertain (per §11.4.17, narrower scope is cheap to widen).

Terminology: when prose references "the parent CLAUDE.md" or "the root Constitution," the referent is the constitution-submodule file at `constitution/<filename>`, never the consumer's file. When it references "the project CLAUDE.md" or "this project's AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>`. AI agents resolve ambiguous references via this rule.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the commit message.

Pre-build gate (recommended) `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution

submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17 (universal-vs-project classification — §11.4.35 defines the file-layer split that §11.4.17 classifies INTO). Reading order: this anchor first, then §11.4.17. Classification: universal (per §11.4.17). No escape hatch.

§ . . — Mandatory install_upstreams on clone/add Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Every clone / add of a Git repository under any consuming project MUST be followed by `install_upstreams` invocation from that repository's root IF its tree contains an `upstreams/` directory (or legacy `Upstreams/` per §11.4.29 transition) populated with `*.sh` recipe files declaring upstream Git SSH URLs.

`install_upstreams` is a host-system utility on operator's `PATH` (exported via `.bashrc` / `.zshrc`), implemented in this constitution submodule (`install_upstreams.sh`). The utility reads recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.

Skipping the invocation when `upstreams/` IS present silently breaks §2.1 (Multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE`

- paired mutation (§1.1).

Automation: `incorporate-submodule` (§11.4.31) and `scripts/init-submodules.sh` patterns auto-invoke `install_upstreams` when applicable. Operator-explicit manual invocation remains supported.

Pre-commit attention: before the first commit in the newly-cloned working tree, verify `git remote -v | grep -c push` reports the expected upstream count.

Classification: universal (§11.4.17). Composes with §2, §2.1, §3, §9.2, §11.4.17, §11.4.20, §11.4.28, §11.4.29, §11.4.30, §11.4.31. See Constitution §11.4.36 for the full mandate.

§ . . — Fetch-before-edit mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!"

The FIRST git-touching action of any session, on any consuming project, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}          # parent
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If `HEAD..@{u}` is non-empty, integrate (ff-merge / rebase / surface to operator per §11.4.4) BEFORE any local edit, scanner run, or test cycle. Multi-agent / multi-upstream codebases (Claude Code + Cursor + Aider + operator sessions in parallel) routinely lap each other; a 30-second fetch prevents the agent from redoing work a parallel session already finished, from filing a false-confidence "completion" of already-done work, and from doubling the multi-upstream conflict surface (§2.1) with sibling commits of the same change.

The check is non-negotiable even when the operator says "do X immediately" — skipping it on the basis of "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: remote state is not knowable without a fetch. The fetch+log output (even if empty) is the captured evidence.

Scope: consuming project root + every owned submodule recursively (§11.4.28) + the constitution submodule itself (§11.4.26 step 1 made this explicit for constitution-side edits; §11.4.37 generalises to ALL edits) + any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36).

Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit's parent — non-aligned parent = FAIL. Paired mutation (§1.1): synthetic commit whose parent was N commits behind the then-current upstream HEAD → gate FAILs.

Classification: universal (§11.4.17). No escape hatch. Composes with §2.1, §11.4.4, §11.4.6, §11.4.20, §11.4.26, §11.4.32. See Constitution §11.4.37 for the full mandate.

§ . . — Installable-Asset Evidence Mandate (User mandate, - -)

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

A PASS without opening the artifact and verifying the asset chain end-to-end is a §11.4 PASS-bluff. The specific failure mode: source file exists → build packages it → post-build checks pass at the source layer → artifact produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the project's standard QA gate.

Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.38 for the full mandate.

§ . . — Full-suite retest before release tag mandate (User mandate, - -)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between batch fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post- build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — typically 12–48h elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests pass but feature broken" failure mode §11.4 prohibits.

Composes with §11.4.4 (per-fix retest still required at fix granularity) + §11.4.7 (full-suite retest is authoritative baseline for closures) + §11.4.39 (per-feature on-device validation runs as step 3 of the full-suite retest).

11 4 41 Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.40 for the full mandate. 2026 05 17

§ . . — Pre-Force-Push Merge-First Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-17):

> "make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!"

Any force-push (`--force` , `--force-with-lease` , `+<ref>` , equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per the appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured- evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. Both required: CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact — `docs/changelogs/<tag>.md` "Force-push merge-first audit" section captures fetch output, per-remote divergence log, integration strategy, conflict- marker scan, test delta, push output with lease SHA, CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Classification: universal (§11.4.17). No escape hatch. Composes with §9.2, §11.4.4, §11.4.6, §11.4.26, §11.4.32, §11.4.37, §11.4.40, CONST-043, CONST-047. See Constitution §11.4.41 for the full mandate.

§ . . — Iteration-discipline mandate (User mandate, - -)

Project work proceeds in priority-ordered iteration cycles. Each cycle has five mandatory steps: (1) select TOP + MIDDLE critical items only (defer LOW until critical batch closed), (2) batch implementation with §11.4.4 four-layer coverage + §11.4.9 batch-source-fixes-before-rebuild, (3) smoke gate (<30 min — batch-touched tests + critical-path regression probe + anti-bluff baseline), (4) ONLY if smoke GREEN and no new operator/user report arrived, run §11.4.40 full retest (12–48 h), (5) release-ready OR loop back to step 1 with new evidence added to queue.

Composes with §11.4.4 (per-fix retest inside step 2), §11.4.7 (closures require same-conditions evidence; step 4 is authoritative baseline), §11.4.9 (source-side batching inside step 2), §11.4.34 (Reopened items attribute source), §11.4.40 (the multi-hour retest IS step 4 — §11.4.42 is the meta-loop conductor).

Anti-bluff coupling: every smoke and full-system PASS MUST carry positive captured evidence per §11.4.2 + §11.4.5. Tests AND HelixQA Challenges bound equally.

No escape hatch — no `--skip-priority-batch`, no `--skip-smoke`, no `--full-suite-only`. Subagents default to the §11.4.42 path.

Classification: universal (§11.4.17). See Constitution §11.4.42 for the full mandate.

§ . . — TDD-Fix-Discipline mandate (User mandate, - -)

Every fix MUST follow the 5-step TDD-fix workflow: **RED** (failing test FIRST, real product defect per §11.4.1, captured evidence per §11.4.2) → **LIVE-ADB-PROBE** (try fix on running device via `adb shell` for mutable surfaces — `setprop persist.*`, `settings put`, `pm clear`, `am ...`, boot-script push; INFEASIBLE for kernel / framework / HAL / vendor / init.rc / sepolicy / ro.* / Android.bp — must rebuild; cite `LIVE_PROBE_INFEASIBLE: <reason>` in commit message) → **GREEN** (source patch achieves same effect, batched per §11.4.9, four-layer coverage per §11.4.4) → **VERIFY** (re-run RED test, must PASS under SAME conditions per §11.4.7, captured positive evidence per §11.4.5, 10-iteration

reliability per §11.4.42) → **DOCUMENT** (Issues.md → Fixed.md with type-aware closure vocabulary per §11.4.33, CLAUDE.md Applied Fixes row, changelog, guides, HelixQA bank, CONTINUATION.md per §12.10 — all in the SAME commit).

No escape hatch — no `--skip-red-test`, `--no-live-probe`, `--skip-verify` flag. "Test added after the fix" is a §11.4 PASS-bluff: it demonstrates the test agrees with the fix, not that the test catches the bug.

Classification: universal (§11.4.17). Composes with §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.7 / §11.4.9 / §11.4.40 / §11.4.42. See Constitution §11.4.43 for the full mandate.

§ . . — Document revision header mandate (User mandate, - -)

Every tracked document in scope (Issues.md, Issues_Summary.md, Fixed.md, Fixed_Summary.md, CONTINUATION.md, docs/guides/**, docs/research/**, docs/scripts/**, docs/changelogs/**, docs/superpowers/plans/**, docs/hardware/**, all other docs/* tracked Markdown) MUST carry a header block directly below the H1 title containing two MANDATORY fields: `**Revision:** N` (monotonic positive integer, never reset, never skipped) and `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` (ISO 8601 UTC). Optional fields (Description, Authority, Maintainer, Scope) encouraged but not gated. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF artifacts are explicitly OUT of scope (revision tracked via VERSION file or auto-derived from source Markdown).

Auto-bump: `scripts/doc_revision_bump.sh <file>` (idempotent), pre-commit hook for automatic staged-doc bumps, `scripts/testing/sync_issues_docs.sh` auto-bumps Issues_Summary / Fixed_Summary after regeneration, `scripts/commit_docs.sh` calls the bump before stage. CONTINUATION.md's existing `Last updated:` line per §12.10 IS the §11.4.44 `Last modified:` line — composed, not duplicated. HTML/PDF exports inherit revision from source Markdown via pandoc pipeline. No escape hatch — no `--skip-revision-bump` flag exists anywhere.

Pre-build gates `CM-DOC-REVISION-HEADER-PRESENT` + `CM-COVENANT-114-44-PROPAGATION` + paired mutations per §1.1. Composes with §12.10 (CONTINUATION.md header reuse), §11.4.12 (Issues_Summary regen), §11.4.22 (commit_docs.sh hook entry), §11.4.23 (HTML colorizer preserves revision), §11.4.18 (companion doc for doc_revision_bump.sh).

Classification: universal (§11.4.17). See Constitution §11.4.44 for the full mandate.
2026 05 18

§ . . — Integration-status-doc maintenance mandate (User mandate, - -)

Every non-trivial domain integration MUST have a `docs/<domain>/<integration>/Status.md` document that (1) exists when more than one fix/test/gate has landed for the integration, (2) carries the §11.4.44 revision header, (3) is auto-synced (HTML

- PDF) on every related test cycle and every fix touching the integration, (4) is auto-colorized per §11.4.23, (5) has a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh` or a per-integration thin shell, (6) lives under `docs/<domain>/<integration>/`, (7) includes a captured-evidence- driven status table per §11.4.5 (every claim cites the test log / recording / sink-probe report that backs it), (8) uses the closed status vocabulary PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED, (9) lists operator-blocked items at the top so operators find action items in O(1), (10) is referenced from `docs/CONTINUATION.md` §3 (Active work) when any item is non- terminal.

§11.4.45 is the generic form of §12.10 (CONTINUATION.md) applied to every integration domain. Without this generalisation each new integration re-invents the same sync wrapper, revision-header discipline, captured-evidence requirement, and operator-blocked surface.

Pre-build gates `CM-COVENANT-114-45-PROPAGATION` + `CM-AF-INTEGRATION-STATUS-DOCS` + paired mutations (propagation strip / Revision-line delete / sync-staleness / vocabulary violation). Composes with §11.4.5 (captured evidence), §11.4.12 (sync wrapper pattern reused), §11.4.13 (sink-side evidence is a specific

instance), §11.4.15 (status vocabulary), §11.4.22 (commit_docs.sh wrapper), §11.4.23 (colorizer), §11.4.44 (revision header), §12.10 (CONTINUATION.md references Status.md paths).

No escape hatch — no `--skip-status-sync`, no `--no-revision-bump-on-status`, no `--allow-stale-html` flag.

Classification: universal (§11.4.17). See Constitution §11.4.45 for the full mandate.
2026 05 18

§ . . — Validate-recent-work-before-post-flash-tests mandate (User mandate, - -)

After every device flash, the orchestrator MUST first run a recent- work validation pass (targeted on-device tests for items currently in Issues.md `In progress` / `Ready for testing` / `Reopened`, Fixed.md items closed within last 7 days, CONTINUATION.md §3 active work). Only if that pass is 100% green does the orchestrator proceed to the full post-flash suite.

Helper: `scripts/testing/recent_work_validate.sh --device <serial>` writes `/data/local/tmp/.recent_work_validated` IFF green; the full suite (`test_all_fixes.sh`) refuses to start without it. Marker is invalidated on reboot (stores device boot epoch).

Composes with §11.4.4 (STOP-on-discovery) + §11.4.6 (no-guessing) + §11.4.7 (demotion-evidence) + §11.4.40 (full-suite gate) + §11.4.42

- §11.4.43 + §11.4.44 + §12.10. Each recent-item fix MUST have a paired §11.4.43 RED-then-GREEN — a GREEN with no prior RED is a bluff.

Pre-build gates `CM-COVENANT-114-46-PROPAGATION` + `CM-AF-RECENT-WORK-VALIDATION-GATE` + `CM-AF-VALIDATION-ARTIFACT-FILE` + paired mutations.

Classification: universal (§11.4.17). No escape hatch — no `--skip-validation`, `--full-suite-always`, `--ignore-recent-work` flag. See Constitution §11.4.46 for the full mandate.

§ . . — Firebase data review mandate (User mandate, - -)

Before every "bigger working round" (pre-build, pre-flash, pre-tag, daily, post-deployment burn-in) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup-maps to existing Issues.md entries via a three-tier algorithm (exact Firebase Issue-ID match → stacktrace- similarity cluster hash → operator merge review), and drafts new Issue entries for unrecognised findings with full Firebase Console URL refs + §11.4.4(a) systematic-debugging output. Skipping the pass is a §11.4 PASS-bluff — Firebase IS the captured evidence from real end-user devices.

Five mandatory elements: (1) 5-trigger cadence (pre-build / pre- flash / pre-tag blocking, daily / burn-in non-blocking), (2) 3-source query (all three sources), (3) Issues.md output with Firebase metadata (Issue IDs + URL + Cluster Hash / KPI / Funnel), (4) 3-tier dedup, (5) comprehensive systematic-debugging output per Issue.

Pre-build gates `CM-COVENANT-114-47-PROPAGATION` + `CM-AF-FIREBASE-REVIEW-CADENCE` + `CM-AF-FIREBASE-ISSUE-XREF` + 3 paired mutations. Composes with §11.4.4 / §11.4.4(a) / §11.4.6 / §11.4.7 / §11.4.10 / §11.4.12 / §11.4.14 / §11.4.15 / §11.4.16 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.46.

No escape hatch — no `--skip-firebase-review`, `--firebase-review-not-applicable`, `--no-issue-from-firebase` flag. Operator MAY filter with `--severity-min` but MUST execute the pass.

Classification: universal (§11.4.17). See Constitution §11.4.47 for the full mandate.

- §11.4.48 — UI-driven video testing mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.4 §11.4.11 §11.4.12 §11.4.13 §11.4.15 §11.4.16 §11.4.19 §11.4.44 §11.4.48 §11.4.53 §11.4.66
- §11.4.49 — Dual-approach testing mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.43 §11.4.48 §11.4.49
- §11.4.50 — Deterministic consistency mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.7 §11.4.50

- §11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.43 §11.4.51
- §11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.52

> "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT a human present to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + JSON result file), headless intent dispatch + state poll (`am start --es`

- `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling), ADB-driven uiautomator (ONLY if `uiautomator dump | grep -c clickable=true` ≥ 1 — near-empty hierarchy proves UI-driven INFEASIBLE and demands fallback to APK/intent), network-side sink probe (Arvus dashboard, Sonos REST, etc. per §11.4.13), HelixQA autonomous QA session (§11.4.27).

Per-feature coverage ledger (§11.4.25) MUST classify each row as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE` . `OPERATOR_ATTENDED_ONLY` is a release blocker until

promoted, citing a tracked migration work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence per §11.4.5, paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation-coverage — §11.4.52 strictens invariants 1+2), §11.4.27 (no-fakes-beyond-unit + 100% type coverage — operational layer closing the gap), §11.4.39 (per-feature on-device end-user validation — refined to mandate autonomous drivability), §11.4.43 (TDD-fix — autonomous path RED before fix, GREEN after), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach — Intent variant often IS the autonomous path), §11.4.50 (deterministic consistency — autonomous paths scale to N iterations), §11.4.51 (live-ADB-first — instrumentation APK is LIVE_ADB_TESTABLE).

Pre-build gates `CM-COVENANT-114-52-PROPAGATION` (anchor literal across canonical files) + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` (coverage-ledger classification column non-empty + valid value). Paired mutations strip the anchor literal AND inject an `OPERATOR_ATTENDED_ONLY` row without a tracked migration item. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.52.

Non-compliance is a release blocker regardless of context.

- §11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.53

> "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations

regardless of whether the underlying `.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, MUST be executable, MUST emit a markdown table whose header columns include `Status` and `Type` per §11.4.19 column-alignment). Auto- sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH Issues_Summary AND Fixed_Summary in one shot (stages 1a + 1b), then exports HTML + PDF (stage 2), then colorizes per §11.4.23 (stage 3), then re-renders the PDFs from the colorized HTML (stage 3b). MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary parity is the sibling rule; §11.4.12 + §11.4.53 are the canonical pair), §11.4.19 (atomic Issues→Fixed migration triggers Fixed_Summary regen — column-aligned structure), §11.4.23 (visual-cue + grouping colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values literally), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee depends on divergent-summary problem NOT existing).

Pre-build gates `CM-FIXED-SUMMARY-SYNC` (6 invariants: Fixed_Summary exists; HTML + PDF mtime ≥ md mtime; Fixed_Summary mtime ≥ Fixed mtime; generator script exists + executable; sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files + per-consumer propagation). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.53.

Non-compliance is a release blocker regardless of context.

- §11.4.54 — ATM-NNN ticket identifier mandate (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.54

> "Every workable item in Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier. ATM- prefix, monotonic, never renumbered, append-only."

Every workable item in `docs/Issues.md` AND `docs/Fixed.md` MUST carry a `[ATM-NNN]` ticket identifier in its heading (form `## §X.Y. [ATM-NNN] <title>`, zero-padded ≥3 digits). Identifiers are allocated by `scripts/testing/assign_atm_ticket_ids.sh` and persisted to an append-only state file `scripts/testing/.atm_ticket_state.json` (jsonl: `atm_id`, `heading_hash`, `type`, `current_location`, `current_status`, `reopens_count`, `created_at`, `last_modified`). Once assigned, an ATM-NNN MUST NEVER be renumbered, reused, decremented, or skipped. `heading_hash` is the binding key — wording reflows preserve the binding via the state-file lookup.

Issues_Summary.md and Fixed_Summary.md MUST carry an `ATM ID` column as the leftmost data column. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`) emit the column so operators / agents can sort + filter on it.

Composes with §11.4.15 (Status), §11.4.16 (Type), §11.4.19 (column-alignment), §11.4.33 (closure vocabulary), §11.4.12 + §11.4.53 (Issues_Summary + Fixed_Summary regen — helper invoked from `sync_issues_docs.sh`), §11.4.55 (per-item Reopens.md path key), §11.4.57 (README.md doc-link cross-reference key).

Pre-build gates `CM-ATM-TICKET-IDS-COMPLETE` (every heading carries `[ATM-NNN]`) + `CM-ATM-TICKET-IDS-UNIQUE` (no duplicates) + `CM-ATM-TICKET-IDS-MONOTONIC` (no gaps) + `CM-COVENANT-114-54-PROPAGATION` (anchor literal across canonical files). Paired mutations strip an `[ATM-NNN]` heading token, dup

an ID in the state file, gap the sequence at NNN=2, strip the anchor literal. No escape hatch — no `--skip-atm-assignment`, `--renumber`, `--no-atm-id-required` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.54.

Non-compliance is a release blocker regardless of context.

- §11.4.55 — Reopens-history tracking + per-item Reopens.md doc (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.55

> "Add a Reopens-count column. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles. Each reopen MUST include By (AI / User), On, Reason, Evidence, and each Fixed-marking with reasoning chain."

Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/<ATM-NNN>/Reopens.md` (+ HTML + PDF). The document MUST contain: (1) §11.4.44 revision header, (2) item identification (ATM ID, Title, Type, Current Status, Current Location, link back to live heading), (3) cycle counters (Total reopens, Total fixed cycles), (4) chronological timeline — one entry per state-change event, each with By (AI/User per §11.4.34), On (ISO date), Event (Opened/Reopened/Fixed/Implemented/ Completed), Reason (closed-vocabulary value from §11.4.34 for reopens; captured-evidence summary for closures), Evidence (path or short description), Outcome, (5) reasoning chain for each closure (root-cause analysis, captured-evidence under same conditions per §11.4.7, gate / mutation pair), (6) most-recent state-change pointer.

Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column; cells with count > 0 hyperlink to the per-item Reopens.md. The §11.4.23 colorizer MAY apply a visual cue when reopens > 2.

Composes with §11.4.34 (per-event capture in current heading — §11.4.55 is the per-item history aggregation), §11.4.54 (ATM-NNN provides the stable path), §11.4.44 (revision header), §11.4.45 (Status.md per-integration analogue), §11.4.53 (Fixed_Summary parity — Reopens column symmetric on both summaries).

Pre-build gates `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` + `CM-REOPENS-DOC-REVISION-HEADER` + `CM-REOPENS-COL-IN-SUMMARIES` + `CM-COVENANT-114-55-PROPAGATION`. Paired mutations delete a Reopens.md for a reopens_count=2 item, strip the revision header, remove the column from Issues_Summary, strip the anchor literal. No escape hatch — no `--skip-reopens-doc`, `--collapse-history`, `--reopens-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.55.

Non-compliance is a release blocker regardless of context.

- §11.4.56 — Status_Summary parity + two-audience format (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.56

> "Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific), page 2 = software engineer summary. Auto-generated after every main Status update."

For every `docs/<domain>/<integration>/Status.md` (§11.4.45) a companion `Status_Summary.md` MUST exist with: (1) §11.4.44 revision header, (2) **Page 1 — For the** — audience- specific heading (audio team for `docs/dolby/*`, video team for `docs/video/*`, etc.) — plain-language summary, What works (1-3 bullets), What's broken or pending (1-3 bullets), Operator / team actions if any. NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names. (3) **Page 2 — For software engineers** — §-letter references, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-references. HTML + PDF exports travel with the markdown.

Generator: `scripts/testing/generate_status_summary.sh <Status.md path>` produces both pages. Invoked from `scripts/testing/sync_integration_status.sh` (§11.4.45 sync wrapper) on every Status.md update.

Composes with §11.4.45 (Status.md per integration — Status_Summary.md COMPLEMENTS, never replaces), §11.4.12 + §11.4.53 (parity discipline), §11.4.44 (revision header), §11.4.23 (colorizer for tracked-item references on page 2), §12.10 (CONTINUATION.md — non-developer stakeholders read Status_Summary; engineers read Status + CONTINUATION + Issues).

Pre-build gates `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` + `CM-STATUS-SUMMARY-TWO-AUDIENCE` + `CM-STATUS-SUMMARY-REVISION-HEADER` + `CM-COVENANT-114-56-PROPAGATION`. Paired mutations delete a Status_Summary.md, remove the Page 1 heading, strip the anchor literal. No escape hatch — no `--skip-status-summary`, `--engineer-only`, `--no-audience-split` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.56.

Non-compliance is a release blocker regardless of context.

- §11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.57

> "Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Every project's top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (heading MUST contain literal `Tracked-Items`). The section is a markdown table with columns: `Document`, `Last modified` (ISO 8601 UTC from §11.4.44 header), `Revision` (integer from same header), `Markdown` link, `HTML` link, `PDF` link. The section MUST link to: Issues.md + Issues_Summary.md (§11.4.12, §11.4.15, §11.4.16), Fixed.md + Fixed_Summary.md (§11.4.19, §11.4.53), CONTINUATION.md (§12.10), every `docs/**/Status.md` + its `Status_Summary.md` pair (§11.4.45, §11.4.56). Status docs are auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator: `scripts/testing/update_readme_doc_links.sh` extracts each doc's `Revision` + `Last modified`, renders the markdown table, replaces the section between markers (`<!-- doc-link-section:begin -->` / `<!-- doc-link-section:end -->`) in README.md. Invoked from `sync_issues_docs.sh` (Issues / Fixed edits) AND `sync_integration_status.sh` (Status edits).

Composes with §11.4.12 + §11.4.19 + §11.4.53 (Issues / Fixed / summaries — README links all four), §11.4.44 (revision header data source), §11.4.45 + §11.4.56 (Status pairs enumeration), §12.10 (CONTINUATION.md explicit link).

Pre-build gates `CM-README-DOC-LINK-SECTION-PRESENT` (literal `Tracked-Items` + section markers) + `CM-README-DOC-LINK-ROWS-COMPLETE` (every canonical doc appears as a row) + `CM-README-DOC-LINK-FRESHNESS` (`Last modified` matches source within sync-wrapper granularity) + `CM-COVENANT-114-57-PROPAGATION`. Paired mutations strip the markers, remove the CONTINUATION row, backdate a `Last modified` cell, strip the anchor literal. No escape hatch — no `--skip-readme-doc-links` , `--collapse-status-rows` , `--no-freshness-check` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.57.

Non-compliance is a release blocker regardless of context.

- §11.4.58 — Parallel-development methodology (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.58

> "We MUST DO one comprehensive research and planning in the background in parallel with current mainstream work: our current methodolgy of the development is very slow. … We MUST CREATE adjusted improoved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! … Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind! Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components: ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file-scope manifest, §11.4.43 RED test, source patch, pre-build gate per §11.4.4(b) layer 1, post-flash test per §11.4.4(b) layer 3, paired §1.1 meta-test mutation, §11.4.4(b) layer 4 HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52.

5-stage pipeline: (Stage 1 DEVELOP — parallel PWU agents in isolated worktrees) → (Stage 2 MERGE — serial conductor via `commit_all.sh` flock

- §11.4.41 4-step merge-first) → (Stage 3 REBUILD+FLASH — parallel where hardware allows) → (Stage 4 VALIDATE — parallel on D3+D4+meta-test+ coverage) → (Stage 5 SWEEP — parallel HelixQA Challenges + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N — the throughput multiplier.

Synchronization mechanism: 4-layer lock hierarchy (L1 parent flock / L2 per-submodule git ops / L3 contention-path advisory locks for the 10 forbidden cross-PWU paths / L4 per-PWU git worktree). Disjoint-scope PWUs run fully parallel. Conflict detection at Stage 2: `git fetch --all --prune --tags` + scope-overlap check → overlap detected rejects PWU back to Stage 1.

Anti-bluff enforcement at merge time (all four required): C1 §11.4.43 RED-test captured-evidence (proves test catches regression), C2 §1.1 paired meta-test mutation (proves gate is not bluff), C3 §11.4.50 deterministic-consistency (3 or 10 iterations identical), C4 §11.4.5 captured-evidence (audio WAV / video screen recording / UI uiautomator dump / HelixQA `result.json`). Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS all REJECTED.

HelixQA mandatory. Every user-visible PWU MUST add a Challenge entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to its on-device test + scoring PASS only on positive captured evidence per §11.4.5.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE` + `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION` (anchor literal across 42 consumer files). Paired mutations strip the lock helpers / forge a READY marker without evidence / bypass the merge queue / spawn a 7th concurrent agent / strip the anchor literal — every mutation FAILs its gate.

No escape hatch — no `--skip-merge-queue`, `--allow-bypass`, `--no-anti-bluff-check`, `--unlimited-agents`, `--sequential-phase-chain- mode` flag. Composes with §11.4.4, §11.4.5, §11.4.6, §11.4.9, §11.4.15, §11.4.16, §11.4.19, §11.4.33, §11.4.41, §11.4.42, §11.4.43, §11.4.45, §11.4.49, §11.4.50, §11.4.52, §11.4.54, §11.4.57, §12.6, §12.7, §12.8, §12.10, §9.2.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

Non-compliance is a release blocker regardless of context.

- §11.4.59 — README always-sync mandate (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.12 §11.4.18 §11.4.44 §11.4.45 §11.4.53 §11.4.56 §11.4.57 §11.4.59
- §11.4.60 — Documentation always-sync composite covenant (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.12 §11.4.15 §11.4.16 §11.4.19 §11.4.23 §11.4.33 §11.4.44 §11.4.45 §11.4.53 §11.4.56 §11.4.57 §11.4.59 §11.4.60

- §11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.44 §11.4.63

MANDATORY HOST-SESSION SAFETY (Constitution §)

Every script, test, helper, and AI agent MUST respect host-session safety. Non-compliance is a release blocker.

Forbidden — directly OR indirectly

1. Suspending the host (`systemctl suspend` , etc.).
2. Hibernating / hybrid-sleeping the host.
3. Logging out the user (`loginctl terminate-session` , `pkill -u <user>` , anything targeting `user@<uid>.service`).
4. Unbounded-memory operations inside `user@<uid>.service` cgroup — any command expected to exceed ~4 GiB RSS MUST be wrapped in a bounded execution scope.
5. Programmatic rfkill toggles, lid-switch handlers, power-button handlers — these cascade into idle-actions.
6. Disabling session managers "to make things faster".

Required safeguards for heavy scripts

1. Source the project's host-safety helper library.
2. Call its pre-flight check and abort if it fails.
3. Wrap any subprocess expected to exceed ~4 GiB RSS in a bounded execution scope.
4. Cap parallelism to fit available RAM.

§ . Memory-Budget Ceiling — % MAXIMUM

Forensic anchor — verbatim user mandate:

> "First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

Project procedures MUST NOT use more than **60%** of total system RAM. The remaining 40% is reserved for the operator's other workloads. No escape hatch — bypassing this is the bluff §11.4 forbids.

§ . Continuation document maintenance

`docs/CONTINUATION.md` MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

MANDATORY ABSOLUTE DATA SAFETY — ZERO RISK (Constitution §)

Every destructive repository operation (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, object pruning) MUST follow the full §9 safety protocol WITHOUT EXCEPTION:

1. **Backup first, always** — hardlinked mirror of `.git` to a sibling backup directory (`cp -al .git <backup>/repo.git.mirror`) is near-instant and uses zero additional disk.
2. **Record metadata** — refs, tags, submodule pointers, HEAD commit, HEAD tree hash, tree content sha256.
3. **Define expected post-op state**.
4. **Run the destructive operation** — never with `--no-verify` , never with `--force` that bypasses hooks, never auto-force on failure.

5. **Post-op gate** — HEAD tree identical, all tags preserved, all submodule pointers intact, per-entry archive integrity 100%, pre-build gates green. If any check fails → restore immediately.
6. **Force-push authorization** — force-push is NEVER automatic. Each force-push event requires explicit human authorization AND requires the post-op gate to have passed.
7. **Audit trail** — every history rewrite gets a `docs/changelogs/` "Force-push audit" section.

Hardlinked backup is so cheap (zero disk, <2 s) that there is NEVER an excuse to skip it.

MANDATORY COMMIT & PUSH CONSTRAINTS

1. **Use the project's official commit wrapper** for the main repo (e.g. `scripts/commit_all.sh`).
2. **NEVER use `git add`, `git commit`, or `git push` directly** on the main repo unless the project Constitution explicitly carves out a use case.
3. **Multi-upstream push is the norm** — every commit pushed to ALL configured upstream remotes. The constitution submodule ships an `install_upstreams.sh` (and an `Upstreams/` directory) that configures all remotes locally.
4. **NEVER skip hooks** (`--no-verify`, `--no-gpg-sign`) unless the user explicitly authorizes it for the session.

MANDATORY TESTING CONSTRAINTS

Tests MUST be run at every stage WITHOUT EXCEPTION:

1. **Pre-build / pre-merge verification** — BEFORE every build.
2. **Post-build / packaging verification** — AFTER every build.
3. **Post-deploy verification** — AFTER every deploy / flash.

NEVER skip tests. NEVER mark a test as "broken — disable for now" without fixing the underlying issue. NEVER ship a release with unresolved WARNs.

Code conventions

Universal conventions applicable to most projects:

- Use the project's preferred language for new code. Don't mix languages in one module without explicit Constitution permission.
- Static analyzers / linters / type-checkers MUST run clean (zero warnings) before commit.
- Style is set by the project's formatter; do not hand-edit style in PRs.
- Public APIs are documented at the source.

When in doubt

- Read `constitution/Constitution.md` for the canonical text.
- Cross-reference the project's CLAUDE.md / AGENTS.md for project- specific extensions.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff.

- §11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.12 §11.4.18 §11.4.23 §11.4.44 §11.4.45 §11.4.53 §11.4.59 §11.4.60 §11.4.63 §11.4.64 §11.4.65
- §11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.7 §11.4.40 §11.4.41 §11.4.42 §11.4.52 §11.4.66
- §11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.4 §11.4.6 §11.4.50 §11.4.51 §11.4.67
- §11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.2 §11.4.5 §11.4.13 §11.4.14 §11.4.46 §11.4.49 §11.4.50 §11.4.52 §11.4.68

- §11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.69

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`, `artifact_not_yet_built` (the last added

2026-07-17 — the NOT-YET-RUNNABLE class: the check is CORRECT and the topology IS present, but its precondition artifact is not built/deployed yet; seam-dependent per §11.4.135 — honest+non-blocking on an intermediate artifact, BLOCKING on the release candidate — NOT flat. Consumer-migration note: adding a member to a CLOSED set means every existing `ab_skip_with_reason` implementation REJECTS the new code at call time until updated — update the helper alongside the constitution pull, per the §11.4.164 seam). Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.

- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post- grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

- §11.4.70 — Subagent-driven execution is the default (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.70

> "Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution via `superpowers:executing-plans` is permitted ONLY when (a) the task is trivial AND fits in a single sub-300-line edit, OR (b) the operator explicitly requests inline execution at brainstorm-handoff time.

Subagents bring an isolated context window per task (no conductor context bloat), a structurally separated review seam (conductor reviews subagent output, eliminating self-review blind spots), parallel-PWU compatibility (§11.4.58 — subagents ARE the parallel work units), and resumability across operator absence (subagents resume from on-disk plan + spec inputs).

Composes with §11.4.4 (four-layer coverage), §11.4.6 (no-guessing — subagent's captured output IS the evidence), §11.4.42 (iteration discipline), §11.4.43 (TDD-fix), §11.4.50 (deterministic consistency), §11.4.51 (LIVE_ADB_FIRST), §11.4.58 (parallel- development PWU).

No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff. Pre-build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired meta-test mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.70.

Non-compliance is a release blocker regardless of context.

- §11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.4 §11.4.5 §11.4.6 §11.4.26 §11.4.32 §11.4.37 §11.4.40 §11.4.41 §11.4.42 §11.4.43 §11.4.71
- §11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.42 §11.4.58 §11.4.72
- §11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.44 §11.4.59 §11.4.61 §11.4.65 §11.4.73
- §11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.10 §11.4.61 §11.4.65 §11.4.73 §11.4.74
- §11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.40 §11.4.41 §11.4.65 §11.4.66 §11.4.67 §11.4.71 §11.4.72 §11.4.73 §11.4.74 §11.4.75
- §11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.28 §11.4.31 §11.4.36 §11.4.71 §11.4.74 §11.4.75 §11.4.76
- §11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.10 §11.4.65 §11.4.66 §11.4.71 §11.4.74 §11.4.75 §11.4.76 §11.4.77
- §11.4.78 — CodeGraph code-intelligence mandate (User mandate, 2026-05-20) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.10 §11.4.12 §11.4.17 §11.4.30 §11.4.35 §11.4.65 §11.4.74 §11.4.77 §11.4.78 §11.4.79
- §11.4.79 — Own-org submodules MUST be included in the CodeGraph index (User mandate, 2026-05-21) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.10 §11.4.74 §11.4.78 §11.4.79

- §11.4.80 — CodeGraph regular-update + sync automation mandate (User mandate, 2026-05-21) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.10 §11.4.45 §11.4.53 §11.4.65 §11.4.78 §11.4.79 §11.4.80
- §11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.2 §11.4.3 §11.4.4 §11.4.5 §11.4.6 §11.4.20 §11.4.27 §11.4.69 §11.4.70 §11.4.81
- §11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.4 §11.4.6 §11.4.9 §11.4.20 §11.4.24 §11.4.42 §11.4.43 §11.4.50 §11.4.51 §11.4.52 §11.4.58 §11.4.70 §11.4.82

§ . . — docs/qa/ end-user evidence mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-22):

> "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run. A feature with no QA transcript is itself a §11.4 / §107 PASS-bluff: it claims to work but has no auditable runtime evidence that an end user actually exercised the feature through the same interface they will use in production.

Operative rule. (1) Every consuming project MUST maintain a `docs/qa/` tree. Each new feature lands under `docs/qa/<run-id>/` where `<run-id>` is monotonic + greppable (timestamp, HRD-NNN, ATM-NNN, other workable-item ID per §11.4.54). (2) Transcripts MUST be full bidirectional — every prompt/command sent + every response received. One-sided is not a transcript. (3) Attached materials MUST be committed in-repo (no external-only links — that is §11.4.13

sink-side violation). (4) Bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact. (5) CI / release gates MUST refuse to tag a version that has any feature-shipping commit without its matching `docs/qa/<run-id>/` directory.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.83.

Non-compliance is a release blocker. No `--qa-evidence-optional` escape hatch.

§ . . — Working-tree quiescence rule for subagent commits (User mandate, - -)

Short tag: `working-tree quiescence` .

Forensic anchor — verbatim user mandate (2026-05-22):

> "no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add` , the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired` , `// always pass` , `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add` , the committing agent MUST grep its own working tree for mutation markers (`MUTATED for paired` , `// always pass` , `return json.Marshal` shortcut paths, `// MUTATION` / `# MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area. Any unexplained file → ABORT.

Lesson (forensic case study). A consuming project's logo-fix subagent (Herald commit `72e81ab` , 2026-05-21) ran in a checkout where a paired §1.1 mutation gate had temporarily introduced an `// always pass` shortcut into a JWT verify path. The subagent's `git add` swept the mutation residue into the same commit

as the unrelated logo fix, and the resulting commit was pushed to all four mirrors before any other agent caught it. The fix (Herald `d5bd360`, "SECURITY FIX: restore commons_auth/middleware.go JWT verify") landed within the hour, but the window during which production-equivalent binaries shipped with a bypassed JWT verify is a real security-defect window. The lesson is now constitutional.

Operative rule. (1) Pre-`git add` MUST grep for mutation markers + cross-check `git status --porcelain` against the subagent's declared scope; unaccounted entries → ABORT. (2) Any active mutation gate MUST be serialised — mutate → assert FAIL → restore → assert PASS — and the working tree MUST be verifiably clean BEFORE any unrelated commit. (3) Concurrent subagents in the SAME checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`); the cleaner solution is `git worktree add` per subagent (composes with §11.4.20/ §11.4.70). (4) Post-commit `mutation-residue-scanner` MUST run before push; any commit containing a mutation marker → push BLOCKED.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: constitution submodule `Constitution.md` §11.4.84.

Non-compliance is a release blocker. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted.

§ . . — Stress + Chaos Test Mandate (User mandate, - -)

Short tag: `stress-chaos-mandate`.

Forensic anchor — verbatim user mandate (2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock, per-iteration latency p50/p95/p99 recorded) + concurrent contention (N ≥ 10 parallel invocations, no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input — every boundary produces a categorised result).

Chaos (closed-set, mechanically auditable, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call, categorised recovery) + network-fault injection (drop/delay/reorder, `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test, detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade gracefully, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault — recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason` per §11.4.69. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT`; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (test files exist + executable + sh -n + bash -n parseable; library exists; fix's pre-build gate cites the stress+chaos test path) + paired meta-test mutation per §1.1 (strip chaos-injection or evidence-capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51,

dispatched against a real device with captured evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden — set-u crashes from chaos step are bluffs), §11.4.5 (captured-evidence content quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations produce identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists.

§ . . — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-25):

> "Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!"

Some Status docs (§11.4.45) are backed by a **tracked roster** (installed apps/components) or a **tracked asset corpus** (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (player app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync **out of the box**, mechanically.

Mechanism (all must hold): (1) **drift-proof fingerprint** — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) **sync helper** that regenerates the fingerprint + re-exports HTML+PDF (via the §11.4.65 exporter), wired so sync is automatic; (3) **pre-build gate** that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) **paired §1.1 mutation** that corrupts the fingerprint and asserts the gate FAILs.

Composes with §11.4.12 / §11.4.45 (roster/corpus specialisation) / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Classification: universal (§11.4.17) — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Canonical authority: constitution submodule `Constitution.md` §11.4.86.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-roster-sync`, `--allow-status-drift`, `--roster-sync-not-applicable` flag exists.

2026 05 26

§ . . — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, - -)

When the operator instructs an AI agent to "continue in endless loop fully autonomously" (or any semantically-equivalent phrasing), the agent MUST treat this as a HARD-CONTRACT covenant: (A) continue working until `docs/Issues.md` Status-column has zero non-terminal entries AND `docs/CONTINUATION.md` §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, "waiting for results" is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs — "physical proofs"); (D) the §11.4 anti-bluff covenant family (§11.4.1/§11.4.2/§11.4.6/§11.4.7/§11.4.27/§11.4.50/§11.4.52/§11.4.68/§11.4.69/§11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally per the historical forensic anchor "we had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and

can't be used"; (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.87.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--idle-OK` (11.4.88), `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag exists.

2026-05-26

§11.4.89 — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, 2026-05-26)

Forensic anchor: 2026-05-26 a single commit_all.sh held its flock ~5 hours because do_push ran synchronously after the commit landed — every subsequent commit was blocked on a slow mirror push that had nothing to do with the local commit's durability. Implementation seam for §11.4.87(B) zero-idle.

The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 &` + `disown` — orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — next autonomous-loop tick checks per §11.4.87(A) "no external dependency in-flight" gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths.

Composes with §2.1 / §9.2 / §11.4.41 / §11.4.42 / §11.4.71 / §11.4.87. Pre-build gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 meta-test mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.88.

Non-compliance is a release blocker. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at execution layer. No escape hatch beyond `--sync-push` for force-push events.

§ . . — Background test execution mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution."

Forensic incident: 2026-05-27 conductor invoked `pre_build_verification.sh` synchronously with foreground `timeout 360`, blocking main stream for 6-7 minutes on §JV/§JW/§JX/§JY scaffolding work. Symmetric anchor to §11.4.88 (background push) at the test-execution layer.

Mandate: (A) Long-running tests (>30s expected: pre_build, meta_test, test_all_fixes, recent_work_validate, HelixQA banks, 4-phase cycles, full-suite retests, audio_loop_supervisor, dual_display_record) MUST run via `nohup ... > <log> 2>&1 &` + `disown` with log placed under known dir (`qa-results/<test_id>_<ts>.log`); (B) Main stream proceeds to §11.4.42 priority queue immediately; (C) Hard-dependency gating: poll exit-status file or `pgrep -af <test>` before steps that need exit code — surface as §11.4.66 interactive options if test still running; (D) Failures land in `<log>` files, next loop tick checks; (E) Foreground execution permitted ONLY for <30s tests OR explicit operator authorisation; (F) Per-script flock serialises same-script invocations, different-script invocations parallel.

Composes with §11.4.42 / §11.4.66 / §11.4.82 / §11.4.84 / §11.4.85 / §11.4.87 / §11.4.88. Pre-build gates `CM-COVENANT-114-89-PROPAGATION` + `CM-BACKGROUND-TEST-EXECUTION-WIRED` + paired §1.1 meta-test mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.89.

1 1 4 9 0

Non-compliance is a release blocker. No escape hatch beyond explicit per-invocation operator authorisation.

2 0 2 6 0 5 2 7

§ . . — Obsolete status + per-item obsolescence audit (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "Bug No 6 - Albums cover etc. seems obsolete after latest request for new behavior when audio and video content is being played ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!"

§11.4.15 Status closed-set extended with terminal `Obsolete (→ Fixed.md)` value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): `superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology | not-reproducible` (`not-reproducible` = a reported defect that does NOT reproduce on the canonical tree/baseline — an environment/isolated-worktree artifact, not a real product defect; triple-check evidence captures the canonical-tree non-reproduction). Every Obsolete heading MUST carry `**Obsolete-Details:**` line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. §11.4.23 colorizer adds `cell-status-obsolete` class — light-gray `#E0E0E0` background + strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 + §11.4.42. Triple-check non-negotiable per operator mandate. Composes §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Pre-build gates `CM-COVENANT-114-90-PROPAGATION` + `CM-ITEM-OBSOLETE-DETAILS` + `CM-OBSOLETE-COLORIZER-WIRED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.90. Non-compliance is a release blocker.

§ . . — Summary-doc clarity mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "Summary docs - Issues_Summary some not clear one line descriptions - like 'Composes with' ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclearity of any kind and no bluff allowed!"

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary page 1+2, all one-liners) MUST contain self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (`Composes with` , `Closure criteria` , `Fix direction` , etc.); bare metadata fragments (`Critical` , `Bug` , `In progress` , etc.); section-marker echoes; §-letter alone. Generators (`generate_issues_summary.sh` / `generate_fixed_summary.sh` / `update_readme_doc_links.sh` / `generate_status_summary.sh`) MUST extract from H1/H2 heading line per §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text. Generators refuse anti-pattern rows — emit `(MISSING DESCRIPTION — fix source heading)` placeholder with visual highlight. Pre-build gate `CM-SUMMARY-CLARITY-DESCRIPTIONS` scans every summary; anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep. Composes §11.4.12 / §11.4.19 / §11.4.23 / §11.4.44 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.74.

Canonical authority: constitution submodule `Constitution.md` §11.4.91. Non-compliance is a release blocker.

§ . . — Multi-pass change-evaluation discipline (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analisys for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!"

Every non-trivial change MUST pass 5-pass evaluation BEFORE commit-ready: **(Pass 1)** Main-task verification — change achieves stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** Regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** Cross-feature interaction analysis — parallel features sharing state/timing/hardware/shell environment audited; **(Pass 4)** Deep-research validation per §11.4.8 — external precedent OR "NO external solution found — original work" + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** Anti-bluff confirmation per §11.4/§11.4.1/§11.4.6/§11.4.27/§11.4.50/§11.4.52/§11.4.69/§11.4.83 — no new bluff surface introduced. Documentation per pass (commit footers OR docs/ entries OR qa-results/evidence). Only AFTER all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND commit message cites exemption explicitly. Composes §11.4.4 / §11.4.5 / §11.4.6 / §11.4.8 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.78 / §11.4.79 / §11.4.82 / §11.4.83 / §11.4.85 / §11.4.87 / §11.4.89. Pre-build gates `CM-COVENANT-114-92-PROPAGATION` + `CM-MULTI-PASS-EVALUATION-EVIDENCE` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.92. Non-compliance is a release blocker.

11 4 93
2026 05 27

§ . . — SQLite-backed single-source-of-truth for workable items (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "There MUST be single source of truth for all of our workable items - SQlite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction".

Text-based Issues/Fixed/Summary/CONTINUATION constellation converted to SQLite-DB-backed single source of truth. DB at `docs/.workable_items.db` (gitignored per §11.4.30 + §11.4.77 regen mechanism). Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40chars + created/modified + composes_with JSON + current_location); item_history (append-only audit per §11.4.34 By/Reason/Evidence); obsolete_details (§11.4.90); operator_block_details (§11.4.21); firebase_metadata

(§11.4.47); meta (schema version + last sync + integrity hash). Go binary at `cmd/workable-items/` : sync md-to-db / db-to-md / diff / validate / add / close. Bidirectional regen byte-identical round-trip (closed-set tolerance for whitespace/section-order). commit_all.sh refuses on diff non-empty. sync_issues_docs.sh invokes Go binary. pre_build runs `workable-items validate` . Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge CME-WORKABLE-ITEMS-001. 6-phase migration: §LA Issues entry → Go binary scaffold + DDL → md-to-db → db-to-md round-trip CI → generator shims → text-direct edits prohibited. Cross-project: Go binary lives in constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Composes §11.4 / §11.4.12 / §11.4.15 / §11.4.16 / §11.4.17 / §11.4.19 / §11.4.21 / §11.4.27 / §11.4.30 / §11.4.33 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.50 / §11.4.52 / §11.4.53 / §11.4.54 / §11.4.55 / §11.4.56 / §11.4.57 / §11.4.58 / §11.4.60 / §11.4.65 / §11.4.74 / §11.4.83 / §11.4.85 / §11.4.86 / §11.4.87 / §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92. Pre-build gates `CM-COVENANT-114-93-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-PRESENT` + `CM-WORKABLE-ITEMS-MD-DB-IN-SYNC` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.93. Non-compliance is a release blocker — text-based-only trackers are §11.4 PASS-bluff at data-architecture layer.

2026 05 27

§ . . — Zero-idle priority-first parallel-by-default operating mode (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!"

§11.4.94 binds §11.4.20+§11.4.42+§11.4.58+§11.4.70+§11.4.72+§11.4.82+§11.4.87+§11.4.88+§11.4.89 into single always-on enforcement: (A) Idle ONLY when every queued item genuinely blocked on external dep (hardware / network upstream / build/test

completion conductor cannot accelerate) OR operator STOP OR §12 host-safety; "don't see what to do" is NEVER valid; (B) before ANY wake/sleep MUST survey parallel-work feasibility per §11.4.42+§11.4.72+§11.4.87, identify non-contending items, dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity+§11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30s wall-clock work via nohup+disown; (F) stability-preserving — composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6-§12.9 host safety; (G) progress updates surfaced when operator asks OR at milestone boundaries. Anti-pattern forbidden: scheduling wake without queue survey; "nothing to do — sleeping" with non-trivial items progressable; serialising parallelisable work; picking lower priority over higher. Pre-build gates `CM-COVENANT-114-94-PROPAGATION` + `CM-PARALLEL-WORK-AUDIT` + paired §1.1 mutations.

11 4 95

Canonical authority: constitution submodule `Constitution.md` §11.4.94. Non-compliance is a release blocker.

2026 05 27

§ . . — Workable-items SQLite DB TRACKED in git, NEVER gitignored (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "We shall not Git ignore our workable items SQlite DB since it is our single source of truth ... workable items SQlite DB regularly commited and pushed to all upstreams!"

§11.4.93's earlier "gitignored per §11.4.30" clause is AMENDED — the DB at `docs/workable_items.db` is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every `workable-items sync md-to-db` that mutates state MUST stage+commit+push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. WAL-checkpoint (`PRAGMA wal_checkpoint(TRUNCATE)`) required before commit-stage so transient `.db-wal` + `.db-shm` sidecars (gitignored per §11.4.30) are safely discardable. §11.4.77 regeneration mechanism does NOT apply — DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization;

§11.4.41 force-push merge-first applies if DB history ever needs rewrite. Pre-build gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.95. Non-compliance is a release blocker.

§ . . — Safe-parallel-work-with-long-build catalogue + mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... If yes, and that was not already clear by our root Constitution, we should add all mandatory details into it ... do as much as possible work in background in parallel with main work stream and oreferrably using subagents-driven approach!"

Operational catalogue for the canonical long-running workload (5-7h AOSP containerised build per §12.9):

SAFE during build: (A) MD/docs work under docs/+README+CLAUDE/AGENTS/QWEN+Status+Issues/Fixed+changelogs+research notes; (B) generator/helper script work under scripts/ scripts/testing/ scripts/llm/ scripts/firebase/; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts under tests/test_*.sh; (E) constitution submodule edits + push to 6 remotes; (F) any submodule commit + push per §11.4.88; (G) D3/D4 read-only live-ADB probes (dumpsys/getprop/cat /proc/asound/screencap/logcat); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) pre-build + meta-test execution backgrounded per §11.4.89.

UNSAFE during build: (α) git checkout/reset --hard/clean -df on source tree (use git worktree instead); (β) mass file deletes/renames under device/+frameworks/+hardware/+vendor/+kernel-5.10/+external/+packages/+bionic/+bootable/+build/+system/+cts/+tools/+prebuilts/; (γ) submodule pointer updates affecting built APKs (defer to between-build windows); (δ) out/ directory mutations; (ε) make clean/m clobber/rm -rf out/; (ζ) container destruction; (η) disk-filling operations breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches.

Conductor responsibility: before EVERY pause point during long build, consult catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, dispatch ≥1 per §11.4.20/§11.4.70 subagent default + §11.4.89 background. "Build running, nothing else to do" NEVER true per §11.4.94+§11.4.96. Pre-build gates `CM-COVENANT-114-96-PROPAGATION` + `CM-PARALLEL-WORK-DURING-BUILD-AUDIT` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.96. Non-compliance is a release blocker.

§ . . — Maximum-use-of-idle-time + progress-update cadence (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27): "keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!"

Operating-mode capstone strengthening §11.4.87+§11.4.94+§11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND not genuinely blocked = §11.4.97 violation; "as much as possible, if not it all but as much as we can" is operative — dispatch CONTINUOUSLY through entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit operator-facing 1-line update at every: commit landed / subagent return / constitutional anchor / captured evidence / milestone closure; no operator prompt required; (C) continuous physical-proof gathering per §11.4.5+§11.4.6+§11.4.69 — every autonomous closure cites captured-evidence; evidence path goes into §11.4.93 item_history.evidence_path when DB lands; (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) idle-only-when-blocked closed-set unchanged from §11.4.94(A). Pre-build gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.97. Non-compliance is a release blocker.

§ . . — Full-Automation Anti-Bluff Mandate — Live tests MUST be re-runnable end-to-end without manual intervention (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-28): "Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!"

Composes with §11.4 + §11.4.2 + §11.4.5 + §11.4.50 + §11.4.85 + §11.4.87 + §11.4.89 + §11.4.94 — closes the **manual-intervention gap** they did not explicitly forbid. A live/integration/e2e/Challenge test requiring a human action during execution (typing a chat message, clicking a UI, hand-triggering a webhook, anything beyond test start + PASS/FAIL report) is **by definition a §11.4 PASS-bluff at the automation layer**, regardless of how thorough the manual run is — cannot run continuously in CI, cannot validate regressions between manual runs, human dependency masks drift.

(A) Binding rule: every test this Constitution governs — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end; reports PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap OUTSIDE test execution (`.env` from vault, shell exports in `~/.bashrc` , OAuth approval at first install, MTProto session activation at first run) — configuration, not test driving. (C) Concrete requirements for live messenger/channel/agent tests: (1) no "operator MUST type a message" prompts — drive programmatically (MTProto for Telegram, real-user-token API for Slack, IMAP-test-account for email, webhook fixture, in-process loopback — never human keystrokes); (2) no hard-coded session UUIDs that collide with active dev session (Herald 2026-05-28 lesson: `claude --resume <UUID>` on same UUID as dev session returns silent exit -1); (3) no 60s human-response windows (§11.4.50 determinism violation); (4) re-runnability proof — PASS at `-count=3` consecutive automated invocations with self-cleaning state; (5) §11.4.98 obsolescence audit — every existing test classified COMPLIANT vs NON-COMPLIANT; (6) no false-positive PASS — silent-skip-reported-as-PASS forbidden, stale-evidence forbidden, SKIP-with-reason per §11.4.3 is correct. (D)

Composes with §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 — together = continuously-validated, fully-automated, non-flake, anti-bluff regime. (E) Inheritance per §11.4.35 — every consuming repo's CLAUDE.md/AGENTS.md/QWEN.md restates citing literal anchor `11.4.98`; pre-build gate `CM-COVENANT-114-98-PROPAGATION` enforces literal presence; paired §1.1 mutations strip → gates FAIL. (F) Enforcement: commit adding manual-action test BLOCKED at release-gate; manual-dependency test not rewritten within 30 days graduates to §11.4.90 Obsolete citing §11.4.98 as obsolescence reason.

Canonical authority: constitution submodule `Constitution.md` §11.4.98. Non-compliance is a release blocker.

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate — instructions/guides/manuals MUST be verified against latest official online sources BEFORE publication (User mandate, 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28): "Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!"

Case study (Herald 2026-05-28): first-draft Herald MTProto setup guide recommended VoIP/Google-Voice/Twilio fallback AND omitted the `recover@telegram.org` pre-login email — both directly contradicted Telegram's official docs + gotd/td maintainer guidance, and following the guide could have caused a permanent Telegram account ban. Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer.

Composes with §11.4.4 + §11.4.5 + §11.4.8 + §11.4.92 Pass 4 + §107 + §11.4.98. Closes the gap §11.4.92 Pass 4 alludes to but does not explicitly mandate. (A) Pre-commit cross-reference: every operator-facing instruction document MUST (1) fetch the LATEST official online docs of the service/library via WebFetch/MCP/direct browsing (NEVER training data, memory, or prior committed docs); (2) cross-reference each instruction against the source; (3) seek secondary authoritative sources when the official docs are sparse/silent (library maintainer SUPPORT.md, official changelogs, vetted community FAQs); (4) cite source URL + date in a "##

Sources verified" footer on the doc; (5) cite the cross-reference in the commit-message footer (`Sources verified <date>: <urls>`). (B) Negative findings: gaps/silences/contradictions MUST be documented explicitly so the next reader doesn't assume absence-of-contradiction is authoritative agreement. (C) Re-verification cadence: documents older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or when an operator reports an error following the guide. (D) Risk-classified services (Telegram/WhatsApp messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting services, package managers) — 90-day max staleness; explicit safety warnings cited against latest policies. (E) Composes with §11.4.92 Pass 4 but is INDEPENDENT — cannot substitute. (F) Inheritance per §11.4.35 — restate literal anchor `11.4.99` in every consumer's CLAUDE/AGENTS/QWEN; pre-build gate `CM-COVENANT-114-99-PROPAGATION` (when implemented) enforces; paired §1.1 mutations strip → gates FAIL. (G) Enforcement: commit without "Sources verified " doc-footer AND commit-message-footer BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete with `Reason=stale-documentation` .

Canonical authority: constitution submodule `Constitution.md` §11.4.99. Non-compliance is a release blocker.

- §11.4.100 — RETIRED.** Demoted to consumer project (a consuming project's video-color/visual-quality fideli… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.17 §11.4.35 §11.4.100

11 4 101

2026 05 28

§ . . – Autonomous-decision-over-blocking mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-28):

> "when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

When operating in an autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and instead make the safe, reliable, reversible decision itself — so work is NOT stalled (e.g. overnight) waiting for input. §11.4.87 says keep working; §11.4.101 says HOW to clear the decision points that would otherwise force a stop-and-wait.

Decision rule (closed-set — proceed autonomously when ALL hold): (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the agent can determine the safe choice from captured evidence per §11.4.6 (no guessing — `LIKELY` is not a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9.

Block-only-when rule (BLOCK via §11.4.66 ONLY when ALL hold): the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending / sending to third parties. `Operator-blocked` per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes.

Maximize-progress-while-blocked: an unavoidable block parks one work unit, it does not pause the loop — the agent MUST keep progressing every NON-blocked item in parallel per §11.4.87 + §11.4.94. Posing the question and going idle is a §11.4.94 + §11.4.97 violation.

Composes with §11.4.6 / §11.4.21 / §11.4.40 / §11.4.41 / §11.4.66 / §11.4.87 / §11.4.94 / §9.2 / §12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-101-PROPAGATION` enforces the literal anchor `11.4.101` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → gate FAILs (gate-code = separate work item). No escape hatch — no `--always-block-on-decision`, `--never-decide-autonomously`, `--skip-decision-rule`, `--block-without-self-resolution` flag exists.

Canonical authority: constitution submodule `Constitution.md` §11.4.101.

Non-compliance is a release blocker regardless of context.

§ . . — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-29):

> "Make sure that we ALWAYS trigger / start the "/superpowers:systematic-debugging" skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / unconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analisys / debugging and determine root causes of all problem or obtain relevant data and information we need! ... we MUST make sure that "/using-superpowers" skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!"

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first.

(A) Mandatory systematic-debugging activation. On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST.** Full four-phase arc: root-cause (read the real failure, reproduce, gather facts) → pattern (classify the defect, search for the same pattern elsewhere) → hypothesis (form a falsifiable cause, prove/disprove with captured evidence) → implementation (design the fix only against the proven root cause). Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes ("probably transient / flaky") WITHOUT a completed investigation are §11.4.102 violations. Real-world signal: calling a failure `transient` / `flaky` / `intermittent` / `probably-timing` without captured forensic evidence is

simultaneously a §11.4.6 (no-guessing) and §11.4.7 (demotion-evidence) violation — §11.4.102 makes the corrective response mechanical (the classification is forbidden until the systematic-debugging arc proves it).

(B) Mandatory always-loaded `using-superpowers`. `superpowers:using-superpowers` (or the platform-equivalent skill-discovery / capability-index discipline) MUST be loaded and applied at session start and consulted before any task. Operative rule: before acting on ANY request, survey available skills; if ANY skill could apply — even at 1% relevance — it MUST be invoked rather than improvised from memory. Skipping skill-discovery at session start, or improvising a task a loaded skill exists for, is a §11.4.102 violation.

(C) Mandatory plugin / dependency availability. Every skill plugin / marketplace package / capability dependency the project relies on (Claude Code, its skill marketplaces, or any other runtime's plugin ecosystem) MUST be installed + loadable BEFORE the dependent work proceeds. A missing plugin that blocks a mandated skill (e.g. `superpowers` absent so `systematic-debugging` cannot launch) is a **release-blocker** until installed + confirmed loadable. Install mechanism: the runtime's own plugin/marketplace install path — for Claude Code the in-session `/plugin` marketplace flow (add marketplace → install plugin → confirm the skill appears in the available-skills list); for other runtimes the documented package/extension installer. Anti-bluff: confirm by observing the skill in the live capability list, never assume install succeeded (install exit 0 ≠ skill loadable, per the §11.4.80 lesson).

(D) AUTOMATIC activation — no operator prompt required (User mandate, 2026-07-13). Verbatim operator mandate (2026-07-13): "Make sure we automatically turn on the systematic-debugging plugin and skills every time we work on any issue! Add this as MANDATORY RULE to our root constitution Submodule and all of its relevant files!" The clause-(A) systematic-debugging activation is AUTOMATIC and the STANDING DEFAULT — the agent MUST auto-activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) on ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour WITHOUT waiting for an operator prompt, request, or per-issue confirmation, engaged the MOMENT an issue is detected (composes §11.4.126 default-autonomous-loop's from-first-prompt standing-default working mode). Before any such investigation proceeds, the `superpowers` plugin +

`superpowers:systematic-debugging` AND `superpowers:using-superpowers` skills MUST be confirmed installed + loadable per clause (C) — observed in the live capability list, never assumed (§11.4.6 anti-bluff). "I did not activate systematic-debugging because the operator did not ask" is itself a §11.4.102(D) violation — clause (A) is NOT a prompt-gated option, it is a standing mechanical default engaged automatically the instant a problem surfaces. STRENGTHENS clause (A) (prompt-independent + standing-default), never weakens it: (A)/(B)/(C) + the four-phase-arc + the Iron-Law ("NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST") preserved in full.

Composes with §11.4.4 (test-interrupt — first action after STOP is launch systematic-debugging), §11.4.6 (no-guessing — (A) is the procedural enforcement producing the facts §11.4.6 demands), §11.4.7 (demotion-evidence — the arc captures same-conditions evidence), §11.4.8 (deep-web-research — inside the hypothesis phase), §11.4.43 (TDD-fix — RED test written against proven root cause), §11.4.70 (subagent-driven — the investigation is a natural subagent dispatch), §11.4.82(A) (generalises Phase-1-forensic-before-speculative-patch to ANY fix + adds skill-discovery + plugin-availability), §11.4.92 (feeds Pass 1 + Pass 4), §11.4.126 (default-autonomous-loop — §11.4.102(D)'s always-on-by-default activation mirrors §11.4.126's from-first-prompt standing default). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-102-PROPAGATION` (literal `11.4.102` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--skip-systematic-debugging`, `--guess-and-retry-OK`, `--symptom-patch-permitted`, `--skip-skill-discovery`, `--plugin-optional`, `--missing-plugin-is-warning` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.102.

Non-compliance is a release blocker regardless of context.
2026 05 29

§ . . — Continuous parallel-stream working routine (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-29):

> "Do this working approach continuously and make it part of regular working routine and add it to the Constitution Submodule documented fully."

Promotes the proven multi-stream operating pattern into the project's **standing default working routine** (not a per-request opt-in). Binds §11.4.87/§11.4.88/§11.4.89/§11.4.94/§11.4.96 and adds the load-bearing invariant: **the main work stream MUST always stay FREE.**

(A) Main stream stays FREE. ALL commit AND push operations run detached (`nohup … &` + `disown`, per §11.4.88 commit-lock-release-immediately + detached-push) — the main stream returns to the priority queue the moment the local commit is durable, never blocking on a push or a slow mirror. **(B) ≥3 parallel background streams at all times + auto-backfill** (User mandate 2026-05-29; raised from ≥2) — run **at least three** subagent-driven background streams (per §11.4.70/§11.4.20, isolated per §11.4.58 PWU worktree + §11.4.84 quiescence) alongside the main stream whenever three-plus non-contending actionable items exist; **the moment any one stream is FULLY done, a new stream MUST immediately start and take its place** (claim next-highest-priority non-contending item), so the active-stream count NEVER drops below 3 while actionable items remain. Idle below 3 is permitted ONLY when no remaining non-contending actionable items OR all remaining are externally blocked (§11.4.94/§11.4.97/§11.4.101). Standing band 3–6, bounded above by §12.6 60% memory + §11.4.58 6-agent cap. **(C) Most-critical + most-visible first; audio always top** per §11.4.72 + §11.4.42 priority order. **(D) Safe-during-build scope only** — while the 42 GB containerised AOSP rebuild (§12.9) or any heavy `gradle` / `m -j` build runs, streams restrict to the §11.4.96 SAFE catalogue (investigation/forensic/docs/test-authoring/gate-authoring/read-only probes/submodule edits/research/DB ops/backgrounded pre-build+meta-test); NEVER a second concurrent heavy build (§12.8). **(E) Heavy anti-bluff on every closure** — root cause proven or `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` per §11.4.6, captured evidence per §11.4.5+§11.4.69, deterministic consistency per §11.4.50, paired §1.1 mutations, §11.4.102 systematic-debugging on any spotted problem. **(F) Idle ONLY when genuinely externally blocked** (hardware/network upstream/in-flight build-

test-push completion) OR operator STOP OR §12 host-safety, per §11.4.94(A) +§11.4.97; "nothing visible to do" with progressable items is NEVER valid; a block parks one work unit, not the loop (§11.4.101).

Composes with §11.4.58 / §11.4.70 / §11.4.72 / §11.4.87 / §11.4.88 / §11.4.89 / §11.4.94 / §11.4.96 / §11.4.97 / §11.4.101 / §11.4.102 / §11.4.42 / §11.4.84 / §12.6 / §12.7 / §12.8 / §12.9 / §9.2. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-103-PROPAGATION` (literal `11.4.103` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--block-main-stream`, `--synchronous-commit`, `--synchronous-push`, `--single-stream-only`, `--skip-parallel-streams`, `--serialise-actionable-work`, `--idle-without-queue-survey` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.103.

Non-compliance is a release blocker regardless of context.

2026 05 31

§ . . — Participant identity, attribution & notification-tagging (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-31):

> "Every supported messenger must relate messages to participants (Subscribers/Users); the same logical person may have a different username on every messenger. Workable items must carry who created them and who they are assigned to. Notifications must @-tag the right participant — but never the operator (who drives the system) and never the system agent."

MANDATORY for every consumer that ships a messenger/notification surface (Herald + flavor binaries are the reference impl; others inherit per §11.4.35). Detailed spec: Herald `docs/design/PARTICIPANT_ATTRIBUTION.md` — restated, not redefined. **(A) Participant identity.** Every messenger MUST relate messages to a **Participant** (logical Subscriber/User); the SAME person MAY have a DIFFERENT username per messenger — modelled as a logical subscriber (canonical messenger-neutral `handle`, `kind ∈ {human,agent,service}`) + per-

channel aliases (`channel` , `channel_user_id` , the `@username` used for tagging). Canonical handle closed set: `Claude` (reserved system-agent sentinel; never tagged) OR a subscriber `@username` . **(B) Operator = env var, not a DB flag** — the one human who drives via the agent CLI, designated by `HERALD_<CHANNEL>_OPERATOR_USERNAME` (e.g. `HERALD_TGRAM_OPERATOR_USERNAME`); a normal Participant whose handle equals that value. **(C) Workable items MUST carry `created_by` + `assigned_to`** (canonical handles): CLI-prompt → Operator; system/agent-detected → `Claude` ; received-message → sender's resolved `@username` . `assigned_to` defaults to Operator, overridable. Legacy items carry `""` and MUST still parse + validate. **(D) Tagging matrix:** tag `assigned_to` if it is a human ≠ Operator; tag `created_by` if it is a human ≠ Operator ≠ `Claude` ; NEVER tag `Claude` (system) or the Operator (no self-ping); de-dup; resolve each handle to the channel `@username` , skip if not on that channel. **(E) Anti-bluff (composes §11.4):** real SQLite round-trip with the new columns byte-identical (incl. legacy fixtures WITHOUT the fields); tagging matrix proven by a truth-table + a cell-flip mutation forcing FAIL; E2E real-event → real-message asserting the exact `@username` s + a NEGATIVE case proving the Operator is NOT tagged; evidence under `docs/qa/<run-id>/` .

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.5 / §11.4.69 / §11.4.50 / §11.4.91 / §11.4.93 / §11.4.95 / §1.1. Classification: universal (§11.4.17) — projects with no messenger surface inherit it latently (binds the moment they ship one, per the §11.4.96 pattern). Propagation gate `CM-COVENANT-114-104-PROPAGATION` (literal `11.4.104` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--skip-attribution` , `--no-participant-tagging` , `--tag-operator-anyway` , `--attribution-later` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.104; detailed spec Herald `docs/design/PARTICIPANT_ATTRIBUTION.md` .

Non-compliance is a release blocker.

2026 05 31

§ . . — Natural-language intent recognition & clarification (User mandate, – –)

Forensic anchor — verbatim user mandate (2026-05-31):

> "Users must NOT need to know command syntax (no `COMMAND: …` prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (`@user …`), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System."

MANDATORY for every consumer that ships a messenger/command surface (Herald + flavor binaries are the reference impl; others inherit per §11.4.35). Detailed spec: Herald `docs/design/INTENT_RECOGNITION.md` — restated, not redefined. **(A) No required command syntax.** Users MUST NOT be required to know any command syntax (no `COMMAND:` prefix); they send plain natural language and the System determines the intent. **(B) Three-tier resolution** (first that succeeds wins): TIER 1 — recognize the System's existing command set from natural language → action (confident deterministic match); TIER 2 — when no command matches, infer the exact intent (LLM dispatch maps language → action), NEVER guessing; TIER 3 — when neither a command nor a confident intent can be determined, REPLY to the message, TAG the sender (`@username` , resolved via the §11.4.104 IdentityResolver) and ask a PRECISE clarifying question NAMING the candidate intents — no guessing, no silent drop. **(C) Never guess, never drop:** a wrong action is worse than a clarifying question (composes §11.4.6 no-guessing); a message is NEVER silently dropped; only genuine ambiguity reaches Tier 3, which always replies-tags-and-asks. **(D) Anti-bluff (composes §11.4):** every tier ships unit + integration + E2E + full-automation tests with real captured evidence — Tier 1 truth-table (natural-language → action+fields, plus conservative negatives that MUST fall through to "no match"); Tier 3 E2E whose recorded reply body is EXACTLY `@<sender> <specific question>` + a NEGATIVE proving a clear command does NOT trigger clarify; a paired §1.1 mutation breaking the confidence guard (false-match) OR dropping the clarify tag MUST FAIL a test; evidence under `docs/qa/<run-id>/` .

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.6 (no-guessing) / §11.4.104 (clarify reply tags the sender) / §11.4.5 / §11.4.69 / §11.4.98 / §1.1. Classification: universal (§11.4.17) — projects with no messenger surface inherit it latently (binds the moment they ship one, per the §11.4.96 pattern). Propagation gate `CM-COVENANT-114-105-PROPAGATION` (literal `11.4.105` across

consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--require-command-syntax`, `--guess-intent-ok`, `--skip-clarify`, `--drop-on-ambiguous` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.105; detailed spec Herald `docs/design/INTENT_RECOGNITION.md`.

Non-compliance is a release blocker.

2026 05 31

§ . . — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, - -)

Forensic anchor — operator mandate (2026-05-31): Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates; consumers MUST use the engine instead of ad-hoc per-project doc-sync scripts, register their chains via per-context YAML, and never accept a faked transform.

MANDATORY for every consumer. Docs Chain (the `vasic-digital/docs_chain` engine — a universal Go bidirectional document-and-database dependency-propagation engine) is the canonical mechanical enforcer of the documentation-sync mandates. Detailed spec: the engine docs `~/Projects/docs_chain` → `docs/CONSTITUTION_INTEGRATION.md` (distribution + inheritance + anchor mapping table) + `docs/USE_CASE_CATALOGUE.md` (chain recipes) — restated, not redefined. **(A) Use the engine, never ad-hoc scripts** — consumed **by reference** (the flat-layout sibling `~/Projects/docs_chain` / the constitution-exposed path), inherited like §11.4.80's `codegraph_*` scripts: referenced, NEVER copied; ad-hoc `sync_*` / `generate_*_summary` / `update_readme_doc_links` scripts are superseded and retired per registered context. **(B) Consumer-owned contexts** — the engine is project-agnostic; the consumer registers its chains as data via `.docs_chain/contexts/*.yaml` (§11.4.28 decoupling); `state.json` + `*.docs_chain.tmp` are gitignored. **(C) Anchors it mechanizes** (per the CONSTITUTION_INTEGRATION mapping table): §11.4.12 / §11.4.53 / §11.4.45 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.86 / §11.4.93 / §11.4.95 / §12.10 / §11.4.44 — with content-hash change detection (NOT mtime, §11.4.86), atomic-rename + SQLite-txn commit + rollback (§9.2), both-dirty `sync` → conflict-not-silent-merge (§11.4.6), `verify` as the deterministic CI/pre-build gate (§11.4.50), per-run captured evidence to `qa-results/docs_chain/<run-id>/`

(§11.4.69). **(D) NOT a replacement for authoring discipline** — the source author still writes the §11.4.44 revision header; the engine only keeps exports in sync. **(E) Anti-bluff (composes §11.4):** the engine NEVER fakes a transform — a missing pandoc/weasyprint surfaces a typed `ToolAbsentError` + honest §11.4.3 SKIP-with-reason, never a fake PASS / partial write; every `sync` / `verify` carries real captured evidence.

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant) / §11.4.6 (no-guessing — conflict not silent merge) / §11.4.28 (engine/context decoupling) / §11.4.80 (inherited-by-reference, never copied) / §9.2 / §11.4.50 / §11.4.69 / §11.4.5 / §1.1, plus the sync anchors it mechanizes (§11.4.12/.53/.45/.56/.57/.59/.60/.65/.86/.93/.95/.44, §12.10). Classification: universal (§11.4.17) — projects with no derived-export/DB-sync surface inherit it latently (binds the moment they ship one, §11.4.96 pattern). Status (§11.4.6): engine Phases 1–3 AND the CLI/YAML loader (Phase 4) IMPLEMENTED+tested — `cmd/docs_chain/main.go` registers `sync` / `verify` / `diff` (alias)/ `doctor` , `go test -count=1 ./...` GREEN 7/7 packages; this CLI/loader is in the working tree but UNTRACKED and the engine is NOT yet a registered git submodule (in-tree at `./docs_chain` , HEAD = parent HEAD); only submodule distribution (Phase 6) remains PLANNED + OPERATOR-GATED — wire as phases land, claim no unshipped behaviour. Propagation gate `CM-COVENANT-114-106-PROPAGATION` (literal `11.4.106` across consumer fleet) + paired §1.1 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch — no `--ad-hoc-sync-ok` , `--skip-docs-chain` , `--fake-transform` , `--sync-evidence-optional` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.106; detailed spec the Docs Chain engine docs `docs/CONSTITUTION_INTEGRATION.md` + `docs/USE_CASE_CATALOGUE.md` (`vasic-digital/docs_chain`).

Non-compliance is a release blocker.

§ . . — Anti-bluff AV/test-validation techniques mandate (User-driven research, - -)

Forensic anchor (genericised, 2026-06-02): a test PASSed on a SINGLE captured frame showing "a picture" on the target output — but the picture was a FROZEN / STALE frame from the previously-played content (stale-producer / stuck-decoder), so the feature was broken for the user while the test was green; a sibling incident FLASHED media on the WRONG output for ~1 s before routing with no test sampling that window; a third class shipped a comparator that PASSed its own deliberately-degraded fixture (the analyzer, not the feature, was the bluff). §11.4.5 mandates captured evidence + a presence pass; §11.4.107 raises the bar to **liveness + correct-routing + self-validated-analyzer**.

Every test asserting audio/video output is genuinely playing/advancing for the end user MUST satisfy ALL of: (1) **a single captured frame is NOT proof** — prove LIVE, ADVANCING frames over a steady-state window via a **freeze-detection oracle** (near-duplicate / `freezedetect`-class filter OR perceptual-hash adjacent-frame distance, **NOT byte-identical compare** — byte-identity is only a zero-cost pre-filter); (2) an **independent frame-advance counter from the platform's compositor/decoder telemetry** must increase across the window (a different-domain oracle — flat counter ⇒ stuck decoder ⇒ FAIL even if pixels appear to move); (3) **loading/buffering is a distinct state** — wait for genuine playback-start before judging liveness, never false-FAIL a still-loading stream nor false-PASS a spinner; timeout+unreachable ⇒ SKIP-with-reason per §11.4.3, timeout+reachable ⇒ FAIL; (4) **not-stale-from-previous cross-check** — new content's first frame ≠ previous content's last frame; (5) **measured FPS / no-lost-frames within tolerance**; (6) **no-flash-on-the-wrong-output** — sample the non-target output at high frequency during a routing transition, any content frame there ⇒ FAIL (content-protection regime classified explicitly, never guessed); (7) **drive through the realistic feed/UI path, not deep-link shortcuts** (shortcuts bypass the transition paths where bugs live; UI-not-introspectable ⇒ operator-attended fallback per §11.4.52, never fake-PASS); (8) **metamorphic relations solve the oracle problem when there is no golden source** (same content on output-A vs output-B must match; paused ⇒ counter stops; 2× speed ⇒ ~2× advance rate); (9) **full-reference quality metrics (SSIM/VMAF/ΔE2000) vs a golden source for owned content**; (10) **mutation-test every analyzer with a golden-good + golden-bad fixture pair** so the analyzer itself provably cannot bluff (an analyzer that PASSes its

golden-bad fixture is a bluff gate — §1.1 applied to the analyzers); (11) **per-channel audio RMS/loudness (EBU R128) + XRUN/underrun census** — a single aggregate RMS misses a dead channel; (12) **OCR overlay/subtitle detection needs a per-word confidence floor + ROI** to avoid BOTH false-positive (false-FAIL) and false-negative (false-PASS); (13) **thresholds calibrated on the project's own fixtures, not hardcoded from literature** (§11.4.6 no-guessing).

Honest gap (§11.4.6): true photon FPS at the sink has no clean software oracle — compositor/decoder counters measure the presentation pipeline; flag the gap, never claim it.

4-layer coverage per §11.4.4(b): pre-build gate (a `CM-AV-LIVENESS-NO-FROZEN-FRAME`-class gate asserting every output-is-playing test references the liveness battery not a single frame + an analyzer-self-validation gate wiring the golden-good/golden-bad fixtures into meta-test) + on-device/runtime test + paired §1.1 meta-test mutation (single-frame-only assertion → gate FAILs; analyzer that PASSes its golden-bad fixture → self-validation FAILs) + HelixQA Challenge. Every PASS via `ab_pass_with_evidence` citing the motion / not-stale / frame-advance / fps / per-channel-loudness / metamorphic / self-validation artefacts (`video_display` / `audio_output` / `subtitle_render` per §11.4.69) — never a single screenshot.

Classification: universal (§11.4.17) — platform-neutral AV/test-validation techniques reusable by ANY project validating media playback or any pixel/audio output; the project supplies its concrete capture mechanism + calibrated thresholds per §11.4.35. Composes with §11.4.5 (its strict expansion), §11.4.6, §11.4.50, §11.4.68, §11.4.69, §11.4.85, plus §11.4.2 / §11.4.3 / §11.4.13 / §11.4.48 / §11.4.52 / §1.1. Propagation gate `CM-COVENANT-114-107-PROPAGATION` enforces the literal anchor `11.4.107` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → gate FAILs (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.107.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--single-frame-proves-playback`, `--skip-liveness`, `--byte-identical-freeze-OK`, `--no-frame-advance-counter`, `--skip-not-stale-check`, `--allow-wrong-output-flash`, `--deep-link-shortcut-OK`, `--unvalidated-analyzer-OK`, `--aggregate-rms-suffices`, `--hardcoded-thresholds-OK` flag exists.

§ . . — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase . , - -)

Forensic anchor (genericised, 2026-06-03): across one batch, multiple fixes were "green" at every gate yet NOT working for the end user — a change present in source and passed by both the source pre-build gate AND the post-build gate never reached the boot-time command-line embedded in the deployed boot artifact (output feature dead despite all-green); sibling fixes correctly built into the system image were masked by stale per-user overlay copies from a previous deployment (the running code was the stale shadow, not the fresh build); deployment did not wipe the mutable overlay so the shadow survived; validation ran against whatever was running — the stale shadow — and reported green on code that was never exercised. Per `superpowers:systematic-debugging` Phase 4.5: each fix revealing a fresh "fixed-but-not-working" in a DIFFERENT place is NOT independent bugs — it is ONE architectural VERIFICATION flaw; patching each symptom is thrashing.

A fix crosses FOUR distinct layers, and "fixed" at one does NOT imply fixed at the next: (1) **SOURCE** (committed in the source file — what a grep-the-source pre-build gate checks), (2) **ARTIFACT** (the change's BYTES actually landed in the produced build artifact — image / bundle / installer / embedded command-line), (3) **RUNTIME-ON-CLEAN-TARGET** (active on a CLEAN/fresh deployment — the layer the end user experiences — with no stale overlay shadowing the deployed code), (4) **USER-VISIBLE** (the feature works for the end user — the §11.4.5/§11.4.69 captured-evidence layer). Green at layer 1 is the cheapest, least conclusive signal and says nothing about layers 2–4. A gate verifying ONLY the source layer is itself a §11.4 bluff surface.

The mandate (ALL must hold): (1) **Runtime-signature-as-definition-of-done** — a fix is DONE only when its declared runtime signature verifies on a CLEAN/fresh deployment; source-committed ≠ artifact-contains-it ≠ active-on-clean-target ≠ user-visible-working. (2) **Every fix declares ONE machine-checkable runtime signature** — a single observable on a clean target proving the fix is BOTH active AND working (a running-system property, a downstream/sink-side report per §11.4.13, a §11.4.5/§11.4.69 captured-evidence assertion, or a counter/state delta — NEVER a re-grep of the source); this **registry of per-fix runtime signatures is**

the SINGLE SOURCE OF TRUTH for "fixed" — it REPLACES "a gate greps the source" as the definition of done. (3) **Gates span all four layers** — source (pre-build), artifact (post-build — assert the change's BYTES landed in the artifact, not merely that the source still contains the change), runtime-on-clean-target (post-deploy — assert the runtime signature on a freshly-deployed clean target), user-visible (§11.4.5/§11.4.69). (4) **Eliminate the stale-deployment / shadow layer by construction** — deployment MUST yield a CLEAN state (wipe the mutable overlay) OR a pre-validation assertion MUST prove `running-artifact == built-artifact` BEFORE any validation runs; validation against possibly-stale deployed state is INVALID and any PASS it produces is a §11.4 PASS-bluff (the test exercised code that was never deployed). (5) **Meta-rule** — ≥ 3 "fixed-but-not-working" discoveries in one cycle signal an architectural VERIFICATION flaw, NOT three independent bugs; on the 3rd, STOP patching symptoms (§11.4.4), fix the VERIFICATION pipeline, and re-certify EVERY item through it on a clean target. (6) **A batch is "validated" only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline** — NOT after the touched items' own tests pass.

Classification: universal (§11.4.17) — platform-neutral verification-pipeline disciplines reusable by ANY project that builds an artifact, deploys it, and validates the deployed result; the project supplies its concrete artifact format, clean-deployment / equal-artifact mechanism, and per-fix runtime-signature observables per §11.4.35. Composes with §11.4.1 / §11.4.2 / §11.4.4 (clause 5's STOP-and-fix-pipeline is its §11.4.108 specialisation; "clean baseline" = clause 4's clean deployment) / §11.4.5 / §11.4.6 (every layer asserted by evidence, never assumed to propagate) / §11.4.27 / §11.4.40 (§11.4.108 adds the cross-layer per-item runtime-signature dimension) / §11.4.46 (clean-baseline / equal-artifact pre-flight) / §11.4.50 / §11.4.52 / §11.4.69 (a runtime signature is a taxonomy-class observable) / §11.4.102 (Phase 4.5 architectural-flaw recognition IS clause 5's trigger). Propagation gate `CM-COVENANT-114-108-PROPAGATION` (literal `11.4.108` across the consumer fleet) + recommended per-fix gate `CM-RUNTIME-SIGNATURE-REGISTRY` + paired §1.1 meta-test mutations (strip the literal → propagation gate FAILs; downgrade a fix to source-only verification → registry gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.108.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--source-green-is-done`, `--skip-artifact-byte-check`, `--validate-against-running-state`, `--no-clean-deployment`, `--skip-runtime-signature`, `--spot-validate-touched-only` flag exists.

§ . . — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

Short tag: `anti-forgetting-enforcement`. **UNCONFIRMED forensic anchor** (pending operator's verbatim mandate quote). Background: emulator subagents ran raw host-direct `emulator` / `adb` because the orchestrator forgot to inject the Containers-submodule rule. A rule forgotten at dispatch is not enforcement. Fix: (A) a `PreToolUse` guard hook (`constitution/scripts/hooks/guard-forbidden-commands.sh`) that blocks host-direct emulator, force-push/bypass, sudo, and host-power commands at the tool-call boundary regardless of agent memory; (B) a canonical `docs/AGENT_GUARDRAILS.md` preamble the orchestrator pastes verbatim into every subagent dispatch; (C) an **ORCHESTRATOR PRE-ACTION CHECKLIST** in the same document. Hook = the floor; preamble = the ceiling.

Consuming projects MUST: (1) wire `constitution/scripts/hooks/guard-forbidden-commands.sh` as a `PreToolUse` hook in `.claude/settings.json` (or equivalent runtime settings); (2) maintain `docs/AGENT_GUARDRAILS.md` containing the `SUBAGENT CONSTITUTIONAL PREAMBLE` and `ORCHESTRATOR PRE-ACTION CHECKLIST` headings, with the anchor literal `11.4.109`; (3) provide a hermetic hook test suite (≥ 20 cases: every blocked class exits 2, every allowed command exits 0, escape hatch fires for non-power classes, host-power rejects even with escape marker). The hook is inherited by reference — NEVER copied locally (a copy diverges silently).

Gates: `CM-ANTI-FORGETTING-ENFORCEMENT` (hook present + wired + guardrails doc present + test present) + `CM-COVENANT-114-109-PROPAGATION` (anchor literal `11.4.109` across every consumer CLAUDE.md / AGENTS.md / QWEN.md). Paired §1.1 mutations: remove hook entry → gate (2) FAILs; delete guardrails doc → gate (3) FAILs; strip hook from constitution → gate (1) FAILs; strip `11.4.109` → propagation gate FAILs.

Classification: universal (§11.4.17). Composes with §11.4.6 / §11.4.10 / §11.4.75 / §11.4.76 / §11.4.78 / §11.4.79 / §11.4.80 / §11.4.81 / §11.4.84 / §11.4.98 / §11.4.102 / §12.

Canonical authority: constitution submodule `Constitution.md` §11.4.109; reference implementation `constitution/scripts/hooks/guard-forbidden-commands.sh` + `constitution/docs/AGENT_GUARDRAILS.md`.

Non-compliance is a release blocker. No escape hatch — no `--skip-pretooluse-hook`[11 4], `--no-guardrails-doc`[110], `--anti-forgetting-optional`, `--single-layer-sufficient` flag exists.

2026 06 03

§ . . — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, - -)

Forensic anchor (genericised, 2026-06-03): a fix shipped a new system-property *read* but introduced no matching security-policy grant + no property-context type entry — the read was silently denied at runtime + the feature was dead, while every pre-build gate stayed green because the gate only grepped the source file. The defect class generalises: a change introduces a new dependency on a *second* artifact (security policy, context file, service registry, interface freeze-snapshot, symbol table, build-graph node) that the pre-build never cross-checks. This is §11.4.108's SOURCE→ARTIFACT gap shifted LEFT to pre-build time — most such clashes are statically catchable from the change diff itself, before any build. The mandate (ALL hold): (1) a single deterministic **READY-FOR-BUILD verdict** gates the rebuild (orchestrator refuses to start the build unless READY); (2) a **diff-driven change-impact + clash detector** cross-checks every newly-introduced second-artifact dependency (new property read ⇄ property-context type + read-grant; new service ⇄ service-context entry; new init service ⇄ security label; new/changed stable interface ⇄ freeze-snapshot updated; new native-lib dep ⇄ module/prebuilt resolves; new policy rule ⇄ every type/attribute defined; two batch changes on the same seam ⇄ collision acknowledged); (3) **coverage-completeness is a gate** — every changed file maps to ≥1 gate + ≥1 deployed-target test + ≥1 paired §1.1 mutation, baseline ratchets upward per §11.4.50; (4) **two-speed honesty** — grep-speed always-on gates vs REQUIRES_BUILD heavy gates (build-graph parse-only dry-run, full neverallow compilation, ABI diff) as diff-gated opt-in stages, bounded per §12.6/§12.7; (5) every gate + wired analyzer is **anti-bluff by paired §1.1**

mutation; (6) **honest boundary** — a READY verdict proves static internal-consistency + ready-to-build, NOT that the feature works on the deployed target; the regime empties the *preventable* defect class, not the *all-defects* class (runtime/USER-VISIBLE remains §11.4.108's job).

Classification: universal (§11.4.17) — platform-neutral pre-build-rigor disciplines reusable by ANY project that builds an artifact from source; the consuming project supplies its concrete property/service/policy registries, build-graph parse-only command, interface-freeze mechanism, and changed-file→gate mapping per §11.4.35. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.9 / §11.4.27 / §11.4.50 / §11.4.67 / §11.4.75 / §11.4.92 / §11.4.108 (§11.4.110 is the SOURCE→ARTIFACT half shifted left to pre-build; §11.4.108 owns RUNTIME-ON-CLEAN-TARGET→USER-VISIBLE — together they span all four layers). Propagation gate `CM-COVENANT-114-110-PROPAGATION` (literal `11.4.110`) + recommended per-family gates `CM-READY-FOR-BUILD-VERDICT` / `CM-CHANGE-IMPACT-CLASH-DETECTOR` / `CM-COVERAGE-COMPLETENESS-GATE` / `CM-BUILDGRAPH-DRYRUN-WIRED` / `CM-SEPOLICY-NEVERALLOW-WIRED` + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.110. Non-compliance is a release blocker. No escape hatch — no `--source-green-is-ready`, `--skip-clash-detector`, `--skip-coverage-gate`, `--no-ready-verdict`, `--grep-proves-neverallow`, `--skip-buildgraph-dryrun`, `--build-without-ready-verdict` flag.

11 4 111 2026 06 03

§ . . — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, - -)

Forensic anchor (genericised, 2026-06-03): a platform bound an audio output to a kernel-enumerated device index (`card=0`); a second device of a different class enumerated FIRST at boot and took slot 0, shifting the intended device to slot 1 — the static index binding now pointed at the wrong device (policy mis-attached the sink, mis-assigned its TYPE, the output switcher labelled the AV receiver "Wired Headphone" + routing collapsed to stereo). The lower layer of the SAME stack already resolved the device correctly **by name** (scanning the controller-name registry) — proving the brittleness was the *index* binding, not the resolution capability. Generalises to every enumerated resource whose ordinal is assigned at

discovery/boot/hotplug time (non-deterministic across reboots, device additions, topology changes). The mandate: any binding to a hardware device / resource handle / enumerated entity (audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input/camera devices, container/process slots) MUST resolve by a **stable identifier** (name / UUID / serial / label / controller-name / content-hash / sink-reported identity) and MUST NOT bind by enumeration index / ordinal / slot, UNLESS the platform documents that ordinal as deterministically pinned AND the pin is itself captured + asserted as part of the binding. Where a stable identifier exists at one layer, every other layer binding the same resource MUST use the same identifier (mixed by-name-here / by-index-there is the structural weak link forbidden here). Honest boundary (§11.4.6): when only an ordinal exists, pin it deterministically via the platform's own mechanism, capture the pin in the binding's §11.4.108 runtime signature, and document the residual fragility as `UNCONFIRMED:`-class risk — never silently trust an unpinned ordinal.

Classification: universal (§11.4.17) — platform-neutral binding-robustness discipline reusable by ANY project binding to enumerated hardware/resources/handles; the consuming project supplies its stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) + the layers that must agree per §11.4.35. Composes with §11.4.6 (no-guessing — "the index is *usually* stable" is the exact guess forbidden) / §11.4.8 (mature stacks resolve by name/UUID — reproducing a known-brittle index binding when the by-name path is documented is a §11.4.8 omission) / §11.4.69 (sink-side evidence verifies the by-name binding's correctness) / §11.4.108 (the by-name binding's runtime signature asserted on a clean target across the topology that broke the index) / §11.4.110 (a new ordinal binding in a diff is a statically-catchable clash class). Propagation gate `CM-COVENANT-114-111-PROPAGATION` (literal `11.4.111`) + recommended gate `CM-RESOLVE-BY-NAME-NOT-INDEX` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.111. Non-compliance is a release blocker. No escape hatch — no `--allow-index-binding`, `--ordinal-is-stable-enough`, `--skip-name-resolution`, `--trust-unpinned-index` flag.

§ . . — Structural-impossibility won't-fix classification mandate (research-derived, - -)

Forensic anchor (genericised, 2026-06-03): deep research (§11.4.8) into relocating protected (content-protection / secure-surface) video to a secondary display PROVED — from authoritative platform/HDCP docs + reproducible captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screencap of a secure layer returns black) — that the goal is **structurally impossible by platform design**, not a missing feature or unsolved engineering problem. Without a durable classification, such a goal is re-investigated every cycle (re-read the same sources, re-run the same probes, re-derive the same impossibility — compounding wasted effort). The mandate: when deep research per §11.4.8 PROVES (cited authoritative sources AND, where applicable, reproducible captured evidence) a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation — NOT merely unimplemented or hard), the goal MUST be: (1) classified `Won't-fix` + closed per §11.4.90 with closure reason `structurally-impossible`; (2) documented with the impossibility evidence — cited authoritative source URLs (per §11.4.99 latest-source verification) + the reproducible probe/captured evidence — in the tracker entry + relevant `docs/` guide; (3) NOT re-attempted in future cycles — a reopen MUST cite NEW evidence the platform constraint changed (per §11.4.34 + §11.4.7), never merely re-derive the same impossibility; (4) paired with the correct posture — the entry states what the project DOES instead, so "impossible" is never confused with "broken/unhandled". Honest boundary (§11.4.6): `structurally-impossible` is reserved for *proven* platform/hardware/protocol impossibility — "could not find a way" / "very hard" / "no time" are `Operator-blocked` (§11.4.21) or open work, NOT won't-fix; mislabelling them to avoid the work is a §11.4 planning-layer bluff. A future platform change can make the impossible possible; the classification is durable but not eternal.

Classification: universal (§11.4.17) — platform-neutral effort-conservation + honesty discipline reusable by ANY project; the consuming project supplies the specific impossible goal, its platform constraint, and the cited evidence per §11.4.35. Composes with §11.4.6 (FACT with cited evidence, never "probably can't") / §11.4.7 (reopening requires NEW positive evidence) / §11.4.8 ("NO external solution found — structurally impossible" is the citation) / §11.4.34 (reopen attribution) / §11.4.90 (`structurally-impossible` is a closure reason in the closed vocabulary) /

§11.4.99 (latest-source so the verdict is not stale). Propagation gate `CM-COVENANT-114-112-PROPAGATION` (literal `11.4.112`) + recommended gate `CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.112. Non-compliance is a release blocker. No escape hatch — no `--wont-fix-without-proof`, `--reattempt-closed-impossible`, `--skip-impossibility-evidence`, `--impossible-equals-broken` flag.

§ . . — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-03): "Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule's main (or master) branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule's upstreams!"

Force-push is STRICTLY FORBIDDEN with NO exception — `git push --force`, `--force-with-lease`, `+<ref>`, or any history-rewriting overwrite of a remote ref, against EVERY repository this Constitution governs (main repo, this constitution submodule, every owned + nested submodule, every upstream). No operator-approval path, no "after a merge-first audit" path. The mandated 6-step integration procedure for any repo/submodule whose local has commits to publish OR whose mirrors diverged: (1) `git fetch --all --prune --tags` all remotes; (2) set the base to the LATEST commit on the canonical `main` / `master` branch (the most-advanced mirror tip); (3) carefully MERGE every change to be published on top of that base — union, preserve BOTH sides, NEVER `-s ours` / rebase / reset that drops commits (per §9 no-commit-loss); (4) resolve every conflict carefully — no conflict markers, no file dropped, gates/tests still pass; (5) commit the merge (stage only intended files, NEVER `git add -A` in a submodule per §11.4.30); (6) push to ALL upstreams — each push is a fast-forward because the merge commit descends from every mirror tip, so NO force is ever needed (if an upstream still rejects, return to step 1 for it, merge its new tip, re-validate, re-push). **TIGHTENS §11.4.41 / §11.4.71 / §9.2 / CONST-043 — the force-push escape hatch is REMOVED:** even WITH operator approval, even after a clean merge-first audit,

force-push is forbidden, because the merge-onto-latest-main path is always available so force is never necessary. Those clauses' merge-first/fetch-first machinery stays in force as the integration discipline; only their terminal "...then force-push" step is struck.

Classification: universal (§11.4.17) — a platform-neutral integration discipline reusable across every repository. Composes with §2.1 (multi-upstream push — step 6 fans out) / §9 / §9.2 (absolute data safety — this is the no-loss push discipline that makes force unnecessary) / §11.4.4 / §11.4.6 (remote state unknowable without the step-1 fetch) / §11.4.26 (constitution-update conflict resolution IS this procedure) / §11.4.37 (fetch-before-edit → fetch-before-push) / §11.4.40 / §11.4.41 (merge-first stays, force-push step removed) / §11.4.71 (same tightening) / §11.4.88 (background-push still fast-forward-only) / CONST-043 (no force-push authorisable). Propagation gate `CM-COVENANT-114-113-PROPAGATION` (literal `11.4.113`) + recommended gate `CM-NO-FORCE-PUSH-ABSOLUTE` (scan tracked scripts/hooks for `push --force` / `--force-with-lease` / `push +<ref>` + reject; a §11.4.109-class PreToolUse guard blocks the class at the tool-call boundary) + paired §1.1 mutation (inject a `git push --force` into a tracked script → gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.113. Non-compliance is a release blocker. No escape hatch — no `--force`, `--force-with-lease`, `--allow-force-push`, `--force-push-authorised`, `--skip-merge-onto-main` flag.

§ . . — Last-known-good-tag regression isolation mandate (. . -dev remediation, - -)

When a previously-working feature/behaviour is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or commit/build/deploy) at which it was KNOWN-GOOD and diff/bisect the broken state against it — BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle: it bounds the search to the commits between good and now (`git diff <good-tag>..HEAD --stat` of the feature's files = captured evidence), tells you it is a regression REPAIR not a from-scratch design problem, and gives each suspect file a behavioural oracle. When the operator volunteers a known-good tag, that lead is load-bearing and MUST be acted on first. Default to a

SURGICAL forward-fix (keep the post-good-tag features, revert ONLY the broken sub-part) over a wholesale revert that loses the batch's other working features — unless the operator prefers wholesale revert. Honest boundary (§11.4.6): "it worked before" is a HYPOTHESIS until the known-good tag is identified AND the feature is confirmed working there; "probably regressed in the last batch" without the diff is a guess; a feature that NEVER worked is not a regression. Composes §11.4.4 / §11.4.6 / §11.4.7 / §11.4.40 / §11.4.43 / §11.4.102 / §11.4.108. Propagation gate `CM-COVENANT-114-114-PROPAGATION` (literal `11.4.114`) + recommended gate `CM-REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.114. Non-compliance is a release blocker. No escape hatch — no `--skip-known-good-diff` (11.4.115), `--root-cause-from-scratch`, `--assume-regression-source`, `--wholesale-revert-without-isolation` (2026-06-03) flag.

§..— RED-baseline-on-the-broken-artifact + polarity-switch mandate (..-dev remediation, --)

Strict refinement of §11.4.43's RED step. Every RED test MUST be authored to REPRODUCE the defect on the CURRENT, pre-fix artifact (the actual broken build/deployment), capturing positive evidence per §11.4.5 / §11.4.69 / §11.4.107 that the defect is genuinely present — never a synthetic failure the fix is then written to agree with. The SAME test source MUST carry a single polarity switch (env flag / parameter, canonical `RED_MODE`, default `1` = reproduce-and-assert-defect-present) that flipped to `0` post-fix converts the test into the GREEN regression-guard asserting the defect is ABSENT. One source, two roles: the bug-catcher IS the regression-guard; no separate happy-path test is authored as the primary guard (that demonstrates only that the test agrees with the fix — the §11.4.43 PASS-bluff). RED-on-broken-artifact then GREEN-on-fixed-artifact (on a clean target per §11.4.108) MUST both be captured. Honest boundary (§11.4.6): if the RED run does NOT fail on the broken artifact, that is a finding (close per §11.4.7 with negative evidence, or fix the test) — a RED test that passes on the known-broken artifact is a blind test. Composes §11.4.1 / §11.4.2 / §11.4.5 / §11.4.69 / §11.4.107 / §11.4.4 / §11.4.7 / §11.4.43 / §11.4.50 / §11.4.108 / §11.4.114. Propagation gate

`CM-COVENANT-114-115-PROPAGATION` (literal `11.4.115`) + recommended gate `CM-RED-POLARITY-SWITCH-PRESENT` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.115. Non-compliance is a release blocker. No escape hatch — no `--green-only-test`, `--skip-red-reproduction`, `--separate-happy-path-suffices`, `--synthetic-red-OK` flag.

2026-06-03

§11.4.116 — Real-time conductor ↔ autonomous-test-framework sync channel mandate (dev remediation, non-blocking)

Any autonomous, long-running test/QA/validation framework an external orchestrator (conductor agent / operator) depends on for real-time decisions MUST expose a real-time sync channel: (1) a structured append-only event stream (JSONL or equivalent, one event per line, never rewritten) emitting at minimum session-start / phase-transition / per-test-or-challenge-start / captured-evidence-path / external-call (LLM / vision / sink-probe) / error / per-item-verdict events; (2) an atomically-rewritten status snapshot (single small file written write-temp-then-rename so a reader never sees a torn write) carrying current session/phase/item + counters + last verdict. Verdicts use the closed vocabulary PASS / FAIL / SKIP / OPERATOR-BLOCKED (§11.4.45). The conductor tails it live so it stays in real-time sync, can §11.4.4-interrupt on a fresh defect, and never idles blindly (§11.4.94 / §11.4.97). Anti-bluff: a verdict event MUST carry the evidence path that backs it (§11.4.69) — a PASS event with no evidence path is a channel-layer PASS-bluff; a snapshot reporting PASS while the stream shows no evidence event for that item is a contradiction → treat as FAIL; an item with no start-event cannot have a verdict-event. When the framework is an owned project-agnostic submodule (§11.4.28), the channel stays project-neutral — the consumer registers its data (endpoints, package ids, sink hosts) at runtime via the public API, never hardcoded. Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.27 / §11.4.28 / §11.4.45 / §11.4.52 / §11.4.89 / §11.4.94 / §11.4.97. Propagation gate `CM-COVENANT-114-116-PROPAGATION` (literal `11.4.116`) + recommended gate `CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.116. Non-compliance is a release blocker. No escape hatch — no `--no-sync-channel`, `--end-of-session-report-suffices`, `--verdict-without-evidence-path`, `--torn-status-write-OK` flag.

2026-06-03

§ . . — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (. . -dev remediation, - -)

Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank/partial/known-unreliable for the app under test (TV-Compose, leanback, canvas/`SurfaceView`/GL, games, custom-rendered UIs), the test MUST fall back to a PIXEL ORACLE: (1) DRIVE input by computer-vision template-match (locate a control by its rendered appearance → tap its screen coordinates), not by a hierarchy node id; (2) ASSERT content by ROI OCR (read the rendered text the user reads — caption strip, title, error overlay) with a per-word confidence floor + region-of-interest per §11.4.107(12), not a hierarchy text attribute that may be absent/stale. The tool MUST both drive input AND read pixels — a hierarchy-only tool is NOT a content oracle. This makes §11.4.52's "near-empty hierarchy → INFEASIBLE" constructive: pixel-drive, not only SKIP. Anti-bluff (§11.4.107(10)): the CV/OCR analyzer is self-validated — golden-good fixture PASSes, golden-bad fixture FAILs, wired into meta-test; thresholds calibrated on the project's own frames, not hardcoded (§11.4.6). Honest boundary: when BOTH hierarchy is blank AND pixel oracle is infeasible (secure surface black-captures per §11.4.112, geo-unreachable), SKIP-with-reason per §11.4.3 + tracked operator-attended migration item per §11.4.52 — never fake PASS. Composes §11.4.5 / §11.4.6 / §11.4.48 / §11.4.49 / §11.4.52 / §11.4.107 / §11.4.112. Propagation gate `CM-COVENANT-114-117-PROPAGATION` (literal `11.4.117`) + recommended gate `CM-CV-OCR-PIXEL-ORACLE-FALLBACK` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.117. Non-compliance is a release blocker. No escape hatch — no `--hierarchy-is-content-oracle`, `--skip-pixel-fallback`, `--unvalidated-ocr-OK`, `--hardcoded-ocr-threshold-OK` flag.

§ . . — Discovery-pressure to confirm known-issue-set completeness mandate (. . -dev remediation, - -)

A remediation/release cycle MUST NOT treat "every reported defect is fixed" as "the build is good" — the reported set is biased by what the operator happened to test ("we only see what we test"). After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems, journeys, and edge cases BEYOND the reported defects — to CONFIRM the reported set is the COMPLETE critical set and surface unreported defects before the end user does. The pass MUST produce PROVABLE coverage: an enumerated list of the subsystems/user-journeys/stress scenarios actually exercised, each with its outcome (no-new-issue, or a new tracker entry per §11.4.15 / §11.4.16). "We found no other issues" is a §11.4 bluff unless accompanied by "here is the enumerated set we exercised" — absence of evidence of looking is not evidence of absence. New findings trigger §11.4.4 interrupt + the §11.4.114/§11.4.115 isolation→RED→fix loop. Honest boundary (§11.4.6): discovery reduces the unknown-unknown surface but does not prove zero remaining defects — the earned claim is "reported set + enumerated discovery set addressed," with un-exercised subsystems stated as honest coverage gaps (§11.4.3), never silently implied clean. Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.25 / §11.4.40 / §11.4.42 / §11.4.52 / §11.4.85 / §11.4.114 / §11.4.115 / §11.4.119. Propagation gate `CM-COVENANT-114-118-PROPAGATION` (literal `11.4.118`) + recommended gate `CM-DISCOVERY-COVERAGE-ENUMERATED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.118. Non-compliance is a release blocker. No escape hatch — no `--reported-set-suffices`, `--skip-discovery-pass`, `--no-issues-without-coverage-list`, `--assume-complete` flag.

§ . . — Single-resource-owner partitioning for parallel hardware testing mandate (. . -dev remediation, - -)

Strict refinement of §11.4.58 / §11.4.103 for hardware contention. When multiple parallel work/test/discovery streams exercise SHARED hardware or any exclusive-access resource (a media-playback path, a single HDMI/audio sink, an exclusive device handle, a serial/JTAG line, a GPU under exclusive capture), exactly ONE stream MUST own each such resource at a time. The exclusive owner drives it (playback, input injection, capture); every other concurrent stream targeting the same resource MUST be READ-ONLY (passive probes — `dumpsys` / `/proc` / `/sys` reads / sink-side network probes / log tails). Parallelism is partitioned by resource: distinct devices/sinks/handles run fully concurrent (stream-per-device), but the same device's exclusive resource is single-owner. Ownership MUST be enforced by an advisory lock/token (§11.4.58 L3 + the hardware analogue of §11.4.84 quiescence), event-driven (claim when the resource frees, release the moment work completes so the next queued stream claims it per §11.4.103 auto-backfill). Why: concurrent drivers of one exclusive resource produce CROSS-CONTAMINATED evidence (sink reports the wrong stream's audio, foreground belongs to whichever `am start` landed last, input events interleave) — a PASS under contention is a §11.4 evidence-integrity bluff. Honest boundary (§11.4.6): a "read-only" probe stream MUST issue NO state-changing command against a device it does not own; when a resource genuinely cannot be partitioned, streams serialize on it (single-owner over time), not run concurrently and hope. Composes §11.4.5 / §11.4.69 / §11.4.13 / §11.4.50 / §11.4.58 / §11.4.82 / §11.4.84 / §11.4.103 / §11.4.118. Propagation gate `CM-COVENANT-114-119-PROPAGATION` (literal `11.4.119`) + recommended gate `CM-SINGLE-RESOURCE-OWNER-PARTITION` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.119. Non-compliance is a release blocker. No escape hatch — no `--allow-concurrent-resource-drivers`, `--skip-ownership-lock`, `--read-only-may-mutate`, `--contended-evidence-OK` flag.

§ . . — Fix-breaks-its-own-gate reconciliation mandate (. . -dev remediation, - -)

When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal the fix landed — it MUST NOT be suppressed by either forbidden response: (1) FAKE-PASSING the gate (editing it to `always pass`, weakening its assertion to a tautology, or deleting it — a §11.4 gate-layer bluff + a §11.4.84 mutation-residue risk); (2) REVERTING the correct fix to satisfy the stale gate (re-introducing the defect). The required response is RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, AND update its paired §1.1 mutation so the mutation breaks the NEW invariant. Discriminator vs bluffing: after reconciliation the gate + mutation still form a valid §1.1 pair (mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL (the assertion became a tautology). The reconciliation MUST be a visible, evidence-cited change, never a silent assertion-weakening. Honest boundary (§11.4.6): a post-fix gate FAIL is NOT automatically "stale gate, reconcile" — investigate per §11.4.102 first; the FAIL may be the gate correctly catching a REGRESSION the fix introduced, in which case the FIX is wrong, not the gate. Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is the intended, evidence-confirmed mechanism. Composes §11.4.1 / §11.4.4 / §11.4.6 / §11.4.84 / §11.4.102 / §11.4.108 / §1.1. Propagation gate `CM-COVENANT-114-120-PROPAGATION` (literal `11.4.120`) + recommended gate `CM-GATE-RECONCILED-NOT-FAKE-PASSED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.120. Non-compliance is a release blocker. No escape hatch — no `--fake-pass-stale-gate`, `--revert-fix-for-gate`, `--weaken-assertion-to-pass`, `--delete-failing-gate` flag.

§ . . — No-commit-while-build-writes-tracked-artifacts mandate (. . -dev remediation, - -)

A commit (especially `git add -A` / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts INTO tracked (version-controlled) directories — doing so races the writer and stages a PARTIAL or stale artifact (a §11.4.108 SOURCE→ARTIFACT integrity failure landed in version control). The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so the tree is quiescent at the artifact layer AND the committed artifacts are the FRESH, whole outputs (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start) — a build still in flight writing tracked dirs is a HOLD on the commit, not a race to win. Build-output analogue of §11.4.84 (no commit while a mutation gate is in flight): both close the "commit captures transient non-final tree state" class at two write-sources. Where build outputs land OUTSIDE version control (gitignored `out/` / `dist/`), the race does not apply. Honest boundary (§11.4.6): "the build probably finished" is not "the build finished" — verify with a completion signal; committing source changes that DON'T touch the build's tracked-artifact directories is fine mid-build. The PROJECT-SPECIFIC instance (which tracked directory + which build steps write it) is recorded in the consuming project's own governance per §11.4.35. Composes §11.4.6 / §11.4.30 / §11.4.58 / §11.4.84 / §11.4.88 / §11.4.96 / §11.4.103 / §11.4.108. Propagation gate `CM-COVENANT-114-121-PROPAGATION` (literal `11.4.121`) + recommended gate `CM-NO-COMMIT-DURING-ARTIFACT-WRITE` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.121. Non-compliance is a release blocker. No escape hatch — no `--commit-during-build`, `--stage-partial-artifact-OK`, `--assume-build-finished`, `--skip-build-completion-check` flag.

§ . . — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!"

Forensic case study (FACT). During the 1.1.8-dev burn-down, two shipped capabilities — F2 (an Apple-TV-class application) and F4 (a Huawei HMS / Mobile-Services component) — were removed from the existing System WITHOUT first asking the operator; the operator reversed both. A removal the operator has to discover and reverse after the fact is a defect of the same severity class as a §11.4 PASS-bluff: the System silently lost a user-facing capability the operator never agreed to drop.

No application, system component, service, package, feature, driver, module, library, prebuilt asset — any already-existing end-user capability of the existing codebase / shipped System — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed, or otherwise made unavailable to the end user) WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. The question MUST be posed through the platform's interactive clarification mechanism per §11.4.66 (`AskUserQuestion` on Claude Code) — NEVER a free-text "should I remove X?" buried in narrative, NEVER a silent removal justified post-hoc, NEVER an autonomous removal decision. A silent removal is a **release blocker** regardless of how well-intentioned the rationale (deduplication, "it was broken anyway", geo-restricted, incompatible, superseded) — the operator decides, the agent asks.

What counts as a removal (non-exhaustive): deleting an app/APK/binary from the build's package set (`PRODUCT_PACKAGES` / `device.mk` / equivalent), removing a service from the init/boot/service-registry set, dropping a kernel module / driver / config from the shipping configuration, un-bundling a prebuilt asset, deleting a submodule or its shipped output, removing a feature flag that gated a live capability,

or any edit whose NET EFFECT is "an end-user capability that shipped before no longer ships." Adding, replacing-with-operator-approved-equivalent, or fixing a capability is NOT a removal. When uncertain whether an edit constitutes a removal, treat it AS a removal and ask (per §11.4.6 no-guessing + §11.4.101 — removal of an existing user-facing capability is high-blast-radius and MUST be operator-confirmed, never autonomously decided). The tracked DROP path: ask → operator approves → mark the item `Obsolete (→ Fixed.md)` with `Obsolete-Details` reason `feature-removed` + an operator-approval citation (§11.4.90) → then remove; the removal never precedes the operator's yes.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project that ships a set of user-facing capabilities; the consuming project supplies its concrete capability-manifest paths per §11.4.35. Composes §11.4.66 / §11.4.101 / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate `CM-COVENANT-114-122-PROPAGATION` (literal `11.4.122`) + recommended gate `CM-NO-SILENT-COMPONENT-REMOVAL` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.122. Non-compliance is a release blocker. No escape hatch — no `--remove-without-asking`, `--silent-removal`, `--autonomous-removal-OK`, `--dedup-removal-exempt`, `--it-was-broken-anyway` flag.

§ . . — Rock-solid-proof-or-deep-research mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-03):

> "Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!"

Forensic case study (FACT). In the 1.1.8-dev remediation the validation method for two feature classes was, at first, genuinely unclear: relocating a `FLAG_SECURE` secure surface to a secondary display (pixel capture returns black) and asserting on-screen content in non-introspectable streaming-app UIs (blank accessibility hierarchy). Rather than declaring them "untestable" or accepting a metadata-only PASS, the cycle performed deep web research (`docs/research/testing_frameworks_20260603/`) that yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107 + §11.4.112 + §11.4.117) — making rock-solid evidence possible where it had appeared impossible. "Unclear how to validate" is a research trigger, NEVER a bluff licence.

Every single reported issue, every fix, and every claimed completion MUST be fully and 100% validated with rock-solid CAPTURED proof per §11.4.5 / §11.4.69 / §11.4.107 before it may be marked fixed / implemented / completed (§11.4.33 closure vocabulary). Nothing may be considered fixed or complete without hard captured evidence — metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1); no false results, no bluff of any kind, at any layer.

The research-or-don't-bluff rule (the operative addition): when the agent is UNSURE how to fully test / validate / verify something — when no obvious evidence-producing method exists OR the candidate method would yield only metadata/config/absence-of-error evidence — it MUST ALWAYS first perform deep web research per §11.4.8 + §11.4.99 (official docs, articles, guides, vendor references, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD a validation method that produces rock-solid evidence and

leaves no space for a false result. Declaring something "untestable" / "not automatable" / accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation — same severity class as a PASS-bluff. The research output (cited source URLs + the evidence-producing method, OR the literal "NO external solution found — original work" per §11.4.8) is the captured proof the path was exhausted. Only after that research genuinely fails may the item be classified `PENDING_FORENSICS:` / `Operator-blocked` (§11.4.21) / `structurally-impossible` won't-fix (§11.4.112) — with the cited research as the evidence the classification is earned, never a convenience.

Classification: universal (§11.4.17) — a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.8 / §11.4.52 / §11.4.69 / §11.4.99 / §11.4.107 / §11.4.118 / §11.4.21 / §11.4.112. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`) + recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.123. Non-compliance is a release blocker. No escape hatch — no `--metadata-pass-suffices`, `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag.

2026-06-04 11.4.124

§ . . — Dead/unwired-code investigate-before-remove mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-04):

> "Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal."

"Zero importers / never called / unwired ⇒ dead ⇒ delete" is a GUESS (§11.4.6), never a finding — a "no references" result proves only *current* non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow` , `git log -S` / `-G` pickaxe across all history, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead — call-site deleted deliberately / by mistake (regression) / never-completed / refactored-unreachable, (3) whether "no references" is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven wiring). The investigation output (cited commits + determination) is the captured evidence. **Removal is conditional:** permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible, composes §11.4.84 quiescence + §11.4.92 multi-pass) with a descriptive message citing the git-history evidence — plus §11.4.122 operator-confirmation when the element is an end-user capability; the §11.4.90 tracked path marks it `Obsolete (→ Fixed.md)` . **No proof ⇒ do NOT delete:** investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27 / §11.4.43 / §11.4.115 — the missing test is part of why it drifted into apparent-deadness). **Extra-caution default:** when uncertain whether removal-proof is sufficient, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; "probably dead" is never sufficient — the bar is captured proof. Classification: universal (§11.4.17) — the consuming project supplies its static-analysis / importer-graph tooling + hidden-

reference mechanisms per §11.4.35. Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 / §11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`) + recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.124. Non-compliance is a release blocker. No escape hatch — no `--zero-importers-means-dead`, `--delete-unwired-on-sight`, `--skip-git-history-investigation`, `--remove-without-proof`, `--bundle-removal-with-other-work` flag.

2026 06 04

§ . . — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-04):

> "After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks."

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/

§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSes on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`) + recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.125. Non-compliance is a release blocker. No escape hatch — no `--skip-code-review`, `--build-without-review`, `--no-review-gate`, `--review-optional`, `--trust-the-author` flag.

§ . . — Default autonomous-loop working mode from first prompt (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-04):

> "Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!"

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-instruction opt-in ("continue in endless loop fully autonomously" or a semantically-equivalent phrasing); §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 multi-upstream push + §11.4.40 full-suite-retest-before-tag + §11.4.113 absolute-no-force-push merge-onto-latest-main); OR (B) **Non-release**

main-stream scope — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working (claim the next priority item, dispatch the next parallel stream, progress every non-blocked item per §11.4.42 / §11.4.72 / §11.4.94 / §11.4.103). The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the current working scope — the upcoming release / current main-stream goal — with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand (the loop yields to host safety unconditionally). Idle-while-blocked parks one work unit, it does not stop the loop — the agent keeps progressing every non-blocked item in parallel per §11.4.101 + §11.4.94 + §11.4.97. Goal — ABSOLUTE EFFICIENCY (no operator-side restart overhead, no idle gaps, no stop-and-wait round-trips); applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN — this composes the entire §11.4 anti-bluff covenant family (§11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); the agent MUST genuinely perform the continuous work and capture positive evidence for every closure, and a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 (the endless-loop covenant — §11.4.126 promotes it from opt-in to always-on default) / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate `CM-COVENANT-114-126-PROPAGATION` (literal `11.4.126` across the consumer fleet) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.126. Non-compliance is a release blocker. No escape hatch — no `--ask-before-continuing`, `--single-turn-only`, `--not-default-loop`, `--mimic-OK` flag.

§ . . – Session-handoff resumption-prompt mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-06): "make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue."

When the agent determines a fresh session is needed (context-window limits, performance degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment and project phase** — self-contained enough that pasting it into a fresh session resumes work with ZERO loss. Two variants on demand: a SHORT first-sentence ("Read `<handoff docs>`, then continue `<terminal goal>` …") AND a FULL detailed block. The prompt MUST: (1) point to the live handoff doc(s) — `.remember/remember.md` if present + `docs/CONTINUATION.md` per §12.10 — read FIRST + `git fetch --all`; (2) state current PHASE + immediate NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs / artifact MD5, device/target serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID, NEVER a generic template. Handoff doc(s) MUST be current BEFORE the prompt is given (§12.10). A missing / stale / generic prompt is a §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-127-PROPAGATION` (literal `11.4.127`) + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.127. Non-compliance is a release blocker. No escape hatch — no `--skip-handoff-prompt`, `--generic-prompt-OK`, `--no-resumption-sentence`, `--handoff-without-state` flag.

§ . . — Always-on device-recording mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-06): we MUST ALWAYS live-record all available data from all devices we use for testing (or known to be under manual testing), EXTRA carefully so it never harms the device / its performance / causes side effects; raw recordings are NOT processed without need (token-conscious) and are ALWAYS git-ignored + code-intelligence-excluded; only curated evidence is committed, and only at release prep.

For EVERY test/debug device the project uses + every device under known manual testing, across EVERY reachable transport (USB / wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all analysable data: activities, all logs, performance metrics (CPU/memory/I/O/thermal/load), every sink-side report per §11.4.13, and any other live-changeable parameter. (1) **Extra-careful, side-effect-free** — non-invasive read-only probes only, bounded sampling, bounded write-volume, an observer-effect budget; a recorder that perturbs the device-under-test is a §11.4.128 violation, NOT evidence. (2) **Background + parallel + subagent-driven** per §11.4.103 + §11.4.70 — never blocks the main stream. (3) **Token-conscious — record-now, analyse-later** — raw data NOT processed without need; the only standing analyse-trigger is release-tag prep (§11.4.40 / §11.4.42) OR explicit operator ask. (4) **Raw is git-ignored (with a §11.4.77 regen-mechanism declaration) AND code-intelligence-excluded (§11.4.78/§11.4.79)** — only CURATED evidence is committed, and only at release prep under `docs/qa/<run-id>/` (§11.4.83). (5) **Deterministic layout** `<recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>`. (6) **Anti-bluff** — a recorder claimed running but with no growing corpus is a §11.4 bluff; every curated finding traces to a real raw-corpus path; recorder health is itself captured evidence per §11.4.5/§11.4.69.

Composes §11.4.2 / §11.4.5 / §11.4.13 / §11.4.69 / §11.4.40 / §11.4.42 / §11.4.70 / §11.4.77 / §11.4.78 / §11.4.79 / §11.4.83 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128`) + recommended gate `CM-DEVICE-RECORDING-ALWAYS-ON` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.128. Non-compliance is a release blocker. No escape hatch — no `--skip-recording`, `--record-without-layout`, `--commit-raw-corpus`, `--index-raw-corpus`, `--analyse-corpus-always`, `--invasive-probe-OK` flag.

§ . . — Huge-blocker release protocol (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-06): when a huge blocker is discovered during release validation we MUST stop all testing, fix ALL discovered issues, process all recorded data from the last session, land rock-solid fixes, author NEW validation+verification tests of ALL supported test types, rebuild, reflash, and RESTART the full validation+verification of every fix/change from the last release tag to now — on both devices in parallel, recorded, with real physical captured proofs and no bluff.

On discovery of a HUGE BLOCKER (release-blocking-severity defect: core user-facing capability broken, regression invalidating the in-flight cycle, or blast radius reaching the batch's other fixes) during release validation, execute in order with NO spot-check shortcut: (1) **STOP all testing** on every device (the §11.4.4 test-interrupt STOP at release granularity — continuing past a huge blocker is the §11.4 PASS-bluff). (2) **Fix ALL discovered issues** — not just the blocker; root-cause each per §11.4.102 + isolate regressions against the last known-good tag per §11.4.114. (3) **Process all recorded data from the last session** — analyse the §11.4.128 raw-corpus slice (this IS the §11.4.128(3) release-prep analyse-trigger). (4) **Land rock-solid fixes** per §11.4.123 + §11.4.43/§11.4.115 + §11.4.9. (5) **Author NEW validation+verification tests of ALL supported test types** per §11.4.27 + §11.4.85, each anti-bluff + paired §1.1 mutation. (6) **Rebuild (full, not module-only) + reflash to a CLEAN target** per §11.4.108. (7) **RESTART the full validation+verification from the last release tag to now** per §11.4.40 — RESTART, never resume — on both/all owned devices IN PARALLEL per §11.4.103/§11.4.119, every run RECORDED per §11.4.128, real physical captured proofs per §11.4.5/§11.4.69/§11.4.107, no bluff. This anchor BINDS the existing release anchors for the huge-blocker case (adds STOP→fix-all→process-recordings→new-tests-all-types→rebuild→reflash→full-restart + the restart-not-resume rule), citing them rather than duplicating.

Composes §11.4.4 / §11.4.40 / §11.4.42 / §11.4.9 / §11.4.27 / §11.4.85 / §11.4.102 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.128 / §11.4.103 / §11.4.119. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-129-PROPAGATION` (literal `11.4.129`) + recommended gate `CM-HUGE-BLOCKER-FULL-RESTART` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.129. Non-compliance is a release blocker. No escape hatch — no `--resume-after-blocker`, `--spot-validate-after-fix`, `--skip-recording-analysis`, `--skip-new-tests`, `--module-only-after-blocker`, `--single-device-restart` flag.

§ . . — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-06): when a blocker discovered during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, we MUST first re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / redistributed / updated, the agent MUST: (1) **re-test the SPECIFIC last-failing features FIRST** (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation; (2) **validate the just-incorporated fixes with real captured evidence** — the §11.4.115 RED test flips GREEN at `RED_MODE=0` on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden per §11.4 / §11.4.1; proof per §11.4.5/§11.4.69/§11.4.107/§11.4.123); (3) **only after the targeted fix is CONFIRMED working** proceed to the §11.4.40 full retest from the last tag to now. Rationale: a first fix attempt may not work / may be incomplete / may regress again under the new artifact — confirming the targeted fix FIRST catches a fix-did-not-take case immediately instead of hours later at the END of a full cycle (then restarting per §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is the §11.4.46 recent-work-validation gate specialised for the post-blocker-reflash case + the targeted-

confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary (§11.4.6): "the fix probably took" ≠ "the fix took" — the RED→GREEN flip + runtime-signature on the new artifact is the proof; a still-FAILing targeted re-test re-enters the §11.4.114/§11.4.115 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4 / §11.4.40 / §11.4.46 / §11.4.108 / §11.4.114 / §11.4.115 / §11.4.123 / §11.4.129 / §11.4.82. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-130-PROPAGATION` (literal `11.4.130`) + recommended gate `CM-FIX-FIRST-AFTER-REDEPLOY` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.130. Non-compliance is a release blocker. No escape hatch — no `--skip-targeted-retest`, `--full-cycle-first`, `--assume-fix-took`, `--validate-fix-at-end`, `--skip-red-green-flip-on-new-artifact` flag.

§ . . — Standing session-resumption file mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-07): "Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!"

Every project MUST maintain a SINGLE canonical, always-current **session-resumption file** at a fixed, project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision). This file is the OUT-OF-THE-BOX entry point for any fresh session: creating a new session requires ONLY pointing the new agent at this one file. §11.4.131 promotes §11.4.127 (PREPARE a resumption prompt on demand) into a STANDING, version-controlled ARTIFACT — ALWAYS present, ALWAYS in sync. (A) **Existence + fixed path** — exists at the declared path at all times, encoded as a literal path in the project-layer instantiation (§11.4.35), never silently moved. (B) **Always written + always synced** — (re)written whenever a fresh session is needed OR the live state materially changes (new HEAD, build/artifact id, phase, device/target state, in-flight job, blocking decision) — the §12.10 trigger set; a stale resumption file is a §11.4.131 violation of the same severity class as a §12.10 stale-CONTINUATION violation. (C) **Content (composes §11.4.127)** — both SHORT + FULL variants;

points to `.remember/remember.md` + `docs/CONTINUATION.md` read FIRST + `git fetch`; embeds exact live-state anchors (HEAD, build/artifact ids + checksums, device serials, in-flight PIDs + log paths, captured-evidence paths); states PHASE + immediate NEXT + terminal goal; restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MOMENT-VALID, never a generic template (§11.4.6). (D) **Export + freshness** — §11.4.65 scope (synchronized `.html` / `.pdf` siblings) + §11.4.44 revision header. (E) **Out-of-the-box resumption** — a fresh session, given ONLY this file's path, fully resumes with zero additional context. Composes §12.10 / §11.4.127 / §11.4.65 / §11.4.44 / §11.4.6 / §11.4.66 / §11.4.126. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + recommended gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.131. Non-compliance is a release blocker. No escape hatch — no `--skip-resumption-file`, `--ephemeral-prompt-only`, `--stale-resumption-OK`, `--generic-template-OK` flag.

§ . . — Risk-ordered validation priority mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-07): "We MUST ALWAYS first test and validate features, functionalities and fixes/changes that have been worked most recently, the ones which were most problematic, which have the most chance to crash or break again, the ones which have been re-opened the most times! Then, after we validate and verify all this with real (physical) proofs and hard evidence, with no false results and bluffs of any kind, we continue with all other existing tests in the test suites! This IS MANDATORY."

Tests / validations / verifications MUST run in **RISK-DESCENDING order** — the highest-risk set FIRST, and ONLY AFTER that set is fully GREEN with real (physical) captured evidence does the remainder of the suite run. Risk ranking is computed from a CLOSED set of factors, highest-risk first: (a) **most-recently-worked** features / fixes / changes; (b) **historically most-problematic** (longest defect history, most prior fixes/failures); (c) **highest crash/break/regress likelihood** (greatest blast radius / complexity / dependency surface); (d) **most-**

reopened per §11.4.55 reopens-count (a high reopen count is the strongest empirical fragility signal). Each item in the highest-risk set MUST pass with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107 — no metadata-only / config-only / absence-of-error / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff (§11.4.6). ONLY AFTER the entire highest-risk set is GREEN with captured proof does the rest of the suite run; running the suite in arbitrary order, or running lower-risk tests before the highest-risk set is GREEN, is a §11.4.132 violation. §11.4.132 REFINES/STRENGTHENS §11.4.130 (generalises "validate the just-fixed items first" to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering within the recent/high-risk set) + §11.4.42 (applies the implementation-layer priority discipline to VALIDATION ordering). Classification: universal (§11.4.17) — the consuming project supplies its recency / problematic-history / reopen-count sources (e.g. §11.4.93 workable-items DB `reopens_count` + `last_modified`) per §11.4.35. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Propagation gate `CM-COVENANT-114-132-PROPAGATION` (literal `11.4.132`) + recommended gate `CM-RISK-ORDERED-VALIDATION-PRIORITY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.132. Non-compliance is a release blocker. No escape hatch — no `--skip-risk-ordering`, `--any-order-OK`, `--suite-order-fixed` flag.

11 4 133
2026 06 08

§ . . — Target-System + hardware safety mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-08): "Make sure that all changes we do to the System are ALWAYS safe for the System itself and for the hardware the system runs on! This is MANDATORY."

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself — MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable — AND (b) the hardware it runs on — MUST NOT exceed safe electrical/thermal/voltage/clock limits or damage panels/storage/radios/regulators. Concrete obligations: (1) reversible-first — verify irreversible high-blast-radius changes (bootloader/U-Boot MD5, partition layout) against known-good

values + capture a pre-op backup (§9.2) BEFORE applying; (2) NO unverified hardware-control writes — never write an unverified value to a voltage/clock/regulator/thermal-throttle/current-limit sysfs node or register that could exceed datasheet limits, the safe range established as FACT (§11.4.6), never guessed; (3) thermal/perf changes (forcing a performance governor, pinning the top OPP, disabling thermal management) MUST respect the device's cooling design, validated by captured thermal evidence; (4) flashing MUST use the sanctioned tool + a freshly-built integrity-verified image — never an ad-hoc partition write or stale/unverified artifact; (5) unprovable-safety ⇒ blocked — a change whose target/hardware safety cannot be established from captured evidence is treated as UNSAFE and blocked (§11.4.6 + §11.4.101 reversible-first + §11.4.123 rock-solid-proof). DISTINCT from §12 host-session safety: §12 protects the DEVELOPER's HOST + session; §11.4.133 protects the TARGET device + its hardware — both apply, neither weakens the other. Classification: universal (§11.4.17) — the consuming project supplies its concrete hardware-control surfaces, datasheet-safe ranges, known-good bootloader/image hashes, and sanctioned flashing tool per §11.4.35. Composes §12 / §11.4.6 / §11.4.101 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-133-PROPAGATION` (literal `11.4.133`) + recommended gate `CM-TARGET-HARDWARE-SAFETY` + paired §1.1 meta-test mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.133. Non-compliance is a release blocker. No escape hatch — no `--unsafe-hardware-write`, `--skip-system-safety`, `--brick-risk-accepted` flag.

§ . . — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-08): "For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review's requests, we MUST RE-RUN code-review AGAIN until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind."

When the §11.4.125 code-review returns ANY finding — BLOCKING, nit, or warning — and the author fixes/improves the batch per that review, the code review MUST BE RE-RUN, and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. A single pass that "addressed the findings" is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings introduced by the very fixes that closed the prior ones — the §11.4.1 fix-A-creates-B failure mode). The loop terminates ONLY on a clean GO (no new findings, no warnings); a residual warning is itself a finding that re-arms the loop. Every round's verdict AND every fix's validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio / video / sysfs / dumpsys / sink-side / runtime-signature) proving the fixed/improved codebase REALLY works as expected — never metadata-only / configuration-only / absence-of-error / grep-without-runtime; no false results, no bluff at any round; a reported GO unbacked by captured physical evidence is itself a §11.4 PASS-bluff at the review-loop layer. §11.4.134 REFINES / STRENGTHENS §11.4.125 (iterate "until no blocking findings remain"): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises termination to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round. Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + recommended gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.134. Non-compliance is a release blocker. No escape hatch — no `--skip-rereview`, `--single-review-pass`, `--warnings-ok`, `--evidence-optional` flag.

- §11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User man… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.4 §11.4.17 §11.4.40 §11.4.43 §11.4.46 §11.4.50 §11.4.107 §11.4.108 §11.4.115 §11.4.118 §11.4.123 §11.4.124 §11.4.130 §11.4.132 §11.4.135

- §11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08).** Refines/strengthens… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.5 §11.4.13 §11.4.17 §11.4.48 §11.4.50 §11.4.69 §11.4.107 §11.4.117 §11.4.123 §11.4.136
- §11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandat… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.5 §11.4.6 §11.4.13 §11.4.17 §11.4.52 §11.4.69 §11.4.107 §11.4.108 §11.4.112 §11.4.115 §11.4.117 §11.4.123 §11.4.137
- §11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, 2026-06-08).** When t… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.17 §11.4.102 §11.4.108 §11.4.115 §11.4.118 §11.4.123 §11.4.135 §11.4.138
- §11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08).** Refines §1… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.17 §11.4.46 §11.4.108 §11.4.130 §11.4.135 §11.4.137 §11.4.139

§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09; arrow form 2026-07-02). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules` ; (3) execute the remainder of the prompt under the expanded instruction. **Five EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash), (5) `ACTION_NAME ---> <rest>` (bare arrow) ≡ `PREFIX::ACTION_NAME ---> <rest>` (namespaced arrow). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x` ≡ `BACKGROUND ---> x` ≡ `DEFAULT::BACKGROUND ---> x` . `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`** , and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar` , YAML `key: value` , URLs) in forms 1/2, the arrow-body separator `" ---> "` (one ASCII space on each side) in form 5, and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A` , re-scan, expand `B` , then the residual is the task); a leading `\` escapes the prefix for the `::` , arrow, and slash forms (`\BACKGROUND :: x` , `\BACKGROUND ---> x` , `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 5 forms) but is NOT registered is NEVER silently

expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **`BACKGROUND`** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). A second registered action **`REMINDER`** re-surfaces previously-scheduled, CRITICAL, status-UNCERTAIN work: FIRST verify the ACTUAL current status from captured evidence (never assume done or not-done — §11.4.6 no-guessing), THEN act on the delta — report the captured proof if genuinely complete, resume from the exact point if partial, action it NOW if not started, or surface the block per §11.4.66/ §11.4.101 — always producing a status verdict (composes §11.4.6 / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.108 / §11.4.130 / §11.4.147).

Severity / handling-priority markers (§11.4.140 extension, 2026-07-16). Three further registered actions tag the REMAINDER of the prompt with a HANDLING PRIORITY (as `BACKGROUND` tags it "run in background"): **`CRITICAL`** (highest-priority / potentially release-blocking — maximum urgency + rigor ahead of lower-priority work, track it, auto-activate §11.4.102 systematic-debugging when an issue is involved, do NOT defer it), **`IMPORTANT`** (high-priority, above routine work but below CRITICAL — track it, full anti-bluff rigor), and **`NOTE`** (capture the remainder as durable CONTEXT — request-history §11.4.208 / §11.4.210 + persistent memory when a non-obvious project fact — applied when relevant, NOT an urgent action unless it contains an explicit action; §11.4.6 record only what the note says, never invent). All six grammar forms work (`CRITICAL: x` ≡ `CRITICAL :: x` ≡ `CRITICAL ---> x` ≡ `/CRITICAL x` ≡ `/DEFAULT::CRITICAL x`). Registering `NOTE` promotes `NOTE:` from an ordinary-prose NO-OP to a registered action (ordinary-prose single-colon examples are now `TODO:` / `WARNING:` / `FIXME:`).

> The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.

- §11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09).** Every project wor… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.7 §11.4.17 §11.4.35 §11.4.50 §11.4.78 §11.4.79 §11.4.96 §11.4.102 §11.4.123 §11.4.125 §11.4.141
- §11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.4 §11.4.5 §11.4.6 §11.4.17 §11.4.20 §11.4.35 §11.4.40 §11.4.69 §11.4.70 §11.4.92 §11.4.108 §11.4.110 §11.4.125 §11.4.133 §11.4.134 §11.4.142
- §11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.6 §11.4.10 §11.4.17 §11.4.35 §11.4.48 §11.4.52 §11.4.107 §11.4.112 §11.4.117 §11.4.136 §11.4.137 §11.4.143
- §11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10).** Direct ope… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.14 §11.4.17 §11.4.21 §11.4.35 §11.4.69 §11.4.101 §11.4.128 §11.4.144
- §11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10).** For EVERY fix… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.5 §11.4.6 §11.4.17 §11.4.20 §11.4.40 §11.4.69 §11.4.70 §11.4.92 §11.4.108 §11.4.125 §11.4.133 §11.4.134 §11.4.145
- §11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User ma… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.1 §11.4.3 §11.4.4 §11.4.5 §11.4.6 §11.4.8 §11.4.17

§11.4.43 §11.4.50 §11.4.69 §11.4.85 §11.4.99 §11.4.102 §11.4.107 §11.4.108 §11.4.115 §11.4.118 §11.4.123 §11.4.130 §11.4.135 §11.4.138 §11.4.139 §11.4.146

- §11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10)… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.17 §11.4.40 §11.4.58 §11.4.69 §11.4.84 §11.4.87 §11.4.94 §11.4.97 §11.4.101 §11.4.102 §11.4.108 §11.4.116 §11.4.125 §11.4.126 §11.4.142 §11.4.144 §11.4.147
- §11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.10 §11.4.15 §11.4.16 §11.4.17 §11.4.21 §11.4.28 §11.4.33 §11.4.35 §11.4.54 §11.4.66 §11.4.69 §11.4.86 §11.4.91 §11.4.93 §11.4.95 §11.4.104 §11.4.106 §11.4.108 §11.4.115 §11.4.123 §11.4.148
- §11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10).** Every workable item MUST… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.6 §11.4.10 §11.4.17 §11.4.27 §11.4.28 §11.4.33 §11.4.35 §11.4.45 §11.4.55 §11.4.65 §11.4.69 §11.4.86 §11.4.93 §11.4.106 §11.4.132 §11.4.148 §11.4.149
- §11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.4 §11.4.6 §11.4.7 §11.4.8 §11.4.17 §11.4.20 §11.4.33 §11.4.34 §11.4.35 §11.4.40 §11.4.42 §11.4.55 §11.4.70 §11.4.89 §11.4.93 §11.4.94 §11.4.95 §11.4.97 §11.4.99 §11.4.101 §11.4.102 §11.4.103 §11.4.108 §11.4.112 §11.4.118 §11.4.123 §11.4.125 §11.4.126 §11.4.142 §11.4.145 §11.4.150
- §11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12).** Verbatim oper… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.28 §11.4.29 §11.4.30 §11.4.35 §11.4.40 §11.4.77 §11.4.113 §11.4.126 §11.4.151

- §11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage m… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.9 §11.4.10 §11.4.17 §11.4.35 §11.4.43 §11.4.47 §11.4.102 §11.4.108 §11.4.115 §11.4.118 §11.4.123 §11.4.135 §11.4.138 §11.4.152
- §11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording c… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.2 §11.4.3 §11.4.4 §11.4.5 §11.4.6 §11.4.17 §11.4.35 §11.4.40 §11.4.44 §11.4.45 §11.4.52 §11.4.56 §11.4.60 §11.4.65 §11.4.69 §11.4.74 §11.4.86 §11.4.102 §11.4.106 §11.4.107 §11.4.108 §11.4.118 §11.4.134 §11.4.146 §11.4.153 §11.4.159
- §11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.2 §11.4.3 §11.4.6 §11.4.10 §11.4.17 §11.4.83 §11.4.86 §11.4.107 §11.4.111 §11.4.122 §11.4.137 §11.4.154
- §11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15).** Verbatim: "A… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.29 §11.4.30 §11.4.35 §11.4.77 §11.4.83 §11.4.107 §11.4.122 §11.4.128 §11.4.151 §11.4.153 §11.4.154 §11.4.155
- §11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User m… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.29 §11.4.40 §11.4.75 §11.4.109 §11.4.156
- §11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.26 §11.4.35 §11.4.65 §11.4.141 §11.4.142 §11.4.155 §11.4.156 §11.4.157
- §11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification man… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.2 §11.4.4 §11.4.17 §11.4.27 §11.4.35 §11.4.69 §11.4.83 §11.4.85 §11.4.107 §11.4.117 §11.4.128 §11.4.153 §11.4.154 §11.4.155 §11.4.158

- §11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.2 §11.4.3 §11.4.10 §11.4.17 §11.4.29 §11.4.69 §11.4.102 §11.4.107 §11.4.137 §11.4.151 §11.4.154 §11.4.159
- §11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21).** Compact summar… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.17 §11.4.35 §11.4.107 §11.4.108 §11.4.112 §11.4.117 §11.4.159 §11.4.160
- §11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21).** Compact summary: every project… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.10 §11.4.17 §11.4.74 §11.4.76 §11.4.112 §11.4.161
- §11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21).** Compact summary: every projec… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.27 §11.4.35 §11.4.48 §11.4.49 §11.4.74 §11.4.107 §11.4.162
- §11.4.163 — Universal Media Validation & Verification Mandate (User mandate, 2026-06-21).** Compact summary… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.17 §11.4.107 §11.4.108 §11.4.117 §11.4.159 §11.4.163
- §11.4.164 — Universal Constitution Auto-Propagation & Hook System (User mandate, 2026-06-21).** Compact sum… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.28 §11.4.32 §11.4.67 §11.4.164
- §11.4.165 — Universal Independent Verification Agent Mandate (User mandate, 2026-06-21).** Compact summary:… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.10 §11.4.17 §11.4.40 §11.4.44 §11.4.65 §11.4.70 §11.4.86 §11.4.92 §11.4.108 §11.4.134 §11.4.142 §11.4.159 §11.4.163 §11.4.165
- §11.4.166 — REPEALED (operator decision, 2026-06-22).** The Universal Semgrep static-analysis mandate is re… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.166

- §11.4.167 — Big-work-item feature work-stream lifecycle (User mandate, 2026-06-23).** Compact summary: ever… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.29 §11.4.35 §11.4.40 §11.4.41 §11.4.58 §11.4.69 §11.4.77 §11.4.84 §11.4.108 §11.4.113 §11.4.116 §11.4.119 §11.4.142 §11.4.145 §11.4.147 §11.4.151 §11.4.167
- §11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate,… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.5 §11.4.6 §11.4.17 §11.4.65 §11.4.69 §11.4.70 §11.4.107 §11.4.112 §11.4.117 §11.4.134 §11.4.135 §11.4.138 §11.4.142 §11.4.168
- §11.4.169 — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, 202… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.4 §11.4.5 §11.4.10 §11.4.17 §11.4.25 §11.4.27 §11.4.50 §11.4.52 §11.4.69 §11.4.76 §11.4.85 §11.4.98 §11.4.107 §11.4.161 §11.4.169
- §11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate, 2026-06-25).** Com… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.3 §11.4.6 §11.4.17 §11.4.107 §11.4.153 §11.4.162 §11.4.168 §11.4.170
- §11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29).**… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.15 §11.4.16 §11.4.17 §11.4.19 §11.4.65 §11.4.91 §11.4.93 §11.4.148 §11.4.171
- §11.4.172 — Mandatory production-readiness planning with realistic timeline projection (User mandate, 2026-… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.40 §11.4.42 §11.4.65 §11.4.93 §11.4.108 §11.4.172
- §11.4.173 — Containerized + distributed build mandate (User mandate, 2026-06-29).** EVERY build of EVERY co… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.24 §11.4.28 §11.4.38 §11.4.40 §11.4.74 §11.4.76 §11.4.82 §11.4.108 §11.4.121 §11.4.161 §11.4.173

- §11.4.174 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29).** On any shared… [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.66 §11.4.101 §11.4.133 §11.4.147 §11.4.174
- §11.4.176 — Conflict-free multi-track work-division + exactly-once claim registry + capability-aware deadlock-proof device-lock (User mandate, 2026-07-02).** Two mandatory multi-track arbitration layers, conflict-free by construction: (A) exactly-once work-item/logical-group claim registry (extends §11.4.147/ §11.4.116; disjoint file-scope §11.4.58 L3 + worktree/CoW §11.4.167; no codebase loss §9.2/§11.4.113/§11.4.84/§11.4.41); (B) capability-aware deadlock-proof device-lock (§11.4.119 single-owner; all-or-nothing + non-blocking + TTL-reap breaks all 4 Coffman conditions; append-only JSONL + atomic snapshot §11.4.116); (C) universal/decoupled (§11.4.17/§11.4.28/ §11.4.35) + evidence-based resource-tuning (§11.4.24/§11.4.6, never guessed). Classification: universal (§11.4.17). Gates CM-WORK-DIVISION-EXCLUSIVE-CLAIM + CM-DEVICE-LOCK-DEADLOCK-FREE + CM-COVENANT-114-176-PROPAGATION + paired §1.1 mutation. [full → Constitution.md] · refs: §11.4.6 §11.4.17 §11.4.24 §11.4.28 §11.4.35 §11.4.41 §11.4.58 §11.4.84 §11.4.85 §11.4.103 §11.4.113 §11.4.116 §11.4.119 §11.4.147 §11.4.167 §12.6 §9.2 §11.4.176
- §12.11 — Maximal dynamic resource utilization for containerized builds (User mandate, 2026-07-03).** The containerized build path (its OWN cgroup + own MemoryMax → OOM-kills ITSELF, never user.slice) MUST use the maximal SAFE fraction of host capacity, computed DYNAMICALLY per host: auto-detect nproc/MemTotal/RLIMIT_NPROC (never hardcode, §11.4.6), reserve explicit host headroom (cores+RAM), scale -j against BOTH the memory model AND the process rlimit (privileged nproc-raise for full -j; EAGAIN-safe -j otherwise), never break/bottleneck/wedge. NO fixed low -j for the containerized path. The §12.6 60% ceiling + bare-metal -j cap REMAIN, SCOPED to user.slice-resident work — §12.11 does NOT weaken §12.6, it scopes it. Classification: universal (§11.4.17). Gate CM-MAXRES-DYNAMIC-BUILD + CM-COVENANT-12-11-PROPAGATION + paired §1.1 mutation. [full → Constitution.md] · refs: §12.1 §12.2 §12.3 §12.6 §12.10 §11.4.6 §11.4.24 §11.4.133 §9.2 §11.4.35 §12.11

§11.4.177 — Developer-tooling project-decoupling + invocation-directory operation (User mandate, 2026-07-04). Compact summary: no project-specific script / hook / alias / binary may be wired (copied / symlinked / PATH-exported /

aliased) into a global or shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the INVOCATION directory (cwd / explicit path / config arg), NEVER a hardcoded project path; a project-specific helper lives in + runs from its own repo, a genuinely-reusable helper is promoted to the shared toolkit ONLY after being stripped of every project-specific assumption (paths, device serials, package names, region endpoints) + taking its target from the invocation context; a project script reachable on the global/shared PATH is a decoupling violation of equal severity to importing project-specific code into a shared submodule (§11.4.28); forensic FACT 2026-07-04 — a project cwd-hook symlinked into the global Claude-Toolkit PATH re-coupled every alias to one hardcoded checkout; honest boundary §11.4.6 — decoupling guarantees project-agnostic, not correct (still needs the tool's own tests). Classification: universal (§11.4.17). Composes §11.4.28/.29/.35/.11/.6. Propagation gate `CM-COVENANT-114-177-PROPAGATION` (literal `11.4.177`) + recommended gate `CM-TOOLING-PROJECT-DECOUPLED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.177. Non-compliance is a release blocker. No escape hatch — no `--global-symlink-ok`, `--hardcode-project-path`, `--wire-into-shared-path` flag.

§11.4.178 — Track-qualified identity for parallel work streams (session-name collision ban) (User mandate, 2026-07-04). Compact summary: parallel work streams sharing hardware / a git backing store / a tmux-session namespace / a lock namespace / any single-owner resource MUST be addressed by a TRACK-QUALIFIED identity (`<project>__<track>__<role>`), NEVER a bare directory basename; multiple checkouts/clones/worktrees sharing basename `atmosphere` collapse into one session/lock/log key + cross-wire tmux targets, lock files, device claims, log dirs (observed collision 2026-07-04); every registry row / session name / lock path / log dir / device claim for a ≥2-concurrent-claimant resource MUST carry the track-qualified key, a bare-basename identity for such a resource is a violation; honest boundary §11.4.6 — a unique identity prevents cross-wiring, it does not itself arbitrate ownership (that is §11.4.119 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor)). Classification: universal (§11.4.17). Composes §11.4.111/.119/.29/.58 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor). Propagation gate `CM-COVENANT-114-178-PROPAGATION` (literal `11.4.178`) + recommended gate `CM-TRACK-QUALIFIED-IDENTITY` + paired §1.1 mutation. **Multi-project-per-track layout clarification (2026-07-10):** a `/mnt/track<N>` track parent is NOT

single-project — MULTIPLE projects coexist on the same track parents, each in its OWN project-named subdirectory `/mnt/track<N>/<project_name>/` (e.g. `/mnt/track1/atmosphere/` AND `/mnt/track1/helix_ota/` side by side); the per-track working copy of project P on track N is ALWAYS `/mnt/track<N>/<project_name>/` (canonical lowercase snake_case dir-name per §11.4.29), never the bare track root and never a shared basename; this is WHY the identity carries the `<project>` component (`<project>__<track>__<role>`) — so two coexisting projects' track-N sessions / locks / logs / device-claims / registry rows never collide; a project adopting the multi-track engine (§11.4.187) MUST create AND register its own `/mnt/track<N>/<project_name>/` per track and MUST NOT assume it owns the track root or that a track hosts only one project. **Canonical authority:** constitution submodule `Constitution.md` §11.4.178. Non-compliance is a release blocker. No escape hatch — no `--bare-basename-ok` , `--skip-track-qualifier` , `--shared-session-name` flag.

§11.4.179 — Corruption-isolated parallel git streams (own-.git independent clones, not shared-common-dir worktrees) (User mandate, 2026-07-04). Compact summary: when corruption-isolation is a requirement, parallel git work streams MUST each be an isolated repo with its OWN `.git` (own object store, own index, own lock namespace), NOT `git worktree` checkouts sharing one common `.git` ; a shared common-dir is a single point of failure — one stale `index.lock` / `.commit_all.lock` / `.push_all.lock` freezes EVERY stream at once (observed 9-h all-track freeze 2026-07-04) + one corrupted object store loses every stream; each stream's `git rev-parse --git-common-dir` MUST resolve INSIDE its own tree, a common-dir resolving to a shared parent is a violation; where CoW/reflink disk-feasibility is the constraint use the §11.4.167 reflink-clone-with-own- `.git` path (btrfs/XFS/ZFS/APFS reflink shares extents on disk while keeping a per-stream `.git` , so isolation + disk-feasibility both hold); honest boundary §11.4.6 — own- `.git` contains lock/corruption blast radius, it does NOT remove per-repo stale-lock reaping (§11.4.180) nor ff-only no-force integration (§11.4.113). Classification: universal (§11.4.17). Composes §11.4.167/.119/.58/§9.2 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor). Propagation gate `CM-COVENANT-114-179-PROPAGATION` (literal `11.4.179`) + recommended gate `CM-GIT-STREAMS-OWN-COMMON-DIR` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md`

§11.4.179. Non-compliance is a release blocker. No escape hatch — no `--shared-common-git`, `--worktree-when-isolation-required`, `--skip-git-isolation` flag.

§11.4.180 — Commit/push (single-writer) wrappers MUST auto-reap provably-stale locks before acquiring (User mandate, 2026-07-04). Compact summary: every commit / push / single-writer wrapper MUST, before acquiring its own lock, auto-reap any PROVABLY-stale git lock — one whose recorded holder PID is DEAD (`kill -0` fails), OR (no PID recorded) older than a defined threshold AND with no live wrapper/git process holding that repo; the reap covers the wrapper's own lock plus the git-internal existence-based locks (`index.lock`, `HEAD.lock`, `refs/**/*.lock`) git does NOT auto-release on holder death; it MUST NEVER remove a lock whose holder is ALIVE (removing a live-held lock corrupts a concurrent writer — §9.2); each reap decision (REAPED / KEPT + reason) is logged as captured evidence (§11.4.6 — liveness PROVEN via `kill -0`, never assumed); a wrapper that blocks indefinitely on a dead-holder lock (observed 9-h freeze 2026-07-04) is a violation; honest boundary §11.4.6 — reaping clears dead-holder deadlock, it does NOT substitute for the own-`.git` isolation of §11.4.179. Classification: universal (§11.4.17). Composes §11.4.84/.88/.6/§9.2/§11.4.116/§11.4.179. Propagation gate `CM-COVENANT-114-180-PROPAGATION` (literal `11.4.180`) + recommended gate `CM-WRAPPER-STALE-LOCK-AUTO-REAP` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.180. Non-compliance is a release blocker. No escape hatch — no `--skip-stale-lock-reap`, `--force-remove-live-lock`, `--block-on-dead-holder` flag.

§11.4.181 — Consistent controlled feature-branch naming: one feature/logic-group ⇒ exactly ONE canonical branch name across the main repo + all owned submodules (User mandate, 2026-07-05). Compact summary: one feature / logical group of workable items MUST map to EXACTLY ONE canonical branch name used IDENTICALLY on the main repo AND every touched owned submodule — NEVER two differently-named branches for one feature/group, NEVER per-repo naming drift; (1) single canonical name per group minted ONCE per the §11.4.167 D scheme (`feature/<slug>` branch + `<prefix>-<base>-feat-<slug>` tag, `<slug>` lowercase snake/kebab §11.4.29), used identically across the main repo + every branched submodule (an untouched submodule stays on its base branch); (2) single source of truth — the canonical name is recorded ONCE in the §11.4.176 exactly-once claim registry / the §11.4.93

workable-items `logic_group → branch` mapping and LOOKED UP, never re-invented (creating a branch whose name ≠ the group's registered canonical name, or minting a 2nd name for a group that already has one, is a §11.4.6 no-guessing violation); (3) mechanical enforcement (§11.4.75) — recommended gate `CM-BRANCH-NAME-CONSISTENCY` FAILs on an unregistered feature-branch name / two divergent names for one group / a submodule branch name diverging from the group's main-repo canonical name, paired §1.1 mutation (register one group under two divergent names → gate FAILs); (4) risk-free reconciliation (§9.2/§11.4.113/§11.4.84) — a mis-named/duplicate branch is reconciled by MERGING its commits onto the canonical branch (union, no commit lost, no `-s ours` /reset), ff-only push to all upstreams (NEVER force-push), retiring the mis-named branch only after the merge is confirmed on all mirrors. Classification: universal (§11.4.17). Composes §11.4.167/.176/.178/.179/.28/.29/.93/.113/.84/.6/.75/§9.2. Propagation gate `CM-COVENANT-114-181-PROPAGATION` (literal `11.4.181`) + recommended gate `CM-BRANCH-NAME-CONSISTENCY` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.181. Non-compliance is a release blocker. No escape hatch — no `--allow-divergent-branch-names` , `--skip-branch-registry` , `--rename-branch-freely` , `--per-repo-branch-name-OK` , `--two-names-one-feature-OK` flag.

§11.4.182 — Track+branch work-stream identity label `(T<N>/<branch>)` on every agent/subagent label + every operator-facing work-stream reference (User mandate, 2026-07-05). Compact summary: EVERY agent / subagent / work-stream label AND every operator-facing reference to a work stream MUST START with a `(T<N>/<branch> - <alias>)` prefix identifying the track + git branch + the claude-toolkit alias in use (e.g. `(T1/main - claude1) ATM-312 MPV drm_prime investigation` , `(T2/feature/mistiq-vader - deepseek) SPK-XXX ...`) so parallel-track work (`/mnt/track1..4` , each on its own branch, each driven by a distinct toolkit alias) is never ambiguous; (1) universal label form on every agent-dispatch description, status report, resume/handoff doc, progress line; (2) DETERMINISTIC derivation never guessed (§11.4.6) — `<N>` from the checkout path `/mnt/track<N>/…` (honest `?` off-track, never a guessed number), `<branch>` from `git rev-parse --abbrev-ref HEAD` , `<alias>` from `CLAUDE_CONFIG_DIR` basename `.claude-<alias>` → `<alias>` (honest `?` when unset/non-matching, never a guessed name) (reference labeler `scripts/multitrack/track_branch_label.sh`); (3) mechanical enforcement (§11.4.75 / §11.4.109-class) — a PreToolUse guard hook (`constitution/scripts/hooks/`

`guard-track-branch-label.sh`, inherited BY REFERENCE per §11.4.177, never copied) BLOCKS any Agent/Task/TaskCreate dispatch whose `description` (or `subagent` label) does not START with `^\(T[0-9]+/[^)]+\)`, non-agent tools untouched; pre-build gate `CM-TRACK-BRANCH-LABEL` (labeler+hook exist/exec/parseable, hook wired in settings, convention doc exists) + propagation gate `CM-COVENANT-114-182-PROPAGATION` (literal `11.4.182`); (4) honest boundary (§11.4.6) — a `?` in the track OR the alias field is a valid honest label, never a bluff, and the enforced numeric-track format surfaces (blocks) an off-track dispatch rather than mislabeling it. Classification: universal (§11.4.17). Composes §11.4.178 (track-qualified identity — the label-prefix instantiation applied to every agent/subagent label + operator-facing reference) / §11.4.29 / §11.4.75 / §11.4.109 / §11.4.177 / §11.4.6 / §11.4.58 / §11.4.103 / §11.4.119 / §11.4.116 / §11.4.147. Propagation gate `CM-COVENANT-114-182-PROPAGATION` (literal `11.4.182`) + recommended gate `CM-TRACK-BRANCH-LABEL` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.182. Non-compliance is a release blocker. No escape hatch — no `--skip-track-label`, `--unlabeled-agent-OK`, `--bare-work-stream-reference`, `--no-track-branch-prefix`, `--label-from-memory` flag.

§11.4.183 — Maximal multi-agent utilization per work-stream + full-constitution-application + zero-bluff mandate (User mandate, 2026-07-07). Compact summary: every track / work-stream MUST maximize multi-agent working approaches — subagent-driven development (§11.4.20/§11.4.70), independent review agents (§11.4.125/§11.4.142/§11.4.165/§11.4.134), parallel background streams with auto-backfill (§11.4.58/§11.4.103), and every other applicable agent form — so every track produces the MOST completed work at the HIGHEST quality in the LEAST time; every track MUST apply the ENTIRE constitution (every rule, mandatory constraint, guide, and derived technology — NOTHING ignored, skipped, avoided, or deemed less-relevant); there MUST NEVER be false / faulty / untested / unverified results or bluff of any kind anywhere (§11.4 anti-bluff covenant, captured evidence §11.4.5/§11.4.69/§11.4.107); honest boundary §11.4.6 — "maximize agents" is bounded by §12.6 60% memory + §11.4.58 parallel-agent cap + genuine parallelizability + §11.4.119 single-resource-owner, spawning contending/redundant agents is NOT maximization, the bar is maximum USEFUL parallel throughput not agent count. Classification: universal (§11.4.17). Composes §11.4.20/.58/.70/.92/.94/.97/.103/.119/.125/.126/.134/.142/.165/§12.6. Propagation gate `CM-COVENANT-114-183-PROPAGATION` (literal `11.4.183`) + recommended

gate `CM-MAX-AGENTS-PER-TRACK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.183. Non-compliance is a release blocker. No escape hatch — no `--single-agent-OK`, `--skip-review-agents`, `--partial-constitution-application`, `--serialise-parallelisable-work` flag.

§11.4.184 — Mandatory SonarQube static-analysis CLI + local tooling installed and PATH-discoverable (User mandate, 2026-07-06). Compact summary: every project/host that runs static analysis MUST have the SonarQube scanner CLI (`sonar-scanner`) installed AND durably discoverable on the system PATH via the shell rc (`.bashrc` / `.zshrc`) — exactly like the Firebase/GitHub/GitLab CLIs — PLUS the shared `constitution/scripts/sonarqube/` tooling (`sonarqube_install_check.sh` / `sonarqube_lib.sh` / `sonarqube_run_scan.sh` / `sonarqube_container.sh` + `compose/`) consumed BY REFERENCE (§11.4.28 decoupled + §11.4.177 project-agnostic + §11.4.80-style inherited, NEVER copied per project); (1) installed + PATH-durable — the scanner resolves in every fresh shell via a durable rc export, not an ephemeral session-only PATH; (2) shared tooling present + GREEN — `sonarqube_install_check.sh` exits 0 (scanner + rootless podman §11.4.161 + podman-compose + curl + ES-ready `vm.max_map_count`) before any scan-dependent work; (3) anti-bluff §11.4.6 — "installed/configured" PROVEN by `command -v sonar-scanner` resolving AND install-check exit 0, NEVER assumed, a claimed-installed-but-absent CLI is a §11.4 tooling-layer bluff; (4) rootless §11.4.161 — the local SonarQube server runs via rootless podman, never rootful docker/sudo; honest boundary §11.4.6 — names the SonarQube SCANNER CLI + shared local-server tooling, and per §11.4.166 (former universal Semgrep static-analysis mandate REPEALED) SonarQube is a DISTINCT operator-mandated tool not a Semgrep re-instatement, mandating TOOLING availability not a scan on every build. Classification: universal (§11.4.17). Composes §11.4.28/.36/.74/.75/.109/.161/.166/.177/.6. Propagation gate `CM-COVENANT-114-184-PROPAGATION` (literal `11.4.184`) + recommended gate `CM-SONARQUBE-CLI-INSTALLED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.184. Non-compliance is a release blocker. No escape hatch — no `--skip-sonarqube-install`, `--sonar-cli-optional`, `--ad-hoc-sonar-path`, `--rootful-sonar-OK` flag.

§12.12 — Process/thread-limit (RLIMIT_NPROC) awareness for parallel subagent/multi-process work (User mandate, 2026-07-07). Compact summary: heavy parallel subagent/multi-process work is bounded by the OS per-user process/thread limit `ulimit -u` (`RLIMIT_NPROC` , Linux counts ALL threads+processes of the real UID, not per-process); the ambient tooling fleet (MCP servers, tokio/V8/libuv worker runtimes, `mongod` / `chrome` / `prisma` -class MCP servers, local LLM servers) consumes a large FIXED fraction before project work starts — FORENSIC FACT (2026-07-07): a host at `ulimit -u 4096` sat at ~3900-4005 threads from the ambient fleet alone (<150 headroom); a 4th Go-heavy subagent (`GOMAXPROCS=nproc` spawns dozens of threads) hit `EAGAIN` / `failed to create new OS thread (errno=11)` and crashed tooling; stopping 3 non-essential MCP servers freed ~1345 threads (3827→2482). Mandate: BEFORE scaling parallel subagents/processes, check thread headroom (`ulimit -u` soft + `ulimit -Hu` hard + live `ps -L --no-headers -u "$USER" | wc -l`) and treat exhaustion as a §12 host-safety event that YIELDS UNCONDITIONALLY — the ORTHOGONAL second exhaustion axis alongside §12.6's memory ceiling (abundant RAM ≠ abundant thread budget); signals `EAGAIN` / `fork: retry` / `failed to create new OS thread` / `cannot allocate memory` on fork/clone (distinct from OOM, §11.4.6 no-guessing — never conflate); low headroom ⇒ SERIALIZE, avoid `GOMAXPROCS=nproc` -class spikes, never dispatch blind (a subagent hitting `EAGAIN` mid- `git push` is a §9.2 data-safety risk). Three raising techniques: (a) no-restart `prlimit --pid <PID> --nproc=<soft>:<hard>` (root, e.g. `su -c`) — new children of a running process inherit the raise; (b) persistent `/etc/security/limits.conf` (`<user> hard/soft nproc <N>`) or systemd `LimitNPROC=<N>` , next login; (c) self-raise `ulimit -u <N>` up to the existing hard ceiling within the current session — `su -c "ulimit -u N"` sets the limit ONLY inside that transient subshell, NOT the already-running session. Freeing threads (stopping non-essential MCP servers) REQUIRES operator authorization (§11.4.122, no autonomous tooling removal); CAUTION `pkill -f` / `pgrep -f` match the killer's OWN command line — always exclude the caller's `$$` / `$PPID` (and conductor PID) from any cmdline-pattern kill list to avoid self-kill. Classification: universal (§11.4.17). Composes §12/.6/.7/§11.4.58/.101/.103/.122/§9.2. Propagation gate `CM-COVENANT-12-12-PROPAGATION` (literal `12.12`) + recommended gate `CM-NPROC-HEADROOM-CHECK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §12.12. Non-

compliance is a release blocker. No escape hatch — no `--ignore-thread-limit`, `--dispatch-blind`, `--skip-headroom-check`, `--autonomous-tooling-kill`, `--assume-ulimit-scope` flag.

§11.4.185 — Manual QA-team testing as the mandatory FINAL confirmation of done (User mandate, 2026-07-07). Compact summary: no set of features / scope of work / release may be considered fully completed until it has received MANUAL TESTING confirmation by the project's QA team as the FINAL confirmation step — everywhere and always, never forgotten; every automated gate (§11.4 anti-bluff covenant, §11.4.5/§11.4.69 captured evidence, §11.4.52 autonomous validation, §11.4.40 full-suite retest, §11.4.108 four-layer verification, §11.4.132 risk-ordered validation) is NECESSARY but NOT SUFFICIENT — manual QA is the FINAL human sufficiency gate; ADDS to (never weakens) §11.4.52's mandatory autonomous-validation-path requirement — a feature needs BOTH the autonomous path AND the manual-QA final confirmation, never one substituting the other; pipeline: autonomous-GREEN → build → flash/deploy → autonomous on-device validation → hand off to QA team → ONLY after QA-team manual confirmation is the scope fully-completed / tag-eligible; the §11.4.126 autonomous-loop release-tag terminal condition now REQUIRES this confirmation; honest boundary §11.4.6 — the agent MUST hand off + WAIT, never self-certify/simulate/infer the manual step, while continuing every other non-blocked item in parallel per §11.4.87/.94/.97/.101 (the wait parks one scope, not the loop). Classification: universal (§11.4.17). Composes §11.4/.5/.6/.40/.52/.69/.108/.126/.129/.130/.132. Propagation gate `CM-COVENANT-114-185-PROPAGATION` (literal `11.4.185`) + recommended gate `CM-MANUAL-QA-FINAL-CONFIRMATION` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.185. Non-compliance is a release blocker. No escape hatch — no `--skip-manual-qa`, `--automation-suffices`, `--self-certify-manual-step`, `--assume-qa-confirmed`, `--autonomous-only-completion` flag.

§11.4.186 — Anti-divergence enforcement: cross-document consistency is a mandatory-before-export/commit gate, never an after-the-fact audit (research-derived, 2026-07-08). Compact summary: any project maintaining more than one representation of the same tracked data (a delivery plan + its MVP brief + a workable-items DB + their summaries + `.md` / `.html` / `.pdf` / `.docx` exports) MUST enforce cross-document CONSISTENCY as a deterministic gate that runs BEFORE any export render, any doc/DB sync `verify`, and any doc-set commit — NEVER as an after-the-fact human audit that runs only when an operator happens

to ask (the forensic FACT: the same NanoKVM/phone-as-screen feature appeared in three plan clusters — one shared ticket SPK-596 across SIX rows split over TWO releases, and a carve-out due to ship 2026-08-31 whose own prerequisite does not START until 2026-10-05 — and NO gate caught it; only an operator's manual read of `Plan_v9.0.xlsx` surfaced it, after the divergent bytes had already exported). (1) **One no-divergence verdict, three seams** — a single deterministic PASS/FAIL/SKIP verdict (built by composing a cross-document integrity validator with the workable-items-DB internal `validate`) gates all three write-seams (export / sync-verify / commit); a FAIL at any seam refuses that seam, naming the exact offending records (§11.4.6 actionable). (2) **Five decidable check families** — DEDUP (no two records describe one feature with divergent timeline/status/release, keyed on ticket OR normalised `(subject, scope)`, NOT bare subject substring), TIMELINE (start ≤ deadline; no deadline-before-dependency; no dependency cycle; GATED exempt-but-marked), CROSS-DOC (the same item across every representation carries identical timeline/status/type against a NAMED authoritative source), INTEGRITY (no orphan refs; Status↔Type §11.4.33; location↔status), STRUCTURAL (required columns; ID uniqueness+monotonicity §11.4.54). (3) **Drift-proof trigger** — a §11.4.86 sha256-of-sorted-members fingerprint of the authoritative inputs re-arms the gate on ANY input change, so a stale verify cannot pass on changed data. (4) **Self-validated analyzer (§11.4.107(10))** — the validator ships golden-good (MUST PASS) + one golden-bad per family (MUST FAIL, pinpointing the offender) + a negative-control (distinct same-subject tasks that MUST PASS — the false-positive guard); a validator that passes its golden-bad, or fails golden-good/the negative-control, is itself a §11.4 bluff and a release blocker. (5) **Reusable + decoupled (§11.4.28/.31)** — packaged as a depth-1 reusable engine under the constitution submodule per the §11.4.28(C) carve-out with a `helix-deps.yaml`; project-specific doc-sets + authoritative-source bindings live in a consumer-owned checkset (data, never engine code). (6) **Supersedes ad-hoc audits** — the per-cycle `*_INTEGRITY_FINDINGS.md` / `RECONCILIATION_*.md` after-the-fact audit docs are retired in favour of the gate; honest boundary §11.4.6 — the gate proves internal CONSISTENCY (no record diverges, no timeline is incoherent, every ref resolves), NOT plan CORRECTNESS (achievability/date-rightness stay operator/management decisions the gate SURFACES, never MAKES). Classification: universal (§11.4.17). Composes §11.4.6/.12/.33/.44/.50/.53/.54/.60/.65/.73/.75/.85/.86/.91/.93/.95/.106/.107/.108/.110/.148/.176. Propagation gate `CM-COVENANT-114-186-PROPAGATION` (literal `11.4.186`) + recommended gate `CM-DOC-INTEGRITY-VALIDATION` (5 invariants: validator

present+executable; consumer checkset present; the no-divergence gate wired into each of the three seams; fingerprint sidecar fresh; `selfcheck` golden-good/ golden-bad/negative-control wired into meta-test) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.186. Non-compliance is a release blocker. No escape hatch — no `--skip-divergence-gate`, `--audit-after-export-OK`, `--cross-doc-check-not-applicable`, `--unvalidated-analyzer-OK`, `--divergent-commit-OK` flag.

§11.4.187 — Automatic multi-track ruler orchestration (User mandate, 2026-07-04; landed 2026-07-09). Compact summary: every project incorporating this constitution MUST ship its multi-track parallel-development orchestration as a UNIVERSAL, AUTOMATIC, OUT-OF-THE-BOX, INHERITED capability — not a per-project reimplementation — via a single conductor session that programmatically SPAWNS, DRIVES, RESUMES, MONITORS, and CONTROLS per-track headless workers. (1) **Headless spawn primitive** — workers launch via native headless `claude -p --output-format stream-json --verbose`; the worker's `session_id` is captured from the FIRST `init` event (never invented); success/failure is read from the terminal event's `result.is_error` field, NEVER the process exit code (a CLI can exit non-zero on a successful `result`, or zero on a failed one — exit-code-as-success-signal is a §11.4.6 guess); continuation of an existing worker uses `--resume <session_id>` invoked from the SAME cwd + env the spawn used (the storage/resume-collision gotcha). (2) **Per-subscription auth + rate-limit fallback** — every spawn `unset`s any ambient `ANTHROPIC_API_KEY` (the precedence trap that silently routes a "subscription" worker onto pay-per-token API billing) and sets the selected alias's `CLAUDE_CODE_OAUTH_TOKEN` / `CLAUDE_CONFIG_DIR`; on a captured rate-limit signature (`isApiErrorMessage` + `apiErrorStatus:429`, quota-exhausted vs transient distinguished by evidence, never guessed) the ruler REBINDS the affected track onto the next healthy alias (cross-track reuse permitted), falls back to a provider alias where configured, and bounded-parks (§11.4.101) rather than hanging when every alias is cooled. (3) **Crash-resilient ruler self-supervisor** — the ruler's OWN state (per-track alias/session_id/worktree/last-verdict) is persisted durably (write-temp-then-rename, NEVER tmpfs) so an external watchdog detects a dead ruler PID and re-hydrates every track's binding from the durable state + the §11.4.116 event stream — no work is silently lost to a ruler crash (§11.4.147 at the ruler layer). (4) **Idempotent bootstrap + auto-install + conductor-stays-home** — one

idempotent bootstrap command wires the whole system out of the box (cwd-hook install, config validation, reconciliation), auto-invoked on every constitution pull via the §11.4.164 `post_update_hook.sh` skill-register seam; the `conductor:`-designated alias/session is NEVER bound into a track worktree (the resolver returns nothing for it) so the conductor always runs from the main checkout. (5) **Project-agnostic engine, consumer-owned data** — the entire mechanism (config loader, alias→worktree resolver, cwd-hook, bind/fallback/cooldown orchestrator, session launcher, rate-limit monitor, supervisor) lives at `constitution/scripts/multitrack/`, inherited BY REFERENCE (§11.4.28(B)/§11.4.177), carrying NO project literal; every consuming project supplies its own per-host `config/multitrack/<hostname>.yaml` (tracks/mounts/serials/aliases/conductor) as DATA, never as an engine edit. Anti-bluff (§11.4/§11.4.108): every piece carries four-layer coverage (pre-build gate + on-host test + paired §1.1 mutation + captured evidence per §11.4.69), and each fix declares a runtime signature verified on a clean host — never a claim the mechanism "should work." Honest boundary (§11.4.6): this anchor mandates the MECHANISM; genuine per-subscription quota-isolation proof requires live tokens exercising real rate-limit boundaries (an operator-scheduled live acceptance window) — a mock-driven GREEN proves the spawn/resume/fallback CONTRACT, never live-quota behaviour, and the two are never conflated or substituted for each other. Classification: universal (§11.4.17). Composes §11.4.20 (subagent-driven-by-default) / §11.4.28 (submodule decoupling + dependency layout) / §11.4.58 (parallel-development PWU pipeline) / §11.4.70 (subagent-driven execution default) / §11.4.101 (autonomous-decision-over-blocking — the bounded-park-on-all-cooled behaviour) / §11.4.103 (continuous parallel-stream routine) / §11.4.111 (resolve-by-stable-name — config by serial, never by index) / §11.4.116 (real-time conductor↔framework sync channel) / §11.4.119 (single-resource-owner partitioning) / §11.4.126 (default autonomous-loop mode) / §11.4.128 (always-on device-recording — the analogous host-safety discipline) / §11.4.131 (standing session-resumption file) / §11.4.147 (crashed-agent respawn-until-complete — the ruler-self-supervisor is its ruler-layer instantiation) / §11.4.164 (constitution auto-propagation hook seam) / §11.4.167 (feature work-stream lifecycle) / §11.4.176 (multi-track work-division + exactly-once claim + device-lock) / §12.6/§12.8 (host-safety memory + concurrency ceilings — the spawn/launch pool is budget-gated). Propagation gate `CM-COVENANT-114-187-PROPAGATION` (literal `11.4.187`) + recommended gates `CM-MULTITRACK-ENGINE-IN-CONSTITUTION` (the generic ruler scripts present under `constitution/scripts/multitrack/`, carrying no

project literal, `bash -n` + `sh -n` clean per §11.4.67) / `CM-MULTITRACK-AUTOINSTALL-WIRED` / `CM-MULTITRACK-BOOTSTRAP-IDEMPOTENT` / `CM-MULTITRACK-BIND-ON-START` / `CM-MULTITRACK-FALLBACK-MONITOR` / `CM-MULTITRACK-HEADLESS-SPAWN` / `CM-MULTITRACK-RULER-SUPERVISOR` + paired §1.1 mutation (strip the literal → propagation gate FAILs; re-introduce a project literal into the engine, or downgrade a rate-limit rebind to exit-code-based success detection → the corresponding mechanism gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.187. Non-compliance is a release blocker. No escape hatch — no `--skip-multitrack`, `--manual-multitrack-only`, `--no-auto-install`, `--per-project-reimpl-OK`, `--exit-code-is-success-signal` flag.

§11.4.188 — Regular main→feature merge cadence: every track / feature-branch MUST FREQUENTLY merge canonical `main` into itself DURING the work, never only at the end (User mandate, 2026-07-09). Compact summary: every long-lived feature branch AND every parallel-development track (§11.4.58/§11.4.103 parallel streams; §11.4.167 feature work-streams; §11.4.176/§11.4.178/§11.4.179 multi-track) MUST regularly `git merge origin/main` (the canonical trunk/master) INTO its own branch THROUGHOUT the work — NOT only at the end — so no branch ever drifts far from main and the eventual back-merge stays a small, low-conflict, low-risk operation. GENERALISES §11.4.167(D) (trunk-merged-INTO-every-BIG-feature-STREAM, at minimum after every trunk tag / daily, `git merge` never rebase, stale-if-not-synced) from the big-work-item feature-stream lifecycle to EVERY feature branch on EVERY track — a small/medium feature branch, or any `git worktree`/CoW track that never earned a full §11.4.167 stream, is STILL bound to this cadence (the §11.4.167(D) discipline promoted to the general standalone rule; §11.4.167(D) remains its big-item specialisation). (1) **WHAT** — fetch the latest canonical `main`/`master` and merge it INTO the working branch, keeping the branch continuously close to trunk; a branch that has NOT integrated trunk within the cadence is flagged STALE (the exact long-lived-branch conflict-storm + integration-risk anti-pattern this forbids). (2) **CADENCE** — merge trunk in FREQUENTLY: after EVERY trunk tag, at minimum DAILY while the branch is live, AND before starting any significant new chunk of work on the branch — MANY SMALL merges, never one big deferred end-of-branch merge (small frequent merges minimise the divergence + conflict surface). (3) **HOW (safe-by-construction)** — MERGE, NEVER rebase a shared/tagged/pushed branch (no commit-loss, no history rewrite — §9/§11.4.113); FETCH-FIRST every remote

before the merge (§11.4.37/§11.4.71 — remote state is unknowable without a fetch, §11.4.6); take a §9.2 hardlinked pre-op backup before any LARGE / conflict-heavy / risky merge; resolve every conflict CAREFULLY — ZERO conflict markers committed, ZERO file silently dropped, union of both sides preserved (§11.4.41 step 3 / §9 no-loss); integrate ONLY when the branch is QUIESCENT (§11.4.84 — no in-flight mutation gate, no unaccounted uncommitted work); prefer running the merge + its verification in the BACKGROUND parallel to the main stream so it never stalls other work (§11.4.88/§11.4.89/§11.4.103); NEVER force-push the merged result (§11.4.113 — the merged branch fast-forwards its own mirrors). (4) **CONSISTENCY + ANTI-BLUFF** — after EVERY merge the branch MUST be PROVEN consistent, never ASSUMED: a pre-build smoke gate runs post-merge (§11.4.42 step-3 smoke) and is GREEN, a `git grep` for conflict markers (`<<<<<<<` / `=======` / `>>>>>>>`) returns EMPTY, and no commit present pre-merge on either side is lost — captured evidence per §11.4.5/§11.4.69; a merge claimed "clean" without the post-merge smoke + marker-scan + no-lost-commit check is a §11.4/§11.4.1 bluff at the integration layer. (5) **HONEST BOUNDARY (§11.4.6)** — regular trunk-integration keeps the branch MERGEABLE + low-conflict; it does NOT by itself prove the branch's own work is correct (each change still crosses §11.4.108 runtime-signature + §11.4.40 retest), and it is NOT the approval-gated back-merge TO trunk (that stays §11.4.167(I)/§11.4.40/§11.4.41 no-merge-until-approved). Classification: universal (§11.4.17) — the consuming project supplies its canonical-trunk ref name, cadence unit, post-merge smoke-gate command, and per-track worktree/CoW mechanism per §11.4.35. Composes §9/§9.2/§11.4.6/§11.4.17/§11.4.37/§11.4.41/§11.4.42/§11.4.71/§11.4.84/§11.4.88/§11.4.89/§11.4.103/§11.4.113/§11.4.167/§11.4.176/§11.4.178/§11.4.179/§11.4.181. Propagation gate `CM-COVENANT-114-188-PROPAGATION` (literal `11.4.188`) + recommended gate `CM-REGULAR-MAIN-MERGE-INTO-FEATURE` (every live feature branch / track has integrated the canonical trunk within the declared cadence; its last integration is a real `git merge` not a rebase; the post-merge smoke + zero-conflict-marker + no-lost-commit checks passed) + paired §1.1 mutation (let a live branch go stale past the cadence, OR record a rebase of a shared branch in place of a merge, OR commit a conflict marker → the gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.188. Non-compliance is a release blocker. No escape hatch — no `--skip-`

`main-merge`, `--merge-only-at-end`, `--rebase-shared-branch`, `--defer-trunk-sync`, `--no-post-merge-smoke`, `--force-push-after-merge`, `--big-items-only-trunk-sync` flag.

§11.4.189 — Most-reopened cases get extra-depth live-testing scrutiny FIRST (User mandate, 2026-07-10). Verbatim operator mandate: "All live testing MUST pay extra attention to cases which have been reopened numerous times and those cases in depth retested, investigated, fully validated and verified with real physical results and no bluff of any kind anywhere!" All LIVE TESTING (on-device / on-target runtime validation, deploy-and-observe) MUST give EXTRA-DEPTH retest + in-depth investigation + FULL validation/verification with REAL PHYSICAL captured evidence (§11.4.5/§11.4.69/§11.4.107 — captured audio/video/sysfs/dumpsys/sink-side/runtime-signature) and NO bluff of any kind ANYWHERE to the cases that have been REOPENED THE MOST TIMES (highest §11.4.55 reopens-count) — the empirically-most-fragile set gets the DEEPEST live scrutiny, and it gets it FIRST. (1) **WHAT** — for the highest-reopens-count cases, live testing is NOT a single pass: it re-runs the case at EXTRA DEPTH (more iterations per §11.4.50 deterministic consistency, wider edge-case + regression coverage), RE-INVESTIGATES the underlying defect per §11.4.102 systematic-debugging + §11.4.146 reproduce-first, and CONFIRMS with rock-solid captured physical evidence per §11.4.123 — never metadata-only / config-only / absence-of-error / grep-without-runtime (§11.4/§11.4.1), never a false result. (2) **ORDER** — the most-reopened set is scrutinised FIRST, ahead of the rest of the live-test suite (the §11.4.132 risk-descending discipline, with most-reopened the load-bearing key). (3) **KEY** — priority is keyed off the §11.4.55 reopens-count (the consuming project supplies its reopens-count source per §11.4.35, e.g. the §11.4.93 workable-items DB `reopens_count`); a high reopens-count is the strongest empirical fragility signal (§11.4.132(d)). (4) **PROOF** — each most-reopened case's live PASS cites its captured-evidence artefact path (§11.4.69/§11.4.107) on a CLEAN/fresh deployment (§11.4.108 runtime-signature), with the §11.4.115 RED→GREEN polarity flip captured where a permanent regression guard exists (§11.4.135). STRENGTHENS/REFINES §11.4.132 (risk-ordered validation factor (d) most-reopened — §11.4.189 promotes most-reopened into its OWN EXTRA-DEPTH mandate at the LIVE-TEST layer, not merely one ranking factor among four) + §11.4.55 (reopens-count as the priority key) + §11.4.129/§11.4.130 (validate-the-just-fixed / highest-risk-first after redeploy). Classification: universal (§11.4.17) — references no project-specific hardware; the consuming project supplies its reopens-count source per §11.4.35.

Composes §11.4.5/§11.4.6/§11.4.55/§11.4.69/§11.4.107/§11.4.108/§11.4.115/§11.4.129/§11.4.130/§11.4.132/§11.4.135/§11.4.146. Propagation gate `CM-COVENANT-114-189-PROPAGATION` (literal `11.4.189`) + recommended gate `CM-MOST-REOPENED-EXTRA-DEPTH-LIVE-TEST` (the live-test suite orders + extra-depth-scrutinises the highest-reopens-count cases FIRST, each with captured physical evidence on a clean deployment) + paired §1.1 mutation (drop the most-reopened extra-depth pass, OR run it AFTER the rest of the suite, OR PASS a most-reopened case on metadata-only evidence → the gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.189. Non-compliance is a release blocker. No escape hatch — no `--skip-most-reopened-depth`, `--reopened-same-as-any`, `--live-test-any-order`, `--metadata-pass-for-reopened-OK`, `--defer-reopened-scrutiny` flag.

§11.4.190 — Website engineering-quality mandate: every project website MUST be fully responsive + completely SEO-optimized + uniquely OpenDesign-authored + bleeding-edge enterprise-quality, each PROVEN with captured evidence (User mandate, 2026-07-10). Verbatim operator mandate: "every website we create inside our projects MUST BE fully responsive - working on all browsers, all operating systems, all platforms and all devices and screen sizes! Every Website MUST BE completely SEO optimized! Layouts and templates done MUST BE unique and everything fully designed from OpenDesign! Make sure this applies to our project Website as well! Everything must be bleeding-edge enterprise quality, stunning visual look and feel, innovative, creative and unique! There MUST BE no false or faulty result or bluff of any kind anywhere!" Every website / web-UI surface a project ships — INCLUDING this project's OWN website — MUST satisfy AND PROVE with captured physical evidence (§11.4.5/§11.4.69/§11.4.107/§11.4.170 — never a bare claim, §11.4.6) ALL of: **(A) FULL RESPONSIVENESS** — correct, usable rendering across ALL major browser engines (Chromium/Firefox/WebKit), all operating systems, all device classes (desktop/laptop/tablet/phone/large-display) and the full range of screen sizes/orientations/pixel-densities; NO broken layout / overflow / clipping / overlap / off-screen or unusable control at ANY breakpoint. **(B) COMPLETE SEO OPTIMIZATION** — semantic HTML + correct document outline; per-page unique `<title>` + meta description; Open Graph + Twitter cards; canonical URLs; structured data (schema.org / JSON-LD); `robots` + XML sitemap; crawlable / indexable markup; WCAG AA accessibility + Core Web Vitals performance (both

feed ranking). **(C) UNIQUE OpenDesign-AUTHORED templates/layouts** — EVERY layout + template designed FROM the OpenDesign design-token system (§11.4.162), NOT generic / off-the-shelf / templated; visually unique, innovative, creative, brand-distinct. **(D) BLEEDING-EDGE ENTERPRISE VISUAL QUALITY** — stunning, polished, professional look-and-feel in BOTH light AND dark themes (§11.4.162 tokens). **(E) ANTI-BLUFF PROOF** — NONE of (A)–(D) may be CLAIMED without captured physical evidence: responsiveness PROVEN by device-independent host-rendered screenshots across the real breakpoint × engine matrix + an OCR/vision layout oracle (no overlap / label-over-label / clip / overflow / off-screen); SEO PROVEN by an automated audit (Lighthouse SEO + structured-data validation + meta / OG / sitemap / robots presence) meeting a defined score floor; visual quality PROVEN by the §11.4.170 device-independent host-rendered pixel proof per screen × state × {light,dark}; uniqueness PROVEN by OpenDesign-token provenance. A website reported responsive / SEO-optimized / unique / enterprise-quality WITHOUT these captured proofs is a §11.4 PASS-bluff at the web-UI layer. §11.4.190 is the strict WEB-quality expansion of §11.4.162 (OpenDesign — it applies §11.4.162's design-token mandate to the full responsive + SEO + enterprise-quality web surface) and binds §11.4.170's device-independent host-rendered pixel proof as the correctness oracle; sibling of §11.4.168 (exported-document independent visual validation) at the web-UI layer. Classification: universal (§11.4.17) — references no project-specific hardware; the consuming project supplies its concrete breakpoint × engine matrix, SEO score-floor, and OpenDesign token set per §11.4.35. Composes §11.4.5/§11.4.6/§11.4.69/§11.4.107/§11.4.162/§11.4.168/§11.4.170. Propagation gate `CM-COVENANT-114-190-PROPAGATION` (literal `11.4.190`) + recommended gates `CM-WEBSITE-RESPONSIVE-PROVEN` (host-rendered screenshots across the breakpoint × engine matrix + layout oracle — no overlap/clip/overflow/off-screen) / `CM-WEBSITE-SEO-OPTIMIZED` (automated SEO audit meets the score floor + structured-data / meta / OG / sitemap / robots present) / `CM-WEBSITE-OPENDESIGN-UNIQUE` (every layout/template's OpenDesign-token provenance, no generic/off-the-shelf template) + paired §1.1 mutation (report a website responsive/SEO/unique/enterprise-quality with NO captured proof, OR ship a generic non-OpenDesign template, OR skip the light/dark host-rendered pixel proof → the recommended gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.190. Non-compliance is a

release blocker. No escape hatch — no `--skip-responsive-proof`, `--seo-optional`, `--generic-template-OK`, `--visual-quality-later`, `--claim-without-render` flag.

§11.4.191 — Work-to-track/branch binding enforcement: a logic-group's work can NEVER be committed OR dispatched onto the wrong track/branch (research-derived, 2026-07-10). Compact summary: every logical group of workable items binds to EXACTLY ONE canonical `(track, branch)` recorded ONCE in the single source of truth (the §11.4.93 workable-items DB `logic_groups.destination` = authoritative branch + a `canonical_track` column + a `group_paths` file-scope manifest), and NO change belonging to a group may LAND on, nor be DISPATCHED to, a track/branch other than that group's canonical one — the file-scope generalisation of §11.4.181/§11.4.182 from branch-NAME consistency to work-PLACEMENT consistency (forensic FACT 2026-07-10: scrcpy/Remote-app work — group `mistiq-vader`, canonical `feature/mistiq-vader` /T2 — was committed onto `main` /T1; §11.4.181's `guard-branch-consistency.sh` never fired because committing onto an existing branch is not a feature-branch CREATE, and no gate checked which FILES land on which BRANCH). (1) **Single source of truth** — the `logic_group → (branch, track, path-globs)` binding is recorded ONCE in the DB and LOOKED UP, never re-invented (a commit/dispatch placing a group's file-scope on a non-canonical branch/track is a §11.4.6 no-guessing violation). (2) **Preventive hook (§11.4.109/§11.4.177)** — a PreToolUse guard `guard-work-track-binding.sh` (inherited BY REFERENCE, never copied) BLOCKS (exit 2) a git-commit-class Bash call whose to-be-committed file set contains a path owned by a group whose canonical branch/track ≠ the current checkout, AND BLOCKS an Agent/Task dispatch whose §11.4.182 label track/branch ≠ the canonical (track, branch) of the ticket's group; fail-closed on unreadable registry, `# guardrails:allow <reason>` audited escape. (3) **Detective gate (§11.4.110/§11.4.108)** — `CM-WORK-TRACK-BINDING-ENFORCED` asserts, on the current branch's own added commits (`merge-base(HEAD,main)..HEAD`), that every `group_paths`-owned file's group destination == current branch (and canonical_track == current track when pinned) — the actual enforcement boundary regardless of how the commit was made, the runtime signature being FAIL-on-wrong-placement / PASS-on-canonical-placement on a clean checkout. (4) **Risk-free reconciliation (§9.2/§11.4.113/§11.4.84)** — a mis-placed change is reconciled by MERGING its commits onto the group's canonical branch (union, no commit lost, no `-s ours`/reset), ff-only push to all

upstreams (NEVER force-push), then removing it from the wrong branch only after the merge is confirmed on all mirrors. Classification: universal (§11.4.17) — references no project hardware; the consuming project supplies its `(track, branch)` map, `group_paths` seed, and commit-wrapper wiring per §11.4.35. Composes §11.4.6/.28/.29/.35/.58/.75/.84/.93/.95/.109/.110/.113/.119/.167/.176/.177/.178/.181/.182/§9.2. Propagation gate `CM-COVENANT-114-191-PROPAGATION` (literal `11.4.191`) + recommended gate `CM-WORK-TRACK-BINDING-ENFORCED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.191. Non-compliance is a release blocker. No escape hatch — no `--allow-cross-track-commit`, `--skip-work-binding`, `--commit-any-branch-OK`, `--dispatch-any-track-OK`, `--no-file-scope-check` flag.

§11.4.192 — Continuous multi-track auto-backfill: a FREE track MUST be IMMEDIATELY re-assigned its next-highest-priority domain work-set, never left idle while its domain has actionable items (User mandate, 2026-07-11). Verbatim operator mandate: "once a Track is free, we shall assign next set of workable items to it ... full continuous multi-track development ... no stopping ... this is MANDATORY". Compact summary: the moment any track's worker COMPLETES its current work-set (its item(s) closed with anti-bluff evidence, or merged), that FREE track MUST be IMMEDIATELY assigned the NEXT-highest-priority actionable work-set from its OWN domain (its §11.4.191 logic_group / canonical branch) — a track is NEVER left idle while its domain still holds actionable workable items, and the conductor/ruler CONTINUOUSLY re-assigns with NO stopping (full continuous multi-track development); PROMOTES the §11.4.103(B) ≥3-parallel-background-streams-with-auto-backfill invariant into the multi-track (§11.4.176/§11.4.178/§11.4.179/§11.4.187) layer (§11.4.103(B) backfills a STREAM the moment it is done; §11.4.192 backfills a TRACK the moment its worker is done, keyed to that track's canonical (track, branch) domain per §11.4.191). (1) **WHAT** — on any track-worker completion the conductor/ruler (§11.4.187) MUST, per that track's domain, look up the NEXT actionable work-set, CLAIM it exactly-once (§11.4.176 exactly-once claim registry, no two tracks claiming one item), and DISPATCH it as the track's new §11.4.182-labelled work stream — without waiting for a scheduled wake or an operator prompt. (2) **PRIORITY** — the next work-set is the track domain's highest-priority actionable item (§11.4.42/§11.4.72 priority order, audio-first where in scope), never an arbitrary pick. (3) **SOURCE-SIDE ADVANCES EVEN WHEN VALIDATION IS GATED** —

when the next item's on-device VALIDATION is blocked on a pending build / device availability (§11.4.108 clean-target / §11.4.185 manual-QA / operator-gated flash), the track MUST still advance the item's SOURCE-SIDE work (investigation, RED-test §11.4.115, source patch, pre-build gate + paired §1.1 mutation, docs) and honestly §11.4.3/§11.4.52 SKIP-with-reason ONLY the genuinely device-gated validation portion — a build/device gate on the validation step is NOT a reason to idle the whole track. (4) **HONEST BOUNDARY (§11.4.6)** — a track idles ONLY when its domain GENUINELY has zero remaining actionable workable items OR every remaining item is externally blocked with no source-side advancement possible (§11.4.94(A)/§11.4.97/§11.4.101 idle-only-when-blocked closed set); "the merges are done" / "the current item is done" / "nothing visible right now" is NEVER a valid idle reason while the domain still holds progressable items (a §11.4.94+§11.4.97 violation); an unavoidable per-item block PARKS one item, it does NOT stop the track (§11.4.101 — the track backfills its next non-blocked domain item). Classification: universal (§11.4.17) — references no project hardware; the consuming project supplies its track→domain map + priority source per §11.4.35. Composes §11.4.42/.58/.72/.94/.97/.101/.103/.167/.176/.178/.179/.182/.187. Propagation gate `CM-COVENANT-114-192-PROPAGATION` (literal `11.4.192`) + recommended gate `CM-CONTINUOUS-MULTITRACK-BACKFILL` (on a track-worker completion the ruler re-assigns that track's next-highest-priority domain work-set; a track with a completed worker AND remaining actionable domain items but no new dispatch → the gate FAILs) + paired §1.1 mutation (let a free track sit idle while its domain holds an actionable item, OR treat "merges done" as a valid idle reason → the gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.192. Non-compliance is a release blocker. No escape hatch — no `--allow-idle-track`, `--skip-backfill`, `--stop-multitrack`, `--merges-done-is-idle`, `--defer-track-reassign` flag.

§11.4.193 — Anti-blind-typing mandate: every UI interaction MUST be "seen" and "understood" via OCR/vision/screenshot proof, NEVER blind-typed (User mandate, 2026-07-13). Blind typing — sending keystrokes/text/input events to a UI surface WITHOUT first capturing AND understanding what is on screen (OCR/vision/uiautomator/screenshot) AND confirming the result after each interaction — is STRICTLY FORBIDDEN. Blind typing IS bluff: the agent claims success without visual proof input landed correctly. Every UI-driving command MUST be preceded

by screen-state capture + OCR verification AND followed by confirmation capture. WebView surfaces (common in streaming-app login) require OCR fallback per §11.4.117. OCR of credential-bearing screens requires redaction per §11.4.10. No escape hatch. Canonical authority: constitution submodule Constitution.md §11.4.193.

§11.4.194 — Exhaustive all-scenario, all-angle code-review + verify-against-captured-runtime-evidence mandate (User mandate, 2026-07-14). Verbatim operator mandate: "CRITICAL: Code review MUST TAKE into the accounts all scenarios, look at the problem or change from all angles, deepest possible analisys! No gaps, shortcomings or false results or bluff is allowed! Extend our root constitution Submodule and all relevant files properly so the review process is more precise and we do not have situations like the last one!" STRENGTHENS §11.4.125/§11.4.134/§11.4.142 — every code-review MUST, before GO: (1) ENUMERATE the FULL input/scenario space of the change AND the defect (every dimension/path/interacting feature, not just the reported scenario); for a MULTI-FACTOR failure mode (a product of N terms, ANY degenerate term — zero/empty/null/default-init/out-of-range — triggers the failure) enumerate + INDEPENDENTLY VERIFY the fix covers EACH factor — verifying one and assuming the rest is the exact forbidden gap (forensic FACT "the last one": a boot-crash fix got a clean zero-finding GO that dismissed a clamp branch as an informational NIT "effectively unreachable ... always valid" WITHOUT PROVING it against the runtime input space; the target STILL crash-looped because the degenerate default-init input the review assumed impossible DID happen — a `size = A × B` product (e.g. `frame_size = channels × bytes_per_sample(format)`) with TWO independent zero-triggers, only ONE factor verified, the OTHER assumed away, the pre-fix crash evidence never cross-checked); (2) PROVE every assumption — "can't happen"/"unreachable"/"always valid"/"caller never sends X" MUST be PROVEN from code+spec+captured evidence, never on faith (§11.4.6); an unproven assumption that proves false is a §11.4 review-layer bluff of PASS-bluff severity; a NIT dismissing a path as unreachable MUST carry the proof of unreachability (a bare "effectively unreachable" NIT is itself a finding); (3) VERIFY the fix against ANY captured runtime evidence (crash log/tombstone/stack-trace/sink-probe/§11.4.108 runtime-signature/reproducer) — confirm the fix ACTUALLY prevents the CAPTURED failure against the exact captured conditions; a GO on a fix that then fails at runtime on a scenario present in the evidence (or one the review chose not to consider) is a bluff; (4) ALL ANGLES — correctness, full input domain (every

factor/boundary/degenerate value), every downstream/interacting must-not-break feature (§11.4.133 target/hardware safety + shared-state features e.g. audio/multichannel/passthrough), the TESTS' OWN per-dimension coverage (a test checking only one factor of a multi-factor bug is itself a gap the review catches), the §1.1 mutation per dimension; (5) HONEST BOUNDARY (§11.4.6) — "exhaustive" = the enumerated + evidence-grounded scenario space, NOT literal omniscience; an un-analysable dimension is stated as an explicit gap (UNCONFIRMED/UNKNOWN/PENDING_FORENSICS + tracked follow-up), NEVER silently assumed safe. Classification: universal (§11.4.17). Composes §11.4.125/.134/.142/.6/.108/.130/.107(10). Propagation gate `CM-COVENANT-114-194-PROPAGATION` (literal `11.4.194`) + recommended gate `CM-EXHAUSTIVE-REVIEW-ALL-SCENARIOS` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.194. Non-compliance is a release blocker. No escape hatch — no `--partial-scenario-review`, `--assume-unreachable-OK`, `--skip-captured-evidence-crosscheck`, `--single-factor-review-suffices`, `--dismiss-branch-without-proof` flag.

§11.4.195 — Branch-structure governance: taxonomy + merge-after-live-QA + flavor/product non-merge (User mandate, 2026-07-14). Verbatim operator rules: branch taxonomy `feat/<NAME_OF_FEATURE>` / `product/<NAME_OF_PRODUCT>` / `flavor/<NAME_OF_FLAVOR>`; tracks on FEATURE branches or `main` MUST — after full confirmation + validation + verification + full LIVE MANUAL TESTING (§11.4.185) — MERGE to `main` COMPLETELY (all submodules + main repo), ff-only, NEVER force-push (§11.4.113), no commit lost (§9.2); tracks with an entirely-new FLAVOR/PRODUCT do ALL work on a `flavor/` AND/OR `product/` branch (both may exist) which does NOT merge to `main` the same way; commit + push ALL branches to ALL upstreams (§2.1), ff-only. Every controlled branch belongs to one of THREE canonical taxonomy classes, each with a distinct integration lifecycle: **(A) Taxonomy** (extends §11.4.181/§11.4.167(D)) — `feat/<slug>` = FEATURE (merges to `main` after live-manual-QA, clause B); `product/<slug>` = PRODUCT line + `flavor/<slug>` = FLAVOR line (do NOT merge to `main` the standard way, clause C); `<slug>` lowercase snake/kebab (§11.4.29); the canonical branch NAME is minted ONCE + used IDENTICALLY across main repo + every touched owned submodule (§11.4.181), recorded once in the §11.4.176 claim registry / §11.4.93 `logic_group → branch` map + LOOKED UP never re-invented (§11.4.6); the prefix STRINGS are operator-canonical, a project already on a different feature-prefix (`feature/…`) is compliant so long as it is the ONE

canonical name per §11.4.181, reconciled by the §11.4.181 risk-free MERGE-onto-canonical path (ff-only, no commit lost, never a rename that drops commits). **(B) FEATURE/main → merge-after-live-QA** — before merging to `main` satisfy IN ORDER automated four-layer (§11.4.4(b)) + §11.4.40 full-suite retest THEN full LIVE MANUAL TESTING by the QA team (§11.4.185 FINAL human gate, automated GREEN necessary NOT sufficient); only after §11.4.185 may the merge proceed + it MUST be COMPLETE (main repo + every touched owned submodule) ff-only onto latest `main` (§11.4.113), NEVER force-push, NO commit lost / NO history rewrite (§9.2/§11.4.41 step 3), pushed to ALL upstreams (§2.1); binds the §11.4.185 gate as an explicit precondition of the §11.4.167(I)/§11.4.188 approval-gated back-merge. **(C) FLAVOR/PRODUCT lines → do-not-merge-to-main-the-same-way** — an entirely-new flavor/product line does ALL work on a `product/<NAME>` AND/OR `flavor/<NAME>` branch (both may coexist), a divergent line with its own release lifecycle that does NOT flow back to `main` via clause B; STILL obeys §11.4.113 (no force-push) / §9.2 (no commit lost) / §2.1 (push all branches all upstreams ff-only) / §11.4.188 (MAY merge `main` INTO itself to stay close — trunk-into-line permitted; only line-into-trunk is gated/withheld). **(D) Push discipline** (§2.1/§11.4.113) — commit + push ALL branches (feature/product/flavor) to ALL upstreams ff-only, NEVER force-push; new-branch creation is inherently ff (§9.2) + non-disruptive from an existing HEAD, permitted in a dirty tree under §11.4.84 provided no commit is in-flight. Honest boundary §11.4.6 — governs branch STRUCTURE + integration DIRECTION, does NOT prove any branch's work correct (still §11.4.108 runtime-signature + §11.4.40 retest + §11.4.185 manual-QA). Classification: universal (§11.4.17) — strict generalisation of §11.4.181 + §11.4.167(D)/(I) + §11.4.188 + §11.4.185. Composes §2.1/§9/§9.2/§11.4.6/§11.4.17/§11.4.29/§11.4.35/§11.4.40/§11.4.41/§11.4.84/§11.4.93/§11.4.113/§11.4.167(D)/(I)/§11.4.176/§11.4.178/§11.4.181/§11.4.185/§11.4.188/§11.4.191. Propagation gate `CM-COVENANT-114-195-PROPAGATION` (literal `11.4.195`) + recommended gate `CM-BRANCH-TAXONOMY-CLASS` (every controlled branch prefix ∈ `feat/` | `product/` | `flavor/` |reserved `main` / `master`; a `product/` | `flavor/` branch is NOT the source of a merge onto `main`; a `feat/` | `main` back-merge cites a §11.4.185 manual-QA confirmation) + paired §1.1 mutation (a `product/*` merged onto `main`, OR a `feat/*` back-merge with no §11.4.185 citation → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.195. Non-compliance is a release blocker. No escape

hatch — no `--any-branch-prefix-OK`, `--merge-product-to-main`, `--skip-live-qa-before-merge`, `--force-push-branch`, `--partial-submodule-merge` flag.

§11.4.196 — Native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery + resource-detection-by-real-identity-not-substring-match (User mandate, 2026-07-14). Verbatim operator mandate: use ALL operational claude-NATIVE aliases FIRST — as long as ≥1 native alias is operational (NOT session-rate-limited, NOT weekly-limit-reached, NOT expired-subscription) — before ANY provider alias; track per alias the session rate-limit / weekly-limit / subscription expiry+renewal from REAL signals (never faked, §11.4.6) + record when each becomes operational AGAIN; the moment a higher-priority native recovers, rebind tracks to it; and fix the host-budget guard footgun where a `pgrep -f` substring-match false-REFUSED every worker spawn. The multi-track alias-binding + rate-limit/subscription-tracking layer of the §11.4.187 ruler orchestration MUST implement ALL of: **(A) NATIVE-ALIAS-FIRST PRIORITY** — every OPERATIONAL native (`claudeN`) alias is selected before ANY provider, UNCONDITIONALLY, while ≥1 native is operational; the guarantee is STRUCTURAL — a native/provider CLASS partition scanned native-class-completely-before-provider-class (§11.4.111 resolve-by-CLASS-not-by-file-position), so a provider NEVER wins while a non-cooled native exists regardless of roster order; within a class the operator-mandated order (native equal-capability, then providers deepseek→xiaomi→opencode→kimi→…) is authoritative decision-DATA (§11.4.28) never re-invented (§11.4.6). **(B) PER-ALIAS REAL-SIGNAL LIMIT + SUBSCRIPTION TRACKING** — "operational?" per alias from REAL captured signals, NEVER faked/guessed (§11.4.6): the §11.4.187 captured 429 signature (`isApiErrorMessage` + `apiErrorStatus:429` / `api_error_status:429`) sub-classified into reason-CLASS closed set `{session | weekly | subscription}` from real markers (reset/weekly-limit/session-limit); each limited alias records its CLASS + an operational-again EPOCH with a DIFFERENT window per class — session (self-heals at reset, short), weekly (~until the weekly reset, long), subscription (INDEFINITE — operational-again UNKNOWN until a REAL renewal signal, parked with a far-future sentinel; §11.4.6 NEVER guesses a renewal date, an operator/config supplies it). A limited alias is NEVER auto-selected until its epoch passes (session/weekly) or a real renewal/recovery clears it (subscription). **(C) AUTO-USE-ON-RECOVERY** — the moment a higher-priority native recovers (window elapsed / renewal marked), rebind tracks UPWARD to it (a

`promote`-class op keyed off a comparable priority-rank: native class before provider, then config order), PRESERVING each track's worktree AND device leases (leases key on the STABLE track id — the alias changes, not the track; §11.4.119/§11.4.187); idempotent. **(D) RESOURCE-DETECTION BY REAL IDENTITY, NOT SUBSTRING-MATCH (RB-02 guard fix)** — a "heavy work in flight" guard MUST identify a real resource by its ACTUAL process command line read from the OS (`/proc/<pid>/cmdline`), NEVER a bare `pgrep -f REGEX` substring match that false-matches a process merely MENTIONING a token — a pattern-CARRIER (a process quoting the whole alternation pattern; a `claude` / worker whose prompt embeds the token; a launcher/monitor/test), a same-tool ENGINE process, or the guard's own process/parent (the §12.12 self/sibling pgrep footgun — forensic FACT 2026-07-13: the guard false-REFUSED EVERY §11.4.187 spawn on a host with ZERO real builds because a live `claude` worker's prompt quoted the pgrep pattern); the guard re-reads each matched PID's real cmdline + excludes those carrier/engine/worker/self classes; an UNREADABLE/ vanished cmdline is conservatively counted as the real resource (REFUSE — safe reversible default, §11.4.101/§12.8), so a genuine build is still caught. **(E) ANTI-BLUFF (§11.4/§11.4.108)** — four-layer coverage each (pre-build gate + on-host test + paired §1.1 mutation + captured evidence §11.4.69) with §11.4.115 RED-polarity + §11.4.50 determinism; honest boundary §11.4.6 — mandates the MECHANISM + REAL-signal derivation of "operational", genuine live per-subscription quota-isolation proof still needs live tokens (operator-scheduled acceptance window), never conflated with a mock/hermetic-contract GREEN. Classification: universal (§11.4.17). Composes §11.4.187/.182/.176/.111/.101/.103/.6/.108/.115/.119/§12.12/§12.6/§12.8. Propagation gate `CM-COVENANT-114-196-PROPAGATION` (literal `11.4.196`) + recommended gate `CM-ALIAS-PRIORITY-LIMIT-TRACKING` (native-first CLASS partition; per-alias reason-class + operational-again tracking; auto-rebind-on-recovery; build-guard resolves by real cmdline not bare substring) + paired §1.1 mutation (swap native/provider priority bases so a provider outranks a native → recommended gate FAILs; flip the guard's strict-cmdline-filter to the pre-fix bare `pgrep` → its RB-02 test FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.196. Non-compliance is a release blocker. No escape

hatch — no `--provider-before-operational-native`, `--fake-limit-state`, `--guess-renewal-date`, `--no-rebind-on-recovery`, `--substring-match-build-guard`, `--skip-real-signal-classification` flag.

§11.4.197 — Research / kicked-off-work completion mandate: every research effort MUST be driven to full, wired, verified completion or explicitly closed — never left un-wired in the backlog (User mandate, 2026-07-15). Verbatim operator mandate: "all research work we start MUST BE fully finished, completely confirmed, validated and verified! It MUST NOT happen for it to stay at the backlog uncompleted and un-wired! Loss of requirements, failure to incorporate them is FORBIDDEN!!! Everything MUST BE fully completed! No exceptions!" Every research effort / kicked-off improvement / design doc / spike / prototype / PoC / partially-landed feature a project STARTS MUST be driven to FULL completion OR explicitly evidence-backed CLOSED — there is no third "sitting un-wired in the backlog" state; "we researched/designed/started it" is NEVER terminal, and a `docs/research/*` doc / design doc / spike / half-wired feature that is neither COMPLETED-and-wired nor explicitly-closed is a §11.4.197 violation of §11.4 PASS-bluff severity at the requirements-integrity layer (the requirement was accepted, work began, then it silently evaporated); loss of requirements + failure to incorporate started work are FORBIDDEN. **COMPLETED** = (1) implemented + (2) WIRED (integrated AND used by default, NOT merely present behind a dead flag / uncalled function / unreferenced module / never-selected path — mere-presence-without-wiring is the §11.4.124 dead-code + §11.4.108 layer-2/3 SOURCE→ARTIFACT→RUNTIME gap) + (3) confirmed/validated/verified with captured physical evidence §11.4.5/§11.4.69/§11.4.107 on a clean deployment §11.4.108 runtime-signature (never metadata-only/config-only/absence-of-error/grep-without-runtime §11.4/§11.4.1) + (4) tracked to a terminal Status §11.4.15/§11.4.33. **Explicitly CLOSED** = deliberately terminated with a documented evidence-backed reason from the closed vocabulary — §11.4.90 Obsolete / §11.4.112 structurally-impossible / §11.4.122 operator-confirmed drop — NEVER silent abandonment ("no time"/"lower priority" = §11.4.21 Operator-blocked or a still-open tracked item, NOT a close). **Mechanical tracker binding (§11.4.93):** every research effort / design doc / kicked-off improvement MUST map to ≥1 tracked workable item (ATM-NNN §11.4.54) whose completion the loop enforces; a research doc with NO implementing item, or a tracked item stalled un-wired past the cadence, is a violation; the autonomous loop (§11.4.87/.94/.97/.103/.126/.192) MUST treat every un-wired research effort as an actionable queue item — driving it

to COMPLETED or explicitly CLOSED is exactly the "keep working until the queue is genuinely empty" contract. Honest boundary §11.4.6 — "fully completed" is bounded by wired+verified OR evidence-backed-closed, NOT "every speculative idea ships"; every STARTED effort reaches a documented terminal state, proven never assumed. Classification: universal (§11.4.17). Composes §11.4.8/.87/.94/.97/.103/.118/.123/.124/.150/.192/.90/.112/.122/.21/.93/.54/.108/.15/.33/.126. Propagation gate `CM-COVENANT-114-197-PROPAGATION` (literal `11.4.197`) + recommended gate `CM-RESEARCH-COMPLETION-TRACKED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.197. Non-compliance is a release blocker. No escape hatch — no `--research-may-stay-unwired`, `--skip-completion-tracking`, `--abandon-without-close`, `--present-but-unwired-is-done`, `--backlog-research-OK`, `--lose-requirement-OK` flag.

§11.4.198 — Default working mechanisms: multi-alias native-first orchestration AND heavy token-optimization are ALWAYS-ON defaults for single- and multi-track work (User mandate, 2026-07-15). Operator mandate: multiple claude-toolkit aliases (native-first) AND heavy token-optimization MUST be the DEFAULT mechanisms, always used with single- and multi-track development and with single or multiple working agents. Two proven mechanisms promoted to STANDING always-on defaults, engaged automatically for EVERY configuration (single- OR multiple-agent, single- OR multi-track), never a per-request opt-in: **(A) MULTI-ALIAS NATIVE-FIRST ORCHESTRATION** — the claude-toolkit multi-alias mechanism (every OPERATIONAL claude-native alias before ANY provider per §11.4.196 native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery, driven by the §11.4.187 automatic multi-track ruler) MUST be the default execution substrate for ALL work, NOT only the multi-track case; a SINGLE-track/SINGLE-agent session STILL binds its worker through the same native-first alias-priority mechanism (best operational native + rebind-on-recovery), a MULTI-track/multiple-agent config uses it across every track (§11.4.176 exactly-once claim + §11.4.182 track+branch+alias label); running on an arbitrary/hardcoded/provider-first alias when an operational native exists, or bypassing the mechanism because "it is just one agent", is a violation. **(B) HEAVY TOKEN-OPTIMIZATION** — the §11.4.141 token-efficiency discipline (compact-anchor layout with full-one-hop-away, targeted reads over whole-file dumps, subagent context isolation §11.4.20/§11.4.70, evidence-hash references not evidence dumps) MUST apply ALWAYS — every read/dispatch/doc-edit — for single- AND multi-track,

single OR multiple agents; token-heavy patterns (re-reading whole large files when a targeted slice suffices, dumping full evidence into context, monolithic non-subagent execution of parallelisable work) when a token-efficient path exists are violations. Honest boundary §11.4.6 — "default" = auto-engaged without an operator request; it does NOT weaken host-safety, the multi-alias/multi-agent fan-out stays under the §12.6 60% memory ceiling + §12.12 thread-headroom + §11.4.58 parallel-agent cap (contending/redundant agents are NOT "the default", §11.4.183 maximum-USEFUL-throughput); §11.4.198 BINDS the two mechanisms as always-on, it does NOT redefine them (mechanics live in §11.4.196/§11.4.187/§11.4.141). Classification: universal (§11.4.17). Composes §11.4.141/.187/.196/.182/.176/.183/.103/.58/.20/.70/.126/§12.6/§12.12. Propagation gate `CM-COVENANT-114-198-PROPAGATION` (literal `11.4.198`) + recommended gate `CM-DEFAULT-MECHANISMS-ALWAYS-ON` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.198. Non-compliance is a release blocker. No escape hatch — no `--single-agent-skips-alias-priority`, `--provider-first-default`, `--token-optimization-optional`, `--heavy-tokens-OK`, `--not-default-mechanism` flag.

§11.4.199 — Exact-reproduction-sequence mandate: when a working reproduction exists, the investigation MUST use ITS exact sequence — a deviating repro proves NOTHING (research-derived, 2026-07-15). Forensic FACT (2026-07-15): an investigation concluded a defensive "cover" mechanism "never engages" (and the defect it guards was therefore unfixed) — the conclusion was WRONG because the hand-rolled reproduction drove the target with a DIFFERENT input than the existing test (a HOME key where the test sent a media-pause key), so it never reached the PRECONDITION the mechanism keys off; re-driven with the TEST'S EXACT sequence the mechanism provably ENGAGED. Every investigation / debugging pass / validation that reproduces a defect or exercises a code path MUST: (1) **use the existing reproduction's EXACT sequence** when a working reproduction already exists (a test/harness/recorded sequence that demonstrably reaches the defect) — same inputs, same order, same timing, same preconditions, same topology (§11.4.3); a hand-rolled approximation is NOT a substitute; (2) **a deviating variant MUST FIRST prove it reaches the precondition** with captured evidence before ANY conclusion is drawn from it; (3) **a repro that misses the precondition proves NOTHING** — it is NOT evidence of absence (§11.4.6), and concluding "defect present"/"defect absent"/"the mechanism never engages" from it is a §11.4/§11.4.1 bluff at the INVESTIGATION layer (a

FAIL-bluff when it wrongly condemns working code, a PASS-bluff when it wrongly clears broken code); (4) **record the exact driving sequence as captured evidence** so it is replayable + auditable, every deviation explicit + justified + precondition-proven. STRENGTHENS §11.4.102 (Phase-2 reproduce) / §11.4.146 (reproduce-first) / §11.4.115 (RED on the broken artifact) — they mandate THAT you reproduce, §11.4.199 mandates HOW: with the sequence PROVEN to reach the defect. Classification: universal (§11.4.17). Composes §11.4.1/.3/.5/.6/.7/.102/.107/.115/.123/.135/.146. Propagation gate `CM-COVENANT-114-199-PROPAGATION` (literal `11.4.199`) + recommended gate `CM-EXACT-REPRO-SEQUENCE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.199. Non-compliance is a release blocker. No escape hatch — no `--approximate-repro-OK`, `--skip-precondition-proof`, `--hand-rolled-repro-suffices`, `--absence-proves-absence`, `--deviating-sequence-conclusion-OK` flag.

§11.4.200 — Non-targetable deploy/flash tooling MUST isolate exactly ONE eligible target and VERIFY-AFTER-WRITE on the INTENDED target (research-derived, 2026-07-15). Forensic FACT (2026-07-15): a flashing tool with no device-selection argument AUTO-SELECTED a device from the bus, silently wrote the freshly-built image to the WRONG board (a sibling still attached), printed SUCCESS, and the INTENDED board kept its OLD build — every downstream test then validated a target that had NEVER received the artifact (the §11.4.108 ARTIFACT→RUNTIME layer silently broken while every tool reported green). Whenever a deploy / flash / install / write tool CANNOT address a specific target, the procedure MUST: (1) **targetability is a PRECONDITION** — EITHER pass an explicit STABLE selector (serial/UUID/stable id per §11.4.111, never an enumeration ordinal) OR GUARANTEE exactly ONE eligible target is present at the moment of the write (siblings disconnected/powered-down/excluded), the enumerated eligible-target set captured as evidence (size == 1); (2) **the tool's own success message is NOT proof** — an "ok"/exit-0 proves only that bytes reached the device the TOOL selected, NEVER the device the OPERATOR intended (§11.4.6); (3) **VERIFY-AFTER-WRITE on the intended target** — read back the deployed artifact's IDENTITY (build id / version / checksum / fingerprint) FROM the intended target and assert it equals the artifact just written; a mismatch or the OLD identity is a FAIL, never a warning (the §11.4.108 ARTIFACT→RUNTIME assertion applied to the deployment step); (4) **no validation on an unverified target** — any test run against a target whose deployed-artifact identity was NOT verified is

INVALID and any PASS is a §11.4 PASS-bluff. Honest boundary §11.4.6 — isolation + read-back prove the INTENDED target holds the INTENDED artifact, NOT that the artifact is correct (§11.4.108/.40/.130). Classification: universal (§11.4.17). Composes §11.4.6/.46/.108/.111/.119/.130/.133/.139. Propagation gate `CM-COVENANT-114-200-PROPAGATION` (literal `11.4.200`) + recommended gate `CM-DEPLOY-TARGET-ISOLATED-AND-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.200. Non-compliance is a release blocker. No escape hatch — no `--auto-select-target-OK`, `--trust-tool-success-message`, `--skip-post-write-verify`, `--validate-unverified-target`, `--multiple-targets-on-bus-OK` flag.

§11.4.201 — Every guard/gate MUST assert the REAL condition: a false-positive refusal is a FAIL-bluff, a false-negative pass is a PASS-bluff (research-derived, 2026-07-15). Forensic FACT (2026-07-15, two independent instances in one cycle): (a) a host-budget guard used a bare `pgrep -f '<pattern>'` and REFUSED every worker spawn on a host with ZERO real builds because a live worker's own command line merely QUOTED the pattern (the §11.4.196(D)/§12.12 carrier footgun); (b) a host-safety pre-flight refused a build on a "swap full" reading that did not reflect a real memory-pressure condition, clearing only after the operator manually cycled swap — BOTH guards refused on a PROXY signal that was NOT the real condition. Every GUARD / GATE / PRE-FLIGHT (host-safety, resource-budget, readiness, lock, capability, availability) MUST assert the REAL condition it CLAIMS, resolved from the AUTHORITATIVE source (the real `/proc/<pid>/cmdline`, the real cgroup/meminfo pressure metric, the real lock-holder liveness via `kill -0`, the real device identity), NEVER from a proxy signal that something which is NOT the condition can satisfy: (1) **a FALSE-POSITIVE REFUSAL is a FAIL-bluff (§11.4.1)** — blocking work when the condition is ABSENT is exactly as forbidden as passing while a defect is present (it halts real work AND teaches operators/agents to bypass guards); (2) **a FALSE-NEGATIVE PASS is a PASS-bluff (§11.4)**; (3) **SELF-VALIDATED (§11.4.107(10))** — every guard ships a golden-TRUE fixture (condition present → guard FIRES) AND a golden-FALSE fixture INCLUDING the known CARRIER/decoy case (condition absent but proxy signal present → guard MUST NOT fire), both wired into the meta-test; a guard firing on its golden-FALSE fixture is ITSELF the defect; (4) **conservative-safe default on an unresolvable signal (§11.4.101/§12)** — refuse the destructive/high-blast-radius action and say so HONESTLY ("could not resolve the condition" is NOT a false positive; claiming a condition never verified IS one,

§11.4.6); (5) **every refusal reports its RESOLVED evidence** (real pid+cmdline, real metric+threshold, real lock holder) so a false positive is diagnosable in one step. GENERALISES §11.4.196(D) (resource-detection-by-real-identity — to EVERY guard) + §11.4.180 (liveness PROVEN before reaping — to every guard's condition). Classification: universal (§11.4.17). Composes §11.4.1/.6/.101/.107(10)/.109/.111/.180/.196(D)/§12/§12.6/§12.8/§12.12. Propagation gate `CM-COVENANT-114-201-PROPAGATION` (literal `11.4.201`) + recommended gate `CM-GUARD-ASSERTS-REAL-CONDITION` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.201. Non-compliance is a release blocker. No escape hatch — no `--proxy-signal-OK`, `--substring-match-guard`, `--false-positive-refusal-acceptable`, `--unvalidated-guard`, `--refuse-without-evidence` flag.

EXTENSIONS landed 2026-07-15 (existing anchors strengthened — full text in `Constitution.md`). §11.4.89(G) subagent long-op discipline + conductor-owns-inherently-long-ops: a SUBAGENT MUST NOT run a single synchronous op exceeding the runtime's no-progress stream watchdog (~5 min) — it backgrounds + polls (each poll IS the progress the watchdog observes); inherently-long ops (full builds / full suites / device flashes) are the CONDUCTOR's to run (no such watchdog); a stall-killed subagent is a CRASH per §11.4.147 (respawned, never absorbed as a completion); watchdog budget read from the runtime, never guessed (§11.4.6). **§11.4.106(F)** write-seam hook enforcement: the doc/DB sync MUST be enforced at the COMMIT seam (hook refuses a commit whose staged set touches a chain source while its exports/DB are stale) + the BUILD seam (pre-build gate) + the CONSTITUTION-PULL seam (§11.4.164 post-update hook) — a stale CONTINUATION (§12.10) / session-resumption file (§11.4.131) / memory artefact while live state moved is a violation; honest boundary §11.4.6 — seam hooks give eventual-consistency-at-every-write, NOT literal real-time (that needs a watch daemon, carried as a tracked §11.4.197 upgrade path, never claimed as shipped). **§11.4.147(e)** API-QUOTA / RATE-LIMIT EXHAUSTION is a FIRST-CLASS CRASH CLASS: an agent killed mid-work by a weekly/session/subscription quota boundary is `crashed` (never `complete`) — record the REAL limit signal + reason-class per §11.4.196(B), REBIND to the next OPERATIONAL alias per §11.4.196(A)/(C) and RESPAWN there (bounded-PARK per §11.4.101 only when EVERY alias is cooled), RESUME from the killed agent's captured transcript + partial state at its EXACT last point, and the §11.4.87/.94/.97/.126 loop done-condition MUST NOT read satisfied while any quota-killed entry is non-`complete`. **§11.4.196(F)** CONFIGURED ≠ IN

USE: forensic FACT — the alias orchestrator was installed with a valid config yet its binding registry was EMPTY while THREE live workers ran outside it, so `promote` rebound ZERO tracks; EVERY spawn path MUST bind THROUGH the orchestrator, and the mechanism's readiness is its RUNTIME SIGNATURE (§11.4.108) — **the binding-registry rows MUST EQUAL the live-worker set** — an empty/partial registry alongside live workers is a FALSE-READY state and claiming "ready/in use" on config alone is a §11.4/§11.4.6 orchestration-layer bluff.

§11.4.142 + §11.4.194 re-affirmed (2026-07-15 audit, no change required): the operator re-insisted 3× this session that EVERY change passes an exhaustive code review, ALWAYS, before build/commit. Audited: §11.4.142 already binds EVERY change to ANY governed repository — "no exception ... no 'just a doc edit' exemption, no 'just a one-liner' exemption, no §11.4.92-self-review substitution, no 'trivial change' carve-out ... BEFORE it is accepted, committed, or built" — with independence load-bearing (§11.4.70/§11.4.20); §11.4.194 supplies the exhaustive all-scenario / prove-every-assumption / cross-check-captured-runtime-evidence precision bar; §11.4.134 iterates the review to a ZERO-finding, ZERO-warning GO; §11.4.125 places the gate before the pre-build sweep + main build. NO carve-out exists in any of them — the family is complete as written and needed no widening this round (§11.4.6: audited, not assumed).

§11.4.202 — Reporting directives: a report MUST auto-create a fully-populated, fully-synced workable item — never a prose acknowledgement (User mandate, 2026-07-15). Verbatim operator mandate: introduce reporting directives `ISSUE` / `BUG` / `TASK` that create a proper workable item filled with all details, with the WHOLE workable-items system automatically up to date and fully in sync — the main DB, all documents, and all external tracking systems; universal, reusable on every project, fully decoupled, recognized immediately by every project that pulls the constitution. Every governed project MUST expose REPORTING DIRECTIVES — the §11.4.140 registered actions **`BUG`** (Type=Bug), **`TASK`** (Type=Task), and **`ISSUE`** (generic report, CLASSIFIED into the §11.4.16 closed set) — such that a plain-language report from the operator is TURNED INTO a real, tracked, fully-synced workable item, automatically, every time. A report that is discussed, acknowledged, answered in prose, or "noted" WITHOUT landing a tracked item is a §11.4.202 violation of §11.4 PASS-bluff severity at the requirements-intake layer: the requirement was accepted and then silently evaporated (the §11.4.197 loss-of-requirements failure, at its entry point). **(1) GRAMMAR (§11.4.140).** The three directives are ordinary registry rows, so EVERY

§11.4.140 form works out of the box (`BUG :: <text>` , `DEFAULT::BUG :: <text>` , `/DEFAULT::BUG <text>` , `BUG ---> <text>`), PLUS a SIXTH form this anchor adds — the **single-colon form** **`NAME: <text>`** (`ISSUE: subtitles are late`), the shape operators actually type. The single-colon form is **REGISTERED-ACTION-ONLY by construction**: a single-colon token that is NOT a registered action is a NO-OP (an ordinary prompt), NEVER an ASK and NEVER a sub-system shortcut — because ordinary English prose (`TODO:` , `WARNING:` , `FIXME:`) shares that shape (`NOTE:` is now a registered §11.4.140 severity marker), and routing it to the §11.4.66 clarify path would question every such sentence (§11.4.6 honest boundary: the other five forms keep their ASK-on-unknown behaviour — their shapes do not occur in prose). Host-command collisions are handled by the EXISTING §11.4.140 conflict mechanism (`slash_bare: auto` + `slash_conflicts: [..]`), which already satisfies the operator's explicit-prefix requirement: a bare `/NAME` that collides with a built-in (e.g. the documented Claude Code built-in `/bug`) is NOT honored, while the namespaced `/DEFAULT::NAME` form is ALWAYS unambiguous — and the `NAME:` / `NAME ::` / `NAME --->` forms are never affected by any host command. **(2) THE ITEM.** The created item MUST carry, on ALL surfaces: a valid Status (§11.4.15, `Queued` at intake) + a Type from the CLOSED set {Bug | Feature | Task} (§11.4.16 — `ISSUE` is a REPORTING CHANNEL, never a fourth type; inventing one is a violation) + a stable auto-incremented id (§11.4.54), and a COMPREHENSIVE structured description (§11.4.148 D2 / §11.4.171): what / affected-scope / reproduction / acceptance. An undetermined section is recorded as an explicit `UNKNOWN:` gap — NEVER invented (§11.4.6). For `ISSUE` , the type is classified from the report's content and stated as FACT; if the content does not determine it, the operator is ASKED (§11.4.66 / §11.4.105) BEFORE creation; ONLY when running autonomously where asking is impossible (§11.4.101) may the §11.4.16 lowest-stakes ambiguity default `Task` be used — and then the defaulted classification MUST be recorded verbatim in the item and surfaced for reclassification, never silently asserted. **(3) THE FULL SYNC (the load-bearing half).** Creating the item is NOT sufficient: the directive MUST drive the WHOLE workable-items system into sync, in one shot — (a) the item lands in the SQLite single-source-of-truth (§11.4.93 / §11.4.95); (b) EVERY derived document is regenerated FROM the DB — the open + closed trackers, their summaries, and their `.md` / `.html` / `.pdf` / `.docx` siblings (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106); (c) the item is pushed to EVERY configured external tracker (§11.4.148

D5). **(4) NEVER A FAKED PUSH (§11.4.10 / §11.4.6 / §11.4.3).** A tracker whose credentials or whose client are absent is SKIPPED with an honest machine-readable reason (`credentials_absent` — names of the unset variables ONLY, never a value; `tracker_client_absent` — PENDING-OPERATOR-INPUT). A tracker PASS may be recorded ONLY after a real push command really exited 0. Fabricating, assuming, or silently omitting a push is a §11.4 bluff at the integration layer; a missing tracker NEVER blocks the item from landing in the DB + docs (the report is never lost because a mirror is unreachable). **(5) DECOUPLED ENGINE, CONSUMER-OWNED DATA (§11.4.28 / §11.4.177).** The item-creation + full-sync engine lives in the constitution submodule (`constitution/scripts/reporting/report_item.sh`), is inherited BY REFERENCE (never copied), and carries ZERO project literals; every project-specific value — DB path, id prefix, sync command, tracker commands + their required env vars — comes from a CONSUMER-OWNED config file (DATA, never an engine edit). The engine FAILS CLOSED with an actionable message when that config is absent — it never guesses a project's paths (§11.4.6). **(6) ANTI-BLUFF (§11.4 / §11.4.107(10)).** The mechanism ships four-layer coverage: a pre-build gate (`CM-REPORTING-DIRECTIVES`) whose FUNCTIONAL invariant SOURCES and RUNS the parser (never a grep-only assertion, §11.4.108), a paired §1.1 mutation test that proves each invariant is genuinely load-bearing, and an end-to-end suite run against a REAL SQLite DB with REAL binaries (no mocks, §11.4.27) that asserts a real row lands with the right Type/Status/id, that the docs are regenerated FROM the DB, and that an absent tracker SKIPs honestly — self-validated with a golden-good + golden-bad + negative-control fixture set (an oracle that passes its golden-bad fixture is itself the bluff). Classification: universal (§11.4.17) — the consuming project supplies its DB path, id prefix, sync command, and tracker bindings per §11.4.35. Composes §11.4.140 (grammar) / §11.4.93 / §11.4.95 (DB SSoT) / §11.4.15 / §11.4.16 / §11.4.54 (status + type + id) / §11.4.148 / §11.4.171 (comprehensive description) / §11.4.106 / §11.4.12 / §11.4.53 / §11.4.65 (doc sync) / §11.4.10 (credentials) / §11.4.66 / §11.4.105 (clarify, never guess) / §11.4.101 (autonomous decision) / §11.4.197 (a started requirement never evaporates) / §11.4.28 / §11.4.177 (decoupling) / §11.4.164 (auto-registration on constitution pull) / §11.4.107(10) / §1.1. Propagation gate `CM-COVENANT-114-202-PROPAGATION` (literal `11.4.202`) + recommended gate `CM-REPORTING-DIRECTIVES` (registry declares ISSUE/BUG/TASK + the single-colon form is registered-only; the engine exists, is parse-clean, and is project-literal-free; the three directives really expand at runtime while prose does not; the honest-skip reasons exist and a tracker PASS is gated on a real exit

0) + paired §1.1 mutation (strip an action row, flip the single-colon form to non-registered-only, strip the parser branch, remove the prose NO-OP branch, make the engine fake a tracker push, or couple the engine to one project → the gate FAILs; strip the literal → the propagation gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.202.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--report-without-item`, `--prose-acknowledgement-OK`, `--skip-item-creation`, `--skip-tracker-sync`, `--fake-tracker-push`, `--invent-a-fourth-type`, `--hardcode-project-in-engine` flag.

§11.4.207 — Instant multi-stream resume engine: whole-fleet continuation state is a durable, content-addressed, atomically-committed snapshot resumed in O(changed) reads, not an O(history) re-read (User mandate, 2026-07-15). Compact summary: the continuation mechanism (§12.10 / §11.4.127 / §11.4.131 / §11.4.205) is EXTENDED — never replaced — by a mechanical engine so a fresh session resumes ALL work (every Track, main stream, and agent) at once, INSTANTLY and at token cost O(changed streams), by reading ONE snapshot manifest + one compact blob per stream, NEVER re-reading the full handoff history. (1) **Content-addressed Merkle store** — each stream's canonical state bytes are named by their sha256 (dedup: unchanged streams share a hash → a global fleet snapshot costs O(changed)); a snapshot is ONE manifest referencing every stream's content-hash (§11.4.205(4)). (2) **O(streams) resume, O(history) NEVER on the resume path** — `RestoreAll` reads the manifest + one blob per stream; the append-only event ledger is audit + lost-update recovery only, never read to resume (§11.4.127). (3) **Tamper-evident + deterministic** — every blob is integrity-checked on read (bytes MUST hash to their id) and a deterministic round-trip (re-serialize must byte-match), no wall-clock in the hashed content, verified by a SELF-VALIDATING oracle: golden-good PASS + golden-bad detected FAIL + negative-control (a legitimately-older-but-valid snapshot) PASS — the false-positive guard proving the verifier distinguishes a LAGGING copy from a TAMPERED one (§11.4.201/§11.4.107(10)/§11.4.206(3)). (4) **Durable + atomic + single-writer + crash-safe** — atomic ref write (temp→fsync→rename→dir-fsync, §11.4.205(6)), append-only ledger (§11.4.205(6)), single-writer-per-stream refusal (§11.4.206), advisory lock with provably-stale reap (`kill -0` liveness, never steals a live lock, §11.4.180/§9.2). (5) **Project-agnostic engine, consumer-owned data** — the engine (`github.com/vasic-digital/continuum` , GitLab mirror)

carries ZERO project literals, fails closed rather than guess a store path (§11.4.6/§11.4.177), and is inherited BY REFERENCE as a depth-1 submodule under the §11.4.28(C) carve-out (`constitution/submodules/continuum/` , `helix-deps.yaml` , zero own-org deps); every consumer supplies its store path / actor / stream naming as DATA. Anti-bluff (§11.4/§11.4.108): the whole engine ships four-layer coverage (unit + integration + e2e + stress + chaos §11.4.85, race-clean) with a measured resume Metric (`blobs_read == 1 + streams` is the O(streams) proof) and the self-validating oracle as captured evidence (§11.4.5/§11.4.69/§11.4.107); honest boundary (§11.4.6): it snapshots the state an agent REPORTS, not its execution image (not a CRIU/durable-execution runtime, §11.4.112), and `est_tokens` is a documented `bytes/4` heuristic, not a tokenizer count. Classification: universal (§11.4.17). Composes §12.10 / §11.4.127 / §11.4.131 / §11.4.205 / §11.4.116 / §11.4.147 / §11.4.180 / §11.4.201 / §11.4.206 / §11.4.107(10) / §11.4.28 / §11.4.177 / §11.4.85 / §9.2. Propagation gate `CM-COVENANT-114-207-PROPAGATION` (literal `11.4.207`) + recommended gate `CM-CONTINUUM-RESUME-ENGINE-PRESENT` (the engine present under `constitution/submodules/continuum/` , `go test -race ./...` green, `continuum selfcheck` = good=PASS/bad=FAIL/negctrl=PASS, zero project literals) + paired §1.1 mutation (strip the literal → propagation gate FAILs; downgrade the resume path to read the ledger, or the oracle to pass its golden-bad → the mechanism gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.207. Non-compliance is a release blocker. No escape hatch — no `--resume-reads-history` , `--skip-integrity-check` , `--unvalidated-oracle-OK` , `--hand-typed-resume-state` , `--per-project-resume-reimpl` flag.

§11.4.208 — Operator-request-history document: every project maintains a project-local, always-in-sync ledger of every operator request/prompt (User mandate, 2026-07-15). Verbatim operator mandate: introduce a request-history document capturing per operator request/prompt — full content, accepted-timestamp with explicit timezone, the track that processed it, the alias that took the workable item, and the model + effort used — always in sync, no false data, no bluff; the operator's load-bearing correction (§11.4.35/§11.4.17): the RULE is universal (→ constitution submodule), the request-history DOCUMENT itself is PROJECT-LOCAL (never placed into the constitution). Every governed project MUST maintain a single project-local, append-only, newest-first **operator-request-history document** at a fixed project-declared path (e.g. `docs/requests/`

`history.md` ; §11.4.35, never moved without a §11.4.66 operator decision), each entry carrying EXACTLY: (1) **Request content** (verbatim where archived, else a faithful complete summary, marked which); (2) **Accepted (when)** — date + time + TIMEZONE stated EXPLICITLY on every entry (consuming project declares its default zone §11.4.35; date-only ⇒ `time UNKNOWN` , never fabricated); (3) **Track** (§11.4.176/.178/.182); (4) **Alias** (§11.4.182/.196); (5) **Model + effort**. (A) **NO INVENTION (§11.4.6)** — an unrecoverable field is recorded literally `UNKNOWN` , never guessed; a fabricated timestamp/alias/model is a §11.4/§11.4.1 bluff at the requirements-record layer. (B) **HONEST RECONSTRUCTION BOUNDARY** — pre-mandate sessions are best-effort reconstructed from the durable dated record with unrecoverable fields marked `UNKNOWN` and the boundary stated explicitly, never silently implied complete. (C) **ALWAYS IN SYNC + EXPORTED** — §11.4.44 revision header, §12.10 append-on-every-new-prompt, §11.4.65/§11.4.73 four-format export (`.md` + `.html` + `.pdf` + `.docx` where applicable, siblings ≥ `.md`). (D) **KEEP-APPLYING MECHANISM** — a helper script and/or a `UserPromptSubmit` -class hook appends a newest-first row per NEW prompt at capture time (track/alias derived deterministically §11.4.182, honest `?` / `UNKNOWN`); a helper landed WITHOUT the auto-capture hook is honestly a partial mechanism (hook wiring = tracked §11.4.197/§12.10 follow-up, never claimed as auto-capture it does not perform). (E) **COMPLEMENTS, NEVER REPLACES** any per-session no-loss ledger (§11.4.197/.202), the workable-items DB SSoT (§11.4.93/.95), or the §12.10/§11.4.131 resumption artefacts; requirement loss at intake is FORBIDDEN (§11.4.197). (F) **RULE-UNIVERSAL / DOCUMENT-PROJECT-LOCAL (§11.4.35)** — the RULE lives in the constitution submodule; the document + its path + default timezone are consumer-owned project DATA (§11.4.28 decoupling), NEVER placed into the constitution. Classification: universal (§11.4.17). Composes §11.4.6/.35/.44/.65/.73/.118/.131/.176/.178/.182/.196/.197/.202/.93/.95/§12.10. Propagation gate `CM-COVENANT-114-208-PROPAGATION` (literal `11.4.208`) + recommended gate `CM-REQUEST-HISTORY-DOC-PRESENT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.208. Non-compliance is a release blocker. No escape hatch — no `--request-history-optional` , `--skip-request-log` , `--no-timezone-stamp` , `--invent-missing-field` , `--request-history-in-constitution` , `--prompt-without-history-entry` flag.

§11.4.209 — Code-review MUST run on the Fable model at xhigh effort (Opus xhigh fallback) (User mandate, 2026-07-15). Verbatim operator mandate: the mandatory independent code-review MUST ALWAYS be performed with the **Fable model at xhigh effort**; if Fable is unavailable, use **Opus at xhigh effort**. Every mandatory independent code-review the constitution requires — the §11.4.125 code-review-agent gate before pre-build + main build, the §11.4.142 universal every-change-review (no exception), the §11.4.194 exhaustive all-scenario/all-angle review, iterated to a zero-finding GO per §11.4.134 — MUST be dispatched on the **Fable model at `xhigh` effort**. (A) **Fallback (closed, ordered)** — ONLY when Fable is genuinely unavailable (established as FACT §11.4.6, never a guessed unavailability) the review runs on **Opus at `xhigh` effort**; a lower-effort or a non-Fable/non-Opus model is NOT an acceptable review substrate, `xhigh` is REQUIRED in both cases (the §11.4.194 all-angle depth is unreachable lower). (B) **Independence preserved (§11.4.70/.20)** — the Fable/Opus-xhigh model is the structurally-separated reviewer, never the author; §11.4.209 sets the review's MODEL + EFFORT, not its independence / iterate-until-GO (§11.4.134) / captured-evidence (§11.4.5/.69/.107). (C) **Honest boundary (§11.4.6)** — a review claimed done that ran on a weaker model / lower effort than Fable-xhigh (or Opus-xhigh on genuine Fable-unavailability) is a §11.4 bluff at the review-substrate layer; the review's captured evidence records which model + effort actually performed it (+ the Fable-unavailability FACT on fallback). STRENGTHENS §11.4.125/.142/.194/.134 — does NOT replace them; it pins the review's execution substrate. Classification: universal (§11.4.17). Composes §11.4.125/.134/.142/.194/.6/.20/.70/.196. Propagation gate `CM-COVENANT-114-209-PROPAGATION` (literal `11.4.209`) + recommended gate `CM-CODE-REVIEW-FABLE-XHIGH` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.209. Non-compliance is a release blocker. No escape hatch — no `--review-any-model`, `--review-lower-effort-OK`, `--skip-fable`, `--non-xhigh-review`, `--fallback-without-fable-unavailability-proof` flag.

§11.4.210 — Zero-loss request/prompt intake: every operator request/prompt MUST be mechanically CAPTURED, TRACKED, and PROCESSED — never skipped, ignored, avoided, or lost (User mandate, 2026-07-15). Verbatim intent: "every request/prompt we issue MUST ALWAYS be respected, taken into account, executed and processed! No avoiding, ignoring, skipping or any form of loss!" Every operator request/prompt MUST be (a) CAPTURED in the §11.4.208 request-history ledger MECHANICALLY at intake via a WIRED verified-runtime-state

`UserPromptSubmit` auto-capture hook (NOT a manual step, NOT a "tracked follow-up" — §11.4.210 PROMOTES §11.4.208(D)'s hook from partial-follow-up to MANDATORY per the §11.4.205 enforced-not-advisory pattern; forensic FACT 2026-07-15: an entire release-drive session's prompts went un-recorded because the hook was left un-wired); (b) TRACKED → a §11.4.202 workable item / §11.4.197 completion, or an explicit evidence-backed no-op-with-reason; (c) PROCESSED — respected/executed, never dropped. Loss at ANY of the three stages is a §11.4 PASS-bluff at the requirements-intake layer of §11.4.197's severity class. Docs-chain-bound (§11.4.106), gated (§11.4.75 — hook verified-installed in the live hook path + content-matched, ledger fresh); the autonomous loop (§11.4.126/.87/.94/.97/.103) processes every captured request (done-condition never satisfied while any captured request is un-processed, §11.4.147(e)); honest §11.4.6 (UNKNOWN never invented). Classification: universal (§11.4.17). Composes §11.4.197/.202/.208/.205/.106/.126/.75/.6/.147(e). Propagation gate `CM-COVENANT-114-210-PROPAGATION` (literal `11.4.210`) + recommended gate `CM-REQUEST-CAPTURE-HOOK-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.210. Non-compliance is a release blocker. No escape hatch — no `--skip-request-capture`, `--manual-ledger-OK`, `--hook-optional`, `--drop-unprocessed-request`, `--ledger-may-be-stale` flag.

§11.4.211 — Merge-conflict resolution during main→feature/product/flavor merges MUST run on the Fable model at xhigh effort (Opus xhigh fallback) (User mandate, 2026-07-16). Verbatim operator mandate: "During the merge of main branch to all feature/product/flavor branches of tracks 2 - 4, if conflicts happen they MUST BE resolved using Fable model with xhigh effort! If Fable is not available, then with Opus with xhigh!" Any merge-CONFLICT resolution performed while integrating the canonical trunk (`main` / `master`) into a feature / product / flavor branch — the §11.4.188 regular main→feature merge cadence, the §11.4.195 branch-taxonomy integration, the §11.4.41 merge-first pipeline, the §11.4.167(D) trunk-into-stream sync AND the §11.4.167(I) approval-gated back-merge — MUST be carried out on the **Fable model at `xhigh` effort**; the merge-resolution analogue of §11.4.209 (which pins the same model + effort for code-REVIEW), pinning it for the DECISION-MAKING of conflict resolution (which side to keep, how to UNION both sides without loss, marker removal, semantic reconciliation of divergent edits). **(A) Fallback (closed, ordered):** ONLY when Fable is genuinely unavailable (not configured / rate-limited / unreachable,

established as FACT §11.4.6 — never a guessed unavailability) the conflict resolution runs on **Opus at `xhigh` effort**; a lower-effort or a non-Fable/non-Opus model is NOT an acceptable substrate for resolving merge conflicts, `xhigh` REQUIRED in both cases (a careless resolution silently drops a commit or commits a marker). **(B) Scope:** the operator named tracks 2–4 (`feature` / `product` / `flavor`); GENERALISES per §11.4.17 to EVERY controlled main→(feature|product|flavor) merge on EVERY track that hits a conflict. **(C) Safety preserved:** sets the MODEL + EFFORT of the conflict-resolution work only, BOUNDED BY §11.4.113 (NEVER force-push) / §9 / §9.2 (no commit lost, hardlinked pre-op backup before any large/risky merge) / §11.4.41 step 3 (ZERO conflict markers committed, ZERO file silently dropped, union of both sides preserved) / §11.4.188(4) anti-bluff (post-merge smoke GREEN + marker-scan empty + no-lost-commit) / §11.4.84 (quiescent-only) / §11.4.37/§11.4.71 (fetch-first). **(D) Honest boundary (§11.4.6):** a merge whose conflicts were resolved on a weaker model / lower effort than Fable-xhigh (or Opus-xhigh on genuine Fable-unavailability) is a §11.4 bluff at the merge-resolution-substrate layer — the merge's captured evidence records which model + effort resolved the conflicts (and, on fallback, the FACT of Fable's unavailability). STRICT EXTENSION of §11.4.209 to merge-CONFLICT resolution; STRENGTHENS §11.4.188/§11.4.195/§11.4.41 by pinning the substrate of their conflict-resolution step. Classification: universal (§11.4.17). Composes §11.4.209/.188/.195/.41/.113/§9/§9.2/.6/.167(D)(I)/.84/.37/.71/.196. Propagation gate `CM-COVENANT-114-211-PROPAGATION` (literal `11.4.211`) + recommended gate `CM-MERGE-CONFLICT-FABLE-XHIGH` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.211. Non-compliance is a release blocker. No escape hatch — no `--merge-conflict-any-model` , `--skip-fable-merge-resolution` , `--non-xhigh-merge-conflict` , `--resolve-conflict-lower-effort-OK` , `--fallback-without-fable-unavailability-proof` flag.

§11.4.212 — Main README is the canonical starting-point / entry point for ALL project documentation — no doc may be an orphan unreachable from README (User mandate, 2026-07-16). Verbatim operator mandate: "Manual tests catalog must have linking all through the main README as the starting point of this and any other Project documentation! This fact and mandatory rule MUST be added into the constitution." The main README (`README.md` at the repository root) is the canonical STARTING POINT / entry point for ALL project documentation, for THIS project and EVERY project incorporating this constitution.

Every project document in the §11.4.65 export scope — INCLUDING the manual-tests catalog (the §11.4.185 manual-QA testing catalog), the §11.4.153 per-feature Status / Status_Summary docs, the Issues / Fixed trackers + their summaries (§11.4.12 / §11.4.53 / §11.4.57), and every guide / research note / changelog / script companion doc — MUST be reachable / linked (DIRECTLY or TRANSITIVELY) from the main README. No project document may be an ORPHAN — a doc that no link-path from README reaches is a §11.4.212 violation. STRICT GENERALISATION of §11.4.57 (which mandated a `Tracked-Items + Status Documents` doc-link section covering the tracker / status set) to EVERY §11.4.65-scope document, so the README is the single navigable root of the project's documentation tree. **(A) Reachability = a link path.** README → guide → sub-doc is fine (transitive); the requirement is that SOME path from README reaches every in-scope doc, not that every doc is linked directly at top level. **(B) Always-sync (§11.4.59 / §11.4.106 docs-chain).** A newly-added §11.4.65-scope doc that no README-reachable path links is a §11.4.212 violation the moment it lands; the manual-tests catalog specifically MUST be linked from the README (the operator's load-bearing example). **(C) Honest boundary (§11.4.6).** Reachability is a link-graph property (every doc FINDABLE from README), NOT a claim about each doc's CORRECTNESS or freshness (those stay §11.4.44 / §11.4.106 / the doc's own gates). **(D) Composition with §11.4.57.** §11.4.57's `Tracked-Items` section remains the mandated README landing block for the tracker / status set; §11.4.212 adds that EVERY OTHER in-scope doc is likewise reachable — §11.4.57 is the specialisation, §11.4.212 the generalisation. Classification: universal (§11.4.17) — the consuming project supplies its README path, its manual-tests-catalog path, and its §11.4.65 doc-set enumeration per §11.4.35. Composes §11.4.57 / §11.4.59 / §11.4.65 / §11.4.106 / §11.4.153 / §11.4.185 / §11.4.44 / §11.4.12 / §11.4.53. Propagation gate `CM-COVENANT-114-212-PROPAGATION` (literal `11.4.212`) + recommended gate `CM-README-DOC-ENTRYPOINT-COMPLETE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.212. Non-compliance is a release blocker. No escape hatch — no `--orphan-doc-OK`, `--readme-not-entrypoint`, `--skip-readme-link`, `--doc-unreachable-OK` flag.

§11.4.213 — FEATURE research-scheduling directive: recognized via all §11.4.140 forms, SCHEDULES (never synchronously executes) a deep, enterprise-grade research + implementation-planning effort as a tracked workable item (operator-directed mandate, 2026-07-16). Compact summary:

extends the §11.4.202 reporting-directive family with a fourth registered action `FEATURE` — recognized via ALL SIX §11.4.140 grammar forms (`FEATURE :: x` / `DEFAULT::FEATURE :: x` / `/FEATURE x` / `/DEFAULT::FEATURE x` / `FEATURE ---> x` / the single-colon `FEATURE: x`, registered-action-only so lowercase prose like "feature: ..." never expands and is never asked about, §11.4.6) — that SCHEDULES rather than synchronously executes a deep research + implementation-planning effort: the directive creates a Type=Task / Status=Queued workable item (§11.4.15/.16/.54) by INVOKING the SAME §11.4.202 `report_item.sh` engine (never a duplicate implementation of the item-creation/ DB-sync/tracker-push machinery — a second divergent tracker-push path would itself be a bluff-surface regression) and appends it to a durable `docs/requests/feature_queue.md` queue (mirroring `BACKGROUND` 's `docs/requests/background_queue.md` pattern) so a scheduled request is never dropped; the item's comprehensive description (§11.4.148/.171) embeds the FULL research mandate as its acceptance-defining work program — deep web research on best incorporation (§11.4.8/.99/.150), systematic-debug of every enumerated weak-spot/ gap/danger-zone with a risk-free rock-solid design per gap (§11.4.102), always enterprise-grade/bleeding-edge/innovative, exhaustive documentation down to lines-of-code + micro-POCs + diagrams + SQL + templates (§11.4.65/.73), reuse-first investigation of vasic-digital/HelixDevelopment components kept decoupled (§11.4.28/.74/.177), full test-type + Challenges + HelixQA planning (§11.4.27/.169), fine-grained phased/task/subtask breakdown, enterprise-scalability + max-performance planning, OpenDesign UI/UX wireframes where applicable (§11.4.162/.190), full CodeGraph integration (§11.4.78/.79/.80), and fully-synced workable items across the SQLite SSoT + every connected external tracker (§11.4.93/.95/.148 D5), honestly SKIPPING any absent tracker (§11.4.10) — NEVER a faked push. The multi-day research itself is EXECUTED by the standing autonomous loop (§11.4.87/.94/.97/.103/.126) when it claims the item, driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 — never left un-wired in the backlog; "scheduled" is never reported as "done" (§11.4.6). Anti-bluff four-layer coverage: pre-build gate `CM-FEATURE-DIRECTIVE` (FUNCTIONAL — sources + RUNS the §11.4.140 parser, never grep-only, §11.4.108) + paired §1.1 mutation + a real-DB end-to-end evidence trail (temp SQLite DB, no mocks, §11.4.27). Classification: universal (§11.4.17). Composes §11.4.140/.202/.197/.8/.99/.150/.102/.65/.73/.28/.74/.177/.27/.169/.162/.190/.78/.79/.80/.93/.95/.14

Propagation gate `CM-COVENANT-114-213-PROPAGATION` (literal `11.4.213`) + recommended gate `CM-FEATURE-DIRECTIVE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.213. Non-compliance is a release blocker. No escape hatch — no `--feature-runs-synchronously`, `--skip-feature-queue`, `--feature-without-research-mandate`, `--reimplement-tracker-push`, `--hardcode-project-in-engine`, `--skip-item-creation` flag.

EXTENSIONS landed 2026-07-17 (existing anchors strengthened — full text in `Constitution.md`; research-derived from a consuming project's status-without-custody forensics). Forensic FACTs (genericised, all measured 2026-07-17): 576 tracked items, 182 claiming a done/ready status, **173 (95%) with NO registered regression guard** — an operator manually sampled 6 done-claiming items and found **6/6 broken**, the expected draw of that arithmetic, not bad luck; recorded reopen-per-recorded-fix **52%** (published elite <10%), itself an UNDERCOUNT because recurrences re-entered the tracker as NEW ids and statuses moved with ZERO history rows (one item: 4 Reopened events, 0 terminal events); the standing guard suite's exit code read ONLY its FAIL count — measured live on a release-candidate target: **61 registered guards → PASS 0 / FAIL 5 / PENDING 56 → exit 0 "not blocked"** — and the release-tag tooling consulted the guard suite **ZERO times**, so to the release gate "never verified" and "verified good" were the same colour; the project's own history selected the discriminator — every fix whose done-claim was preceded by an on-device RED→GREEN polarity flip that HELD recorded zero reopens, and every fix confirmed at source-green bounced. **No new anchor number was minted**: §11.4.115 and §11.4.146 already said what the operator mandates and did not bind because they are prose consulted by agents, not conditions checked by seams — the deliverable is the MECHANICAL BINDING (the §11.4.205 test applied: "if I documented this and nobody implemented the hook, would anything catch it?"). **§11.4.115(F) — machine-written verdict pairs + guard-viability:** RED and GREEN are harness-written verdict files, never prose — carrying the item id, guard identity, polarity, exit code, the target's artifact fingerprint READ FROM THE TARGET AT RUN TIME, iterations (≥3, §11.4.50), and evidence files whose CLASS matches the defect's layer (a source-grep transcript can never satisfy a user-visible/pixel/audio class — the wrong-layer oracle dies by construction); GREEN requires a prior RED with exit ≠ 0 on the PRE-fix artifact and a DIFFERENT fingerprint (identical fingerprints prove the fix was never deployed); a guard never observed FAILing on the genuinely-

broken artifact is unvalidated instrumentation and mints no verdicts (a guard structurally unable to FAIL — or to PASS — satisfies nothing); the canonical §1.1 mutation for a landed fix is the fix-commit's REVERT, and a mutation whose diff only deletes the literal strings the gate greps for is a refused tautology. Recommended gate `CM-GUARD-VERDICT-MACHINE-WRITTEN`. **§11.4.135 (verdict-coverage extension) — ABSENCE of a verdict blocks exactly as a FAIL does:** the suite's exit semantics MUST incorporate COVERAGE, never only the FAIL count; the release seam refuses the tag while `uncovered = registered ∧ topology-present ∧ no-verdict-for-the-candidate-fingerprint` is non-empty OR any FAIL exists; the release-tag tooling MUST consult the suite/verdict store mechanically; REQUIRES-REBUILD PENDING is honest on intermediate artifacts and self-clearing on the candidate (re-run on the candidate, or the item leaves the release's done-set — both forced, neither a permanent exemption); topology-absent guards are exempt ONLY via a checked-in target-pool topology map and MUST be enumerated in the release changelog as honest gaps (§11.4.3/§11.4.118) — never silent; a naive "PENDING always blocks" is REJECTED as a §11.4.201 FAIL-bluff; pre-existing unguarded backlogs adopt via a ONE-TIME monotone-decrease ratchet snapshot so day one is green and the disease cannot spread. Recommended gate `CM-RELEASE-VERDICT-COVERAGE` (self-tested with golden-good / golden-bad / negative-control per §11.4.205/§11.4.107(10)). **§11.4.146(D3) — status-custody mechanical binding:** a done/ready status write is REFUSED without the file chain `registry row keyed by the item's EXACT stable id (checked-in alias map for legacy keys — a guard keyed to a non-item id fails the join and satisfies nothing) → executable registered guard → §11.4.115(F) RED+GREEN verdict pair → class-matched evidence` — prose appears nowhere in the chain; Seam A = the workable-items engine (§11.4.93) refuses the write (project-agnostic per §11.4.28, custody paths are consumer DATA per §11.4.35); Seam B = a FULL-TABLE sweep gate on every pre-build run (not a diff check — catches raw-DB writes that bypass Seam A); Seam C = the §11.4.135 release seam; §11.4.205 construction order — the executable sweep + its golden-bad meta-fixture (a temp SSoT overlay with a chain-less terminal item; the meta-test RUNS the sweep and requires FAIL, and asserts the sweep is WIRED) land in the SAME commit as any governance text citing them. Recommended gate `CM-STATUS-CUSTODY`. Honest boundary (§11.4.6), carried explicitly: the binding proves no status outruns its evidence and the REPORTED defect cannot silently return under the guard's conditions — it does NOT prove the

feature bug-free (family dedup §11.4.186 remains), does NOT make a miscalibrated oracle honest (§11.4.107(10) goldens are necessary, not sufficient), does NOT replace the §11.4.185 manual-QA sufficiency gate, and makes target availability a HARD dependency of status progress BY DESIGN (a visible block replaces a silent pass). All Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their registry, verdict-store, sweep, ratchet, and topology-map paths as DATA per §11.4.35. Propagation gates for the literals `11.4.115` / `11.4.135` / `11.4.146` are unchanged and stay GREEN; the three recommended mechanism gates + paired §1.1 mutations are gate-code = separate work item.

EXTENSIONS + NEW ANCHORS landed 2026-07-17 (round 2 — the carrier/measurement-integrity harvest). Research-derived from one cycle's forensics in a consuming project, where EVERY false finding had ONE shape: **a token that MENTIONS X was matched as X, or an instrument returned silence that was read as data.** Genericised FACTs: a removal's completeness was measured by counting references to the removed component and the count matched the AUDIT-TRAIL COMMENTS documenting the removal (a completed removal read as failed); a pattern's escapes were eaten by an intervening shell so the far side searched a LITERAL and matched nothing (a false "the component is missing" that would have implied the target could not boot — it had booted); `\|` under an extended-regex dialect is a LITERAL pipe so a mandated-fields check searched a string nobody wrote and nearly reported the fields absent (all were present); a `check | head` pipeline returned the PAGER's exit status so a parse-check reported "clean ✓" three times without checking anything; a line-start-anchored pattern missed an indented, colour-escaped line and read a present verdict as no-verdict; a binary extractor split a multi-byte character and reported a present string as absent. §11.4.196(D)'s process-carrier and §12.12's self-match had ALREADY anchored this shape TWICE and it kept recurring — that recurrence is the argument for the general rule. The self-demonstrating instance: a mutation-residue scan run over the governance repo flagged the very anchor that DEFINES the mutation markers, because that anchor quotes them in its rule text (proven a carrier, not residue, by an identical HEAD-vs-worktree count). **The fix, proven in-session: the CONTROL NEEDLE** — an artifact check returned 0 hits; instead of reporting absence it ran a string KNOWN present through the SAME path, got 1 hit, and only THEN reported the zero as real.

§11.4.201 (6)(7)(8) — scope widened to EVERY load-bearing measurement + carrier-vs-thing + control needle + metric-validation. (6) clauses (1)-(5) bind guards (decision seams that refuse/allow) with a refusal-shaped taxonomy; (6)-(8) bind **every measurement whose result is load-bearing** (grep/count/parse/exit-code/scan/metric/artifact-inspection) with a MEASUREMENT-shaped taxonomy — a **FALSE-NULL** (a zero read as absence when the instrument could not see) and a **FALSE-MATCH** (a hit on a carrier) are §11.4/§11.4.1 bluffs at the measurement layer; the null is more dangerous because a broken instrument and a clean artifact return the identical quiet zero. (7)(a) MATCH STRUCTURE, NOT SUBSTRING — distinguish the THING from a CARRIER that MENTIONS it. (7)(b) **A NULL IS NOT EVIDENCE until a CONTROL NEEDLE proves the instrument can see** — same instrument, same path, same artifact, a known-present needle MUST return non-null; needle absent ⇒ the instrument is blind and the zero says NOTHING (report the blindness, never the absence §11.4.6). **The needle MUST share the certified query's LOAD-BEARING FEATURES** (same dialect constructs / anchoring / quoting / encoding): a bare-literal needle certifies only the layers a bare literal crosses — the (7)(c) dialect FACT refutes the naive form directly, since a literal needle sails through the same tool+path and would 'certify' a zero from a query still searching a string nobody wrote, whereas a needle carrying the SAME alternation dies with it. Earned inference is therefore BOUNDED: needle found ⇒ the path can see the needle's QUERY CLASS ⇒ a zero from that same class is evidence; a zero from a query using features the needle never exercised needs a needle of its own class. Generalises §11.4.115(F)'s validate-the-detector from guards to every measurement, PER-QUERY — **golden fixtures cannot substitute** (§11.4.146(D3): goldens necessary-not-sufficient), an instrument can pass every fixture on the authoring host and be blind on the real path. (7)(c) **THE PATH IS PART OF THE INSTRUMENT** — the tool PLUS every layer the query crosses (wrapper-shell quoting, regex dialect, pipeline exit status, anchoring, encoding); each returns a CLEAN CONFIDENT WRONG answer without crashing, so §11.4.1's script-bug clause (which covers crashes-into-FAIL) never fires and §11.4.67's parse-check (parse-time only) cannot see it — the control needle is the ONLY mechanism that tests the path. (8) **A METRIC MUST BE VALIDATED AGAINST THE DEFINITION OF DONE** — construct the correct end-state and evaluate the metric there; if the correct end-state does not move it to target, the metric is INVALID and MUST NOT gate work (a metric the correct outcome REFUTES will, if enforced, drive work AWAY from correctness); honest boundary §11.4.6 — this constitution already ships one such gating metric (§11.4.135's

adoption ratchet, blocking at the RELEASE-TAG seam) whose validation against the definition of done is OWED, not claimed, and which every consuming project operating a ratchet MUST TRACK as a §11.4.197 item (stated as the obligation it is — this anchor asserts NO project has already filed it; asserting an unverified tracker state would be the §11.4.6 violation the clause forbids). **Interim precedence:** until that validation lands the §11.4.135(5) ratchet KEEPS GATING — clause (8) does NOT disarm it (an unvalidated ratchet blocking a release is bounded + visible + operator-resolvable; disarming the only adoption mechanism would silently re-open the unguarded-backlog hole §11.4.135 closes — the §11.4.101 reversible-safe choice); clause (8) binds every NEW metric from this anchor forward, and the ratchet's validation as owed work — never as a licence to switch it off. Gate contract landed in the canonical file: `CM-GUARD-ASSERTS-REAL-CONDITION` EXTENDED to the measurement layer ((i) a reported absence cites a class-matched control-needle result; (ii) the needle's class matches the certified query's load-bearing features) + NEW recommended sibling `CM-METRIC-VALIDATED-AGAINST-DONE` (every gating/ranking metric carries a recorded definition-of-done evaluation reaching target) + paired §1.1 mutations (bare-literal needle on a dialect-dependent query → (ii) FAILs; needle stripped → (i) FAILs; metric whose correct end-state moves AWAY from target → FAILs). Gate-code = separate work item — the contract is landed, the code is NOT claimed shipped (§11.4.6).

§11.4.112 (5) — impossibility verdicts are BOUNDED and MUST say where the edge is. FACT: a TRUE verdict ("relocating another application's protected video to a secondary display *while that application's UI stays in place, without its cooperation* is impossible") LEAKED to an ADJACENT VIABLE goal (whole-task placement) for six weeks, while the platform had shipped a public permission-free SDK call for the adjacent goal several major versions earlier and its own connected-display docs PRESCRIBED that call. The anchor was ASYMMETRIC: clause (3) hardens a verdict against reopening while nothing bounded its REACH. Now every `structurally-impossible` verdict MUST (a) STATE ITS EXACT SCOPE (the narrowest claim the evidence supports — every load-bearing qualifier is part of the verdict; a qualifier dropped is a verdict widened), (b) ENUMERATE THE ADJACENT GOALS IT DOES NOT COVER (unexamined neighbour ⇒ UNKNOWN + tracked follow-up, never silently absorbed), (c) BE CITATION-FENCED (citing it beyond its fence, in planning/investigation/design/status, without its OWN §11.4.150 research + proof is a §11.4.6 violation + a verdict-layer bluff of the class §11.4.199 names). Honest boundary — outside the fence ≠ viable, it

means UNDECIDED, earned by its own evidence never by inheritance in either direction. Why a clause not a narrative: §11.4.150 already documented this class in prose and REQUIRES the research pass before a verdict is EARNED — the leak still happened, because §11.4.150 fires where a verdict is MINTED and this leak happened where one was CITED as a premise, crossing no closure seam. Gate contract landed: `CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` EXTENDED (scope statement + adjacent-goals-not-covered enumeration required; citing a verdict outside its stated scope without that goal's own §11.4.150 proof → FAIL) + paired §1.1 mutations. Gate-code = separate work item (§11.4.6).

§11.4.120 (seam-placement) — a gate whose precondition the gated work itself produces is on the WRONG SEAM. §11.4.120's trigger is a gate that FAILs (a gate that RAN); its sibling is a gate that CANNOT RUN in the imposed ordering, and the response is not reconciliation but re-seating. FACT: work was ordered "run checks E1-E4, then implement P1 only if they pass" — but the gate's own text defined E2 as running ON a P1-gated build, so E2 could not exist before P1; the source of truth gated ENABLING (building behind a default-off flag was permitted), and the gate had been imposed on AUTHORSHIP. (a) if gate G requires artifact A and A is produced by the work G gates, G belongs at a LATER seam (activation/enablement/release), never authorship — imposing it there is a deadlock, not rigor; (b) RE-READ the source of truth before re-seating (the gate's own text names its real seam and governs over any restatement — a restatement that moved it from enabling to writing is a §11.4.6 misstatement; obey the source, never weaken the gate); (c) the forbidden responses are §11.4.120's own (no fake-pass/delete/tautology, no build-then-retro-declare, and the work is NOT abandoned as "blocked" when the gate merely sits on the wrong seam). Honest boundary — an unrunnable gate is NOT automatically mis-seated (§11.4.102 first); it may be correctly seated with a genuinely-missing precondition = the §11.4.69 `artifact_not_yet_built` NOT-YET-RUNNABLE state. §11.4.110's clash detector cannot catch this class (its input is a DIFF; an ordering instruction is not a diff). The "report the circularity, don't improvise" half is ALREADY covered by §11.4.6/§11.4.66/§11.4.101/§11.4.201(4) — this clause claims only the seam-placement rule. Gate contract landed: `CM-GATE-RECONCILED-NOT-FAKE-PASSED` EXTENDED (an imposed gate whose precondition the gated work itself produces → FAIL naming the circularity, re-seat at activation) + a golden-FALSE fixture that MUST include a correctly-seated gate merely awaiting

its artifact (§11.4.69 `artifact_not_yet_built`) which the gate MUST NOT fire on, else it becomes the §11.4.201(1) false positive + paired §1.1 mutation. Gate-code = separate work item (§11.4.6).

§11.4.69 (closed reason-set) — `artifact_not_yet_built` added. The NOT-YET-RUNNABLE class (check CORRECT, topology PRESENT, precondition artifact absent — fix not built/deployed) had NO legal code, so a §11.4.135 REQUIRES-REBUILD PENDING had to misuse a false code (`hardware_not_present` / `feature_disabled_by_config` — both untrue, a §11.4.6 misstatement) or fail the helper at call time. Reporting it FAIL is a §11.4.1 FAIL-bluff (sends teams to fix healthy code); PASS is a §11.4 PASS-bluff. Its seam-dependence is §11.4.135's, NOT flat — honest+non-blocking on an intermediate artifact, BLOCKING on the release candidate; a flat "always blocks" is REJECTED per §11.4.135 as itself a §11.4.201 FAIL-bluff. **Audited, NO new anchor:** the operator-candidate "BLOCKED as a third verdict state" is ALREADY fully covered — §11.4.135's 2026-07-17 verdict-coverage extension states it more precisely (seam-dependent, and it explicitly rejects the flat framing), §11.4.3 forbids the PASS, §11.4.201(1) forbids the FAIL, §11.4.116/§11.4.45 already carry the verdict vocabulary; only the reason-code was missing.

§11.4.214 (NEW) — Recurrence-links-not-mints: a defect that returns MUST reopen its existing item, never enter as a new id. FACT: recurrences of already-"fixed" defects re-entered as BRAND-NEW ids (≥5 chains, one three deep), so the ORIGINAL's reopen counter never incremented — and the §11.4.55 `reopens_count`, the exact signal §11.4.132(d)/§11.4.189 use to rank the most-fragile work for the deepest scrutiny, **stays quiet precisely on the items that keep breaking**; the recorded 52% reopen rate was a KNOWN UNDERCOUNT for this reason; ~15 items mapped to ~9 real root causes, so "fix all 15" would have forked duplicate/conflicting fixes on one cause. Before minting a NEW id from ANY intake path, check for an existing item describing the SAME defect → RESOLVE the match THROUGH its `duplicate-of` links to the CANONICAL CHAIN HEAD (§11.4.90's closure vocabulary includes `duplicate-of`, and clause (5) MANUFACTURES such items — reopening one would resurrect a non-canonical copy beside its live canonical item, forking one defect into two and crediting the §11.4.55 signal to the WRONG item), then LINK it, and REOPEN (§11.4.34) **iff the resolved head is in a TERMINAL state**; multi-match ⇒ resolve every match to its head and act on the SINGLE survivor, >1 distinct head or a broken/circular link ⇒ UNDECIDED (ask / mint-with-link, never guess §11.4.6). **Open-original case:** a

matched item still OPEN gets LINK-ONLY — 'reopen an open item' is undefined in the §11.4.15 status machine and §11.4.34's `Reopened` presupposes a closure to demote FROM (§11.4.7); the reopen path exists to correct the specific lie 'closed as fixed and it is not', and an open item is not telling it. **§11.4.202 precedence (explicit, never left to a consumer):** §11.4.202's mandate is that no report is ever LOST — SATISFIED IN FULL by a SAME-DEFECT report landing as link + (terminal-only) reopen; it forbids the report evaporating into prose, it does NOT require a fresh id per report. §11.4.214 refines WHICH item a report lands on; §11.4.202 owns THAT it lands; DISTINCT/UNDECIDED verdicts run §11.4.202's mint path unchanged. (1) EVERY intake path binds (reporting directives §11.4.202, AI-detected test/gate failures, regression-guard verdicts, cycle re-discovery, operator manual testing — §11.4.34's own reason vocabulary enumerates them; wiring dedup ONLY into the reporting channel under-covers exactly the machine-driven paths that recur most). (2) Key on ticket else normalised `(subject, scope)` — NEVER a bare subject substring (§11.4.186). (3) **A wrong merge silently DELETES a real defect — so `distinct-but-similar` is a FIRST-CLASS verdict**: SAME-DEFECT ⇒ link+reopen, DISTINCT ⇒ mint (correct + expected), UNDECIDED ⇒ ask (§11.4.66/.105) or mint-WITH-a-candidate-duplicate-link; autonomous default (§11.4.101) is mint-with-link over merge — a spurious id is cheap + recoverable, a wrongly-merged defect is LOST (the asymmetry decides). (4) Self-validated oracle (§11.4.107(10)/§11.4.186): golden-good + golden-bad + a **negative-control** (two genuinely DISTINCT same-subject items that MUST NOT merge) — an oracle without it is a false-merge engine. (5) Id stability preserved (§11.4.54 — reopen the original, never renumber/reuse/retire; an already-minted duplicate is LINKED via §11.4.90 duplicate-of, never silently deleted). (6) The restored signal IS the point — an undercounting reopen counter is a §11.4/§11.4.1 bluff at the PRIORITISATION layer: it reports the fragile set as stable and aims the deepest scrutiny at the wrong items. Honest boundary §11.4.6 — dedup makes the signal TRUE, not the item correct, and a same-symptom recurrence with a different root cause is real (§11.4.102 decides; the linked+reopened item is where that is recorded, never a reason to fork). Gate `CM-RECURRENCE-LINKS-NOT-MINTS` (every CONFIRMED-duplicate link whose target was TERMINAL at link time has a reopen event on that target) — **its own scope is §11.4.201-bound**: it MUST NOT fire on an OPEN-target link (LINK-only is correct there) nor on a clause-(3) UNDECIDED candidate-duplicate link (un-reopened BY DESIGN, and the mandated autonomous default), or it becomes exactly the §11.4.201(1) false-positive refusal this same round harvests against; its golden-

FALSE set MUST include both + paired §1.1 mutation. Why NEW: §11.4.55 CONSUMES the count, §11.4.34 presupposes the status is already `Reopened`, §11.4.54 forbids only one-id-two-items, §11.4.186's DEDUP needs DIVERGENT metadata and gates export/sync/commit not INTAKE, and §11.4.202's operative text, read WITHOUT this anchor, reads as mint-every-time — the reading the precedence paragraph above corrects (they are NOT in conflict: §11.4.202 owns THAT a report lands, §11.4.214 refines WHICH item it lands on).

§11.4.215 (NEW) — A doc that BINDS work MUST live, tracked, in the repository where that work happens. FACT: the document DEFINING a binding acceptance gate was UNTRACKED and lived in an out-of-tree scratch directory belonging to a different checkout — so the gate could not be honoured (unreadable), reviewed (no diff/history/authorship), version-controlled (no revision/rollback), or reconciled when it contradicted the source of truth. A binding artifact that lives nowhere is a rumour with an imperative mood. (1) **BINDING ⇒ TRACKED, and un-tracked ⇒ NOT BINDING** — a binding claim sourced from an untracked/out-of-tree doc MUST NOT be enforced; land the doc or decline the gate, never obey a citation of it. (2) IN THE REPOSITORY THE WORK HAPPENS IN — a sibling checkout / scratch dir / chat message / issue comment is NOT tracked for this purpose (the test: can someone who clones ONLY that repo read the binding text at its declared path?); multi-repo binding docs are inherited BY REFERENCE from a tracked shared dependency (§11.4.28/§11.4.177), never copied, never out-of-tree. (3) **The prior question §11.4.212 cannot ask** — §11.4.212 ENUMERATES the in-repo scope and checks reachability, so a doc in no checkout is never enumerated and its gate cannot FAIL; §11.4.212 answers "is it findable?", §11.4.215 answers "is it here at all?" (once in-repo, §11.4.212/.44/.65/.73/.106 all attach as normal — this is their precondition, not their competitor). (4) **The generalisation of §11.4.95** (whose entire content is "authoritative ⇒ tracked, NOT a build artefact, IS authoritative source data" for ONE named artifact), exactly as §11.4.212 was minted as the generalisation of §11.4.57. §11.4.11 is the counterweight and stays intact — logs/forensics/scratch remain out-of-tree and untracked by DEFAULT; **binding is what pulls a document into the repository**, and a doc nobody cites as binding is governed by §11.4.11, not this anchor. Honest boundary §11.4.6 — in-repo presence proves AVAILABLE + ATTRIBUTABLE + VERSION-CONTROLLED, never CORRECT/current (§11.4.44/.106/.186), and never well-seated (§11.4.120's seam clause — whose forensic case was carried by

this SAME out-of-tree document: two defects, one artifact, which is itself the argument that an unreviewable binding doc is where bad gates hide). Gate `CM-BINDING-DOC-IN-REPO` + paired §1.1 mutation.

All Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their intake wiring, id scheme, normalisation function, and binding-doc paths as DATA per §11.4.35. Propagation gates `CM-COVENANT-114-214-PROPAGATION` + `CM-COVENANT-114-215-PROPAGATION` (literals `11.4.214` / `11.4.215`); the literals `11.4.201` / `11.4.112` / `11.4.120` / `11.4.69` are unchanged and their propagation gates stay GREEN. All recommended mechanism gates + paired §1.1 mutations = gate-code, a separate work item.

DESIGN-METHODOLOGY MANDATES landed 2026-07-22 (§11.4.216–§11.4.223 — full text in `Constitution.md`; proving project: the Helix Thready MVP design package at `docs/public/research/mvp/design/`). The complete design methodology built and verified end-to-end in Helix Thready is codified as eight universal anchors so every governed project inherits it out of the box. All Classification: universal (§11.4.17) — consumers supply brand values, platform lists, paths, and submodule pointers as DATA per §11.4.35. Propagation gates `CM-COVENANT-114-216-PROPAGATION` … `CM-COVENANT-114-223-PROPAGATION` (literals `11.4.216` – `11.4.223`) + recommended mechanism gates + paired §1.1 mutations = gate-code, a separate work item.

§11.4.216 — Canonical machine-readable design-token source (User mandate, 2026-07-22). Compact summary: every UI-shipping project maintains its design tokens as ONE canonical machine-readable source — CSS custom properties, light in `:root {}`, dark via the `[data-theme="dark"]` override (composing the sanctioned dark mechanisms) — and EVERY surface binds via `var(--…)` or a GENERATED binding produced by codegen FROM that file, never a hand-kept copy (hand-copies drift silently); NO INVENTED VALUES (§11.4.6 applied to design): every color/size/duration traceable to captured evidence — brand colors to a recorded eyedrop/measurement of the canonical brand asset (proving project: brand `#B6E376` = mean of 1,625,855 logo pixels), accent pairs to MEASURED WCAG ratios against their PAIRED background (6.03:1 light / 13.56:1 dark), structural values to the documented shared scale; a hex resolving to no token is a violation — use the nearest token, never mint one; the token source is the white-label seam (§11.4.217 closed override set); not-yet-built codegen bridges

are [OPEN]-tracked (§11.4.223), never claimed shipped. Composes §11.4.6/.162/.170/.190/.197/.217/.223. Propagation gate `CM-COVENANT-114-216-PROPAGATION` + recommended gate `CM-DESIGN-TOKEN-SINGLE-SOURCE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.216. Non-compliance is a release blocker. No escape hatch — no `--hand-copied-palette-OK`, `--inline-hex-OK`, `--invent-token-value`, `--light-only-tokens`, `--platform-keeps-own-palette` flag.

§11.4.217 — OpenDesign brand contract: 9-section DESIGN.md + tokens.css twin, disjoint color roles, AA-pinned accents, locked attribution footer (User mandate, 2026-07-22). Compact summary: every UI-shipping product carries under `design/opendesign/` (or its declared equivalent §11.4.35) a `DESIGN.md` in the VERIFIED OpenDesign 9-section schema (H1 + `> Category:` blockquote immediately after the H1; number-prefix-parsed sections; a "Font labels for catalog extraction" block; real hex; CSS variables wrapped in `:root {}` with dark via `[data-theme="dark"]`; real component CSS with `prefers-reduced-motion` targeting specific elements, never a global `*`; `:focus-visible` on every interactive component — format verified against the tool's CURRENT source per §11.4.99, never from memory) + the machine-readable `tokens.css` twin (the §11.4.216 canonical file or a compiled view; prose and tokens never diverge); THREE DISJOINT color roles by contract — brand DECORATIVE only (never body text), accent INTERACTIVE + AA-PINNED (measured ≥ 4.5:1 per mode), semantic STATE never overridden by any brand (a brand color can never mask an error signal); white-label = a CLOSED override set (brand tokens, accent pair, logo, slogan) with accent overrides SERVER-VALIDATED to WCAG AA or REJECTED carrying the measured ratio + an AA-passing suggestion; structural/neutral/semantic tokens NOT account-overridable; the organization attribution footer is LOCKED — present on every surface, never removed even fully white-labeled. Composes §11.4.6/.99/.162/.190/.216/.220/.223. Propagation gate `CM-COVENANT-114-217-PROPAGATION` + recommended gate `CM-OPENDESIGN-BRAND-CONTRACT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.217. Non-compliance is a release blocker. No escape hatch — no `--skip-brand-contract`, `--prose-only-design-md`, `--brand-as-text-OK`, `--unvalidated-accent-override`, `--remove-attribution-footer`, `--semantic-tokens-brandable` flag.

§11.4.218 — Living design-library catalogue + per-cell-provenance reusability matrix (User mandate, 2026-07-22). Compact summary: every UI-shipping project maintains a LIVING component catalogue rendering EVERY component × declared state × light/dark from SELF-CONTAINED artifacts (zero external requests — no CDNs, vendored libs inline, honest font fallbacks), token-bound (§11.4.216), with REAL interactive behaviors (real dialog/sort/toggle; `prefers-reduced-motion` honored per §11.4.221) — proving project: 14 groups, ~38 components, ≈118 state cells, ≈236 rendered states, both themes auditable via a real persisted toggle; PLUS a components × platforms reusability matrix where EVERY cell carries closed-set provenance — VERIFIED (re-checked at source, dated) / PROPOSED ([DEFAULT — adjustable]) / ASSUMED (labeled inference); an unmarked cell reads ASSUMED; a scaffold cell claiming VERIFIED is a §11.4 PASS-bluff at the design-planning layer; the matrix is read BEFORE implementing any component on any platform. Spec'd-but-unrendered components/states are tracked gaps (§11.4.223). Composes §11.4.6/.162/.170/.216/.221/.223. Propagation gate `CM-COVENANT-114-218-PROPAGATION` + recommended gate `CM-DESIGN-LIBRARY-LIVING-CATALOGUE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.218. Non-compliance is a release blocker. No escape hatch — no `--cdn-in-catalogue`, `--single-theme-catalogue`, `--mocked-behaviors-OK`, `--unmarked-matrix-cell`, `--scaffold-claims-verified` flag.

§11.4.219 — Per-app screen catalogues: full-IA coverage, per-platform chrome, honesty markers (User mandate, 2026-07-22). Compact summary: every shipped surface (web / mobile / desktop / TUI / marketing / …) has a screen catalogue covering EVERY node of its information architecture (an IA node with no catalogue screen is a tracked gap, never silently skipped); per-platform CHROME VARIANTS rendered where one design serves several hosts (mobile OS chrome toggle, per-OS window chrome, terminal-width constraints); each screen SELF-CONTAINED, token-bound (§11.4.216), light + dark, populated with REALISTIC product data and a per-screen state contract (loading/empty/error/populated); UNBUILT or UNDECIDED behavior is HONESTLY marked — `[GAP: id]` / `[OPEN: id]` (§11.4.223), labelled placeholders, inline notes, disabled controls — NEVER a designed-as-if-real fake flow (a fake flow presented as settled is a §11.4 PASS-bluff at the product-design layer). Proving project: 13 web + 11 mobile (Android/iOS chrome toggle) + desktop shell + 10 TUI + 3 marketing screens, every unbacked flow marked. Composes §11.4.6/.162/.170/.185/.190/.216/.218/.223.

Propagation gate `CM-COVENANT-114-219-PROPAGATION` + recommended gate `CM-SCREEN-CATALOGUE-FULL-IA` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.219. Non-compliance is a release blocker. No escape hatch — no `--partial-ia-coverage`, `--one-chrome-fits-all`, `--fake-flow-OK`, `--lorem-ipsum-content`, `--single-theme-screens` flag.

§11.4.220 — Open-first design tooling: self-hosted PenPot primary via the PenPot submodule; proprietary tools import/export only; in-repo open-format sources (User mandate, 2026-07-22). Compact summary: the collaborative design platform is an open-source self-hostable tool — PenPot as the standing default — run SELF-HOSTED and consumed ONLY via the dedicated reusable PenPot submodule (working name `vasic-digital/PenPot`, final repository name pending — the generic referent is normative meanwhile) carrying ALL PenPot mechanisms/utilities (deployment recipe, RPC client, access-token minting, import tooling, browser-hydration automation, MCP wiring), itself consuming the Containers submodule (§11.4.76, rootless §11.4.161) and DECLARING that dependency in its §11.4.31 `helix-deps.yaml`; projects MUST NOT hand-roll PenPot deployment/client code (§11.4.28/§11.4.74 — extend the submodule upstream); the submodule carries mechanisms ONLY, never project design content. Proprietary tools (Figma or equivalent) are OPTIONAL import/export/review bridges, NEVER the source of truth — no design value may exist only inside a proprietary tool/cloud; design sources live IN-REPO in open, diffable formats (SVG masters, CSS tokens, JSON manifests/IR/variables payloads, Lottie JSON, self-contained HTML — §11.4.215 applied to design). Tooling capability claims are RUNTIME-PROBED (§11.4.201/§11.4.99) with a headless-vs-operator-interactive matrix + an honest-limits section — a cloud push claimed without a real credentialed run is a fabrication. Operational rules (research-verified): the HEADLESS write path is the token-authenticated RPC API — access tokens (minted under the `enable-access-tokens` backend flag, handled per §11.4.10) are the MANDATED automation auth mechanism, never interactive OAuth relays; API-created PenPot files REQUIRE a browser hydration pass before handoff (thumbnails/text layout materialize in the browser runtime, not the write API) — an unhydrated API-created file handed off as render-ready is a §11.4 PASS-bluff at the design-handoff layer. Composes §11.4.6/.10/.28/.31/.69/.74/.76/.99/.161/.162/.201/.215/.216/.223. Propagation gate `CM-COVENANT-114-220-PROPAGATION` + recommended gate `CM-OPEN-DESIGN-`

`TOOLING-PRIMARY` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.220. Non-compliance is a release blocker. No escape hatch — no `--figma-is-source-of-truth`, `--cloud-only-design`, `--proprietary-format-source`, `--hand-rolled-penpot-stack`, `--oauth-relay-automation`, `--skip-hydration-pass`, `--unprobed-tooling-claim`, `--rootful-design-stack` flag.

§11.4.221 — Motion discipline: token-bound timing + machine-readable manifest + reduced-motion static fallbacks (User mandate, 2026-07-22). Compact summary: every duration/easing binds to the §11.4.216 motion tokens (an ad-hoc timing literal bypassing the token layer violates §11.4.6); every playable animation ships a STATIC reduced-motion fallback DERIVED FROM ITS OWN KEYFRAMES (poster frame for loopers, final frame for one-shots) in an open format, named per animation in a versioned MACHINE-READABLE motion manifest (id, loop mode, poster, reduced-motion mode, `reducedMotionFallback`) — a deliberate instant-cut is an EXPLICIT null-with-reason, never a silent absence; `prefers-reduced-motion: reduce` honored on EVERY surface via targeted component gates (never a global `*`), swapping in the manifest-named fallback; NO animation is claimed working without a REAL render in a real runtime (parse + structural schema + rendered geometry with progression assertions — the §11.4.107(10) analyzer discipline applied to motion assets), unexercised runtimes [OPEN]-tracked. Proving project: 6 hand-authored Lottie animations + 5 keyframe-derived fallback SVGs + manifest v2, lottie-web/jsdom + headless-Chromium validated, mobile runtimes honestly [OPEN]. Composes §11.4.6/.107(10)/.162/.170/.216/.218/.223. Propagation gate `CM-COVENANT-114-221-PROPAGATION` + recommended gate `CM-MOTION-TOKEN-BOUND-REDUCED-FALLBACK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.221. Non-compliance is a release blocker. No escape hatch — no `--ad-hoc-duration`, `--no-reduced-motion-fallback`, `--global-star-motion-kill`, `--manifest-optional`, `--unrendered-animation-claimed-working` flag.

§11.4.222 — Design-completion export wave: a design is not done until the exports land, honestly (User mandate, 2026-07-22). Compact summary: a design (system / brand / screen set / change wave) is NOT done until its EXPORT WAVE runs with verified outputs — (1) per-platform token codegen regenerated from the canonical source (or [OPEN]-tracked); (2) the FULL icon/asset raster matrix, documented per OS surface and locked 1:1 to its generator, produced by an

HONEST-OUTPUT pipeline: every candidate renderer PROBE-TESTED with a content-verified probe render before use (§11.4.201 — presence ≠ correctness), fall-through on probe failure, absent tools SKIP-with-note (§11.4.3/§11.4.69), NEVER a placeholder/blank/faked output, every raster verified non-degenerate (size floor + non-blank; multi-resolution bundles opened + verified per §11.4.38) — proving project: 83 real rasters, zero faked; (3) both-theme screen renders (deterministic theme forcing, every render verified non-blank) + the bound design book; (4) hand-off bundles with per-leg HONEST statuses (produced / deferred-operator-only / impossible-with-reason); (5) the §11.4.65/§11.4.73 doc twins regenerated. A design wave reported complete without its export wave — or with any faked/unverified export output — is a §11.4 PASS-bluff at the design-delivery layer. Composes §11.4.3/.6/.38/.65/.69/.73/.201/.216/.219/.220. Propagation gate `CM-COVENANT-114-222-PROPAGATION` + recommended gate `CM-DESIGN-EXPORT-WAVE-COMPLETE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.222. Non-compliance is a release blocker. No escape hatch — no `--skip-export-wave`, `--unprobed-renderer`, `--placeholder-raster-OK`, `--single-theme-renders`, `--fake-bundle-status`, `--stale-doc-twins` flag.

§11.4.223 — Provenance-marker discipline for design docs + central open-item registry (User mandate, 2026-07-22). Compact summary: every design document carries inline provenance markers from a closed, project-declared vocabulary whose MINIMUM CORE is `[VERIFIED]` (with evidence/check date) / `[OPERATOR]` / `[DEFAULT — adjustable]` / `[OPEN: id]` (extensions: `[RESEARCH]`, `[ASSUMED]`, `[BUILD-NEW]`, `[GAP: id]`, `[IN-HOUSE: …]`, `[CONSTITUTION §x]` per §11.4.35); an unmarked load-bearing claim reads UNVERIFIED, and an assumption presented as verified is a §11.4.6 violation of §11.4 PASS-bluff severity at the design-documentation layer; every unresolved decision is an `[OPEN: id]` with a stable, never-renumbered id (§11.4.54 discipline) owned by a named file; the design area index maintains the SINGLE CANONICAL open-item registry, TREE-VERIFIED in both directions (every tree id in the registry, no registry ghosts — the §11.4.186 consistency discipline applied to design opens), closures citing the closing revision, narrowed items recording exactly what closed and what remains; registry items map to tracked workable items (§11.4.15/§11.4.54/§11.4.93) so the §11.4.197 completion mandate applies — no design open is silently abandoned. Proving project: 41 registered ids across nine families, tree-verified, 2 closed with citations. Composes

§11.4.6/.15/.44/.54/.61/.93/.123/.186/.197/.212. Propagation gate `CM-COVENANT-114-223-PROPAGATION` + recommended gate `CM-DESIGN-DOC-PROVENANCE-MARKERS` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.223. Non-compliance is a release blocker. No escape hatch — no `--unmarked-claims-OK`, `--assumption-as-verified`, `--unregistered-open-item`, `--registry-optional`, `--silently-abandon-open` flag.

§11.4.224 — Test-first (TDD) for ALL work, not only fixes + a minimum code-coverage floor that is NECESSARY-never-sufficient (User mandate, 2026-07-22). Verbatim operator mandate: "Any work we do MUST START by writing the test! We MUST FULLY comply and follow the TDD - The test driven development! It MUST BE applied for any change, implementation, wiring, tests, challenges and any other form of codebase we write! Bash scripts as well! Coverage with TDD MUST BE close to 100%, not less than 85% !!!" **The gap closed:** §11.4.43's operative text binds FIXES only ("Every fix MUST follow the 5-step TDD-fix workflow"), so a NEW feature / wiring / gate / Challenge / BASH SCRIPT inherited no test-first obligation; and §11.4.27/§11.4.169's "100%" is test-TYPE breadth (which KINDS of test exist per feature × platform), stating no CODE-coverage figure anywhere — §11.4.224 closes exactly those two holes and claims nothing else as new. **(A) TEST-FIRST BINDS ALL WORK** — every work product with an executable surface (change, feature, implementation, wiring, refactor, gate, §1.1 mutation, Challenge, test-harness code, AND bash/shell scripts): the test is written FIRST, run FIRST, and OBSERVED TO FAIL for the right reason before the implementation exists (RED on the broken artifact per §11.4.115 where one exists; RED against the absent behaviour where the work is new); a test authored AFTER its implementation proves only that it AGREES with the code, never that it CATCHES the defect (the §11.4.43 PASS-bluff generalised). Per-work-class test named explicitly (§11.4.6): bash/shell script → an executing test through its REAL invocation path asserting exit status + output + state delta, NEVER a `bash -n` parse-check alone (parse-cleanliness is §11.4.67's separate obligation and proves nothing about behaviour); a pre-build GATE → its paired §1.1 mutation IS the test-first artifact, authored and observed to make the gate FAIL BEFORE the gate is trusted (§11.4.115(F) — observation-before-trust, NOT authorship order); a Challenge → its scored assertion against captured evidence (§11.4.5/§11.4.69); a behaviour-bearing config/data change (any committed data an executing consumer reads to decide behaviour) → an executing test that drives that CONSUMER

through the changed value and asserts the changed observable behaviour. **(B) CODE-COVERAGE FLOOR ≥ 85%, ~100% TARGET**, applying to the executable codebase INCLUDING bash/shell (the operator's "Bash scripts as well!"). **(C) NECESSARY, NEVER SUFFICIENT — anti-gaming (§11.4.201(8)/§11.4/§1.1):** MEASURED FACT (in-session, 2026-07-22) — a line-granular instrument reported an `if/else` line COVERED while the `else` branch was NEVER taken, and an assertion-free test raises the number identically to a proving one; therefore the REAL bar remains catch-its-own-negation proven by a paired mutation (a line executed by an assertion-free test is NOT covered in the sense that matters), the floor is a MINIMUM ON A PROXY, and citing "≥ 85% covered" as proof a change works is a §11.4 PASS-bluff at the metric layer; the floor NEVER stands alone (composes §1.1 + §11.4.5/§11.4.69/§11.4.107 captured evidence + §11.4.108 runtime signature). **(D) METRIC VALIDATION (§11.4.201(8)) — SPLIT VERDICT, honestly stated:** directional validation PERFORMED 2026-07-22 (at the correct end-state — every behaviour test-driven, every test catching its negation — the implementing lines ARE executed by their tests, so the metric DOES reach target; the floor is VALID as a NECESSARY condition); the CONVERSE is REFUTED by the clause-(C) measured gameability fact; and the EMPIRICAL per-corpus calibration of the 85% threshold is explicitly OWED, NOT claimed — every consuming project enforcing the floor MUST track it as a §11.4.197 item (this anchor asserts no project has already filed it — asserting an unverified tracker state would be the §11.4.6 violation the clause prevents); INTERIM PRECEDENCE mirroring §11.4.201(8)'s own §11.4.135-ratchet precedent — the floor KEEPS GATING while the calibration is owed (an uncalibrated floor blocking a merge is bounded, visible, operator-resolvable; disarming it silently re-opens the untested-code hole — the §11.4.101 reversible-safe choice), never a licence to switch it off. **(E) HOW MEASURED, per language, honestly (§11.4.6/§11.4.35):** the consuming project supplies its per-language instrument as DATA; the universal rule is that an instrument MUST exist and MUST actually be RUN — a floor nobody can measure is an unmeasurable gate and itself a §11.4.201 bluff, and a percentage nobody measured is a §11.4.6 fabrication. MEASURED host reality (2026-07-22): Go built-in coverage (`go test -coverprofile` + `go tool cover`) + `gcov` PRESENT, but `kcov` / `bashcov` / `bats` / `shellspec` / `shunit2` ALL PROBED ABSENT for bash (`shellcheck` IS present but is a LINTER, not a coverage instrument, and satisfies nothing here). PROVEN-FEASIBLE zero-tooling bash mechanism, measured in-session never asserted: `PS4='+COV:${BASH_SOURCE##*/}:${LINENO}:' bash -x <script>` emits executed line numbers, and executed ÷

executable (non-blank, non-comment) source lines yields line coverage — a probe correctly identified an uncovered function body, and subshell bodies ARE traced. HONEST LIMITS (measured, stated as fact): LINE coverage NOT branch coverage (the clause-(C) same-line `if/else` counted covered with one branch never taken); `set +x` regions and traps unaccounted — a FLOOR-measuring instrument only, so branch-level assurance requires a real tool (e.g. `kcov`) or an honestly-recorded shortfall. Where NO instrument exists for a language: INSTALL one, or record an §11.4.3/§11.4.69 SKIP-with-reason + a tracked §11.4.197 item — NEVER an invented percentage, NEVER a silently-dropped floor. EXCLUSION-LIST FENCE (amendment, remediation round of 2026-07-22 — the §11.4.135 exemption pattern applied to the coverage corpus): a corpus exclusion is legal ONLY via a CHECKED-IN exclusion list, every entry justified from the closed class set {generated-code (output of a committed generator per §11.4.77) | vendored-third-party | non-shipping-fixtures-and-golden-assets} and enumerated as honest gaps never silent; excluding FIRST-PARTY executable code additionally REQUIRES a tracked §11.4.197 item naming the plan to bring it into scope — an unlisted, unjustified, or un-tracked-first-party exclusion VOIDS the measured figure (a consumer that excludes most of its first-party executable code and truthfully measures ≥ 85% on the remainder has satisfied the floor in letter and voided it in substance — the silent-evasion channel this fence closes). **(F) HONEST BOUNDARY / SCOPE (§11.4.6):** "all work" = work products with an EXECUTABLE surface; a pure prose/governance/documentation edit has no executable surface and is NOT coverage-scoped (its discipline is §11.4.44/§11.4.65/§11.4.106/§11.4.212 + the §11.4.142 universal review — demanding a coverage percentage there would be a §11.4.201(1) false-positive refusal), BUT a governance change that SHIPS an executable artifact (gate, hook, script, mutation) is FULLY in scope for (A) and (B); test-first PRECEDES §11.4.108 four-layer verification / §11.4.40 full-suite retest / §11.4.185 manual-QA, never replaces them; and the CODE-coverage axis is DISTINCT from §11.4.27/§11.4.169's TEST-TYPE breadth — both apply, neither substitutes, satisfying one is never evidence for the other. Classification: universal (§11.4.17) — the consuming project supplies its per-language instruments, corpus scope, CHECKED-IN clause-(E)-fenced exclusion list, and threshold calibration per §11.4.35. Composes §11.4/.1/.5/.6/.27/.35/.43/.50/.66/.67/.69/.77/.85/.107/.108/.110/.115/.122/.135/.142/.146/.169/.185/. §1.1. Propagation gate `CM-COVENANT-114-224-PROPAGATION` (literal `11.4.224`) + recommended gates `CM-TEST-FIRST-ALL-WORK` + `CM-COVERAGE-FLOOR` (contract amended, remediation round of 2026-07-22: the measured numerator

counts ONLY RED-CAPABLE tests — every contributing test carries a recorded failing-first run OR a paired mutation observed to make it FAIL, zero-assertion contributors rejected by construction, sampling-based mutation verification an acceptable stated implementation — so ONE real RED test plus assertion-free padding to 85% FAILs the gate; and the gate FAILs on any exclusion that is unlisted, outside the clause-(E) closed class set, or first-party without its tracked §11.4.197 item; brownfield adoption is DELIBERATELY UNDEFINED — the owed gate-code item MUST surface the adoption question to the OPERATOR per §11.4.66 with concrete options {immediate hard floor | one-time §11.4.135-style monotone-decrease ratchet | per-corpus phase-in | changed-code-only with a scheduled full-corpus deadline} BEFORE first enforcement, recording the answer as consumer DATA — never an invented ratchet, a §11.4.122-class decision the operator owns) + paired §1.1 mutations; gate-code = a separate work item — the CONTRACT is landed, the gate CODE is NOT claimed shipped (§11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.224. Non-compliance is a release blocker. No escape hatch — no `--test-after-OK`, `--skip-red-first`, `--bash-exempt-from-tests`, `--coverage-not-applicable`, `--below-floor-OK`, `--coverage-proves-correct`, `--unmeasured-percentage-OK`, `--assertion-free-test-counts` flag.

§11.4.225 — Scheduler-quota burst-throttling telemetry + interactive-scope isolation from bursty fleets (research-derived, 2026-07-23). Compact summary: progressive / bursty / load-correlated interactive degradation MUST be diagnosed from the scheduling scope's THROTTLING ACCOUNTING — on cgroup v2, `cpu.stat` `nr_throttled` / `throttled_usec` as DELTAS over a measured window under load, as a fraction of enforcement periods, WITH a quiet-phase control — NEVER from averages: quota enforcement is per-period, so a workload far below quota ON AVERAGE can still throttle a large fraction of periods and stall every co-resident process, while CPU average / load average / RSS / thread counts all read HEALTHY (their quiet zeros are §11.4.201(6) FALSE-NULLS; forensic FACT 2026-07-22: a session scope at quota 9.6 CPU/100ms on a 64-CPU host, average demand 4.35 — UNDER quota — yet 16.9% of periods throttled during active phases, 0% quiet, worsening with session age, the co-located terminal-multiplexer server freezing ≥100ms per throttled period into multi-second interactive stalls); an "averages look healthy" verdict without throttle-accounting deltas is a §11.4.6 guess, and the same discipline binds every quota'd scheduler (container CPU limits, VM caps/steal). DESIGN half: an interactive-latency-sensitive process

(terminal server / TUI / editor / operator shell) MUST NOT share a fixed-quota NO-BURST scope with a bursty agent/build fleet — grant a measured burst allowance, isolate the interactive process in its own scope, or both; quota values are HOST-ADAPTIVE (computed from detected capacity per §12.11, §11.4.6 never hardcode — sibling FACT: a FIXED 2-CPU default quota on a 64-core host → 18.8% of periods throttled, 1757s forced idle, fixed by host-adaptive computation proven with a four-quadrant polarity test reading the LIVE cgroup back per §11.4.108); the THIRD orthogonal host-exhaustion axis beside §12.6 memory + §12.12 threads — no axis's instrument substitutes for another's. Honest boundary §11.4.6 — throttle telemetry proves WHERE periods were lost, not that throttling is the only degrader (RTT floors / GC load ranked separately in the founding investigation), and the design half never licenses removing a quota host policy requires (§12). Classification: universal (§11.4.17). Composes §11.4.6/.24/.102/.108/.128(1)/.201/§12.6/§12.8/§12.11/§12.12. Propagation gate `CM-COVENANT-114-225-PROPAGATION` (literal `11.4.225`) + recommended gates `CM-QUOTA-THROTTLE-TELEMETRY` + `CM-INTERACTIVE-SCOPE-NOT-QUOTA-STARVED` + paired §1.1 mutations (averages-only verdict on a progressive-degradation issue → telemetry gate FAILs; fixed low quota literal on a detected many-core host, or terminal server co-located with the fleet in a no-burst quota scope → isolation gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.225. Non-compliance is a release blocker. No escape hatch — no `--averages-prove-healthy`, `--skip-throttle-telemetry`, `--fixed-quota-default-OK`, `--interactive-in-fleet-scope-OK`, `--no-burst-needed`, `--skip-quiet-phase-control` flag.

EXTENSIONS landed 2026-07-23 (the systematic-debugging / measurement-semantics harvest — full text in `Constitution.md`; every lesson verified against captured evidence or re-demonstrated under control conditions before codifying, §11.4.6). §11.4.115(G) — defect-identity binding for CONSTRUCTED reproductions: a RED test's PRECONDITION must be TRACEABLE to the reported defect's real evidence (the reporter's sequence §11.4.199, a captured failure trace, or a probe proving the precondition occurs in the reported environment), recorded with the RED verdict as a precondition-provenance field { `observed` | `constructed` }; when NO working repro exists and the precondition is CONSTRUCTED, the fix mints at most DEFENSIVE HARDENING — labelled as such, NEVER closing the reported item (it stays OPEN §11.4.197); systematic-debugging refutation of every mechanism a pending fix

addresses REVOKES its done-claim on the spot (forensic FACT 2026-07-22: an exit-hang fix nearly shipped on a hand-made-lock-holder RED→GREEN; real-PTY §11.4.199-shape re-driving refuted every addressed mechanism — 6/6 clean exits — and the change was honestly re-labelled in-source "defensive hardening ... NOT a fix for the reported exit-hang"); strengthens (A) from artifact-synthetic to MECHANISM-synthetic; gate `CM-RED-PRECONDITION-DEFECT-TRACEABLE` (a `constructed` provenance on a defect-CLOSING claim FAILs; the same provenance on an honest hardening item PASSes — the §11.4.201(1) false-positive guard) + paired §1.1 mutation. **§11.4.201(9)(10)(11)(12)** — measurement-semantics quartet: (9) FIELD-IDENTITY + UNITS + SEMANTIC CLASS {capacity|occupancy|rate|cumulative|high-water} verified against the authoritative source definition before ANY summary-read metric gates or propagates — CAPACITY≠OCCUPANCY is the named canonical instance (forensic FACT: `rcv_wnd` 181120, a window-CAPACITY ceiling, misread as a "~1.4s standing send queue"; real Send-Q a few hundred bytes `app_limited`; propagated into THREE agent briefs + operator reports before the raw field was re-read, then publicly withdrawn — a downstream consumer of an unverified metric INHERITS the bluff); (10) OBSERVER DECONTAMINATION — an instrument contributing to its own measured population (probe sockets in a socket census, sampler load in a load profile) identifies + EXCLUDES its footprint AND re-proves the decontaminated instrument still sees via a (7)(b) control needle (FACT: TIME-WAIT 68–80 raw → 4–5 decontaminated, ~16× observer-induced, while the same scan still caught a real +264MB/1→93-socket step); composes §11.4.128(1) observer-effect budget + §11.4.119 single-resource-owner for measurement arms; (11) ARTIFACT-USABILITY-NOT-PREREQUISITE-PRESENCE — a readiness/install/health gate probes the ARTIFACT through its real invocation path (§11.4.224(A)), never a build prerequisite's presence (FACT: a `command -v <compiler>` gate turned a host holding a WORKING prebuilt into a false "cannot launch" exit-1 — "[ok] verified" and "[FAILED] cannot launch" in ONE transcript, review-graded BLOCKING; the proxy fails BOTH directions — clause (1) and (2) at once); (12) the SHELL-INSTRUMENT FOOTGUN CHECKLIST — NEW standing reference `docs/guides/shell_instrument_footguns.md` (inherited by reference §11.4.28/§11.4.177): exec-wrapper classes (`timeout` / `nice` / `env`) cannot run shell functions/aliases (rc=127 false refusal), `pgrep -f` self/carrier match (§11.4.196(D)/§12.12), delimiter-keyed extractors TRUNCATING the unit under test into a FALSE PASS, `bash -x` xtrace BLINDED by the code under test's

`exec ... 2>/dev/null` (countermeasure `BASH_XTRACEFD`), and a background watchdog subshell spawned INSIDE a `$(...)` command-substitution — it inherits the substitution's pipe WRITE-END, and an early disarm (`kill`) ORPHANS its `sleep` grandchild which keeps that write-end OPEN, so the `$(...)` BLOCKS for the FULL timeout budget after the real work completed instantly, every verdict and exit code CORRECT (countermeasure: redirect the watchdog subshell's fds away from the pipe — `( … ) >/dev/null 2>&1 &` — or have the probe write to a file) — all five DEMONSTRATED under control conditions 2026-07-23; every shell-touching §11.4.142/§11.4.209 review consults it, every shell-instrument zero stays bound by the (7)(b) class-matched needle; `CM-GUARD-ASSERTS-REAL-CONDITION` EXTENDED with invariants (iv) metric-semantics-verified / (v) census-decontaminated / (vi) artifact-probed-not-prerequisite / (vii) checklist-consulted + paired §1.1 mutations (gate-code = separate work item). **§11.4.67(6)** — sourced-function `exec` redirection-scoping: a command-less `exec` applies EVERY redirection on its line to the CURRENT shell PERMANENTLY, so in sourced/library functions an fd-manipulation `exec` MUST brace-scope its side redirections (`{ exec 9>>"$f"; } 2>/dev/null`); a bare `exec N>file ... 2>/dev/null` in sourced code is a review defect + greppable gate class (forensic FACT 2026-07-22: a sourced lock helper's bare `exec 9>>lock 2>/dev/null` silenced the operator's interactive-shell stderr for the shell's LIFETIME — one provider launch left the terminal unable to print any error — while the paired unlock had used the correct brace form all along; marker-probe proven pre/post-fix); gate `CM-SHELL-EXEC-REDIRECTION-SCOPED` (brace-scoped forms and `exec`-with-command exempt — the §11.4.201(1) false-positive guard) + paired §1.1 mutation. **§11.4.142** — teaching forensic FACT added (2026-07-22): a transport-hardening change was committed AND pushed to `main` BEFORE any independent review (honestly flagged); the post-hoc §11.4.209-substrate review returned NO-GO with ONE BLOCKING finding — an unsafe security-relevant default, already public on every mirror — plus 2 Important + 4 Minor, and remediation consumed a full fix-forward cycle a pre-push review would have folded into the original change; the rule needed NO change (§11.4.142/.125/.134/.209 already bind it with no carve-out) — the incident is cited as its newest forensic anchor; remediation fix-forward per §11.4.113, never a history rewrite. ALREADY-COVERED verdicts this round: measurement-arm single-resource-ownership + observer-effect budget (§11.4.119 + §11.4.128(1) — the decontamination instance landed as §11.4.201(10), not a new anchor); review-before-push as a rule (bound in full by §11.4.142 family — teaching

FACT only). All Classification: universal (§11.4.17) — every toolchain literal genericised; consumers supply scopes, quotas, instruments, and paths as DATA per §11.4.35. The literals `11.4.115` / `11.4.201` / `11.4.67` / `11.4.142` are unchanged — their propagation gates stay GREEN.

§11.4.226 — Evidence-class-at-closure + standing detection pressure: machinery presence does NOT predict whether a fix holds — the EVIDENCE CLASS at closure (runtime vs source) under real DETECTION PRESSURE does; prose does not bind, seams do (research-derived, 2026-07-23). Compact summary: the decisive finding of the 2026-07-22 reopen/first-touch root-cause forensics, confirmed across four independent evidence lines (internal forensics; a 76-source external literature pass; one cross-project corpus WITHOUT tracking machinery; one WITH exemplary machinery): **every fix whose done-claim was preceded by an on-target RED→GREEN polarity flip that held recorded ZERO reopens; every fix confirmed only at source-green bounced** (runtime-evidence corpus: median attempts-to-stick 1, ~0.3% corrected reopen — a floor under weak detection pressure; source-green corpus: 52% recorded, itself an undercount; the healthy corpus's ONLY bouncing family was a wrong-layer echo-assertion; 69 PENDING-BUILD claim-moves; the last audited release entered manual QA with 1/25 items user-layer-validated — the operator was the FIRST user-layer oracle most claims ever met, so first-touch rediscovery was the deterministic pipeline output, not bad luck). The mandate — the STRICT FORMALIZATION of §11.4.115(F)/§11.4.146(D3)/§11.4.108: (1) closed evidence-class RANKING {runtime > artifact > source} with a per-defect-layer floor at every closure seam — a user-visible/runtime defect can NEVER close on artifact/source-class evidence (source-on-source stays legal, the §11.4.201(1) false-refusal guard); (2) class proven by MACHINE FIELDS never a label (runtime ⇒ target fingerprint read FROM the target + runtime observable; artifact ⇒ path + content hash; source ⇒ ref; label-without-fields = SHAPE-INCOMPLETE, unreadable = BLIND); (3) ANTI-ECHO — a "runtime observable" that IS a grep/source-scan transcript is WRONG-LAYER, refused (a grep can never observe a pixel; the pixel-defect-by-five-greps closure dies at this seam); (4) per-status evidence PRECONDITIONS — every §11.4.15/§11.4.33 status carries a checked precondition, not only the terminal write (FACT: a penultimate "ready"-class status parked 79 items — 42% of the done-claiming population — as design-complete-but-UNVERIFIED; the operator's 6/6-broken sample was drawn from exactly that set); (5) FIX-KIND honesty — a state-only repair/mitigation cannot close a defect item as fixed (sibling-corpus FACT: 11

recurrence chains, median 3 / max 7 attempts, dominated by state-only "fixes"); (6) STANDING DETECTION PRESSURE — pressure is a CAUSAL INPUT to every measured quality rate (a rate without its pressure regime is incomparable, §11.4.6; the 52%-vs-~0.3% split is explained by pressure + closure class, not machinery; the healthy corpus's one recurrence surfaced at 24 days = the next SWEEP not the next use): every registered, topology-present guard carries a FRESHNESS CONTRACT (verdict on the CURRENT artifact fingerprint within a declared staleness budget); the never-executed/stale/over-budget set feeds a STANDING risk-ordered re-run queue (most-reopened-first §11.4.189, then stalest-first) drained by the zero-idle loop (§11.4.94/§11.4.97) as regular development-cycle work — the §11.4.135 release seam is the blocking BACKSTOP, never the only executor (FACT: two standing guards, finally executed during the forensics, immediately emitted the latent FAILs — one reproduced the operator's loudest defect 3/3; registration was treated as coverage while execution was nobody's job); topology-absent guards enumerated never queued, an empty registry is BLIND never "all fresh". Binding: applies during EVERY regular development cycle; CHECKED through every code-review per §11.4.194(6); on every constitution pull the §11.4.32 sweep + §11.4.164 hook analyze + apply the additions and violations are tracked + cleared (§11.4.15) — no parallel mechanism. Honest boundary §11.4.6: a measured correlation with a stated mechanism and zero counter-cases — NOT a randomized trial (partially-censored exposure carried openly); fabricated machine fields stay closed by §11.4.115(F) harness-written verdicts + the §11.4.135 candidate-fingerprint join (layered, not eliminated); oracle calibration stays §11.4.107(10); the layer taxonomy is authored data (mislabelling is the §11.4.194(6) (a) review class's detective target); re-running a weak guard proves little — oracle strength stays §1.1/§11.4.115(F), orthogonal discovery stays §11.4.85/§11.4.118. Reference mechanisms (proven, NOT wired §11.4.205): SOL-01/02/04/09 RED-first POCs with golden-good/golden-bad/negative-control fixtures under `docs/research/quality/solutions/` (65/65 GREEN, deterministic second iteration) — no force until their seams are wired by tracked §11.4.197 work items. Classification: universal (§11.4.17) — consumers supply layer bindings, evidence stores, verdict registries, and staleness budgets as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-226-PROPAGATION` (literal `11.4.226`) + recommended gates `CM-EVIDENCE-CLASS-AT-CLOSURE` + `CM-GUARD-FRESHNESS-SCHEDULER` + paired §1.1 mutations (close a runtime-layer item on source evidence / label-without-fields / echo-observable / stale guard with no queue entry → the gates FAIL; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped

§11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.226. Non-compliance is a release blocker. No escape hatch — no `--source-green-closes-runtime-defect`, `--label-is-class`, `--echo-observable-OK`, `--registration-is-coverage`, `--rate-without-pressure-regime`, `--mitigation-closes-defect`, `--skip-freshness-queue` flag.

§11.4.227 — Governance-corpus self-custody: the rule corpus is bound by the same custody it imposes — every named gate implemented-or-registered-deferral under a monotone ratchet, and propagation gates count anchor BLOCKS (exactly-once + lockstep-identical), never bare literals (research-derived, 2026-07-23). Compact summary: measured on THIS corpus 2026-07-22 (commands + control needles preserved in the root-cause analysis): 10,735 canonical lines, 220 distinct anchors; **413 named CM-* gates — 241 (58%) with no implementation in either canonical gate site — plus 85 literal "gate-code = separate work item" deferrals**; the custody seam designed 2026-07-17 was still prose on 2026-07-22 (the corpus's highest-value remediation fell into its own gap within five days); the fix-confirmation anchors predate the failures they failed to prevent by five-plus weeks — while **every fully-mechanised rule in the record either prevented or exposed its failure mode** (the PENDING-semantics + release verdict-coverage seams landed within a day of diagnosis and now block): the gap is not rule quality, it is the prose-to-seam conversion rate. The SHAPE defect: the propagation-gate family — the only family that runs everywhere — asserts literal-presence ≥ 1, mechanically rewarding restatement over implementation ("the corpus measures its own health in words present, and so it grows words"); consequences measured: two anchors duplicated verbatim-divergently in ALL FOUR mirrors while every presence-gate stayed green (the F7 incident); one anchor NUMBER heading two different mandates (the §11.4.140/ §11.4.141 collision, independently re-detected by the block-integrity instrument — resolution stays an operator-owned re-mint); two canonical entry-point mandates contradicting (the mandated resumption file unreachable from the mandated README); external corroboration: mandated-checklist adoption without workflow embedding moved nothing across 101 hospitals (the Ontario null). The mandate: **(A) NAMED-GATE LEDGER + RATCHET** — every `CM-*`-class token named in the corpus is, at every build, either IMPLEMENTED (token in an EXECUTABLE gate site; prose carriers never count §11.4.201(7)(a)) or covered by a REGISTERED DEFERRAL row pointing at a tracked §11.4.197 item; silent gate debt refused; the unimplemented count is MONOTONE-DECREASING (validated

against the definition of done per §11.4.201(8) — at the correct end-state it is 0); a vanished gate NAME without an explicit removal citation FAILs (the delete-names-to-lower-the-count gaming channel closed; repeals stay legal + visible, the §11.4.166 pattern); **an anchor's "done" state is its SEAM landing, not its TEXT landing** — a rule authored without its seam is in §11.4.205's worse-than-nothing state, and the ledger makes that state visible, counted, and shrinking; brownfield baseline = operator §11.4.66 decision (the §11.4.224(E) adoption-fence pattern), never an invented ratchet. **(B) ANCHOR-BLOCK INTEGRITY** — every propagation-class gate counts BLOCK-STARTS (structural line-anchored heading patterns), never bare literals: exactly ONE block per anchor per governance file (0 = absence, >1 = DUPLICATION, FAIL naming anchor + file); content-hash EQUALITY across the declared §11.4.157 lockstep mirror set (divergent copies FAIL naming both files; compact-vs-full variance ACROSS layers stays legitimate — canonical-vs-mirror drift is the §11.4.186 family's job); mid-body citations are CARRIERS and never count; ZERO blocks extracted = BLIND-OR-EMPTY never clean (§11.4.201(6)); an anchor NUMBER heading two mandates is a COLLISION and FAILs (§11.4.54 never-reuse applied to anchor ids); the extractor captures full dotted ids so a sub-anchor never prefix-matches its parent (a measured §11.4.201(1) false positive in the founding instrument, fixed RED-first). **(C)** semantic contradiction between two DIFFERENT anchors stays undecidable-by-hashing — only the decidable projections are mechanized (fenced scopes §11.4.112(5), seam declarations §11.4.120), the rest is §11.4.142/§11.4.194 review territory, stated never claimed; this anchor adds the enforcement-ratio INSTRUMENT, not a growth ban. Binding: the ledger regenerates + diffs on every governance edit + every build (regular development cycles, never an on-demand audit); review-checked per §11.4.194(6)(c); on every constitution pull the §11.4.32 sweep + §11.4.164 hook run the ledger + block-integrity checks and violations are tracked + cleared. Reference mechanisms (proven, NOT wired §11.4.205): SOL-05 + SOL-06 RED-first POCs under `docs/research/quality/solutions/` (SOL-05's live run reproduced the 413-gate census exactly; SOL-06's live run confirmed the four mirrors clean post-F7 + re-detected the 140/141 collision) — no force until wired by tracked work items. Classification: universal (§11.4.17) — consumers (and this governance repo itself) supply gate-site lists, deferral registries, and lockstep mirror sets as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-227-PROPAGATION` (literal `11.4.227` — itself REQUIRED block-start-shaped per (B)) + recommended gates `CM-GATE-LEDGER-RATCHET` + `CM-ANCHOR-BLOCK-INTEGRITY` + paired §1.1 mutations (name a gate with no implementation + no deferral / delete a name

without citation / duplicate a block / diverge two lockstep copies / re-mint a number → the gates FAIL; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.227. Non-compliance is a release blocker. No escape hatch — no `--name-gate-without-seam`, `--silent-gate-debt-OK`, `--delete-name-to-lower-count`, `--presence-gte-1-suffices`, `--duplicate-anchor-OK`, `--mirror-drift-OK`, `--reuse-anchor-number` flag.

EXTENSION landed 2026-07-23 (round 2 — the reopen/first-touch root-cause harvest; full text in `Constitution.md`). §11.4.194(6) — the quality-harvest review classes: every code-review (§11.4.142 universal / §11.4.125 gate / §11.4.209 Fable-xhigh substrate, iterated to GO per §11.4.134) MUST additionally CHECK, per reviewed change: (a) **closure-evidence class vs defect layer** (§11.4.226) — a terminal/ready-class status moving on evidence below the defect layer's floor, a user-visible property "verified" by file-existence + greps, an echo-assertion, or a state-only repair closing a defect as fixed is a FINDING; (b) **needled measurements** — every load-bearing measurement in the change/its tests satisfies §11.4.201(6)–(12), and NO count was converted into a finding without reading the underlying lines — a COUNT IS A LEAD, THE LINES ARE THE FINDING (forensic FACT: twice in one measured cycle a count was read as state — a "mixed half-written doc" that was actually coherent, and an inflated FAIL total that counted its own summary line); (c) **governance-and-gate integrity** (§11.4.227) — a change NAMING a gate lands its implementation or registered deferral in the same change; a governance edit preserves anchor-block integrity; (d) **reviewer-authored adversarial mutations** — the author's self-authored mutation set is NECESSARY but NOT SUFFICIENT (forensic FACT, from the dated forensic corpus cited in the 2026-07-22 root-cause analysis: an author's six self-authored mutations were ALL caught while an independent reviewer's FIRST THREE different mutations ALL sailed through, each restoring a user-visible defect); the reviewer MUST attempt ≥1 mutation the author did not write (or record honest infeasibility per §11.4.3) — a guard whose reviewer-authored mutation survives is unvalidated instrumentation per §11.4.115(F) and a FINDING. The literal `11.4.194` is unchanged — its propagation gate stays GREEN.

§11.4.228 — Cross-agent extension lifecycle mandate: per-platform compatibility, source tracking, documentation with compatibility matrix, and auto-wiring on constitution pull (research-derived from Extensions Catalog forensics, 2026-07-24). Compact summary: every governed extension (Skill, MCP,

ACP, LSP, plugin) MUST (A) ship a per-platform compatibility declaration (Claude Code, OpenCode, Gemini CLI, Qwen Code minimum) with closed vocabulary `{SUPPORTED|UNTESTED|UNSUPPORTED|CONFIG-ONLY}`, NEVER cause loops/ broken behavior in any platform, and carry a per-platform liveness test §11.4.169; (B) be traceable to its exact canonical origin (repo + commit SHA + path, recorded in `EXTENSION_SOURCE.yaml` — symlinks preferred over copies, blind copies without provenance are §11.4.124 dead-code-in-waiting), with a per-project extensions catalog (`docs/extensions/EXTENSIONS_CATALOG.md`) as the single source of truth, §11.4.86 roster/corpus-backed fingerprint; (C) the extensions catalog MUST carry a per-platform compatibility matrix + liveliness-testing evidence per extension × platform per §11.4.69 `feature_class=extension_loading`, four-format export per §11.4.65, auto-synced via §11.4.106 docs-chain; (D) the §11.4.164 `post_update_hook.sh` is EXTENDED to auto-detect all governed agent platforms, symlink constitution skills into each (`.claude/skills/` , `.opencode/skills/` , platform equivalents), register MCP servers into each platform's config, self-validate, and be idempotent — closing the gap where 4 constitution skills exist in source but are un-wired to any platform (the §11.4.108 source-present-runtime-absent gap). Honest boundary §11.4.6 — wiring guarantees REACHABILITY, never runtime acceptance. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-228-PROPAGATION` + recommended gates `CM-EXTENSION-PER-PLATFORM-COMPAT` / `CM-EXTENSION-SOURCE-TRACKED` / `CM-EXTENSION-CATALOG-DOCUMENTED` / `CM-CONSTITUTION-EXTENSIONS-AUTO-WIRED` + paired §1.1 mutations. Full text in `Constitution.md` §11.4.228.

§11.4.229 — Live in-session task/todo tracker MUST always be up to date and fully in sync with the real work state — never stale, never completed-shown-as-pending nor pending-shown-as-done (User mandate, 2026-07-25). Verbatim operator mandate: "make sure it is ALWAYS up to date fully in sync! CRITICAL: This MUST BE one of mandatory rules / constraints! Add it with all relevant details into the constitution Submodule and relevant related files." The agent's LIVE IN-SESSION TASK/TODO TRACKER — the harness-provided task list / `TodoWrite`-class todo tool / any equivalent moment-to-moment execution-state surface the runtime exposes DURING a session — MUST ALWAYS be kept up to date and FULLY IN SYNC with the real live work state: never stale, never showing completed work as pending, never showing pending / blocked / abandoned work as done or in-progress; updated the MOMENT any tracked task's state changes

(started → in-progress, completed → done, blocked, abandoned, split, re-scoped, newly discovered), never batched to a later checkpoint, never reconstructed from memory at the end, never left reflecting a superseded plan — a live tracker that lags the real work is a §11.4 PASS-bluff at the execution-transparency layer (an operator or resuming conductor per §11.4.116 makes decisions on a false picture of what is done / in-flight / owed). (A) every state-change updates it IMMEDIATELY (exactly one in-progress where the surface models that; no completed-marked-pending, no pending-marked-done, no ghost task for dropped work — dropped work is closed with a reason); (B) NEVER stale, NEVER inverted (§11.4.6 — the tracker is captured FACT, falsely-done = PASS-bluff / falsely-pending = §11.4.1 FAIL-bluff, drift corrected the moment observed); (C) DISTINCT artifact that COMPLEMENTS never REPLACES §12.10 (committed CONTINUATION doc) / §11.4.131 (standing committed session-resumption FILE) / §11.4.97 (operator-facing milestone progress cadence) / §11.4.106(F) (doc/DB sync at commit/build/constitution-pull write-seams) / §11.4.202 / §11.4.208 (workable-item DB + request-history ledgers) — those bind committed/cross-session/operator-report/tracked-work artefacts, NONE binds the live in-session tracker itself; it is the in-session analogue of §12.10's always-current mandate applied to the harness task list, the truth those artefacts are materialised FROM; (D) honest boundary §11.4.6 — where no such surface exists this is honestly N/A (§11.4.3 SKIP-with-reason, latent-binding per §11.4.96), and "in sync" = reflects the real work state as the agent knows it now, not omniscience about unobserved work. Classification: universal (§11.4.17). Composes §12.10 / §11.4.131 / §11.4.97 / §11.4.106(F) / §11.4.126 / §11.4.116 / §11.4.202 / §11.4.208 / §11.4.6 / §11.4.1. Propagation gate `CM-COVENANT-114-229-PROPAGATION` (literal `11.4.229`) + recommended gate `CM-LIVE-TASK-TRACKER-IN-SYNC` + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6). No escape hatch — no `--tracker-may-lag`, `--batch-todo-updates`, `--stale-task-list-OK`, `--reconstruct-todos-at-end`, `--skip-task-tracker-sync`, `--pending-may-be-done`, `--completed-may-stay-pending` flag. Full text in `Constitution.md` §11.4.229.

§11.4.230 — Parallelized-pipeline methodology: build↔validate overlap + affected-stages-only re-run + parallel multi-device testing/recording + fan-out-subagents-always (research-derived, 2026-07-26).

Forensic motivation (genericised): the sequential pipeline shape — build FULLY completes → THEN validation begins → on ANY discovery, restart the WHOLE cycle from scratch → all devices tested one-at-a-time — serialises the operator's

wall-clock onto build latency + one-device-at-a-time evidence gathering, so a multi-hour containerised build followed by serial per-device validation and a from-scratch restart-on-discovery makes time-to-a-manual-QA-capable-release a multiple of what the genuinely-parallel work requires. §11.4.58's PWU pipeline already overlaps Stage 1 of round N+1 with Stages 4–5 of round N; §11.4.89 already backgrounds long tests; §11.4.103 already mandates ≥3 auto-backfilled parallel streams; §11.4.119 already partitions shared hardware by single-owner; §11.4.128/§11.4.144 already mandate always-on recording + availability-following. §11.4.230 BINDS those into one operating discipline and adds the three invariants they did not each state on their own: (A) the build↔validate CONCURRENCY seam + affected-stages-only re-run-on-discovery, (B) all-devices-concurrent + worker-pool recording/processing, (C) fan-out-subagents as the standing default. It is the parallelization capstone the way §11.4.103 is the continuous-parallel-stream capstone — a strengthening bind, never a weakening.

(A) PIPELINE-OVERLAP PARALLELIZATION. Validation/verification work MUST run CONCURRENTLY WITH the build wherever a validation stage does NOT depend on the build's own not-yet-produced artifact — never deferred until the build finishes when it could have run alongside it. Deploy the moment the build is done (build-completion is the deploy trigger, per §11.4.121 no-commit-while-build-writes-tracked-artifacts + §11.4.108 artifact-must-exist ordering — the deploy waits for the artifact, the deploy-independent validation does not). Concretely, WHILE a build runs, the following run in parallel per the §11.4.96 SAFE-during-build catalogue: pre-build source-layer gates + meta-test mutation sweeps + coverage/clash checks (§11.4.110), the §11.4.125/§11.4.142/§11.4.194 independent code-review on the batch's SOURCE, doc/DB sync (§11.4.106), and on-device validation of the PRIOR build (device-independent of the in-flight build, §11.4.119). On any NON-huge-blocker discovery, the cycle MUST: (1) bump the version code (a new, distinctly-identifiable candidate artifact — never re-use the failing artifact's id, §11.4.111 stable-name + §11.4.200 target-identity); (2) fix at the proven root cause (§11.4.102 systematic-debugging first, §11.4.115 RED-on-the-broken-artifact); (3) RE-RUN ONLY THE AFFECTED STAGES, PARALLELIZED — the stages whose inputs the fix changed (its own targeted validation FIRST per §11.4.130, then the affected regression set), NOT a serial restart-from-scratch of the entire cycle. The explicit efficiency target is a **4–5× reduction in time-to-a-manual-QA-capable-release** versus the fully-sequential shape — a measured target, tracked as build-and-validate wall-clock deltas (§11.4.24 build-resource stats), never an assumed one

(§11.4.6). **HONEST BOUNDARY (§11.4.6 / §11.4.108 / §11.4.120):** the CURRENT build's ARTIFACT-layer (§11.4.108 layer 2) and RUNTIME-ON-CLEAN-TARGET-layer (layer 3) validation CANNOT precede its own artifact — a gate whose precondition is the build's output belongs at a post-build seam, never overlapped onto authorship (§11.4.120 seam-placement; §11.4.69 `artifact_not_yet_built`). "Concurrent with the build" therefore means the deploy-INDEPENDENT stages overlap; the CURRENT-artifact runtime validation begins the moment that artifact exists, in parallel across devices per (B). **"Affected-stages-only re-run" is bounded to NON-huge-blocker discovery**: a HUGE BLOCKER (release-blocking-severity defect / regression invalidating the cycle / blast radius reaching the batch's other fixes) still triggers the §11.4.129 STOP → fix-all → full-RESTART protocol (restart, never resume) — §11.4.230(A) does NOT weaken §11.4.129, §11.4.40 full-suite-retest-before-tag, §11.4.130 validate-the-fix-first, §11.4.132 risk-ordered validation, or §11.4.185 manual-QA-final; it narrows the from-scratch restart to exactly the cases those anchors require it, and parallelises everything else.

(B) PARALLEL MULTI-DEVICE TESTING + RECORDING/PROCESSING. Every available, reachable test device MUST be exercised CONCURRENTLY — one validation stream per device, never one-device-at-a-time when the devices are independent — each stream the SINGLE EXCLUSIVE OWNER of that device's exclusive resources per §11.4.119 (the owner drives; any concurrent stream targeting the same device is read-only), so per-device evidence is never cross-contaminated (§11.4.119 evidence-integrity). Every device is ALWAYS-RECORDED per §11.4.128 (non-invasive, side-effect-free, deterministic layout) and AVAILABILITY-FOLLOWED per §11.4.144 (a dropped device logs an honest offline event + reconnect-and-resume, never a silent corpus hole, never a faked continuity). Recording CAPTURE + ANALYSIS/PROCESSING MUST be fanned out through a BOUNDED WORKER-POOL (a Go worker-pool or platform equivalent) so N devices' recordings are captured + analysed in parallel and per-device evidence never serialises onto a single processor — the pool bounded by §12.6 memory + §12.12 thread headroom. Per-device PASS cites its own captured-evidence artefact (§11.4.5/§11.4.69/§11.4.107) on a CLEAN/fresh deployment (§11.4.108 runtime-signature, §11.4.200 verify-deployed-artifact-identity-per-device); the highest-risk / most-reopened cases (§11.4.132/§11.4.189) get the deepest scrutiny FIRST on every device. **HONEST BOUNDARY (§11.4.6):** genuinely-shared exclusive resources (one HDMI/audio sink, one media-playback

path) serialise per §11.4.119 single-owner-over-time even across devices; "all devices concurrent" means independent devices, never two owners of one exclusive resource.

(C) FAN-OUT-SUBAGENTS-ALWAYS. Subagent fan-out (§11.4.20/§11.4.70 subagent-driven; §11.4.187 multi-track ruler; §11.4.192 auto-backfill) is the STANDING DEFAULT for EVERY genuinely-parallelizable effort — batch investigation, test-authoring, gate-authoring, per-device validation (B), per-PWU development (§11.4.58), doc/research work — not a per-request opt-in and not reserved for "big" tasks. A parallelizable effort executed monolithically in the foreground when a fan-out path exists is a §11.4.230(C) violation (the §11.4.94/§11.4.97/§11.4.103 zero-idle discipline applied to HOW work is dispatched, not only WHETHER it runs). **HONEST BOUNDARY (§11.4.6 / §11.4.183):** "always fan out" is BOUNDED by (1) genuine parallelizability (contending/redundant/serially-dependent subagents are NOT fan-out — they are waste, and spawning them is NOT maximization per §11.4.183 maximum-USEFUL-throughput); (2) §12.6 60% memory ceiling; (3) §12.12 `RLIMIT_NPROC` thread headroom; (4) §11.4.58 ≤6 concurrent-agent cap; (5) §11.4.119 single-resource-owner (two subagents may not both drive one exclusive device); (6) critical-state sequencing (commits/pushes/tags stay serial per §11.4.88/§11.4.113). The bar is maximum USEFUL parallel throughput, never agent count; fan-out that exceeds a host-safety ceiling or contends a single resource is a violation, not compliance.

Classification: universal (§11.4.17) — a platform-neutral parallelization discipline reusable by ANY project that builds an artifact, validates it across multiple targets, and records device evidence; the consuming project supplies its concrete device set + serials, build/flash tooling, recording layout, and worker-pool implementation per §11.4.35. Composes §11.4.20 / §11.4.24 (wall-clock stats prove the 4–5× target) / §11.4.40 / §11.4.58 (the PWU pipeline this binds + extends) / §11.4.69 / §11.4.70 / §11.4.88 / §11.4.89 / §11.4.94 / §11.4.96 (SAFE-during-build catalogue) / §11.4.97 / §11.4.102 / §11.4.103 / §11.4.108 (artifact/runtime layer ordering — the honest boundary) / §11.4.110 / §11.4.115 / §11.4.119 (single-resource-owner — B's load-bearing partition) / §11.4.120 (seam-placement — A's honest boundary) / §11.4.121 / §11.4.125 / §11.4.126 / §11.4.128 (always-on recording — B's evidence source) / §11.4.129 (huge-blocker full-restart — the un-weakened carve-out) / §11.4.130 / §11.4.132 / §11.4.144 (availability-following — B's drop-handling) / §11.4.183 (maximum-USEFUL-parallelism — C's honest boundary) / §11.4.185 (manual-QA-final — the un-weakened terminal gate) / §11.4.187 / §11.4.192 /

§11.4.200 (per-device deployed-artifact verification) / §12.6 / §12.12. Propagation gate `CM-COVENANT-114-230-PROPAGATION` (literal `11.4.230` across the consumer fleet) + recommended gates `CM-PIPELINE-OVERLAP-VALIDATION` (a build that runs with zero deploy-independent validation stages running concurrently when SAFE-catalogue work is queued → FAIL; a NON-huge-blocker discovery that restarts the whole cycle from scratch instead of re-running only the affected stages → FAIL) + `CM-PARALLEL-DEVICE-WORKER-POOL` (available independent devices validated serially instead of concurrently, OR recording capture/analysis serialised onto one processor when a worker-pool path exists → FAIL) + `CM-FAN-OUT-SUBAGENTS-DEFAULT` (a genuinely-parallelizable effort run monolithically in the foreground when a fan-out path exists AND all host-safety ceilings have headroom → FAIL) + paired §1.1 meta-test mutations (per §1.1; gate-code = separate work item, NOT claimed shipped §11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.230.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--sequential-build-then-validate`, `--restart-whole-cycle-on-discovery`, `--serial-device-testing`, `--serialise-recording-processing`, `--no-fan-out`, `--monolithic-parallelizable-work` flag exists.

§11.4.231 — Nano-precision model-tier selection: lightest-capable-model-first with dynamic mid-task escalation (User mandate, 2026-07-26, `CRITICAL`).

Verbatim operator mandate: every work effort MUST use the LIGHTEST CAPABLE model, escalating tier only as complexity rises — a nano-precision, per-task model-tier choice, NOT "everything on the heaviest model." Forensic motivation (genericised, §11.4.6): dispatching every task — a trivial doc-sync, a one-line status read, a mechanical format fix — on the heaviest available model wastes capacity + token budget + wall-clock that a lighter tier would spend identically well, while the OPPOSITE failure (a hard multi-factor root-cause investigation dispatched on a tier too light to reason through it) produces exactly the shallow, guessed, or wrong-conclusion work §11.4.6 / §11.4.102 / §11.4.194 forbid. Neither "always heaviest" nor "always lightest" is compliant; the correct discipline is a PER-TASK, EVIDENCE-BASED tier choice that tracks the task's own complexity, re-evaluated as that complexity becomes known — the nano-precision selection this anchor mandates.

(A) LIGHTEST-CAPABLE-FIRST + ESCALATE-ON-COMPLEXITY. Every dispatched work effort — main-stream or subagent (§11.4.20/§11.4.70), single-track or multitrack (§11.4.187/§11.4.192) — is assigned the LIGHTEST model tier from the closed, capability-ordered ladder `haiku < sonnet < opus` genuinely sufficient to complete it correctly (the review/merge-conflict-resolution substrate is a SEPARATE, already-pinned tier per clause (E), not a fourth rung of this ladder). Tier selection is a DECIDABLE heuristic, not a guess (§11.4.6) — evaluated against, at minimum: (i) **task scope** (single mechanical edit / focused single-file change / multi-file-or-multi-component change); (ii) **reasoning depth** (a lookup-and-apply action vs. a multi-factor root-cause investigation per §11.4.102/§11.4.194); (iii) **blast radius** (a change touching one isolated file vs. one crossing shared state, hardware safety §11.4.133, or governance/gate seams §11.4.227); (iv) **novelty** (a previously-solved, pattern-matched task vs. a genuinely new problem with no established precedent). Closed-set tier bindings (illustrative, non-exhaustive — the consuming project supplies its own precise mapping per §11.4.35): **`haiku`** — trivial/mechanical work (simple greps/searches, doc-sync regeneration, status/field reads, format/lint fixes, boilerplate scaffolding); **`sonnet`** — moderate work (a focused single-component debug, a bounded single-file implementation, straightforward research or audit reads); **`opus`** — complex work (architecture/design decisions, multi-factor reasoning per §11.4.194(1), cross-feature blast-radius analysis per §11.4.92 Pass 2/3, hard root-cause investigation per §11.4.102 that resists a first-pass hypothesis). A task whose complexity is genuinely UNDECIDABLE from available evidence starts at the NEXT tier UP from the best available estimate (the safe-reversible default per §11.4.101) rather than guessed downward.

(B) DYNAMIC MID-FLIGHT SWITCHING. Tier assignment is NOT a one-time, start-of-task decision — it is RE-ASSESSED continuously as the task's real complexity becomes known during execution. A task DISPATCHED as simple that PROVES harder mid-flight (an unexpected multi-factor interaction surfaces, a root cause resists the first two hypotheses, the blast radius turns out to cross a governance/hardware-safety seam) MUST ESCALATE to the next-heavier tier for its remainder — continuing on an under-powered tier past the point complexity is KNOWN is a clause-(D) false economy, not compliance. Conversely, a task DISPATCHED as heavy that DECOMPOSES into genuinely light, bounded, low-blast-radius subtasks (mechanical file edits following an already-designed pattern, doc regeneration, per-item status reads) dispatches THOSE subtasks on a

LIGHTER tier via subagent fan-out (§11.4.20/§11.4.70/§11.4.183) rather than running every subtask on the parent's heavier tier by default — capability-matching applies at EVERY dispatch granularity, not only the top-level task.

(C) ALWAYS-ON COMPOSITION, NEVER A PER-REQUEST OPT-IN. Nano-precision tier selection is a STANDING DEFAULT mechanism, engaged automatically for EVERY work effort from the first prompt of a session onward (mirrors §11.4.126 default-autonomous-loop / §11.4.198 default-mechanisms-always-on) — applied TOGETHER WITH, never as a substitute for: §11.4.141 (heavy token-optimization — a lighter model dispatched with a bloated context is not the efficiency this anchor targets); §11.4.183 (maximum-USEFUL-parallelism — tier-matching each parallel stream, not merely running more streams on one fixed tier); §11.4.187/§11.4.192 (the multi-track ruler orchestration + continuous auto-backfill — each track's worker is tier-assigned per this anchor, not defaulted to one blanket tier project-wide); and §11.4.196/§11.4.198 (native-alias-first priority + always-on default mechanisms — this anchor governs WHICH TIER within whichever alias/account is selected, composing with, never replacing, §11.4.196's ALIAS-layer priority). None of these five composed anchors is weakened, duplicated, or re-specified by §11.4.231 — this anchor adds the TIER axis to their existing ALIAS/PARALLELISM/TOKEN axes.

(D) HONEST BOUNDARY (§11.4.6) — "LIGHTEST CAPABLE" MEANS CAPABLE, NOT CHEAPEST-REGARDLESS. "Lightest capable" is a CONJUNCTION, not a discount: a tier choice that fails to actually complete the task correctly — producing a shallow investigation, a guessed root cause, an under-verified fix, or ANY form of bluff per the §11.4 anti-bluff covenant — is NOT "lightest capable," it is UNDER-POWERED, and under-powering a task that then fails, bluffs, or requires costly rework is a FALSE ECONOMY and a §11.4 violation of equal severity to any other PASS-bluff (the token/time saved at dispatch is spent many times over in re-investigation, re-review, and the risk of a shipped defect). Capability determinations are made from the clause-(A) decidable heuristic + observed task outcomes — EVIDENCE-BASED, never guessed (§11.4.6): "this always works fine on `haiku`" without a captured basis is exactly the guess §11.4.6 forbids, and a tier's genuine capability for a task class is established by the same anti-bluff discipline (§11.4.5/§11.4.69/§11.4.107) that governs every other claim in this constitution. A platform-chosen serving model outside the `haiku < sonnet <`

`opus` ladder (e.g. `fable` served for general non-review work) is NOT a §11.4.231 dispatch-tier violation — the ladder binds the DISPATCHER's tier ASSIGNMENT, not the platform's own per-turn serving choice.

(E) HARD PINS OVERRIDE THIS DEFAULT FOR THEIR SCOPES. §11.4.231 sets the DEFAULT working tier for NON-review, non-merge-conflict-resolution work. It does NOT relax, override, or compete with the two existing hard model-substrate pins: §11.4.209 (every mandatory independent code-review — §11.4.125/ §11.4.142/§11.4.194 iterated to GO per §11.4.134 — runs on **Fable at `xhigh` effort**, **Opus at `xhigh`** on genuine Fable-unavailability, NEVER a lower tier or lower effort) and §11.4.211 (every main→feature/product/flavor merge-CONFLICT resolution runs on the SAME Fable-xhigh / Opus-xhigh substrate). Those two anchors are STRICTER, SCOPE-SPECIFIC pins that this anchor's general ladder does not reach — review and merge-conflict-resolution are NOT eligible for haiku/ sonnet/opus-by-complexity dispatch; they run on their pinned substrate ALWAYS, regardless of how "simple" the reviewed change or the conflict appears. A review or merge-conflict resolution dispatched on any tier other than the §11.4.209/§11.4.211 pin (absent a genuinely-proven Fable-unavailability fallback) is a violation of THOSE anchors, not a valid application of §11.4.231's general ladder.

**(F) EFFORT-TIER SELECTION — the SAME nano-precision discipline applied to the reasoning-EFFORT axis (operator IMPORTANT mandate, 2026-07-26; verbatim: "Besides dynamic determining proper models to use, we MUST decide the effort as well and switch between the effort settings same as we do now for chosen model! Labeling MUST BE extended for this details as well. Example: (T1/main - claude4 - opus - high)."). ** Every dispatched work effort is assigned not only a model tier per clauses (A)-(E) but ALSO the LIGHTEST reasoning-EFFORT setting from the closed ladder `low < medium < high < xhigh < max` genuinely sufficient to complete the task correctly — chosen by the SAME clause-(A) decidable heuristic (scope / reasoning-depth / blast-radius / novelty), NOT a guess (§11.4.6). Illustrative bindings (non-exhaustive; the consuming project supplies its precise mapping per §11.4.35): `low` / `medium` — trivial / mechanical work (doc-sync regeneration, status / field reads, format / lint fixes, boilerplate); `high` — moderate-to-complex reasoning (focused single-component debug, multi-factor analysis per §11.4.102 / §11.4.194); `xhigh` / `max` — the HARDEST verification / merge-conflict / architecture work. (F.1) DYNAMIC MID-TASK ESCALATION — effort, like model tier per clause (B), is RE-ASSESSED continuously as the task's real complexity becomes known; a task

dispatched at a lower effort that PROVES harder mid-flight MUST ESCALATE its effort for the remainder (continuing under-effort past the point complexity is KNOWN is the clause-(D) false economy), and a heavy task decomposing into light bounded subtasks dispatches THOSE at a lighter effort — effort-matching applies at EVERY dispatch granularity, not only the top-level task. (F.2) HONEST HARNESS BOUNDARY (§11.4.6, load-bearing — do NOT overstate the mechanism) — effort-tiering is REAL where the execution path exposes an effort control and a DOCUMENTED CAPABILITY-GAP where it does not: the current Claude Code **Agent tool** (subagent dispatch) exposes a `model` parameter but **NO `effort` parameter**, so a subagent's effort is NOT presently settable via that path (the §11.4.182 `<effort>` label field degrades honestly to `?` there); the **Workflow tool's `agent()`** DOES take an `effort` argument, so effort IS settable there — the label reports whichever path is live, never fakes one. This is the SAME aspirational-vs-harness state the clause-(E) §11.4.209 / §11.4.211 Fable-`xhigh` pins already occupy: the pins mandate the effort, and whether a dispatch path can SET it is a harness capability reported honestly, never faked (§11.4.182). (F.3) HARD-PIN FLOOR PRESERVED — clause (E)'s hard pins are ALSO effort pins: every §11.4.209 mandatory independent code-review AND every §11.4.211 merge-conflict resolution runs at `xhigh` effort (Fable, or Opus at `xhigh` on genuine Fable-unavailability), and this general effort ladder NEVER tiers those two scopes below `xhigh` regardless of how "simple" the reviewed change or the conflict appears — a review / merge-conflict resolution run below `xhigh` is a §11.4.209 / §11.4.211 violation, not a valid application of this ladder. (F.4) HONEST BOUNDARY (§11.4.6) — "lightest capable effort" is a CONJUNCTION, not a discount: an effort setting that yields a shallow / guessed / under-verified result is UNDER-POWERED, a FALSE ECONOMY, and a §11.4 violation of PASS-bluff severity, exactly as an under-powered MODEL tier is per clause (D); capability determinations are evidence-based, never guessed (§11.4.6).

Classification: universal (§11.4.17) — a platform-neutral model-capacity-matching discipline reusable by any project dispatching work across a tiered model roster; the consuming project supplies its own precise tier↔task-class mapping and its own model-name ladder per §11.4.35 (the `haiku` / `sonnet` / `opus` ladder and the Fable/Opus-xhigh pins are this project's current concrete instantiation, not a universally-fixed vocabulary — a consumer with a differently-named tier roster maps its own ladder onto the SAME lightest-capable-first + dynamic-escalation + always-on-composition + honest-boundary + hard-pin-override structure).

Composes §11.4.6 (no-guessing — capability determinations are evidence-based) / §11.4.20 / §11.4.70 (subagent-driven dispatch — the granularity at which tier-matching applies) / §11.4.92 (blast-radius analysis feeds the clause-(A) heuristic) / §11.4.101 (safe-reversible default on undecidable complexity — escalate up, never guess down) / §11.4.102 / §11.4.194 (reasoning-depth signals for the heuristic) / §11.4.126 / §11.4.141 / §11.4.183 / §11.4.187 / §11.4.192 / §11.4.196 / §11.4.198 (the always-on composition set, clause (C)) / §11.4.209 / §11.4.211 (the hard-pinned review/merge-conflict substrate this anchor does NOT touch, clause (E)). Propagation gate `CM-COVENANT-114-231-PROPAGATION` (literal `11.4.231` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the consumer fleet per §11.4.227(B)) + recommended gate `CM-MODEL-TIER-LIGHTEST-CAPABLE` (a dispatched effort whose logged/declared model tier exceeds the clause-(A) heuristic's indicated tier for a task later confirmed trivial-and-successful-at-the-lighter-tier is a WARN/candidate finding — never a hard FAIL on tier choice alone, since tier-adequacy is only provable in retrospect per clause (D); the gate instead HARD-FAILS the two decidable violations: (i) a review or merge-conflict-resolution effort NOT run on the §11.4.209/§11.4.211 pinned substrate, and (ii) a task whose complexity was PROVEN mid-flight [an escalation event was logged, or a downstream failure/bluff was traced back to it] that did NOT escalate tier before continuing) + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped — §11.4.6; mutation plan: (1) dispatch a code-review labeled as running on `sonnet` instead of Fable-xhigh → gate MUST FAIL; (2) inject a logged mid-task complexity-escalation event with NO corresponding tier-escalation record → gate MUST FAIL; (3) golden-FALSE fixture: a task that stays on its initially-assigned tier throughout, with no escalation event logged → gate MUST NOT fire, per §11.4.201(1) the false-positive guard).

Canonical authority: constitution submodule `Constitution.md` §11.4.231.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--always-heaviest-model`, `--always-lightest-model`, `--skip-complexity-reassessment`, `--fixed-tier-per-session`, `--relax-review-pin`, `--relax-merge-conflict-pin`, `--per-request-tier-opt-in` flag exists.

§11.4.232 — Anti-mess long-op orchestration: every long-op is registered, single-owned-per-purpose, liveness-PROVEN, handed-off-on-stop, safely-reaped, consistency-checked, and escape-bounded — the single source of truth for what is running, never inferred (research-derived, 2026-07-26).

Forensic anchor (genericised): in one cycle a verification sweep produced five distinct mess states — (1) multiple concurrent same-purpose sweeps ran unintentionally (no single-owner-per-purpose claim); (2) a sweep HUNG on a consumer-blocked-on-an-unclosed-producer pipeline and was undetected because liveness was inferred from process-alive (a §11.4.201(6) FALSE-NULL: a wedged op and a progressing op returned the identical "process alive" signal); (3) stopping the owning subagent SILENTLY KILLED its child long-op because ownership was implicit (no handoff-on-stop); (4) a stale/non-growing log was read as live progress (the same false-null, reader-side); (5) an un-completable tooling gate blocked a release for hours (no escape to a terminal state). Per §11.4.102 Phase 4.5 these are ONE architectural state-management flaw, not five bugs: **there was no single source of truth for what long-op is running, who owns it, its state, and its PROVEN liveness** — the §11.4.196(F) configured≠in-use gap generalised to project long-ops, so every mess state was un-observable, and an un-observable bad state is a §11.4/§11.4.1 bluff at the orchestration layer.

The mandate — a **long-op orchestration layer** binding every long-op (any op expected to exceed the runtime background-execution threshold per §11.4.89 — verification sweep, meta-test sweep, build, flash/deploy, test/QA suite, dispatched subagent), ALL of which hold:

(A) SINGLE SOURCE OF TRUTH — the long-op lifecycle registry. Every long-op MUST register BEFORE it runs and reach a terminal state (`complete|failed|reaped|handoff|blocked-escape`) when it ends, in a durable (never-tmpfs, write-temp-then-rename, §11.4.116 append-only event stream + atomic snapshot) registry recording `{op_id, purpose_key, owner, pid|pgid, cwd, state, last_heartbeat, heartbeat_seq, progress_offset, log_path, verdict, evidence_path}` . Success is read from the op's terminal verdict/ `result` , NEVER its process exit code. A long-op running with no registry record — or a record with no live op — is a FALSE-READY drift state.

(B) SINGLE-OWNER-PER-PURPOSE. No two same-`purpose_key` long-ops run at once; acquisition is a probe-race-safe claim keyed on `purpose_key` (§11.4.119). A live claim forces exactly ONE of REUSE (attach) or explicit SUPERSEDE (graceful handoff of the running owner, logged), never a silent second run.

(C) LIVENESS TRUTH-SOURCE. Progress is PROVEN, never inferred from process-alive: every long-op emits a monotonic heartbeat and/or an advancing progress offset; a watchdog reads the DELTA over a window and flags HUNG on no-advance past the op's declared no-progress budget (the §11.4.201(6)-(7)/ §11.4.89 discipline — `kill -0` is a necessary-not-sufficient pre-filter, a stale heartbeat/non-growing log is NEVER "progressing"). A consumer blocked on an unclosed producer write-end (§11.4.201(12)) is caught by the flat heartbeat regardless of internal state.

(D) OWNERSHIP-HANDOFF-ON-STOP. Stopping/killing an owner MUST hand its long-ops to the registry (`handoff` , re-adoptable per §11.4.147) or deliberately reap them (`reaped` , reason logged) — NEVER silently orphan or accidentally cascade-kill; long-ops launch detached (own process-group) so an owner stop does not cascade-signal them, and the stop path records each owned op's disposition. The loop done-condition is not satisfied while a `handoff` op is un-adopted.

(E) SAFE REAPING. An orphaned/hung op is reaped ONLY on PROVEN staleness — holder PID dead (`kill -0` fails) OR no-progress-budget exceeded per (C) — resolved by the process's real `/proc/<pid>/cmdline` (§11.4.196(D)/§12.12), never a bare `pgrep` /substring that could match a carrier or the reaper itself; a LIVE, ADVANCING op is NEVER reaped (§9.2); every REAPED/KEPT decision + resolved evidence is logged (§11.4.180).

(F) CONSISTENCY INVARIANT SWEEP. A fast invariant check runs before every gated transition (build/flash/tag) and on a standing cadence, asserting: no duplicate owner per purpose, no orphan, no stale lock (§11.4.180), no hung op (C), no registry-vs-reality drift (live-long-op set == registry live rows — the §11.4.196(F) check generalised). Any violation is surfaced (named, actionable) and the gated transition is REFUSED until resolved. The sweep is itself a guard (§11.4.201): golden-TRUE + golden-FALSE-with-carrier fixtures, conservative-safe REFUSE on an unresolvable signal, resolved evidence printed on every refusal.

(G) UN-COMPLETABLE-BLOCKER ESCAPE. A gate that cannot reach a terminal verdict MUST degrade, after a bounded budget, to an explicit evidence-backed decision — NEVER an indefinite silent block: §11.4.102 first (hung/(C) vs duplicate/(B) vs wrong-seam/§11.4.120 re-seat-at-activation); NOT-YET-RUNNABLE → §11.4.69 `artifact_not_yet_built` (non-blocking on an intermediate artifact, blocking only on the release candidate per §11.4.135); genuinely un-completable → §11.4.101 (autonomous reversible SKIP-with-reason + tracked §11.4.197 item, ELSE a bounded §11.4.66 operator decision) — the loop keeps progressing every other non-blocked item, it does not idle, and the escape NEVER fabricates a PASS.

Honest boundary (§11.4.6). This layer reduces orchestration mess; it does NOT make product bugs impossible, does NOT make a gate correct (§11.4.108/§11.4.201 still bind each gate), and does NOT remove the operator from genuine blocks — it converts an indefinite silent stall into a bounded, evidence-backed, operator-visible decision. A heartbeat proves ADVANCE not CORRECTNESS; the registry proves what CLAIMS to run, (F) proves registry==reality. **Reuse-not-rewrite:** it is the multitrack per-track agent registry (§11.4.147/§11.4.116/§11.4.187) generalised from per-track workers to any long-op and bound through, per §11.4.196(F) — no parallel mechanism is invented.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their long-op set, purpose keys, no-progress budgets, and stop-hook wiring as DATA per §11.4.35. Composes §11.4.89 / §11.4.101 / §11.4.116 / §11.4.119 / §11.4.120 / §11.4.135 / §11.4.147 / §11.4.180 / §11.4.187 / §11.4.196(D)(F) / §11.4.201(1)(6)-(12) / §11.4.207 / §11.4.69 / §12.6 / §12.12 / §9.2. Propagation gate `CM-COVENANT-114-232-PROPAGATION` (literal `11.4.232`) + recommended mechanism gates `CM-LONGOP-REGISTRY-SINGLE-SOURCE` (A) / `CM-LONGOP-SINGLE-OWNER-PER-PURPOSE` (B) / `CM-LONGOP-LIVENESS-HEARTBEAT` (C) / `CM-LONGOP-OWNERSHIP-HANDOFF-ON-STOP` (D) / `CM-LONGOP-SAFE-REAP-PROVEN-STALE` (E) / `CM-ORCHESTRATION-CONSISTENCY-SWEEP` (F) / `CM-UNCOMPLETABLE-BLOCKER-ESCAPE` (G) + paired §1.1 mutations (run two same-purpose sweeps with no reuse/supersede → (B) FAILs; infer liveness from `kill -0` alone / drop the heartbeat-delta check → (C) FAILs; a stop that leaves an owned op with no terminal/handoff record → (D) FAILs; reap a live-advancing op OR reap by bare-`pgrep` substring → (E) FAILs; a live long-op with no registry row passes the consistency sweep → (F) FAILs; an un-completable gate blocks with no escape/decision → (G) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6). No escape hatch — no `--allow-`

`duplicate-longop`, `--infer-liveness-from-pid`, `--silent-orphan-on-stop`, `--reap-live-op`, `--skip-consistency-sweep`, `--block-without-escape`, `--longop-without-registry` flag.

EXTENSION landed 2026-07-26 — §11.4.182 model-in-label + flawless alias/provider↔model consistency + real-time always-in-sync (User mandate, 2026-07-26). Verbatim operator mandates (2026-07-26): "Extend our logging to present models used since we will now have dynamic models choice and switching! From (example) (T1/main - claude4) into: (T1/main - claude4 - model_used), model MUST be dynamically obtained so we present name equivalents - short names - like: haiku, opus, sonnet, fable. However we MUST fully support all Claude Toolkit providers aliases, so real model used names MUST BE obtained in this case(s) as well! Examples: (T1/main - claude4 - haiku), (T1/main - claude4 - fable), (T1/main - claude4 - opus), (T1/main - claude4 - deepseek), (T1/main - claude4 - xiaomi) ... Labels MUST BE everywhere ALWAYS up to date in real time and in sync!" and "Adding model names into labels considers that providers names / aliases will be up to date too and in relation to model used since models belong to providers / aliases !!! Consistency MUST BE flawless!" This EXTENDS the §11.4.182 label discipline (it does NOT re-mint the anchor — the literal `11.4.182` and its propagation gate `CM-COVENANT-114-182-PROPAGATION` are UNCHANGED and stay GREEN; this carrier block cites §11.4.182, it is not a second §11.4.182 block-start per §11.4.227(B)). (1) **LABEL FORM EXTENDED** — the mandated §11.4.182 label is extended from `(T<N>/<branch> - <alias>)` to `(T<N>/<branch> - <alias> - <model> - <effort>)` (the trailing `<effort>` field added 2026-07-26 per §11.4.231 clause (F) — see the EFFORT-FIELD clause (7) below), `<model>` recording which model is CURRENTLY answering for that alias: for a native Claude alias, the SHORT name (`opus` / `sonnet` / `haiku` / `fable`, case-insensitive-substring-derived from the model id the agent platform reports for the most recent turn — DYNAMIC, obtained live at label-emission time, never a fixed/assumed value since native aliases have no persistent per-alias model and the serving model can legitimately switch mid-session per §11.4.231); for a claude-toolkit PROVIDER alias (`deepseek`, `xiaomi`, `kimi-for-coding`, `opencode`, ...) the REAL provider model id as declared in that provider's host-config registry (the SAME value the alias's own launch mechanism uses — a single source of truth, never re-derived). (2) **MODEL-FIELD DERIVATION (§11.4.6, never guessed)** — a closed two-case split on the SHAPE of the already-derived `<alias>`: (a) native-alias → read the model id from the CURRENT session's own transcript (the platform's per-

project per-session transcript; LAST assistant-turn's reported model → short name); (b) provider-alias → read the model id from the host multi-account-toolkit provider registry (`CMA_PROVIDER_MODEL`). Either lookup that cannot resolve yields the honest `?` — NEVER a fabricated or last-known value; an id obtained but matching none of the four native short-names is printed VERBATIM (real-but-unrecognised is more honest than a false `?`). The 3-field legacy form (no `<model>`) remains a VALID, ACCEPTED label during migration — the enforcing guard-hook `LABEL_RE` admits both forms, and NEVER treats an honest `?` model, or an absent model field, as a block-cause. (3) **FLAWLESS ALIAS/PROVIDER↔MODEL CONSISTENCY INVARIANT (operator NOTE, load-bearing)** — the `<alias>` / provider field and the `<model>` field MUST be MUTUALLY CONSISTENT AT ALL TIMES: the model shown MUST be a model that genuinely BELONGS to the alias/provider shown (models belong to providers/aliases), and the alias/provider name MUST itself be current — never a stale alias beside a fresh model, never a model that does not belong to the shown alias. This is guaranteed BY CONSTRUCTION, not by coincidence: the native model is read from THAT alias's own live session transcript, and the provider model is read from THAT provider's own registry entry — so the derived `<model>` is definitionally a model of the shown `<alias>` ; a derivation that ever emits a model not belonging to the shown alias is a §11.4.6 / §11.4.182 consistency violation, never a valid label. (4) **REAL-TIME, ALWAYS-IN-SYNC (§11.4.229/§11.4.106(F))** — the label is STATELESS and RE-COMPUTED at EVERY emission (every human-invoked labeler run AND every synchronous PreToolUse guard-hook firing on every Agent/Task/TaskCreate dispatch) against the LIVE environment (current `CLAUDE_CONFIG_DIR` , current session id, current transcript tail, current provider `.env`) — so a mid-session model switch (`/model` , an autonomous §11.4.231 escalation) is reflected in the VERY NEXT emission with no cache to go stale: real-time BY CONSTRUCTION, not by a sync daemon. Honest boundary (§11.4.6): the live label has NO persisted artifact that could drift, so the Docs-Chain (§11.4.106) sync engine — which propagates PERSISTED documents/DBs — does not apply to the stateless label itself; the operator's "use/extend Docs Chain for real-time responsiveness" ask attaches to any PERSISTED label-history / model-switch LOG surface (a §11.4.197 follow-up to be tracked), which, if adopted, IS §11.4.106-Docs-Chain-synced and §11.4.208-request-history-adjacent — never faked as already-shipped. (5) **HONEST SUBAGENT-INTROSPECTION LIMIT (§11.4.6/§11.4.197)** — the native-alias transcript lookup is proven reliable when invoked from the DISPATCHING process's own shell (the

guard-hook's actual execution context at dispatch time); it is NOT proven reliable when a background-dispatched subagent introspects its OWN model from inside its own subprocess on hosts where the subagent shares its parent's session-transcript with no subagent-vs-conductor marker — recorded as an open limitation to be tracked, never silently claimed solved. (6) **DIVERGENT-LABELER RECONCILIATION (§11.4.227)** — the LIVE labeler wired via `.claude/settings.json` is `constitution/scripts/multitrack/track_branch_label.sh` (both it and the top-level `scripts/multitrack/` duplicate carry the model-field extension (functionally equivalent on the alias+model fields, not byte-identical — the constitution copy additionally carries the 2026-07-26 effort field the top-level copy does not yet mirror)); the two copies are reconciled to a single source of truth (top-level a thin wrapper exec-ing the constitution copy, §11.4.177 inherited-by-reference) as a reconciliation to be tracked, never a silent divergence. (7) **EFFORT FIELD (added 2026-07-26 per the operator IMPORTANT mandate — verbatim: "Besides dynamic determining proper models to use, we MUST decide the effort as well and switch between the effort settings same as we do now for chosen model! Labeling MUST BE extended for this details as well. Example: (T1/main - claude4 - opus - high).").** The 5-field label appends `<effort>` after `<model>`, recording which reasoning-EFFORT setting is CURRENTLY in force for that work-stream, drawn from the closed set `{low | medium | high | xhigh | max}` (the SAME effort ladder §11.4.231 clause (F) selects). (a) DERIVATION (§11.4.6, never guessed) — `<effort>` is read from whatever LIVE effort signal the execution path exposes (the Workflow-tool `agent()` `effort` argument, a future Agent-tool effort parameter, an env/config effort setting); a path exposing NO effort signal, OR any value outside the closed set, yields the honest `?` — EXACTLY like the `<model>` field's `?` fallback, NEVER a fabricated / assumed / last-known value. (b) HONEST HARNESS BOUNDARY (§11.4.6, load-bearing — do NOT overstate the mechanism) — effort-tiering is REAL where the execution path exposes it and a DOCUMENTED CAPABILITY-GAP where it does not: the current Claude Code **Agent tool** (subagent dispatch) exposes a `model` parameter but **NO `effort` parameter**, so a subagent's effort is NOT currently settable via that path and its `<effort>` field degrades honestly to `?`; the **Workflow tool's `agent()`** DOES take an `effort` argument, so effort IS settable + derivable there — the label reports whichever path is live, never invents one. This is the SAME aspirational-vs-harness state the §11.4.209 / §11.4.211 Fable-`xhigh` review/

merge pins already occupy: the pins mandate the effort, and whether a given dispatch path can SET it is a harness capability the label reports honestly and NEVER fakes. (c) LEGACY-FORM ACCEPTANCE (migration) — the 3-field `(T<N>/<branch> - <alias>)` and 4-field `(T<N>/<branch> - <alias> - <model>)` label forms remain VALID and ACCEPTED; the guard `LABEL_RE` admits all three shapes and NEVER treats an absent `<effort>` field, or an honest `?` effort, as a block-cause (§11.4.6 — mirroring the §11.4.182 track/alias/model honest-`?` rule). (d) CONSISTENCY — the effort field is selected TOGETHER WITH the model by §11.4.231 (both tiers chosen for one task) and never contradicts it; a stale effort beside a fresh model, or an effort outside the closed set, is a §11.4.6 / §11.4.182 consistency violation, never a valid label. (e) REAL-TIME, ALWAYS-IN-SYNC (§11.4.229) — like `<model>`, `<effort>` is STATELESS and RE-COMPUTED at EVERY label emission against the LIVE environment, so a mid-task §11.4.231 effort escalation is reflected in the VERY NEXT emission with no cache to go stale. Propagation: `CM-TRACK-BRANCH-LABEL` is EXTENDED to also assert the labeler emits a parseable `<model>` field AND a parseable `<effort>` field, the guard `LABEL_RE` accepts the 3-field / 4-field / 5-field label forms (legacy 3-field and 4-field ALWAYS remain accepted during migration; a `?` effort is NEVER a block-cause), and the alias/provider↔model consistency holds; paired §1.1 mutation: strip the `<model>` derivation OR emit a model not belonging to the shown alias → the extended gate FAILs (gate-code = separate work item, NOT claimed shipped §11.4.6). No new escape hatch — the existing §11.4.182 "no escape hatch" list applies to the extended label unchanged.

§11.4.233 — Standing anti-mess control plane: a level-triggered, idempotent, invariant-catalogue reconciliation loop over the WHOLE persistent System state, orchestrating every per-domain enforcer, refusing gated transitions on un-reconciled drift (research-derived, 2026-07-26).

Forensic anchor (genericised): every existing consistency enforcer in a multi-track / multi-agent / multi-submodule / multi-upstream System is EDGE-TRIGGERED — a PreToolUse hook fires on an action, a check runs at a seam (lock-acquisition / merge-time / build-time / commit-time), a gate runs when a build runs — so a mess that arises WITHOUT an edge firing is un-observed until it causes harm (two live examples captured 2026-07-26: a large uncommitted-batch pile-up sat unreconciled through a session, and multiple provably-stale VCS/wrapper locks were held by dead holders with no acquisition attempt to reap them — the §11.4.205 "a hook never installed / a check that runs only when asked" forensic

generalised to the whole persistent state). Per §11.4.102 Phase 4.5 the ~19 persistent-state mess classes (Plane R repository: uncommitted pile-up, stale/duplicate locks, orphaned worktrees, half-applied cross-repo commits, drifted submodule pointers, partial merges, divergent mirrors, mis-placed work; Plane G governance: divergent mirrors, pointer drift, gate-ledger drift, anchor-number collision; Plane S tracking: doc/DB drift, cross-track SSoT divergence, ledger drift, handoff-doc drift; Plane P runtime: the §11.4.232 long-op set + un-reaped streams + dangling review state) are ONE architectural flaw — there is no LEVEL-TRIGGERED control plane that re-derives the actual persistent state and reconciles it against a declared clean state; an un-observed drift is a §11.4/§11.4.1 bluff at the organization layer ("the System is clean" while a mess silently accumulates).

The mandate — a standing **anti-mess control plane** (the desired-state/actual-state reconciliation-loop + control-plane/data-plane pattern of distributed systems), ALL of which hold:

(A) DECLARED INVARIANT CATALOGUE. A single machine-readable consumer-owned catalogue (DATA per §11.4.28/§11.4.35) enumerating every consistency invariant across the persistent planes, each carrying `{id, plane, desired-condition, detector, reconcile-class ∈ {auto-safe | operator-gated | owned-by <anchor>}, evidence-shape}` ; the catalogue IS the desired clean state; an un-catalogued mess class is an honest tracked GAP (§11.4.6/§11.4.197), never silently claimed covered.

(B) LEVEL-TRIGGERED LOOP. The loop re-derives the ACTUAL state (not an event) and diffs it against the catalogue BEFORE every gated transition (commit-batch/build/flash/merge-to-trunk/tag/constitution-pull) AND on a standing cadence (§11.4.94/§11.4.103) — catching drift regardless of how it arose (the Kubernetes level-triggered property); actual-vs-desired uses a §11.4.86 fingerprint diff drilling to the offending record (anti-entropy), never a whole-scan where a fingerprint suffices.

(C) IDEMPOTENT RECONCILE-OR-REFUSE. Each drift takes exactly one dedup-keyed action: RECONCILE-AUTO-SAFE (reversible + evidence-determinable + bounded-blast per §11.4.101 — reap a provably-dead lock §11.4.180, regen a stale derived doc §11.4.106, scope-group-commit a placement-clean batch §11.4.191, retire a merged worktree §11.4.167), idempotent (a no-op on a clean state); OR REFUSE-THE-GATE + a bounded operator decision (irreversible/high-blast/indeterminate — id-colliding SSoT §11.4.206, anchor collision §11.4.227, partial

cross-repo merge, divergent mirror) per §11.4.101/§11.4.66; a gated transition NEVER proceeds while a catalogued invariant is violated and un-reconciled; the loop keeps progressing every other non-blocked item (park-one-not-the-loop).

(D) SAGA FORWARD-RECONCILE (no-loss). A partially-applied cross-repo operation is reconciled complete-forward under a §9.2 pre-op backup (finish the remaining legs / bump the pointer / complete the merge zero-markers-zero-drop §11.4.41 step 3 / ff the lagging mirror) — NEVER force-push/reset/ `-s ours` (§11.4.113/§9.2); compensation is §9.2 backup-restore only; the saga reaches fully-completed, never rolled-back-with-loss.

(E) THE LOOP IS A GUARD (§11.4.201). Every detector ships golden-TRUE + golden-FALSE-with-carrier fixtures (§11.4.107(10)); a false-positive REFUSAL of a clean state is a §11.4.201(1) FAIL-bluff exactly as a false-negative PASS is a §11.4 PASS-bluff; every "no drift" null is control-needle-proven (§11.4.201(6)-(12)); conservative-safe REFUSE on an unresolvable signal, resolved evidence printed on every refusal.

(F) CONTROL-PLANE / DATA-PLANE SEPARATION. The control plane DETECTS + RECONCILES state and NEVER performs the data-plane work (the actual build/commit/merge/long-op — §11.4.232 + the build/flash/commit wrappers); the two never merge; the control plane's own state is durable (§11.4.205(6) temp→fsync→rename, never tmpfs, may reuse §11.4.207).

Honest boundary (§11.4.6). Converges the CATALOGUED invariants; does NOT make an un-catalogued mess class impossible (extensible catalogue, never "provably mess-free"), does NOT make a gate correct (§11.4.108/§11.4.201), does NOT auto-fix irreversible drift (operator-gated §11.4.101/§11.4.66), and is eventual-consistency-at-every-gate-and-cadence NOT a real-time daemon (§11.4.205(7)). **Reuse-not-rewrite:** it is the level-triggered COMPOSER over the existing per-domain enforcers (§11.4.180/§11.4.186/§11.4.106/§11.4.191/§11.4.206/§11.4.227/§11.4.232/§11.4.147/§11.4.167), inheriting each's detector — no parallel enforcer is invented (§11.4.227). It composes §11.4.232 (the RUNTIME data-plane long-op orchestrator — §11.4.232 clause (F) is Plane P of this catalogue) as the plane it does not touch.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their invariant catalogue, plane detectors, gated-transition list, cadence, and reconcile-class bindings as DATA per §11.4.35. Composes §11.4.6 /

§11.4.26 / §11.4.28 / §11.4.32 / §11.4.84 / §11.4.86 / §11.4.101 / §11.4.106 / §11.4.108 / §11.4.116 / §11.4.121 / §11.4.147 / §11.4.164 / §11.4.167 / §11.4.176 / §11.4.179 / §11.4.180 / §11.4.181 / §11.4.186 / §11.4.191 / §11.4.196(F) / §11.4.201 / §11.4.205 / §11.4.206 / §11.4.207 / §11.4.227 / §11.4.232 / §12.6 / §9.2 / §2.1. Propagation gate `CM-COVENANT-114-233-PROPAGATION` (literal `11.4.233`) + recommended mechanism gates `CM-ANTIMESS-INVARIANT-CATALOGUE` (A) / `CM-ANTIMESS-LEVEL-TRIGGERED-LOOP` (B) / `CM-ANTIMESS-RECONCILE-OR-REFUSE` (C) / `CM-ANTIMESS-SAGA-FORWARD-NO-LOSS` (D) / `CM-ANTIMESS-LOOP-IS-A-GUARD` (E) / `CM-ANTIMESS-CONTROL-DATA-PLANE-SPLIT` (F) + paired §1.1 mutations (drop the standing-cadence trigger so the loop is edge-only → (B) FAILs on the pile-up/stale-lock fixture; let a gated transition proceed on a catalogued un-reconciled drift → (C) FAILs; a saga that force-pushes/resets to reconcile a mirror → (D) FAILs; a detector that REFUSES a clean golden-TRUE fixture, or PASSes a golden-FALSE-with-carrier → (E) FAILs; the control plane invokes a data-plane build/commit → (F) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). No escape hatch — no `--edge-triggered-only`, `--skip-consistency-catalogue`, `--proceed-on-drift`, `--reconcile-with-force-push`, `--unvalidated-detector`, `--control-plane-does-work`, `--assume-clean-without-reconcile` flag.

§11.4.234 — Dedicated hook-validation script: hook checks run as an explicit stage of a dedicated commit/push script, and the commit/push mechanism is ALWAYS unblocked (operator mandate, 2026-07-26).

Forensic anchor (genericised): a consumer project wired its full local CI gate as an automatic git `pre-push` hook (`core.hooksPath` → hook → multi-minute build/test/sweep stage on EVERY push). The result, captured 2026-07-26: routine pushes took many minutes each, and when one gate stage inside the hook failed or stalled, the push was rejected outright — the commit/push mechanism itself became blocked, with pending commits stranded locally and no operator-usable path forward short of a forbidden `--no-verify` bypass. The gate logic was sound; wiring it as an uninterruptible implicit hook on the commit/push code path was the architectural flaw — an edge-triggered enforcer (§11.4.233) placed directly on the operator's only forward path, violating the §2 single-entrypoint discipline by making the entrypoint unusable.

The mandate, ALL of which hold:

(A) DEDICATED SCRIPT AS THE SINGLE COMMIT/PUSH ENTRYPOINT. Every consumer MUST ship a dedicated, executable, idempotent script (Lava binding: `scripts/commit-push-all.sh` ; consumers bind their own path as DATA per §11.4.35) that runs, in order: (1) submodule/workspace sync, (2) commit of all pending state, (3) the hook validations as an EXPLICIT stage — the unmodified hook logic fed the real push range, Layer-cheap checks always, long gates stage-separated, (4) push to ALL configured upstreams (§2.1), (5) a closing green/clean verification (clean status + zero unpushed commits, recursive over submodules). The script prints per-stage evidence and is safe to re-run; direct `git commit` / `git push` on the main repo ceases to be the routine workflow (§2).

(B) HOOKS MUST NOT BLOCK THE MECHANISM. Automatic git hooks wired via `core.hooksPath` MUST NOT gate routine commit/push in a way that can leave the repository uncommittable or unpushable. Disconnecting means: `core.hooksPath` unset, the hook file itself preserved UNMODIFIED in the repo, and `--no-verify` NEVER used as the routine path. Validation relocates from the implicit hook to the explicit script stage (A) and to tag/release-time gates — it is never deleted.

(C) NO GATE IS LOST. Every check the disconnected hook performed MUST remain executed — by the dedicated script's validation stage, by the tag/release gate, or by a documented scheduled run. Dropping a check silently because the hook no longer fires is a §11.4 PASS-bluff at the process layer ("the gate still exists" while nothing invokes it); the script's stage list MUST name every check it runs and every check it defers, with the deferral target recorded.

(D) ALWAYS-UNBLOCKED INVARIANT. The commit/push mechanism MUST always be able to complete. A failing validation yields a clear per-check report plus a documented remediation path — never an opaque hung or rejected push. Long gates (multi-minute build/test/sweep stages) MUST be separable from the cheap checks and skippable via an explicit, documented flag (Lava binding: `LAVA_SYNC_SKIP_CI=1`), with every skip recorded in the commit message or evidence pack so the deferred gate is tracked, never forgotten. The escape hatch is a recorded deferral, not a bypass: the skipped gate remains owed and is caught at the next full run or tag gate.

Honest boundary (§11.4.6). This anchor relocates WHERE validation executes (implicit hook → explicit script stage + release gates); it does NOT weaken any substantive gate — the Sixth/Seventh-Law-equivalent anti-bluff covenant, the tag-

time full gate, and §11.4.32's sweep-after-constitution-pull all still bind at their own seams. It does NOT mandate deleting hooks (the hook file is preserved), does NOT authorise `--no-verify` as routine practice, and does NOT make a failing gate pass — it makes the mechanism reachable while the failure is remediated. Composes §2 / §2.1 (single entrypoint + all-upstreams push — this anchor keeps that entrypoint usable), §11.4.32 (sweep still runs, now as a named stage), §11.4.113 (no force-push history-rewrite — the unblocked path never rewrites), §11.4.201 (the script is a guard: a false GREEN verification is a FAIL-bluff), §11.4.227 (amendment discipline), §11.4.233 (the commit/push path is a gated transition the anti-mess control plane reconciles).

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their script path, upstream-remote list, cheap-vs-long gate split, and skip-flag name as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-234-PROPAGATION` (literal `11.4.234`) + recommended mechanism gates `CM-DEDICATED-HOOK-VALIDATION-SCRIPT` (A) / `CM-HOOKS-NEVER-BLOCK-PUSH` (B) / `CM-NO-GATE-LOST-ON-HOOK-DISCONNECT` (C) / `CM-COMMIT-PUSH-ALWAYS-UNBLOCKED` (D) + paired §1.1 mutations (wire the full gate back as an automatic blocking pre-push hook → (B) FAILs; disconnect the hook AND drop a check from every stage → (C) FAILs; a validation failure that strands commits with no remediation report → (D) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). No escape hatch beyond the recorded (D) deferral — no `--no-verify-routine`, `--skip-all-gates-silently`, `--delete-blocking-hook`, `--push-without-any-validation` flag.

§11.4.235 — Build-and-deploy the moment source fixes are proven-correct (test-side hardening is a parallel stage, never a build gate) + post-deploy version increment (operator mandate, 2026-07-27).

(A) BUILD-AND-DEPLOY THE MOMENT THE SOURCE FIXES ARE PROVEN-CORRECT — TEST-SIDE WORK IS A PARALLEL STAGE, NEVER A BUILD GATE. The moment the SOURCE fixes of a batch are done AND proven-correct — proven-correct = the mandatory independent SOURCE-correctness review has returned a clean GO (§11.4.125/§11.4.142/§11.4.194 on the source diff, on the §11.4.209 Fable-`xhigh` substrate, iterated to zero-finding/zero-warning GO per §11.4.134) — the build MUST be triggered, pre-build + post-build tests EXECUTED (§11.4.108 layer-1 before, layer-2 after), and the artifact deployed. IN PARALLEL

WITH the build AND the manual-QA session (§11.4.185), NEVER gating the build: (1) gate-instrumentation hardening + new-gate authoring (§11.4.110); (2) paired §1.1 mutations + four-layer §11.4.4(b) coverage + HelixQA Challenges; (3) "gates and checks of all kinds" (§11.4.96); (4) the re-review of THAT test-instrumentation work. The build's GO-SIGNAL is source-correctness alone; test-guard hardening (future-regression coverage) + its re-review are a deploy-independent parallel stage per §11.4.230(A). Holding the build sequentially — waiting for the test-instrumentation buildout + its re-review — when the source is already proven-correct is a §11.4.235(A) violation.

(B) POST-DEPLOY VERSION INCREMENT + MANUAL-QA-DEPLOY CYCLE BOUNDARY (extended 2026-07-28 — operator mandate, verbatim: "Increase upcoming release to 0.1.5 since new cycle starts every time we deploy release for manual QA testing! This is how we will be working this ALWAYS! Extend the constitution!"). The moment image(s)/artifact(s) deploy, the version code/version is INCREMENTED (next build targets the next version; every deployed artifact carries a distinct, monotonic, greppable id — §11.4.151/§11.4.108/§11.4.200). Two deployments sharing one version id defeats per-deploy artifact identity → §11.4.235(B) violation. EVERY deployment of a release FOR MANUAL QA TESTING (§11.4.185) is a DEVELOPMENT-CYCLE BOUNDARY: the manual-QA deploy CLOSES the outgoing cycle and STARTS a NEW development cycle, and the version code/version is incremented AT THAT POINT — the new cycle's work targets the NEXT monotonic version (§11.4.151 project-prefixed naming preserved) — ALWAYS, the STANDING working model per clause (C), never a per-request opt-in. The manual-QA deploy is the canonical clause-(B) increment trigger (a deploy-for-manual-QA IS a deploy), and it SHAPES the §11.4.126 release-cycle framing: fixes for QA findings against the deployed artifact land in the NEW cycle on the NEW version id, never patched onto the already-QA-deployed version id. Starting a new cycle's work still targeting the QA-deployed version id is a §11.4.235(B) violation.

(C) STANDING DEFAULT (§11.4.126/§11.4.230/§11.4.103), never a per-request opt-in.

Honest boundary (§11.4.6): relocates the TEST-INSTRUMENTATION re-review + gate/check hardening OFF the build critical path; does NOT weaken any substantive gate. The SOURCE-correctness review (§11.4.125/§11.4.142/§11.4.194 Fable-`xhigh` §11.4.209 → GO §11.4.134) STILL PRECEDES + gates

the build (it IS "source proven-correct"); pre/post-build tests (§11.4.108 L1-2) run at their seams; §11.4.185 manual-QA-final un-weakened; §11.4.129/§11.4.40 un-weakened. "Test-side work is parallel, never a build gate" is bounded to future-regression instrumentation — a check that proves the SHIPPED artifact wrong is a source/artifact-correctness gate (§11.4.108 L1-2), stays on the critical path.

Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-235-PROPAGATION` (literal `11.4.235`) + recommended `CM-BUILD-ON-SOURCE-PROVEN-NOT-TEST-SIDE` (FAIL: build held past source-GO w/ unfinished test-instrumentation; FAIL: build started with NO source-review GO — the §11.4.201(1) dual-assertion guard) + `CM-VERSION-INCREMENT-ON-DEPLOY` (FAIL: new-artifact deploy shares a version id; FAIL: work of the cycle opened by a manual-QA deploy still targets the QA-deployed version id — the clause-(B) cycle-boundary increment skipped) + paired §1.1 mutations; gate-code = separate work item (§11.4.6/§11.4.227).

Non-compliance is a release blocker. No escape hatch — no `--build-behind-test-hardening`, `--test-side-gates-the-build`, `--skip-source-correctness-review`, `--deploy-without-version-increment`, `--not-default-speed-mode`, `--serialise-build-and-validate`.